The 2007 Federated Conference on

Rewriting, Deduction and Programming

Paris, France

June 25 – 29, 2007



WFLP'07

16th International Workshop on Functional and (Constraint) Logic Programming

June 25th, 2007

Proceedings

Editor :

Rachid Echahed

Contents

Preface iv

Sebastian Fischer, Josep Silva, Salvador Tamarit and German Vidal
Towards a Safe Partial Evaluation of Lazy Functional Logic Programs 1

Bernd Brassel, Sebastian Fischer and Frank Huch
Declaring Numbers 17

Emilio Jess Gallego Arias, Julio Mario and Jose Maria Rey
A Generic Semantics for Constraint Functional Logic Programming .. 31

Murdoch gabbay and Michael gabbay
a-logic with an arrow 47

Javier lvez and Paqui Lucio
A Decision Procedure for Non-failure in CLP 63

Wolfram Kahl
Call-By-Undetermined-Value in the Pattern Matching Calculus 79

Wolfgang Lux
Declarative Debugging Meets the World 97

Rafael Caballero-Roldan, Yolanda Garca-Ruiz and Fernando Senz-Prez
A New Proposal for Debugging Datalog Programs 109

Ron van Kesteren, Olha Shkaravska and Marko van Eekelen
Inferring static non-monotonically sized types through testing 123

Tetsuo Ida, Mircea Marin, Hidekazu Takahashi and Fadoua Ghourabi
Computational Origami Construction as Constraint Solving and Rewriting 139

Michele Baggi and Demis Ballis
PHIL: A Lazy Implementation of a Language for Approximate Filtering of XML Documents 153

Alexei Lescaylle and Alicia Villanueva
Using tccp for the Specification and Verification of Communication Protocols 169

Preface

This volume contains the proceedings of the 16th International Workshop on ( ) ('). The workshop was held in Paris, France on June 25th, 2007, as part of the fourth edition of the International , , (').

The aim of WFLP workshop series is to bring together researchers interested in functional programming, (constraint) logic programming, as well as the Integration of the two paradigms. It promotes the cross-fertilizing exchange of ideas and experiences among researchers and students from the different communities interested in the foundations, applications, and combinations of high-level, declarative programming languages and related areas. The previous WFLP editions took place in Madrid (2006), Tallinn (2005), Aachen (2004), Valencia (2003), Grado (2002), Kiel (2001), Benicassim (2000), Grenoble (1999), Bad Honnef (1998), Schwarzenberg (1997, 1995, 1994), Marburg (1996), Rattenberg(1993) and Karlsruhe (1992).

These proceedings contain 12 papers, selected out of 21 submitted abstracts. All submissions were subject to careful refereeing, with at least three reviews for each paper. The topics of selected papers range over a wide spectrum, including language design, semantics, logics, testing and debugging as well as applications of declarative programming languages. Selected papers from these proceedings will be published as a volume of the series Electronic Notes in Theoretical Computer Science (ENTCS).

I would like to thank all the people who contributed to the success of WFLP'07, especially the Program Committee and the additional reviewers for their valuable contributions to the selection process as well as the contributing authors without whom this volume would not exist.

I would like also to express my gratitude to all members of the RDP07 Organizing Committee as well as the institutional sponsors of RDP'07: the Conservatoire des Arts et Métiers (CNAM), the Centre National de la Recherche Scientifique (CNRS), the École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise (ENSIEE), the GDR Informatique Mathématique, the Institut National de Recherche en Informatique et Automatique (INRIA) unit Futurs, and the Région Île de France.

The organization of WFLP'07 has been partly funded by the project - of the French .

June, 2007 Rachid Echahed

Program committee of WFLP'07

Sergio Antoy	Portland State University, USA
Rachid Echahed	CNRS,laboratoire LIG, Grenoble, France (Chair)
Santiago Escobar	Technical University of Valencia, Spain
Moreno Falaschi	Universita di Siena, Italy
Michael Hanus	Christian-Albrechts-Universitt zu Kiel, Germany
Tetsuo Ida	University of Tsukuba, Japan
Herbert Kuchen	Westflische Wilhelms-Universitt Mnster, Germany
Francisco J. Lpez-Fraguas	Universidad Complutense de Madrid, Spain
Wolfgang Lux	Westflische Wilhelms-Universitt Mnster, Germany
Mircea Marin	University of Tsukuba, Japan
Jos Meseguer	University of Illinois at Urbana-Champaign, USA
Juan J. Moreno-Navarro	Universidad Politecnica de Madrid, Spain
Germn Vidal	Technical University of Valencia, Spain

Additional Reviewers

Demis Ballis
Bernd Brassel
Marco Comini
Sebastian Fischer
Pedro Lopez-Garcia
Sunita Marathe
Julio Marinno
Susana Munoz-Hernandez
Susana Nieva
Maria Jose Ramirez
Miguel A. Salido
Jaime Sanchez-Hernandez
Thierry Boy de la Tour
Rafael del Vado Virseda

Towards a Safe Partial Evaluation of Lazy Functional Logic Programs

Sebastian Fischer[1]

Department of Computer Science, University of Kiel
Olshausenstr. 40, D-24098, Kiel, Germany

Josep Silva[2] Salvador Tamarit[2] Germán Vidal[2]

DSIC, Technical University of Valencia
Camino de Vera S/N, E-46022 Valencia, Spain

Abstract

Partial Evaluation is a well-known technique for specializing programs w.r.t. a given restriction of their input data. Although partial evaluation has been widely investigated in the context of functional and functional logic languages like Haskell or Curry, current schemes are either overly restrictive or destroy sharing through the specialization process, which may produce incorrect specializations when non-deterministic functions are considered. In this work, we present a new partial evaluation scheme for lazy functional logic programs that preserves sharing through the specialization process and still allows the unfolding of arbitrary functions. Furthermore, our approach ensures that sharing is also preserved across non-deterministic computations.

Keywords: Partial evaluation, laziness

1 Introduction

Partial evaluation is a well-known technique for the specialization of programs w.r.t. part of their input data. Consider a program P whose input data are split into *static* (i.e., known), say $\mathbf{s}$, and *dynamic* (i.e., unknown) data, say $\mathbf{d}$. Then, the partial evaluation of P w.r.t. the static data $\mathbf{s}$ returns a *residual* program $P_{\mathbf{s}}$ specialized for $\mathbf{s}$ which is equivalent to the original program w.r.t. the dynamic data, i.e., $P(\mathbf{s}, \mathbf{d}) = P_{\mathbf{s}}(\mathbf{d})$ for all $\mathbf{d}$.

Partial evaluators fall in two main categories, *online* and *offline*, according to the time when termination issues are addressed. Online partial evaluators include a single, monolithic stage which performs partial computations, termination tests, generation of residual rules, etc. Offline partial evaluators, on the other hand, include

[1] Email: sebf@informatik.uni-kiel.de
[2] Email: {jsilva,stamarit,gvidal}@dsic.upv.es

two separate stages. The first stage—called Binding-Time Analysis (BTA)—mainly propagates the static data through the entire program and often includes a static termination analysis. The outcome of the BTA is a copy of the original program with a number of *annotations* to guide the specialization process (i.e., whether a function call should be unfolded or not, whether a function argument should be generalized or not, etc). The second stage is usually an extension of an interpreter for the considered programming language: it takes an annotated program and evaluates it as much as possible according to the annotations.

An offline partial evaluation scheme for first-order functional and functional logic programs has recently been proposed in [12]. Unfortunately, this scheme does not include a special treatment for sharing, which means that either the unfolding strategy should be rather restrictive or it might destroy sharing through the specialization process, which may produce incorrect specializations when non-deterministic functions are considered. Let us illustrate this problem with a simple example. Consider the following program (natural numbers are built from `Z` and `S`):

```
main       = double e
double x   = add x x
add Z y    = y
add (S x) y = S (add x y)
...
```

where `e` is an expression whose evaluation is expensive. Here, if we allow the unfolding of function `double`, we could get the following specialized program:

```
main       = add e e
add Z y    = y
add (S x) y = S (add x y)
...
```

where the evaluation of `e` is now duplicated, which is unacceptable. Observe that this duplicated evaluation does not occur in the original program since the two occurrences of variable `x` in the right-hand side of function `double` are *shared* in current lazy functional languages like Haskell or Curry.

Furthermore, if the evaluation of `e` is non-deterministic, then the specialized program would not be sound. Consider, for instance, that `e` $\equiv$ `coin` with

```
coin = Z
coin = S Z
```

Then, `main` can be non-deterministically reduced to `Z` and `S (S Z)` in the original program, while it could also be reduced to `S Z` in the specialized one.

Clearly, one can easily avoid this situation by forbidding the unfolding of functions whose right-hand side is not *linear* (i.e., whose right-hand side contains multiple occurrences of the same variable). However, this turns out to be rather restrictive

$$
\begin{array}{rcll}
P & ::= & D_1 \ldots D_m & \text{(program)} \\
D & ::= & f(x_1, \ldots, x_n) = e & \text{(function definition)} \\
e & ::= & x & \text{(variable)} \\
 & | & c(x_1, \ldots, x_n) & \text{(constructor call)} \\
 & | & f(x_1, \ldots, x_n) & \text{(function call)} \\
 & | & \mathit{let}\ \{\overline{x_k = e_k}\}\ \mathit{in}\ e & \text{(let binding)} \\
 & | & x_1\ \mathit{or}\ x_2 & \text{(disjunction)} \\
 & | & \mathit{case}\ x\ \mathit{of}\ \{\overline{p_k \to e_k}\} & \text{(case expression)} \\
p & ::= & c(x_1, \ldots, x_n) & \text{(pattern)}
\end{array}
$$

Domains

$$
\begin{array}{rcll}
P_1, P_2, \ldots & \in & \mathit{Prog} & \text{(Programs)} \\
x, y, z, \ldots & \in & \mathit{Var} & \text{(Variables)} \\
a, b, c, \ldots & \in & \mathcal{C} & \text{(Constructors)} \\
f, g, h, \ldots & \in & \mathcal{F} & \text{(Functions)} \\
p_1, p_2, p_3, \ldots & \in & \mathit{Pat} & \text{(Patterns)}
\end{array}
$$

Fig. 1. Syntax for normalized flat programs

in practice. To the best of our knowledge, there is no approach to partial evaluation that includes a non-trivial treatment of sharing during the specialization process.

In this work, we present a first step towards the development of a partial evaluation scheme for lazy functional logic programs that is not overly restrictive and still ensures that sharing is not destroyed. For this purpose, we extend a natural semantics for lazy functional logic languages so that it becomes appropriate to perform *partial* computations. Then, we describe how residual programs can be extracted from the partial computations performed with the extended semantics.

2 Preliminaries

In this work we consider *flat* programs [9], a convenient standard representation of functional logic programs which makes explicit the pattern matching strategy by case expressions. This flat representation constitutes the kernel of modern functional logic languages like Curry [8] or Toy [11].

Furthermore, we assume that flat programs are *normalized*, i.e., *let* constructs are used to ensure that the arguments of function and constructor calls are always variables (not necessarily pairwise different). As in [10], this is essential to express sharing without the use of graph structures. A normalization algorithm can be found in [1]. Basically, normalization introduces one new let construct for each non-variable argument, e.g., $f(e)$ is transformed into "$\mathit{let}\ \{x = e\}\ \mathit{in}\ f(x)$".

The syntax for normalized flat programs is shown in Figure 1, where we write $\overline{o_n}$ for the *sequence of objects* $o_1, \ldots, o_n$. A program consists of a sequence of function definitions such that the left-hand side has pairwise different variable arguments. The right-hand side is an expression composed by variables, data constructors, function calls, let bindings (where the local variables $\overline{x_k}$ are only visible in $\overline{e_k}$ and e), disjunctions (e.g., to represent non-deterministic operations), and case expressions of the form $\mathit{case}\ x\ \mathit{of}\ \{c_1(\overline{x_{n_1}}) \to e_1; \ldots; c_k(\overline{x_{n_k}}) \to e_k\}$, where x is a variable, $c_1, \ldots, c_k$ are different constructors, and $e_1, \ldots, e_k$ are expressions. The *pattern variables* $\overline{x_{n_i}}$ are introduced locally and bind the corresponding variables of the subexpression e_i.

Laziness of computations will show up in the description of the behavior of function calls and case expressions. In a function call, parameters are not evaluated but directly passed to the body of the function. In a case expression, the outermost symbol of the case argument is required. Therefore, the case argument should be evaluated to *head normal form* [5] (i.e., a variable or an expression with a construc-

tor at the outermost position). Consequently, the operational semantics that we consider will describe the evaluation of expressions only to head normal form. This is not a restriction since the evaluation to normal form or the solving of equations can be reduced to head normal form computations (see, e.g., [9]).

Following a recent proposal in [4,7], we assume in this work that logical variables are modeled by means of non-deterministic *value generators*. For instance, in order to write `add (S Z) x`, where `x` is a logical variable, we write `add (S Z) genNat` instead, where the auxiliary function `genNat` is defined as follows:

```
genNat = Z or (S genNat)
```

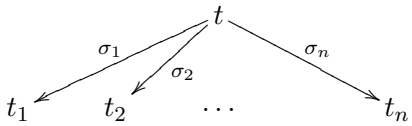
This allows one to consider a simpler semantics by ignoring logical variables. In particular, in this work we consider the semantics of [6] which extends the operational semantics of [1] in two ways: 1) logical variables are represented by value generators and 2) it preserves sharing even across non-deterministic computations.

3 Partial Evaluation of Lazy Functional Logic Programs

In this section, we introduce the core of a new scheme for the partial evaluation of lazy functional logic programs. Basically, at the heart of an offline partial evaluator, we usually find

- a non-standard semantics, which is used to perform symbolic computations (according to the program annotations) and
- a method to extract residual rules from partial computations.

For instance, within the original *narrowing-driven* partial evaluation framework [3], one proceeds as follows: given a term-rewriting system $\mathcal{R}$ and a term t, a *finite (possibly incomplete) narrowing tree* for t in $\mathcal{R}$ is built:



Then, for every (possibly incomplete) narrowing derivation $t \leadsto_{\sigma_i} t_i$, we produce an associated residual rule of the following form: $\sigma_i(t) = t_i$.

This simple approach, however, is not applicable to the (normalized) flat language. Here, the standard semantics (the LNT calculus [9]) is not appropriate and, thus, a *residualizing* version, the RLNT calculus, has been introduced in [2]. The RLNT semantics does not compute bindings but encodes them by means of case expressions (which are considered "residual" code). For instance, given a case expression of the form

$$case\ x\ of\ \{\overline{p_k \to e_k}\}$$

where x is a logical variable, the RLNT semantics performs the following step:

$$case\ x\ of\ \{\overline{p_k \to \sigma_k(e_k)}\}$$

(Lookup) $$\frac{\Gamma\backslash\{(x,\Gamma[x])\} : \Gamma[x] \;\Downarrow\; \Delta : v}{\Gamma : x \;\Downarrow\; \Delta[x \mapsto v] : v}$$

(Val) $$\Gamma : v \;\Downarrow\; \Gamma : v$$

(Fun) $$\frac{\Gamma : \sigma(e) \;\Downarrow\; \Delta : v}{\Gamma : f(\overline{x_n}) \;\Downarrow\; \Delta : v}$$ where $f(\overline{y_n}) = e \in P$ and $\sigma = \{\overline{y_n \mapsto x_n}\}$

(Let) $$\frac{\Gamma[\overline{y_k \longmapsto \sigma(e_k)}] : \sigma(e) \;\Downarrow\; \Delta : v}{\Gamma : \mathit{let}\ \{\overline{x_k = e_k}\}\ \mathit{in}\ e \;\Downarrow\; \Delta : v}$$ where $\sigma = \{\overline{x_k \mapsto y_k}\}$ and $\overline{y_k}$ are fresh variables

(Select) $$\frac{\Gamma : e \;\Downarrow\; \Delta : c(\overline{y_n}) \qquad \Delta : \sigma(e_i) \;\Downarrow\; \Theta : v}{\Gamma : \mathit{case}\ e\ \mathit{of}\ \{\overline{p_k \to e_k}\} \;\Downarrow\; \Theta : v}$$ where $p_i = c(\overline{x_n})$ and $\sigma = \{\overline{x_n \mapsto y_n}\}$

(OR) $$\Gamma : x_1\ \mathit{or}\ x_2 \;\Downarrow\; \Gamma : \mathit{OR}\ r\ [x_1, x_2]$$ where r is fresh

(Lift) $$\frac{\Gamma : e \;\Downarrow\; \Delta : \mathit{OR}\ r\ [\overline{x_n}]}{\Gamma : \mathit{case}\ e\ \mathit{of}\ \{\overline{p_k \to e_k}\} \;\Downarrow\; \Delta[y_n \mapsto \mathit{case}\ x_n\ \mathit{of}\ \{\overline{p_k \to e_k}\}] : \mathit{OR}\ r\ [\overline{y_n}]}$$ where $\overline{y_n}$ are fresh

Fig. 2. Core semantics

with $\sigma_i = \{x \mapsto p_i\}$, $i = 1, \ldots, k$, and continues with the evaluation of the different branches $\sigma_1(e_1), \ldots, \sigma_k(e_k)$.

In our approach, however, a more complex extension is required since—in contrast to the LNT/RLNT calculi—the considered semantics [6] copes with sharing (even across non-deterministic computations) and deals with *configurations* of the form $\Gamma : e$, where Γ is a *heap*—an updatable set of bindings, which is essential to model sharing—and e is an expression.

Therefore, in the following section, we introduce an extension of this semantics which is appropriate for performing partial computations. Then, in Section 3.2, we describe the extraction of residual rules from partial computations.

3.1 Partial Evaluation Semantics

First, we introduce the core of the considered semantics [6], whose rules are shown in Fig. 2. Here, a *heap*, denoted by Γ, Δ, or Θ, is a partial mapping from variables to expressions (the *empty heap* is denoted by $[\,]$). We let $\mathcal{D}om(\Gamma)$ denote the set $\{x \mid x \mapsto e \in \Gamma\}$. The value associated to variable x in heap Γ is denoted by $\Gamma[x]$. $\Gamma[x \mapsto e]$ denotes a heap with $\Gamma[x] = e$, i.e., we use this notation either as a condition on a heap Γ or as a modification of Γ. A *value* v is a constructor-rooted term $c(\overline{e_n})$ (i.e., a term whose outermost function symbol is a constructor symbol).

We use judgments like "$\Gamma : e \;\Downarrow\; \Delta : v$" which are interpreted as "in the context of heap Γ, the expression e evaluates to value v producing a new heap Δ." Let us briefly explain the rules of the semantics:

- In order to evaluate a variable x in the context of a heap Γ, rule Lookup first reduces $\Gamma[x]$ to a value v in the context of the heap $\Gamma\backslash\{(x,\Gamma[x])\}$; note that the binding for x is removed from Γ in order to detect *black holes* (a self-dependent infinite loop). Then, Γ is updated with the computed value for x. Therefore, if the value of x is required again in the computation, the value v is immediately returned so that the effects of sharing are achieved.

$$\text{(Hnf-Val)}\quad \frac{\Gamma : e \;\Downarrow\; \Delta : c(\overline{x_n})}{\Gamma : hnf\ \rho\ e \;\Downarrow\; \Delta : c(\overline{x_n})} \qquad \text{where } c \neq OR$$

$$\text{(Hnf-Choose)}\quad \frac{\Gamma : e \;\Downarrow\; \Delta : OR\ r\ [\overline{x_n}] \qquad \Delta : hnf\ \rho\ x_{\rho(r)} \;\Downarrow\; \Theta : v}{\Gamma : hnf\ \rho\ e \;\Downarrow\; \Theta : v} \qquad \text{if } r \in \mathrm{Dom}(\rho)$$

$$\text{(Hnf-Or)}\quad \frac{\Gamma : hnf\ \rho\ x \;\Downarrow\; \Delta_1 : OR\ r\ [\overline{x_n}] \qquad \overline{\Delta_n : hnf\ (\rho \cup \{(r,i)\})\ x_n \;\Downarrow\; \Delta_{n+1} : v_n}}{\Gamma : hnf\ \rho\ x \;\Downarrow\; \Delta_{n+1} : OR\ r\ [\overline{x_n}]} \qquad \text{if } r \notin \mathrm{Dom}(\rho)$$

Fig. 3. HNF rules

- Rule Val is used to evaluate a value and its definition is self-explanatory.
- Rule Fun is used to perform a function unfolding; here, we assume that the considered program P is a global parameter of the calculus.
- Rule Let evaluates a let expression by adding its bindings to the heap and then proceeding with the evaluation of its expression. Note that we introduce fresh names in order to avoid variable name clashes.
- Rule Select is used to evaluate a case expression whose argument reduces to a constructor-rooted term. In this case, we select the matching branch and, then, proceed with the evaluation of the expression in this branch.
- Rules OR and Lift are the main novelty of this calculus w.r.t. the original semantics of [1]. Here, non-deterministic choices of the form *or* $x_1\ x_2$ are not evaluated but replaced by $OR\ r\ [x_1, x_2]$, where OR is a fresh constructor symbol and r is a fresh *reference* (e.g., a natural number) to identify this particular occurrence of OR (see below).

  The occurrences of OR in an expression (if any) are intended to denote the *non-deterministic branches* of the considered evaluation,[3] e.g., an expression of the form $OR\ r_1\ [OR\ r_2\ [x_1\ x_2],\ x_3]$ denotes a search tree with three branches x_1, x_2 and x_3. References in OR expressions are useful to record previous choices. For instance, given an expression like $OR\ r_1\ [OR\ r_2\ [x_1\ x_2],\ x_3]$, if $r_1 = r_2$, then it should actually be $OR\ r_1\ [x_1,\ x_3]$, i.e., we replace $OR\ r_2\ [x_1\ x_2]$ by x_1 because we are inside the first branch of another OR expression with the same reference.

  Since OR constructors are used to represent the structure of the non-deterministic search tree, we want them to be the topmost symbols in an evaluated expression. Here, rule Lift is used to push computations inside the branches of an OR expression (i.e., to lift the OR constructor one level up).

  Thanks to these rules, sharing is not only preserved along a deterministic computation, but also across different non-deterministic branches.

In principle, given a normalized expression e, we could evaluate e using the initial configuration $[\,] : e$ and then applying the rules of the semantics. However, in its current formulation, only the topmost constructor of an expression is computed. While this is fine when the computation is deterministic, it is not acceptable when it is non-deterministic since every evaluated expression would be headed by the constructor OR and, thus, its branches would not be evaluated at all.

In order to overcome this limitation, we now introduce a set of rules that are

[3] Therefore, they are only used internally but should not be part of the computed values.

used to evaluate an expression as much as needed to have a head normal form in every branch.[4] The new rules are shown in Figure 3.

Now, the computation for e in P will start with an initial configuration of the form "$[\,] :\ hnf\ \{\}\ e$", where $\{\}$ is a mapping which is used to keep track of the non-deterministic branches found so far in the computation (thus it is initially empty). The new rules are the following:

- Rule Hnf-Val is used to compute the head normal form of a deterministic expression. Here, the result is a value and, thus, ρ is simply ignored.
- Rule Hnf-Choose applies when the considered expression evaluates to an OR expression *with the same reference of a previously encountered OR*. Therefore, we delete the OR constructor, choose the same branch as before, and continue with its evaluation.
- Finally, rule Hnf-Or is used when the expression evaluates to an OR expression *with a reference not found so far*. This means that the expression being evaluated is actually non-deterministic and, thus, the computed result should include different values. These values are obtained by applying function *hnf* to each branch of the OR expression with $\rho \cup \{(r, i)\}$ to record the fact that the i-th branch has been chosen.

Compared to [6], we lose the possibility to employ different search strategies and implicitly evaluate non-deterministic branches of the computation in depth-first order. We can do this because we assume that the use of a binding-time analysis ensures the termination of computations by placing appropriate annotations in the source program (cf. Section 1).

The appendix shows an example which illustrates the rules of the new semantics.

3.2 Extraction of Residual Rules

In this section, we describe the extraction of a residual rule associated to a computation using the rules of the semantics shown in Figures 2 and 3.

In principle, one could think that the following naive approach would be appropriate: given a derivation of the form

$$[\,] :\ let\ \{\ x = main\ \}\ in\ hnf\ \{\}\ x \Downarrow \Gamma : v$$

the associated residual rule would be

$$f(\overline{x_n})\ =\ let\ \overline{\Gamma}\ in\ v$$

where f is a fresh function symbol, $\overline{x_n}$ are the "logical variables" in the evaluation of *main* (i.e., those variables that were bound to value generators), and $\overline{\Gamma}$ denotes the set of bindings of Γ in equational form. Unfortunately, these rules would not be appropriate for a number of reasons:

- the computed value v may contain *illegal* symbols (e.g., occurrences of OR) and

[4] This is done in [6] by means of a primitive function `searchTree` which builds the non-deterministic search tree for a given expression in a lazy manner.

- v may also contain calls to value generators that should rather be parameters of the function (i.e., variables from $\overline{x_n}$).

In order to overcome these drawbacks, now we introduce a number of transformations that eventually produce a right-hand side with the appropriate form.

First, we need the following preliminary definition.

Definition 3.1 (reachable variables) *Given a heap Γ and two variables $x, y \in \mathcal{D}om(\Gamma)$, we say that y is reachable from x if $y \in reach(\Gamma, x)$, where function reach is defined as follows:*

$$reach(\Gamma, x) = \{x\} \cup \bigcup_{y \in Var(\Gamma[x])} reach(\Gamma, y)$$

Intuitively, a variable x is reachable from another variable y if y is bound to an expression that contains x, or to an expression that contains a variable w that is bound to an expression that contains x, and so on.

Now, we can already define the extraction of residual rules. Given a derivation of the form

$$[\,] : \; let \; \{ \; x = main \; \} \; in \; hnf \; \{\} \; x \Downarrow \Gamma : v$$

we first construct a residual rule as explained before:

$$f(\overline{x_n}) \;=\; let \; \overline{\Gamma} \; in \; v$$

We consider, for instance, the computation shown in the appendix. In this case, the initial residual rule is as follows: [5]

```
f x6 = let { x6 = OR 1 x10 x11,  x8 = S x9,  x9 = Z,  x10 = Z,  x11 = S x18
           ; x18 = genNat,  x19 = add x18 x6,  x20 = add x19 x8 }
       in OR 1 (S x9) (S x20)
```

Then, we apply the following transformations to the right-hand side of the rule:

(i) First, we move the outermost bindings to every *leaf* of the computed expression. Moreover, the bindings should be modified according to the branch where they appear, e.g., a binding `x` $\mapsto$ `OR 1 y z` should be replaced by `x` $\mapsto$ `y` within the first branch of an `OR` labeled with reference 1 and by `x` $\mapsto$ `z` within the second branch of the same `OR`.

In our running example, the residual rule is transformed as follows:

```
f x6 = OR 1 (let { x9 = Z } in S x9)
            (let { x6 = x11,  x8 = S x9,  x9 = Z,  x11 = S x18,
                 ; x18 = genNat,  x19 = add x18 x6,  x20 = add x19 x8 }
             in  S x20)
```

[5] We only show in this section the relevant bindings. The complete example can be found in the appendix.

(ii) The next step consists in transforming those `OR` expressions that come from the evaluation of a parameter in the left-hand side by a case expression.

For instance, in our example, we have a single parameter `x6` in the left-hand side of the residual rule. In the final heap of the computation, we had the following binding:

$$\texttt{x6} \mapsto \texttt{OR 1 x10 x11}$$

Therefore, every expression of the form `OR 1 e1 e2` in the right-hand side of the residual rule should be replaced by

$$\texttt{case x6 of } \{\texttt{p}_1 \mapsto \texttt{e1};\ \texttt{p}_2 \mapsto \texttt{e}_2\}$$

where $\texttt{p}_1$ and $\texttt{p}_2$ are obtained by dereferencing `x10` and `x11`.

In this way, we obtain the following rule:

```
f x6 = case x6 of
       {    Z ↦ let { x9 = Z } in S x9;
        S x18 ↦ let { x8 = S x9,  x9 = Z,  x19 = add x18 x6,
                      x20 = add x19 x8 } in S x20 }
```

(iii) In the next step, we remove those bindings that are not reachable (using function *reach*) from the variables in the different branches.

We do not show the application of this step here since non-reachable variables have been removed from the beginning to improve readability. The interested reader is referred to the appendix.

(iv) Now, we should move outwards those bindings that are common to several branches, so that sharing is enforced.

In our example, we can only move outwards the binding `x9 = Z` which appears in both branches of the residual rule:

```
f x6 = let { x9 = Z } in
       case x6 of { Z ↦ S x9;
                    S x18 ↦ let { x8 = S x9,  x19 = add x18 x6,
                                  x20 = add x19 x8 } in S x20 }
```

(v) Finally, we should replace every remaining occurrence of $OR\ n\ x_1\ x_2$ by a standard *or* expression of the form $or\ x_1\ x_2$ (thus losing the connections given by references).

In our example, no remaining `OR` appears and, thus, no further transformation is required.

4 Conclusions and Future Work

This work has introduced a new partial evaluation scheme which is particularly well suited for specializing *lazy* functional logic programs. In particular, it is not overly

restrictive since every function can be unfolded (even if it is not right-linear) and still preserves sharing, thus avoiding the introduction of redundant computations in the residual program.

As future work, we first plan to formalize the extraction of residual rules and prove its correctness. Then, we will extend the semantics of Figures 2 and 3 in order to cope with the additional features of the language, e.g., higher-order functions, constraints, etc. Finally, we undertake the development of a partial evaluator for a significant subset of a lazy language like Haskell or Curry following the ideas presented so far.

References

[1] Albert, E., M. Hanus, F. Huch, J. Oliver and G. Vidal, *Operational Semantics for Declarative Multi-Paradigm Languages*, Journal of Symbolic Computation **40** (2005), pp. 795–829.

[2] Albert, E., M. Hanus and G. Vidal, *A Residualizing Semantics for the Partial Evaluation of Functional Logic Programs*, Information Processing Letters **85** (2003), pp. 19–25.

[3] Albert, E. and G. Vidal, *The Narrowing-Driven Approach to Functional Logic Program Specialization*, New Generation Computing **20** (2002), pp. 3–26.

[4] Antoy, S. and M. Hanus, *Overlapping Rules and Logic Variables in Functional Logic Programs*, in: *Proc. of the 22snd Int'l Conf. on Logic Programming (ICLP'06)* (2006), pp. 87–101.

[5] Barendregt, H., "The Lambda Calculus—Its Syntax and Semantics," Elsevier, 1984.

[6] Braßel, B. and F. Huch, *On a Tighter Integration of Functional and Logic Programming* (2007), personal communication.

[7] de Dios-Castro, J. and F. López-Fraguas, *Elimination of Extra-Variables in Functional Logic Programs*, in: *Proc. of the 6th Spanish Conf. on Programming and Languages (PROLE'06)* (2007).

[8] (ed.), M. H., *Curry: An Integrated Functional Logic Language*, Available at: `http://www.informatik.uni-kiel.de/~mh/curry/`.

[9] Hanus, M. and C. Prehofer, *Higher-Order Narrowing with Definitional Trees*, Journal of Functional Programming **9** (1999), pp. 33–75.

[10] Launchbury, J., *A Natural Semantics for Lazy Evaluation*, in: *Proc. of the ACM Symp. on Principles of Programming Languages (POPL'93)* (1993), pp. 144–154.

[11] López-Fraguas, F. and J. Sánchez-Hernández, *TOY: A Multiparadigm Declarative System*, in: *Proc. of the 10th Int'l Conf. on Rewriting Techniques and Applications (RTA'99)* (1999), pp. 244–247.

[12] Ramos, J., J. Silva and G. Vidal, *Fast Narrowing-Driven Partial Evaluation for Inductively Sequential Systems*, in: *Proc. of the 10th ACM SIGPLAN International Conference on Functional Programming (ICFP 2005)* (2005), pp. 228–239.

A An Example

We illustrate the rules of the semantics in Figures 2 and 3 with an example. Consider the following flat program:

```
data Nat = Z | S Nat

add x y = case x of { Z -> y;
                      S n -> let { m = add n y } in S m }

double x = add x x

genNat = let { n = Z, m = let { x = genNat } in S x } in or n m

main = let { n = genNat, x = double n, y = S m, m = Z } in add x y
```

and the initial configuration:

$$[\,] : \texttt{let } \{ \texttt{ x} = \texttt{main } \} \texttt{ in hnf } \{\} \texttt{ x}$$

The evaluation produced by the semantics is shown in Figures A.1 and A.2, together with the final heap of the computation.

The initial residual rule associated to the computation shown in Figures A.1 and A.2 can be found in Figure A.3.

Then, we apply the following transformations to the right-hand side of the rule:

(i) First, we move the outermost bindings to every *leaf* of the computed expression. Moreover, the bindings should be modified according to the branch where they appear, e.g., a binding $\texttt{x} \mapsto \texttt{OR 1 y z}$ should be replaced by $\texttt{x} \mapsto \texttt{y}$ within the first branch of an `OR` labeled with reference 1 and by $\texttt{x} \mapsto \texttt{z}$ within the second branch of the same `OR`.

In our running example, the residual rule is transformed as follows:

```
f x6 = OR 1 (let {  x5 = x14,  x6 = x10,  x7 = x12,
                    x8 = S x9,  x9 = Z,  x10 = Z,
                    x11 = S x18,  x12 = x10,  x13 = S x19,
                    x14 = x16,  x15 = S x20,  x16 = S x9,
                    x17 = case x11 of {Z ↦ x8;
                                       S x3 ↦ let {x4 = add x3 x8} in S x4},
                    x18 = genNat,  x19 = add x18 x6,
                    x20 = add x19 x8 } in S x9)
            (let { x5 = x15,  x6 = x11,  x7 = x13,
                    x8 = S x9,  x9 = Z,  x10 = Z,
                    x11 = S x18,  x12 = x11,  x13 = S x19,
                    x14 = x17,  x15 = S x20,  x16 = Sx9,
                    x17 = case x11 of {Z ↦ x8;
                                       S x3 ↦ let {x4 = add x3 x8} in S x4},
                    x18 = genNat,  x19 = add x18 x6,
                    x20 = add x19 x8 } in S x20)
```

(ii) The next step consists in transforming those `OR` expressions that come from the evaluation of a parameter in the left-hand side by a case expression.

For instance, in our example, we have a single parameter `x6` in the left-hand side of the residual rule. In the final heap of the computation, we have the following binding:

$$\texttt{x6} \mapsto \texttt{OR 1 x10 x11}$$

$[\,] : \texttt{let } \{\texttt{x0} = \texttt{main}\} \texttt{ in hnf x0}$

$\Rightarrow_{\mathsf{Let}}$ $\Delta_2 \equiv \Delta_1[\texttt{x5} \mapsto \texttt{main}] : \texttt{hnf x5}$

$\Rightarrow_{\mathsf{Hnf-Or}}$ $\Delta_2 : \texttt{x5}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_3 \equiv 2\backslash[\texttt{x5} \mapsto \texttt{main}] : \texttt{main}$

$\Rightarrow_{\mathsf{Fun}}$ $\Delta_3 : \texttt{let x1} = \texttt{genNat}, \texttt{x2} = \texttt{double x1}, \texttt{x3} = \texttt{S x4}, \texttt{x4} = \texttt{Z in add x2 x3}$

$\Rightarrow_{\mathsf{Let}}$ $\Delta_4 \equiv \Delta_3[\texttt{x6} \mapsto \texttt{genNat}, \texttt{x7} \mapsto \texttt{double x6}, \texttt{x8} \mapsto \texttt{S x9}, \texttt{x9} \mapsto \texttt{Z}] : \texttt{add x7 x8}$

$\Rightarrow_{\mathsf{Fun}}$ $\Delta_4 : \texttt{case x7 of } \{\texttt{Z} \mapsto \texttt{x8};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x8}\} \texttt{ in S x4}\}$

$\Rightarrow_{\mathsf{Lift}}$ $\Delta_4 : \texttt{x7}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_5 \equiv \Delta_4\backslash[\texttt{x7} \mapsto \texttt{double x6}] : \texttt{double x6}$

$\Rightarrow_{\mathsf{Fun}}$ $\Delta_5 : \texttt{add x6 x6}$

$\Rightarrow_{\mathsf{Fun}}$ $\Delta_5 : \texttt{case x6 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\}$

$\Rightarrow_{\mathsf{Lift}}$ $\Delta_5 : \texttt{x6}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_6 \equiv \Delta_5\backslash[\texttt{x6} \mapsto \texttt{genNat}] : \texttt{genNat}$

$\Rightarrow_{\mathsf{Fun}}$ $\Delta_6 : \texttt{let } \{\texttt{x2} = \texttt{Z}, \texttt{x3} = \texttt{let } \{\texttt{x1} = \texttt{genNat}\} \texttt{ in S x1}\} \texttt{ in or x2 x3}$

$\Rightarrow_{\mathsf{Let}}$ $\Delta_7 \equiv \Delta_6[\texttt{x10} \mapsto \texttt{Z}, \texttt{x11} \mapsto \texttt{let } \{\texttt{x1} = \texttt{genNat}\} \texttt{ in S x1}] : \texttt{or x10 x11}$

$\Rightarrow_{\mathsf{Or}}$ $\Delta_7 : \texttt{OR 1 x10 x11}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_8 \equiv \Delta_7[\texttt{x6} \mapsto \texttt{OR 1 x10 x11}] : \texttt{OR 1 x10 x11}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_9 \equiv \Delta_8[\texttt{x12} \mapsto \texttt{case x10 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\}] : \texttt{OR 1 x12 x13}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{10} \equiv \Delta_9[\texttt{x7} \mapsto \texttt{OR 1 x12 x13}] : \texttt{OR 1 x12 x13}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{11} \equiv \Delta_{10}[\texttt{x14} \mapsto \texttt{case x12 of } \{\texttt{Z} \mapsto \texttt{x8};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x8}\} \texttt{ in S x4}\},$
$\texttt{x15} \mapsto \texttt{case x13 of } \{\texttt{Z} \mapsto \texttt{x8};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x8}\} \texttt{ in S x4}\}] : \texttt{OR 1 x14 x15}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{12} \equiv \Delta_{11}[\texttt{x5} \mapsto \texttt{OR 1 x14 x15}] : \texttt{OR 1 x14 x15}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{12} : \texttt{hnf } \{(1, 1)\} \texttt{ x14}$

$\Rightarrow_{\mathsf{Hnf-Choose}}$ $\Delta_{12} : \texttt{x14}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{13} \equiv \Delta_{12}\backslash[\texttt{x14} \mapsto \texttt{case x12 of } \{\texttt{Z} \mapsto \texttt{x8};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x8}\} \texttt{ in S x4}\}] :$
$\texttt{case x12 of } \{\texttt{Z} \mapsto \texttt{x8};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x8}\} \texttt{ in S x4}\}$

$\Rightarrow_{\mathsf{Lift}}$ $\Delta_{13} : \texttt{x12}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{14} \equiv \Delta_{13}\backslash[\texttt{x12} \mapsto \texttt{case x10 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\}] :$
$\texttt{case x10 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\}$

$\Rightarrow_{\mathsf{Select}}$ $\Delta_{14} : \texttt{x10}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{15} \equiv \Delta_{14}\backslash[\texttt{x10} \mapsto \texttt{Z}] : \texttt{Z}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{16} \equiv \Delta_{15}\backslash[\texttt{x10} \mapsto \texttt{Z}] : \texttt{Z}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{16} : \texttt{x6}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{17} \equiv \Delta_{16}\backslash[\texttt{x6} \mapsto \texttt{OR 1 x10 x11}] : \texttt{OR 1 x10 x11}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{18} \equiv \Delta_{17}[\texttt{x6} \mapsto \texttt{OR 1 x10 x11}] : \texttt{OR 1 x10 x11}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{19} \equiv \Delta_{18}[\texttt{x12} \mapsto \texttt{OR 1 x10 x11}] : \texttt{OR 1 x10 x11}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{20} \equiv \Delta_{19}[\texttt{x16} \mapsto \texttt{case x10 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\},$
$\texttt{x17} \mapsto \texttt{case x11 of } \{\texttt{Z} \mapsto \texttt{x8};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x8}\} \texttt{ in S x4}\}] : \texttt{OR 1 x16 x17}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{21} \equiv \Delta_{20}[\texttt{x14} \mapsto \texttt{OR 1 x16 x17}] : \texttt{OR 1 x16 x17}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{21} : \texttt{hnf } \{(1, 1)\} \texttt{ x16}$

$\Rightarrow_{\mathsf{Hnf-Val}}$ $\Delta_{21} : \texttt{x16}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{22} \equiv \Delta_{21}\backslash[\texttt{x16} \mapsto \texttt{case x10 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\}] :$
$\texttt{case x10 of } \{\texttt{Z} \mapsto \texttt{x6};\ \texttt{S x3} \mapsto \texttt{let } \{\texttt{x4} = \texttt{add x3 x6}\} \texttt{ in S x4}\}$

$\Rightarrow_{\mathsf{Select}}$ $\Delta_{22} : \texttt{x10}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{23} \equiv \Delta_{22}\backslash[\texttt{x10} \mapsto \texttt{Z}] : \texttt{Z}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{24} \equiv \Delta_{23}[\texttt{x10} \mapsto \texttt{Z}] : \texttt{Z}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{24} : \texttt{x8}$

$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{25} \equiv \Delta_{24}\backslash[\texttt{x8} \mapsto \texttt{S x9}] : \texttt{S x9}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{26} \equiv \Delta_{25}[\texttt{x8} \mapsto \texttt{S x9}] : \texttt{S x9}$

$\Rightarrow_{\mathsf{Val}}$ $\Delta_{27} \equiv \Delta_{26}[\texttt{x16} \mapsto \texttt{S x9}] : \texttt{S x9}$

Fig. A.1. Evaluation using the semantics of Figures 2 and 3

$\Rightarrow_{\mathsf{Val}}$ Δ_{27} : S x9
$\Rightarrow_{\mathsf{Val}}$ Δ_{27} : S x9
$\Rightarrow_{\mathsf{Val}}$ Δ_{27} : hnf {(1, 2)} x15
$\Rightarrow_{\mathsf{Hnf-Val}}$ Δ_{27} : x15
$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{28} \equiv \Delta_{27}\backslash$[x15 $\mapsto$ case x13 of {Z $\mapsto$ x8; S x3 $\mapsto$ let {x4 = add x3 x8} in S x4}] :
case x13 of {Z $\mapsto$ x8; S x3 $\mapsto$ let {x4 = add x3 x8} in S x4}
$\Rightarrow_{\mathsf{Select}}$ Δ_{28} : x13
$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{29} \equiv \Delta_{28}\backslash$[x13 $\mapsto$ case x11 of {Z $\mapsto$ x8; S x3 $\mapsto$ let {x4 = add x3 x8} in S x4}] :
case x11 of {Z $\mapsto$ x8; S x3 $\mapsto$ let {x4 = add x3 x8} in S x4}
$\Rightarrow_{\mathsf{Select}}$ Δ_{29} : x11
$\Rightarrow_{\mathsf{Lookup}}$ $\Delta_{30} \equiv \Delta_{29}\backslash$[x11 $\mapsto$ let x1 = genNat in S x1] : let x1 = genNat in S x1
$\Rightarrow_{\mathsf{Let}}$ $\Delta_{31} \equiv \Delta_{30}$[x18 $\mapsto$ genNat] : S x18
$\Rightarrow_{\mathsf{Val}}$ $\Delta_{32} \equiv \Delta_{31}$[x11 $\mapsto$ S x18] : S x18
$\Rightarrow_{\mathsf{Val}}$ Δ_{32} : let {x4 = add x18 x6} in S x4
$\Rightarrow_{\mathsf{Let}}$ $\Delta_{33} \equiv \Delta_{32}$[x19 $\mapsto$ add x18 x6] : S x19
$\Rightarrow_{\mathsf{Val}}$ $\Delta_{34} \equiv \Delta_{33}$[x13 $\mapsto$ S x19] : S x19
$\Rightarrow_{\mathsf{Val}}$ Δ_{34} : let {x4 = add x19 x8} in S x4
$\Rightarrow_{\mathsf{Let}}$ $\Delta_{35} \equiv \Delta_{34}$[x20 $\mapsto$ add x19 x8] : S x20
$\Rightarrow_{\mathsf{Val}}$ $\Delta_{36} \equiv \Delta_{35}$[x15 $\mapsto$ S x20] : S x20
$\Rightarrow_{\mathsf{Val}}$ Δ_{36} : S x20
$\Rightarrow_{\mathsf{Val}}$ Δ_{36} : OR 1 (S x9) (S x20)

with

Δ_{36} = [x5 $\mapsto$ OR 1 x14 x15,
x6 $\mapsto$ OR 1 x10 x11,
x7 $\mapsto$ OR 1 x12 x13,
x8 $\mapsto$ S x9,
x9 $\mapsto$ Z,
x10 $\mapsto$ Z,
x11 $\mapsto$ S x18,
x12 $\mapsto$ OR 1 x10 x11,
x13 $\mapsto$ S x19,
x14 $\mapsto$ OR 1 x16 x17,
x15 $\mapsto$ S x20,
x16 $\mapsto$ S x9,
x17 $\mapsto$ case x11 of {Z $\mapsto$ x8; S x3 $\mapsto$ let {x4 = add x3 x8} in S x4},
x18 $\mapsto$ genNat,
x19 $\mapsto$ add x18 x6,
x20 $\mapsto$ add x19 x8]

Fig. A.2. Evaluation using the semantics of Figures 2 and 3 (continued)

Therefore, every expression of the form `OR 1 e1 e2` in the right-hand side of the residual rule should be replaced by

$$\texttt{case x6 of } \{\texttt{p}_1 \mapsto \texttt{e1};\ \texttt{p}_2 \mapsto \texttt{e}_2\}$$

where $\texttt{p}_1$ and $\texttt{p}_2$ are obtained by dereferencing `x10` and `x11`.

In this way, we obtain the following rule:

```
f x6 = let {  x5 = OR 1 x14 x15,
              x6 = OR 1 x10 x11,  x7 = OR 1 x12 x13,
              x8 = S x9,  x9 = Z,  x10 = Z,
              x11 = S x18,  x12 = OR 1 x10 x11,
              x13 = S x19,  x14 = OR 1 x16 x17,
              x15 = S x20,  x16 = S x9,
              x17 = case x11 of {Z ↦ x8; S x3 ↦ let {x4 = add x3 x8} in S x4},
              x18 = genNat,  x19 = add x18 x6,
              x20 = add x19 x8 }
           in OR 1 (S x9) (S x20)
```

Fig. A.3. Initial residual rule for the computation of Figures A.1 and A.2

```
f x6 = case x6 of
       { Z ↦ let {   x5 = x14,  x6 = x10,  x7 = x12,
                     x8 = S x9,  x9 = Z,  x10 = Z,
                     x11 = S x18,  x12 = x10,  x13 = S x19,
                     x14 = x16,  x15 = S x20,  x16 = S x9,
                     x17 = case x11 of {Z ↦ x8;
                                        S x3 ↦ let {x4 = add x3 x8} in S x4},
                     x18 = genNat,  x19 = add x18 x6,
                     x20 = add x19 x8 } in S x9;
         S x18 ↦ let { x5 = x15,  x6 = x11,  x7 = x13,
                     x8 = S x9,  x9 = Z,  x10 = Z,
                     x11 = S x18,  x12 = x11,  x13 = S x19,
                     x14 = x17,  x15 = S x20,  x16 = S x9,
                     x17 = case x11 of {Z ↦ x8;
                                        S x3 ↦ let {x4 = add x3 x8} in S x4},
                     x18 = genNat,  x19 = add x18 x6,
                     x20 = add x19 x8 } in S x20 }
```

(iii) In the next step, we remove those bindings that are not reachable from the variables in the different branches.

For instance, in our example, the first branch is `S x9`. Here, we have

```
reach([ x5 ↦ x14, x6 ↦ x10, x7 ↦ x12, x8 ↦ S x9, x9 ↦ Z, x10 = Z,
        x11 = S x18, x12 = x10, x13 = S x19, x14 = x16, x15 = S x20,
        x16 = S x9, x17 = case x11 of {...}, x18 = genNat,
        x19 = add x18 x6, x20 = add x19 x8], x9) = {x9}
```

and, therefore, only the equation `x9 = Z` is needed.

Analogously, the set of reachable variables from the second branch, `S x20`, is the following:

```
reach([ x5 ↦ x15, x6 ↦ x11, x7 ↦ x13, x8 ↦ S x9, x9 ↦ Z, x10 = Z,
        x11 = S x18, x12 = x11, x13 = S x19, x14 = x17, x15 = S x20,
        x16 = S x9, x17 = case x11 of {...}, x18 = genNat,
        x19 = add x18 x6, x20 = add x19 x8], x20) = {x8, x9, x19, x20}
```

Observe that we exclude from the above list the variable *parameters* (i.e., `x6` and `x18`) since their bindings should not appear anymore. Therefore, the transformed residual rule is now as follows:

```
f x6 = case x6 of
       {Z ↦ let {     x9 = Z } in S x9;
        S x18 ↦ let { x8 = S x9,
                      x9 = Z,
                      x19 = add x18 x6,
                      x20 = add x19 x8 } in S x20 }
```

(iv) Now, we should move outwards those bindings that are common to several branches, so that sharing is enforced.

In our example, we can only move outwards the binding `x9 = Z` which appears in both branches of the residual rule:

```
f x6 = let { x9 = Z } in  case x6 of
                          {Z ↦ S x9;
                           S x18 ↦ let { x8 = S x9,
                                         x19 = add x18 x6,
                                         x20 = add x19 x8 } in S x20 }
```

(v) Finally, we should replace every remaining occurrence of $OR\ n\ x_1\ x_2$ by a standard *or* expression of the form $or\ x_1\ x_2$ (thus losing the connections given by references).

In our example, no remaining `OR` appears and, thus, no further transformation is required.

Declaring Numbers

Bernd Braßel, Sebastian Fischer, Frank Huch[1,2]

Institute of Computer Science, University of Kiel
Olshausenstr. 40, 24098 Kiel, Germany

Abstract

Most implementations of functional and functional logic languages treat numbers and the basic numeric operations as external entities. The main reason for this is efficiency. However, this basic design decision has many unfortunate consequences for all programs using numbers. We present an approach to model numbers in a declarative way and argue that the loss in efficiency is compensated by the newly gained possibilities. Functional logic languages benefit the most from this proposal because all the numeric operations become fully narrowable. This enables the solving of simple equations on numbers in an efficient way without having to resort to external constraint solvers.
The presented approach can either be used as a library for purely declarative numbers or it can be employed as a basic data type of functional (logic) languages. Indeed, we have integrated the presented data structures as the only numbers available in our compiler for the functional logic language Curry and have experienced several months of satisfactory performance.

1 Introduction

Implementations of functional and functional logic languages usually provide numbers as an external data type and reuse the default implementation of the underlying language (e.g. C) for the implementation of operation like `(+)`, `(-)`, `(<=)`, `(==)`. This provides a very efficient implementation of complex arithmetic computations within the high-level language.

However, in the context of functional logic languages, this approach results in a major drawback: numbers cannot be guessed by means of narrowing. As a consequence, semantic extensions, like *residuation* [4,5], have been proposed which allow the user to use some restricted logical features in combination with numbers. The idea is that all (externally defined) functions on numbers suspend on free variables as long as their value is unbound. These residuated functions have to be combined with a generator which specifies all possible numbers of a free variable. As an example, we consider Pythagorean triples (a, b, c) with $a^2 + b^2 = c^2$. This problem can be implemented in PAKCS by means of the test and generate pattern [1]:

[1] This work has been partially supported by the German Research Council (DFG) under grant Ha 2457/5-2.
[2] Email: {bbr,sebf,fhu}@informatik.uni-kiel.de

```
pyt | a*a+b*b=:=c*c &
      c =:= member [1..] & b =:= member [1..c] & a =:= member [1..c]
    = (a,b,c)
  where a,b,c free

member (y:ys)  = y ? member ys
```

First, the search tests whether a combination of a, b and c is a Pythagorean triple. Since PAKCS cannot guess natural numbers this test is suspended by means of residuation for external functions `(+)` and `(*)`. Now the programmer has to add good generator functions which efficiently enumerate the search space.

If, like in PAKCS, a depth first search strategy is used, it is especially important to restrict the search space for a and b to a finite domain. In practical applications, finding good generator functions is often much more complicated than in this simple example.

Moreover, residuation sacrifices completeness and detailed knowledge of internals is required to understand why the following very similar definition produces a run-time error.

```
pyt' | a*a + b*b =:= c*c = generate a b c where a,b,c free
generate a b c
  | c =:= member [1..] & b =:= member [1..c] & a =:= member [1..c]
   = (a,b,c)
```

On the other hand, the most beautiful standard examples for the expressive power of narrowing are defined for *Peano numbers*:

```
data Peano = O | S Peano

add O     m = m
add (S n) m = S (add m n)

mult O     _ = O
mult (S n) m = add m (mult m n)
```

These functions cannot only be used for there intended purpose. We can also invert them to define new operations. For instance, the subtraction function can be defined by means of `add`:

```
sub n m | add r m =:= n = r
  where r free
```

Note, that we obtain a partial function, since we did not define a Peano representation of negative numbers. By means of narrowing, a solution for the constraint `add r m =:= n` generates a binding for `r`. This binding is the result of `sub`.

For a solution of the Pythagorean triples, it is much easier to use Peano numbers instead of predefined integers. We can easily define a solution to this problem as follows:

```
pyt | (a `mult` a) `add` (b `mult` b) =:= (c `mult` c) = (a,b,c)
  where a,b,c free
```

It is not necessary to define any generator functions at all. In combination with breadth first search, all solutions are generated.

Furthermore, if the result c of the Pythagorean equation is already known (e.g., $c = 20$) and you are only interested in computing a and b, then it is even possible to compute this information with depth first search. The given equation already restricts the search space to a finite domain.

```
pyt | c =:= intToPeano 20 &
      (a `mult` a) `add` (b `mult` b) =:= (c `mult` c) = (a,b,c)
  where a,b,c free
```

We obtain the solutions: $(a, b) \in \{(0, 20), (12, 16), (16, 12), (20, 0)\}$.

Unfortunately, using Peano numbers is not appropriate for practical applications, as a simple computation using Peano numbers in PAKCS shows: computing the square of 1000 already takes 7 seconds. As a consequence, the developers of Curry [7] proposed the external type `Int` in combination with residuating, external functions for numbers.

In this paper we present an alternative approach which is as flexible as Peano numbers but is, in practical applications, almost as efficient as externally defined numbers: a binary encoding of natural numbers. In a second step we extend these natural numbers with an algebraic sign and add zero to represent arbitrary integers.

We do not intend to provide means of solving complex numeric equations ("number crunching") by narrowing. The purpose of the presented approach is to improve the integration of numbers into the language. Using our representation of numbers, the programmer does no longer have to worry about floundering goals when applying functional logic programming techniques like those presented in [1].

2 Numbers and Numeric Operations

2.1 Natural Numbers

Our representation of natural numbers, i.e., numbers greater than zero, is defined as follows. We consider zero and negative numbers in Subsection 2.2.

```
data Nat = IHi | O Nat | I Nat
```

Natural numbers are represented in binary notation with the *least significant bit* first. The *most significant bit* is always one and, therefore, denoted `IHi`. The main reason to define natural numbers without zero is the unique representation. When zero is present as a terminal constructor of type `Nat` any number is representable in infinitely many ways with leading zeros. This is a serious obstacle for functional logic languages since the search spaces to derive numbers are always infinite in that case. The values of type `Nat` can be related to natural numbers as follows:

$$\begin{aligned} \texttt{IHi} &\;\hat{=}\; 1 \\ \texttt{O}\ (nat(n)) &\;\hat{=}\; 2n \\ \texttt{I}\ (nat(n)) &\;\hat{=}\; 2n+1 \end{aligned}$$

It is straight forward to define the successor function on this data type.

```
succ :: Nat -> Nat
succ IHi   = O IHi          1 + 1 = 2 · 1
succ (O n) = I n            2n + 1 = 2n + 1
succ (I n) = O (succ n)     (2n + 1) + 1 = 2 · (n + 1)
```

After each rule in the definition of `succ` we give an equational reasoning for its correctness. This is one part of the proof that our functions implement the intended semantics. In general, the proof consists in combining three considerations:

- completeness of pattern matching
  This is in most cases directly observable in the given definitions.
- correctness of defined rules
  This is demonstrated by equations added to each rule. To check the equations, it is important to remember that all occurring variables stand for *positive natural numbers*.
- termination
  Because of laziness our functions are normally also defined on infinite numbers. Therefore, we distinguish two categories of termination. In this section these categories only apply to deterministic expressions. Considerations with respect to narrowing will follow in Section 3.

(i) nf-termination:
We call an expression nf-terminating iff it evaluates to a *normal form* in finitely many steps. We call a function f "nf-terminating for arguments of categories $C_1 \ldots C_n$" iff the application of f to arguments of that categories is a nf-terminating expression. For instance, we will show below that `succ` nf-terminates if its argument is nf-terminating. All of the defined functions nf-terminate for nf-terminating arguments because they inductively descend on at least one of these arguments.

(ii) hnf-termination:
We call an expression hnf-terminating iff it evaluates to a *head normal form* of arbitrary depth in finitely many steps. Analog to nf-termination we will also say that a function hnf-terminates for certain categories of arguments. E.g., `succ` hnf-terminates if its argument hnf-terminates.

The function `succ` nf-terminates for nf-terminating input since it descends on its only argument. Moreover, each rule directly produces a head normal form, yielding that `succ` hnf-terminates for hnf-terminating input.

The function `succ` is used to easily build `add` on natural numbers.

```
add :: Nat -> Nat -> Nat
IHi `add` m   = succ m              1 + m = m + 1
O n `add` IHi = I n                 2n + 1 = 2n + 1
O n `add` O m = O (n `add` m)       2n + 2m = 2 · (n + m)
O n `add` I m = I (n `add` m)       2n + (2m + 1) = 2 · (n + m) + 1
I n `add` IHi = O (succ n)          (2n + 1) + 1 = 2 · (n + 1)
I n `add` O m = I (n `add` m)       (2n + 1) + 2m = 2 · (n + m) + 1
I n `add` I m = O (succ n `add` y)  (2n + 1) + (2m + 1) = 2 · (n + 1 + m)
```

For implementations making use of symmetric pattern matching, the first equation should be spelled out to match the second and fifth equation plus a case for `IHi` and `IHi`. The languages considered here all perform a left-to-right matching for this example and, hence, the given definition is sufficient.

Function `add` nf-terminates for nf-terminating input as it inductively descends both arguments. Hnf-termination is also given for hnf-terminating arguments, as each rule directly produces an hnf or calls the hnf-terminating function `succ`.

```
mult :: Nat -> Nat -> Nat
IHi `mult` m = m                          1 · m = m
O n `mult` m = O (n `mult` m)             (2n) · m = 2 · (n · m)
I n `mult` m = O (n `mult` m) `add` m     (2n + 1) · m = 2 · (n · m) + m
```

This definition is clearly asymmetric. However, the following consideration shows that the result symmetrically relies on both arguments.

`mult` nf-terminates iff both arguments nf-terminate as it descends the first argument and finally yields the second argument as a result. Hnf-termination is also given for hnf-terminating arguments, since either one of those arguments is returned (first rule) or a constructor is produced directly (second rule) and because `add` directly produces an hnf for the call `O` e_1 `` `add` `` e_2 if e_2 is hnf-terminating, regardless of e_1 (third rule).

Next is comparing two numbers. Traditionally, comparing in a functional logic language maps to the ordering type.

```
data Ordering = LT | EQ | GT
```

Respectively, the constructors denote the results "less than", "equal" and "greater than". We also use the tests corresponding to this data definition, i.e., `isEQ`, `isLT`, `isGT :: Ordering -> Bool`. It is convenient to simply map two arguments on this type and formulate the relations `(<=),(<),(>=),(>)` in terms of a comparison function like `compNat` or `compare` which is defined later. `compNat` compares two natural numbers as follows. We denote by $\circ$ any of the relations $<, =, >$.

```
compNat :: Nat -> Nat -> Ordering
compNat IHi IHi   = EQ                 1 = 1
compNat IHi (O _) = LT                 1 < 2n
compNat IHi (I _) = LT                 1 < 2n + 1
compNat (O _) IHi = GT                 2n > 1
compNat (O n) (O m) = compNat n m      2n ∘ 2m ⇔ n ∘ m
compNat (O n) (I m)
  | isEQ cmpnm = LT                    n = m ⇒ 2n < 2m + 1
  | otherwise  = cmpnm                 (n > m ⇒ 2n > 2m + 1)∧
 where cmpnm = compNat n m             (n < m ⇒ 2n < 2m + 1)
compNat (I _) IHi = GT                 2n + 1 > 1
compNat (I n) (O m)
  | isEQ cmpnm = GT                    n = m ⇒ 2n + 1 > 2m
  | otherwise  = cmpnm                 (n > m ⇒ 2n + 1 > 2m)∧
 where cmpnm = compNat n m             (n < m ⇒ 2n + 1 < 2m)
compNat (I n) (I m) = compNat n m      (2n + 1) ∘ (2m + 1) ⇔ n ∘ m
```

Because the result type `Ordering` is not a complex type hnf-termination and nf-termination coincide and we simply speak of termination. Since `compNat` *simultaneously* descends on both arguments, it terminates whenever at least one of its arguments is nf-terminating and the other is at least hnf-terminating. Note that the comparison only takes as many steps as the size of *the smaller* argument. This is a nice property helping on efficiency.

Now we define `(<=),(<),(>=),(>)` as follows:

```
n <  m = isLT (compare n m)
n >  m = isGT (compare n m)
n <= m = not (n > m)
n >= m = not (n < m)
```

For Curry, note that by defining the relations on base of `compare`, we obtain a narrowable definition of the comparing operations instead of a rigid one. We think that having numbers as external base type is one of the main reasons why these operations where defined rigidly in Curry in the first place.

We can do better, however, than defining

```
n == m = isEQ (compare n m)
n /= m = not (n == m)
```

by sticking to the usual definition of `(==)` which directly compares the constructors. For non-equal values this implementation can even yield a result without evaluating any of its arguments completely.

We now turn to the last operator solely defined on natural numbers. This is the function `pred` which computes the predecessor of a given natural number. Of course, this operator can only be defined partially. This partiality will then lead to the extension of our numbers to *integers* in the next section.

`pred (O IHi)`	`= IHi`	$2-1=1$
`pred (O n@(O _))`	`= I (pred n)`	$2\cdot 2n-1=2\cdot(2n-1)+1$
`pred (O (I n))`	`= I (O n)`	$2\cdot(2n+1)-1=2\cdot 2n+1$
`pred (I n)`	`= O n`	$2\cdot n+1-1=2\cdot n$

`pred` nf-terminates on nf-terminating input because it descends on its argument. For hnf-terminating input it hnf-terminates as each rule produces a constructor immediately. `pred` fails if its argument is `IHi`.

The function `pred` is a good example to illustrate that our categorization captures subtle differences. Should we, for instance, use the alternative definition

```
pred' x | succ y =:= x = y where y free
```

hnf-termination for hnf-terminating input would not be given. The reason is that `(=:=)` demands the full evaluation of `x` to normal form.

`pred` is important for two reasons. For one it can be used to define the useful `(n+1)` patterns in an unproblematic way. These patterns can either simply be taken as syntactic sugar for a *function pattern* [2] `(succ n)` or be translated to a simple case distinction:

```
f (n+1) = e   ⇒   f (I n')   = let n=O n'    in e
                  f n'@(O _) = let n=pred n' in e
```

Note, that in the context of *positive* natural numbers, this pattern has a slightly different semantics than in Haskell: the smallest value which `n` can be bound to is 1, not 0.

Secondly, `pred` is used in the definition of subtraction. Subtraction naturally leads to the domain of integers.

2.2 Integers

We define integers as *signed* natural numbers.

```
data Int = Pos Nat | Zero | Neg Nat
```

We first define the basic operations $(+1), (-1)$ and $(\cdot 2)$ from which we will build the more complex operations. In the equations we employ the usual signed notation for integers, i.e., $-n$ for the negative number with absolute value n.

```
inc, dec, mult2 :: Int -> Int
inc Zero = Pos IHi                  0 + 1 = 1
inc (Pos n) = Pos (succ n)          n + 1 = n + 1
inc (Neg IHi) = Zero                -1 + 1 = 0
inc (Neg (O n)) = Neg (pred (O n)) -2n + 1 = -(2n - 1)
inc (Neg (I n)) = Neg (O n)         -(2n + 1) + 1 = -2n

dec Zero = Neg IHi                  0 - 1 = -1
dec (Neg n) = Neg (succ n)          -n - 1 = -(n + 1)
dec (Pos IHi) = Zero                1 - 1 = 0
dec (Pos (O n)) = Pos (pred (O n)) 2n - 1 = 2n - 1
dec (Pos (I n)) = Pos (O n)         2n + 1 - 1 = 2n

mult2 Zero    = Zero                0 · 2 = 0
mult2 (Pos n) = Pos (O n)           n · 2 = 2n
mult2 (Neg n) = Neg (O n)           -n · 2 = -(2n)
```

All three functions `inc`, `dec` and `mult2` perform a simple pattern matching before directly returning a head normal form. They therefore inherit hnf- and nf-termination from their arguments and the functions `pred` and `succ`, respectively.

Employing the three basic operations we can define subtraction on natural numbers in a concise way. Subtraction maps two natural numbers to an integer.

```
sub :: Nat -> Nat -> Int
IHi `sub` m   = inc (Neg m)             1 - m = (-m) + 1
O n `sub` IHi = Pos (pred (O n))        2n - 1 = 2n - 1
O n `sub` O m = mult2 (n `sub` m)       2n - 2m = 2 · (n - m)
O n `sub` I m = dec (mult2 (n `sub` m)) 2n - (2m + 1) = 2 · (n - m) - 1
I n `sub` IHi = Pos (O n)               2n + 1 - 1 = 2n
I n `sub` O m = inc (mult2 (n `sub` m)) 2n + 1 - 2m = 2 · (n - m) + 1
I n `sub` I m = mult2 (n `sub` m)       2n + 1 - (2m + 1) = 2(n - m)
```

If both arguments nf-terminate then `sub` also nf-terminates as `sub` descends on both arguments and calls only functions which also nf-terminate for this category

of arguments. Moreover, if the first argument nf-terminates and the second at least hnf-terminates then `sub` hnf-terminates because it descends on the first argument, finally calling the hnf-terminating function `inc` (first rule) and it calls only hnf-terminating functions on the right hand sides of the remaining rules.

Employing `add,sub,mult` on natural numbers, we can simply define their counterparts on integers.

```
(+), (-), (*) :: Int -> Int -> Int
Pos n + Pos m = Pos (n `add` m)     n + m = n + m
Neg n + Neg m = Neg (n `add` m)     −n + (−m) = −(n + m)
Pos n + Neg m = n `sub` m           n + (−m) = n − m
Neg n + Pos m = m `sub` n           −n + m = m − n
Zero  + n     = n                   0 + n = n
n@(Pos _) + Zero = n                n + 0 = n
n@(Neg _) + Zero = n                n + 0 = n

n - Neg m = n + Pos m               n − (−m) = n + m
n - Pos m = n + Neg m               n − m = n + (−m)
n - Zero  = n                       n − 0 = n

Pos n * Pos m = Pos (n `mult` m)    n · m = n · m
Pos n * Neg m = Neg (n `mult` m)    n · (−m) = −(n · m)
Neg n * Neg m = Pos (n `mult` m)    −n · (−m) = n · m
Neg n * Pos m = Neg (n `mult` m)    −n · m = −(n · m)
Zero  * _     = Zero                0 ∗ n = 0
Pos _ * Zero  = Zero                n ∗ 0 = 0
Neg _ * Zero  = Zero                n ∗ 0 = 0
```

All termination properties are inherited from the operations on natural numbers.

Likewise we can build `compare` on integers on the comparison of natural numbers.

```
compare :: Int -> Int -> Ordering
compare Zero    Zero    = EQ          0 = 0
compare Zero    (Pos _) = LT          0 < n
compare Zero    (Neg _) = GT          0 > −n
compare (Pos _) Zero    = GT          n > 0
compare (Pos n) (Pos m) = compNat n m n ∘ m ⇔ n ∘ m
compare (Pos _) (Neg _) = GT          n > −n
compare (Neg _) Zero    = LT          −n < 0
compare (Neg _) (Pos _) = LT          −n < m
compare (Neg n) (Neg m) = compNat m n −n ∘ −m ⇔ m ∘ n
```

Again all termination properties are inherited.

2.3 Division and Modulo

Last we want to introduce division and modulo operations in the following sense.

$$n = m * d + r \wedge r < m \Leftrightarrow div(n, m) = d \wedge mod(n, m) = r$$

We only add these definitions for completeness but we are not yet satisfied with the implementation. Therefore we omit completeness, termination and narrowing results for the division and modulo operation.

Analogous to the previous sections we first define simple basic operations to build on. First the partially defined shift operation corresponds to what is known as a "logical right shift".

```
shift :: Nat -> Nat
shift (O n) = n
shift (I n) = n
```

Obviously, shift nf- (hnf-)terminates whenever its argument nf- (hnf-) terminates.

Next, we define the special case of modulo 2.

```
mod2 :: Nat -> Int
mod2 IHi   = Pos IHi   1 = 2 · 0 + 1 ∧ 1 < 2
mod2 (O _) = Zero      2n = 2n + 0 ∧ 0 < 2
mod2 (I _) = Pos IHi   2n + 1 = 2n + 1 ∧ 1 < 2
```

`mod2` nf-terminates if its argument hnf-terminates.

We can now specify the general operation which computes div and mod for given natural numbers.

```
divmod :: Nat -> Nat -> (Int,Int)
divmod x y
  | y==IHi    = (Pos x,Zero)            x = x · 1 + 0 ∧ 0 < 1
  | otherwise =
      case compNat x y of
        EQ -> (Pos IHi,Zero)            x = y ⇒ x = 1 · y + 0 ∧ 0 < y
        LT -> (Zero, Pos x)             x < y ⇒ x = 0 · y + x ∧ x < y
        GT -> case divmod (shift x) y of
                (Zero,_) -> (Pos IHi,x `sub` y)
                (Pos d,Zero) -> (Pos (O d),mod2 x)
                (Pos d,Pos m) ->
                  case divmod (carryover x m) y of
                    (Zero,m')   -> (Pos (O d),m')
                    (Pos d',m') -> (Pos (O d `add` d'),m')
  where
    carryover (O _) n = O n
    carryover (I _) n = I n
```

3 Narrowing

With respect to narrowing we are interested in the kind of equations we are able to solve. For that purpose we make some conjectures enabling us to deduce solvability properties from the termination properties which were given with the operation definitions. The general discussion of these conjectures is beyond the scope of this paper.

First, we extend our categories of termination.

- We call an expression *finitely branching* if its evaluation will induce only a finite number of non-failing non-deterministic branches.
- We add to the notion of nf-termination of an expression e the requirement that each branch induced by the evaluation of e nf-terminates.
- We add to the notion of hnf-termination of an expression e the requirement that the evaluation of e only introduces a finite number of non-deterministic branches before producing an hnf in each non-failing branch.

Note that the search space of an hnf-terminating expression may still be infinite. For example the expression `nat` with the following definition is hnf-terminating.

```
nat = 0 ? S nat
```

In the following we will denote the combination "nf-terminating and finitely branching" by simply saying "finite". A simple example of an expression which is nf-terminating but not finite is `zeros` with the definition

```
zeros = 0 ? zeros
```

Moreover, because free variables can only be bound to finite ground terms, we classify them as nf-terminating, although not as "finitely branching".

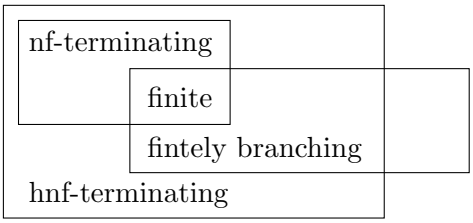


Fig. 1. The Categories and their Relations of Inclusion

For convenience, Figure 1 displays the relations between the categories introduced.

Conjecture 3.1 (Terminating Search)
Let e be a finite expression. Then a depth first search will produce all values of e in finite time and finally terminate.

For example, the unification `(=:=)` built-in for Curry is finite if one of its arguments is finite and the other one is hnf-terminating.

If the conjecture holds, information about nf-termination directly corresponds to solvability of equations. For ease of reference we have collected this kind of information about the numeric operations presented in this paper in the following table.

Expression built with	Argument requirement for nf-terminating result
`succ, inc, pred, dec, mult2`	nf-terminating
`add, (+)`	both nf-terminating
`mult, (*)`	both nf-terminating
`compNat, compare`	any nf-terminating, other hnf-terminating
`(<), (<=), (>), (>=)`	any nf-terminating, other hnf-terminating
`sub, (-)`	both nf-terminating
`(=:=)`	any nf-terminating, other hnf-terminating

Conjecture 3.2 (Complete Search)
Let e be an nf-terminating or hnf-terminating expression. Then an or-fair strategy like breadth first search will produce all values of e but may not terminate after producing the last value.

`(=:=)` terminates (hnf and nf coincide for type `Success`) if one of its arguments nf-terminates and the other one is hnf-terminating.

The following table lists the hnf-termination properties of the defined numeric operations.

Expression built with	Requirement to Argument for hnf-terminating result
`succ, inc, pred, dec, mult2`	hnf-terminating
`add, (+)`	both hnf-terminating
`mult, (*)`	both hnf-terminating
`compNat, compare`	any nf-terminating, other hnf-terminating
`(<), (<=), (>), (>=)`	any nf-terminating, other hnf-terminating
`sub, (-)`	first nf-terminating, other hnf-terminating
`(=:=)`	any nf-terminating, other hnf-terminating

If both conjectures hold, we can now infer whether or not certain equations can be solved and by which strategy. For example, we again consider the Pythagorean triples. Breadth first search completely finds the values of the following expression, where we assume `a,b,c` to be free variables:

```
a*a + b*b =:= c*c &> (a,b,c)
```

For a fixed `c`, e.g. beforehand bound by `c =:= 20`, all solutions are also computed by depth first search.

On the other hand, the following expression does not terminate after all solutions

are computed:

```
x+x=:=x
```

4 Experiments

We have made some experiments to evaluate the efficiency of the arithmetic operations. Our benchmarks were run on an AMD Athlon^{TM} XP 3000+ with 2 GHz. It is no surprise that our implementation of integers is clearly outperformed by primitive arithmetic operations. It is, however, efficient enough to justify its use in a functional logic language – given the additional possibilities outlined in the previous sections. We think that high performance of arithmetic computations is not as important as a seamless integration of arithmetics in the functional logic paradigm.

We have compared a Haskell implementation of our operations with GHC's native implementation of `Integers`[3]. In addition, we have compared a Curry implementation of our operations with the primitive arithmetic operations integrated in PAKCS [6]. As we do not aim at comparing the Haskell with the Curry implementation, we use different input values to obtain significant results in both systems. To measure only the time necessary for arithmetic operations, we employ the following test function:

```
test _  x 1     1     = True
test op x 1     (m+1) = test op x x m
test op x (n+1) m     = eval (op n m) && test op x n m

eval n = n==n

bench op x = test op x x x
```

We have called `bench` with the arithmetic operators `(+)`, `(-)`, `(*)`, `div` and `mod`. To measure the overhead imposed by this test function, we have called `bench` with the tuple constructor `(,)` and subtracted the runtime for this computation from all runtimes for the arithmetic operations.

We have employed the latest versions of GHC and PAKCS for our benchmarks. For GHC we have instantiated the parameter `x` of `bench` with 4000 and for PAKCS with 300 to obtain significant results – they are depicted in Table 1. The run times of our arithmetic operations and the built-in functions – after subtracting overhead – are shown in one column. The slowdown of our library is depicted next to the run times.

The measured slowdown imposed by multiplication using GHC is significantly larger than the slowdown for the other operations. We believe that it stems from the larger results of the computation that have to be consumed by `eval`. It also suggests that our definition of multiplication is far from optimal and we are open to suggestions. In PAKCS – where the largest computed value is significantly smaller – there is no such slowdown.

We can see that our arithmetic library is roughly up to 20 times slower than

[3] Note that the size of `Int` values is limited in Haskell.

	GHC `bench op 4000`		PAKCS `bench op 300`	
`op`	seconds	slowdown	seconds	slowdown
`(+)`	9.74/0.45	21.6	3.35/1.03	3.25
`(-)`	9.39/1.15	8.17	7.03/1.01	6.96
`(*)`	62.1/0.52	119	12.8/1.02	12.5
`div`	12.9/2.24	5.77	19.4/1.02	19.0
`mod`	20.5/2.24	9.13	21.9/1.04	21.0

Table 1
Experimental Results

primitive arithmetic operations. Also, our library compiled with GHC is faster than the primitive arithmetic operations of PAKCS. As Curry programmers do not complain about the arithmetic performance of PAKCS, we feel confident that they would also not complain about a Haskell-based Curry system [3] that exclusively uses our narrowable arithmetic operations. Because of the additional support for narrowing, it could even be an option for PAKCS to use our library instead of a primitive implementation of integers. In addition, the time spent in arithmetic computations is usually only a fraction of the run time of functional (logic) programs. Even in our benchmarks, which were designed to make mostly arithmetic operations, the overhead measured with `(,)` as `op` was much larger than the time used to compute the result of the arithmetic computations in most cases.

5 Conclusions

We propose a different implementation of arithmetic operations for functional logic programming languages. Currently, declarative programming languages employ external functions to do arithmetics, which results in serious limitations in the context of functional logic programming. Free variables cannot be guessed in arithmetic operations and need to be generated explicitly.

We show that a binary encoding of integers as algebraic datatype eliminates this need. We define all usual arithmetic operations on this datatype and show that our implementation can be used to narrow unknown arguments to these operations. Practical experiments show that our implementation produced significant overhead compared with primitive operations. However, we feel that programmers willingly accept this overhead given the new opportunities of narrowing.

Our implementations of the arithmetic operations are fairly straightforward. We curiously expect suggestions to improve them in future work. Also, our conjectures w.r.t. the termination analysis may lead to interesting future research.

References

[1] Antoy, S. and M. Hanus, *Functional logic design patterns*, in: *Proc. of the 6th International Symposium on Functional and Logic Programming (FLOPS 2002)* (2002), pp. 67–87.

[2] Antoy, S. and M. Hanus, *Declarative programming with function patterns*, in: *Proceedings of the International Symposium on Logic-based Program Synthesis and Transformation (LOPSTR'05)* (2005), pp. 6–22.

[3] Braßel, B. and F. Huch, *Translating Curry to Haskell*, in: *Proc. of the ACM SIGPLAN 2005 Workshop on Curry and Functional Logic Programming (WCFLP 2005)* (2005), pp. 60–65.

[4] Hanus, M., *On the completeness of residuation*, in: *Proc. Joint International Conference and Symposium on Logic Programming* (1992), pp. 192–206.

[5] Hanus, M., *The integration of functions into logic programming: From theory to practice*, Journal of Logic Programming **19&20** (1994), pp. 583–628.

[6] Hanus, M. et al., *PAKCS: The Portland Aachen Kiel Curry System (version 1.8.0)*, Available at URL `http://www.informatik.uni-kiel.de/~pakcs/` (2007).

[7] Hanus (ed.), M., *Curry: An integrated functional logic language (vers. 0.8.2)*, Available at `http://www.informatik.uni-kiel.de/~curry` (2006).

A Generic Semantics for Constraint Functional Logic Programming

Emilio Jesús Gallego Arias[a,1,2] Julio Mariño Carballo[a,3]
José María Rey Poza[b,4]

[a] *Babel Research Group*
Universidad Politécnica de Madrid
Madrid, Spain
https://babel.ls.fi.upm.es

[b] *Telefónica I+D*

Abstract

We propose an operational semantics for lazy constraint functional programs that is *generic* in the sense of allowing the integration of different constraint solvers into an existing kernel language in a rather *clean* way. The design of the semantics has been driven by two principles often contradicting each other: the need to support laziness in the constraint side, and the need to adhere to the *black box* principle of constraint programming as much as possible. We start by taking a previous semantics for Curry with equality and disequality constraints over Herbrand terms and showing how to decouple constraint handling from a *neutral* kernel semantics. Then, we define a *Generic Constraint Solver Interface* that allows us to add new solvers, thus showing the applicability of the method. The paper contains also discussions on the role played by the type system and on how to incorporate constraint solver cooperation in this setting.

Keywords: constraint programming, functional-logic programming, Curry, Sloth

1 Introduction

Functional logic programming (FLP) and, particularly, the language Curry [5], proposes a standardization of multiparadigm declarative programming where syntax, type discipline and (most of the) declarative semantics are borrowed from Haskell and deduction methods such as *needed narrowing*, *residuation* or *encapsulated search* provide the operational semantics necessary to cover most of the functionality of existing logic programming languages.

One of the most powerful and interesting features that FLP can adopt from logic programming is constraint programming. However, this integration is not

[1] Author fully supported by CAM/ESF grant 3060/2006
[2] Email: egallego@babel.ls.fi.upm.es
[3] Email: jmarino@fi.upm.es
[4] Email: jmrey@tid.es

as seamless as desired, due in part to the lazy semantics of Curry. Shortly, lazy evaluation imposes a demand driven strategy to the evaluation of expressions that is somehow aware of the internal structure of expressions being evaluated, while constraint programming as incorporated in CLP [7], follows a black-box approach. This conflict has precluded the treatment of constraint terms as first class citizens in the execution model of FLP.

Thus, the treatment of constraints in FLP has either dropped the black box requirement or is essentially ad-hoc. Some examples:

- Domain-specific semantics for Curry plus one constraint domain – e.g. [10,4] – are hard to extend and difficult to implement/verify.
- The use of CRWL to specify a CFL programming language as a whole (TOY [8]) is an interesting formal exercise, but the specifications tend to be quite lengthy and, of course, this gives up the black box principle.
- On the practical side, there are a number of implementations (PAKCS, Sloth) where constraint domains are added interfacing existing solvers with a FL host system. However, this is usually done in an ad-hoc manner and it is hard to justify the correctness of the resulting implementation.

Our position is that the idea of interfacing Curry with external (perhaps existing) solvers is a very practical one that should not be given up,[5] but that it must be done in a uniform way, so that existing semantics do not have to suffer modifications in order to accomodate a new constraint domain.

This is mainly motivated by our own experience implementing Curry (Sloth [3]). Currently, we have support for several constraint extensions in Sloth, taking advantage of the solvers available in the underlying Ciao Prolog system. Linear and finite domain constraints use, essentially, the existing solver as is, but we do not have a formal semantics for these extensions. On the other hand, our implementation of disequality constraints [4] has a domain-specific semantics. Incorporating the other solvers in that semantics would have made it unmanageable. However, a close inspection of the semantics showed that most of the semantics was unaware of constraints and that interaction occured at certain specific points. Isolating those points was the key for a modular semantics.

Our proposal rests on previous work [11] where a trivalued framework for constraint domains was introduced. This means that the black-box principle, as defined in [7] is modified, but not abandoned: only one extra basic operation is assumed for constraint domains. The reader is referred to that paper for a discussion on why this extension is needed in order to have solvers aware of lazy evaluation.

The paper is organized as follows. Next section reviews some basic definitions on constraints and the syntax of our language. In Section 3 we introduce a presentation of an operational semantics for Curry where the Herbrand ($\mathcal{H}$) operators are completely decoupled from a constraint-neutral core. Then, in section 4 we introduce a generic constraint solver interface and show how to integrate new constraint solvers in Curry using this approach. Section 5 discusses how to – partially – achieve solver cooperation within our framework and what influences it has from

[5] The Curry report advocates this, but does not say how to build such an interface.

other cooperation frameworks. Section 6 points some places where the language or its type system needs to be modified to support this framework. Section 7 concludes and points issues for future research.

2 Preliminaries

A number of presentations exist for constraint and constraint logic programming. The following pretends to be as general as possible, and is liberally based on Jaffar and Maher's survey [7]. Let $\Sigma = (PS, FS)$ be a signature composed of a set PS of *predicate symbols* and a set FS of *function symbols*. A *primitive constraint* has the form $p(t_1, \ldots, t_n)$, where $p \in PS$ and the t_i's are (open) terms made from functors in FS and variables. A *constraint* is a (first-order) formula built from primitive constraints. Generally, only a proper subset $\mathcal{L}$ of these are considered admissible constraints for a given domain. A Σ-*structure* $\mathcal{D}$ consists of a domain D and an interpretation for every predicate and function symbol in Σ. The pair $(\mathcal{D}, \mathcal{L})$ is called the *constraint domain*.

The language $\mathcal{L}$ contains constraints which are, respectively, identical to true and false in $\mathcal{D}$, and is usually closed under variable renaming, conjunction and existential quantification. In order to be computationally useful, every constraint domain is expected to support, at least, the following operations and tests on constraints:

(i) Test for *consistency* or *satisfiability*, $\mathcal{D} \vDash \tilde{\exists} c$, where $\tilde{\exists} c$ denotes the existential closure of c.

(ii) The *implication* or *entailment* of one constraint by another: $\mathcal{D} \vDash c \rightarrow c'$, or more generally, the implication of a disjunction of constraints by another: $\mathcal{D} \vDash c \rightarrow c_1 \vee \cdots \vee c_n$.

(iii) The *projection* of a constraint c onto variables $\bar{x}$ in order to obtain an equivalent, hopefully simpler, c' such that $\mathcal{D} \vDash c' \iff \exists_{-\bar{x}} c$, where $\exists_{-\bar{x}} c$ denotes the existential closure of c on all its variables except those in $\bar{x}$.

(iv) Detecting that, given a constraint c, there is a single value for x consistent with c, i.e. $\mathcal{D} \vDash \exists z. \forall x, \bar{y}. c(x, \bar{y}) \rightarrow x = z$. We say x is *grounded* by c.

Accommodating these definitions in a language with *types*, *lazy semantics* and *higher order features* requires a number of changes which should be kept to a minimum in order to preserve the good properties of the traditional framework. The most obvious is that now the domain D must be an information domain, e.g. a *cpo*, so that *undefined* or *partially defined* values can be dealt with appropriately. Elements of FS will be interpreted as (well typed) continuous functions. Elements of PS will be interpreted as typed functions to a domain with a *bottom* element and two *total* values, e.g. $\{\bot, \texttt{Success}, \texttt{Failure}\}$. Notice that this forces the introduction of three-valued interpretations for constraints.

Having a small, standard set of operations like those listed above is what allows CLP to be such a flexible framework. For our lazy scheme we will just require an additional test on the constraint domain:

5. Test for *strictness*: a constraint c is strict in variable x iff replacing $\bot$ for x in c gives an indefinite constraint. In other words, $\mathcal{D} \vDash \tilde{\exists} c[\bot/x] = \bot$.

Remember that existential quantification must be interpreted in a three valued setting – i.e. as a least upper bound – etc. We are not working out the details of this extension here.

3 Decoupling Curry from the Herbrand domain

Given the considerations above, our first step will be to take Curry to a state with no built in constraint operators — not even equality of data terms. This semantics will be refined stepwise in order to obtain a definition generic enough to accommodate other constraint systems.

3.1 The core Curry semantics

This semantics is intended to resemble the one in the Curry report.[6] However, we have split the standard $\Rightarrow$ relation into two new relations $\Downarrow, \downarrow$ (*both* big step) relating terms with its normal and head normal forms respectively.[7] This allows us to have a standard HNF evaluation mechanism to refer to when integrating lazy evaluation of constraints into the semantics. Also, matching against a definitional tree has been renamed from $\mathsf{Eval}[\![]\!]$, to an auxiliary relation labelled as $\mathsf{Match}[\![,]\!]$.

The core semantics is shown in Figures 1 (reduction to HNF) and 2 (reduction to NF).

Answers and constraints

The standard operational semantics in the Curry report defines an element of an answer set as a tuple $\{\sigma \| e\}$, namely a term in normal form and an associated substitution. Following [4] we consider both an answer substitution and a constraint

$$\{c; \sigma \| e\}$$

but now c will be a constraint store possibly made of heterogeneous constraints (i.e. constraints from different domains). Moreover, c is intended to be treated as an abstract data type (defined in 3.2), i.e. it will only be accessed from the semantic rules by means of a restricted set of operators. This is an immediate *space saver*, as adding new domains does not make the core bigger.

Auxiliary functions

The replacement of a position in a term by a disjunctive expression (resulting in another disjunctive expression) is carried out by the auxiliary function *replace*:

$$replace(e, p, \{\gamma_1; \sigma_1 \| e_1, \ldots, \gamma_n; \sigma_n \| e_n\}) = \{\gamma_1; \sigma_1 \| \sigma_1(e)[e_1]_p, \ldots, \gamma_n; \sigma_n \| \sigma_n(e)[e_n]_p\}$$

Function *clean* removes inconsistencies from the constraint stores. These inconsistencies can arise after any reduction step. Function *clean* can be defined as follows:

- $clean(\emptyset) = \emptyset$

[6] In fact, it is an evolution of the semantics in our paper on disequality constraints [4].
[7] NF and HNF henceforth

Computation step for expressions:

$$\frac{\mathsf{Eval}[\![e_i]\!] \Downarrow E_i}{\mathsf{Eval}[\![e_1 \& e_2]\!] \downarrow replace(e_1 \& e_2, i, E_i)} \quad i \in \{1,2\}$$

$$\frac{}{\mathsf{Eval}[\![c(e_1,\ldots,e_n)]\!] \downarrow \{\epsilon; id[\![c(e_1,\ldots,e_n)\}} \quad c \in DC$$

$$\frac{\mathsf{Match}[\![f(e_1,\ldots,e_n), T]\!] \downarrow D}{\mathsf{Eval}[\![f(e_1,\ldots,e_n)]\!] \downarrow D} \quad T \in DT(f)$$

Computation step for operation-rooted expression e:

$$\frac{dt(e,T) = e' \quad \mathsf{Eval}[\![e']\!] \downarrow D}{\mathsf{Match}[\![e,T]\!] \downarrow D}$$

$dt(e, rule(l{=}r)) = \{\epsilon; \sigma[\![\sigma(r)\}$ if $\sigma(l) = e$

$dt(e, branch(\pi, p, T_1, \ldots, T_k)) =$

$$\begin{cases} D & \text{if } e|_p = c(e_1,\ldots,e_n),\ pat(T_i)|_p = c(x_1,\ldots,x_n) \text{ and} \\ & \quad \mathsf{Match}[\![e,T_i]\!] \downarrow D \\ \emptyset & \text{if } e|_p = c(\ldots) \text{ and } pat(T_i) \neq c(\ldots),\ i = 1,\ldots,k \\ \bigcup_{i=1}^{k} \{\epsilon; \sigma_i[\![\sigma_i(e)\} & \text{if } e|_p = x \text{ and } \sigma_i = \{x \mapsto pat(T_i)|_p\} \\ replace(e,p,D) & \text{if } e|_p = f(e_1,\ldots,e_n) \text{ and } \mathsf{Eval}[\![e|_p]\!] \downarrow D \end{cases}$$

Computation step for answers

$$\frac{\mathsf{Eval}[\![e]\!] \downarrow \{c_1; \sigma_1[\![e_1 \ldots c_n; \sigma_n[\![e_n\}\quad \mathsf{Eval}[\![D]\!] \downarrow D'}{\mathsf{Eval}[\![\{c; \sigma[\![e\} \cup D]\!] \downarrow clean(\{\sigma_1 c \wedge \sigma c_1; \sigma \cdot \sigma_1[\![e_1 \ldots \sigma_n c \wedge \sigma c_n; \sigma \cdot \sigma_n[\![e_n\} \cup D')}$$

Fig. 1: Head normal form Curry operational rules

$$\frac{\mathsf{Eval}[\![e_i]\!] \Downarrow D_i}{\mathsf{Eval}[\![c(e_1,\ldots,e_n)]\!] \Downarrow replace(c(e_1,\ldots,e_n), i, D_i)} \quad i \in \{1,\ldots,n\}$$

$$\frac{\mathsf{Eval}[\![e]\!] \downarrow D \quad \mathsf{Eval}[\![D]\!] \Downarrow c}{\mathsf{Eval}[\![e]\!] \Downarrow c} \text{ if } e \neq c(t_1,\ldots,t_n),\ c \notin DC$$

Fig. 2: Normal form Curry operational rules

- $clean(\{c; \sigma[\![e\} \cup D) = clean(D)$ if c is inconsistent.
- $clean(\{c; \sigma[\![e\} \cup D) = \{c; \sigma[\![e\} \cup clean(D)$ otherwise $c = \bigwedge c_I, I$ is an index for the set of constraint solvers present in the system.

3.2 Constraints and constraint stores

Following [4], we assume a syntactic category CP of *constraint predicate* symbols, so that *atomic constraints* take the form of Curry terms $p(e_1^{\tau_1}, \ldots, e_n^{\tau_n})$, with $p \in CP$, and type $p : \tau_1 \to \cdots \to \tau_n \to Success$. In order to deal with different

constraint domains, CP can be seen as a disjoint sum of several CP_i. For the sake of simplicity, in the following we will assume that the CP_i are pairwise disjoint. As usual, constraints (type $Constraint$) are first order formulae containing the atomic constraints and closed (at least) under conjunction and existential quantification.

As we previously said, we assume an opaque type which represent constraint stores CS, and we define the Extended Constraint Stores (ECS) type as the disjoint sum of:

$$ECS = CS + eval_{Expr,Pos} + fail$$

As every constraint store CS for domain constraint D[8] is dealt as a black box, we require the solvers to provide the following operations:

- $\epsilon_D : CS$ (empty constraint store).
- $tell_D : (CS \times Constraint) \rightarrow Set(ECS)$: This function is meant to tell a constraint. It returns the list of modified constraints stores if successful or:
  - $eval_{e,p}$, if the term e at position p needs to be further evaluated in order to post the constraint. Note that the position refers to the position the term was in the constraint predicate, so the term can be evaluated, substituted in the CP and we can call it again.
  - $fail$, if the constraint cannot be satisfied.

Notice that $tell_D$ return type is not a new constraint store, but a *set* of constraints. This is needed in order to allow solvers to return a set of disjoint constraint stores. Keep in mind that this allow solvers – such as the Herbrand disequality one – to return a set of answers, some of them can be successful and other could demand more evaluation:

$$tell_{\mathcal{H}}\ \epsilon\ \texttt{c(a,b)=/=c(x,f(m))} = [Success, eval_{f(m),2}]$$

with `x` a variable, then a choice is `a=/=x` or to evaluate more `f(m)` ($eval_{f(m),2}$), which as we'll see, they get transformed in two paths of computation.

As we have said above, we are requiring constraint solvers to be able to detect those constrained positions where more evaluation is needed. This idea is borrowed from the trivalued framework of [11], where a strictness test for constraints is introduced. This test consists in computing the existential closure of a constraint on all its free variables but one which is replaced by $\bot$. In order to implement this test, the solvers must interpret any symbol not in their signature as if it were $\bot$. This allows us to fed the solvers any Curry term, regardless of how normalized it is. We'll denote any term not in the solver's signature as $\boxed{\texttt{t}}$, meaning that the solver cannot "break-in" and see what's inside.

In practice, implementing this reflective feature in a real solver is not very hard, as most practical domains (arithmetic, finite domains, etc) are essentially strict, i.e. unknowns in the input produce unknowns in the output. More specifically:

- Solvers *must* interpret any symbol not in their signature as $\bot$.
- Solvers must be able to accept *all* their parameters as $\bot$.

[8] D will be substituted for a concrete constraint domain when we are speaking of a concrete one

$$\frac{tell\ \epsilon\ e = \{c_1, \ldots, c_n\}}{\mathsf{Eval}[\![e]\!] \downarrow \{c_1; id[\![Success, \ldots, c_n; id[\![Success\}} \quad e = p(e_1, \ldots, e_n)$$

$$\frac{tell\ \epsilon\ e = fail}{\mathsf{Eval}[\![e]\!] \downarrow \emptyset} \quad e = p(e_1, \ldots, e_n)$$

$$\frac{tell\ \epsilon\ e = eval_{s,p} \quad \mathsf{Eval}[\![s]\!] \downarrow D \quad \mathsf{Eval}[\![replace(e, p, D)]\!] \downarrow D'}{\mathsf{Eval}[\![e]\!] \downarrow D'} \quad e = p(e_1, \ldots, e_n)$$

Fig. 3: Constraint predicates operational rules

- Solvers must return both the arguments position and the unrecognized expression as $eval_{e,p}$. The solver should only request *one* argument to be evaluated.

Most of the above requirements are done in the name of efficiency. For instance, all the above requirements could be substituted for probing with $\perp$ each one of the constraint predicate arguments, and we'd get the same result.

3.3 Selecting the right domain

So far, we have defined the extended constraint stores and their operations, but for this scheme to support multiple solvers, we need to define $c \in [ECS]$, the sequence of ECS stores *indexed* by constraint domain.

This way, we get one element in the sequence for each solver in the system. To refer an specific constraint store, we'll use the standard sequence notation $c[D]$, so for example, to address the constraint store belonging to $\mathcal{H}$ we'll use $c[\mathcal{H}]$.

So then we can introduce the function $tell : (CS \times Constraint) \to [ECS]$, which selects the right $tell_D$ function:

$$tell\ c\ e = \begin{cases} eval_{e,p} & \text{if } tell_D\ c[D]\ e = eval_{e,p} \\ fail & \text{if } tell_D\ c[D]\ e = fail \quad \text{where } e = op_D(e_1, \ldots, e_n) \\ c[D] := c_D & \text{if } tell_D\ c[D]\ e = c_D \end{cases}$$

This means that the set of constraint predicates should be disjoint. We can overcome this problem as shown in section 6.

3.4 Operational rules for constraint solver integration

The integration of constraint solving in this framework is achieved by means of new operational rules (figure 3) dealing with constraint predicates.

The two first rules respectively deal with the case of a satisfiable constraint or a failing one. The interesting one is the third, which is needed when the constraint predicate needs its arguments to be more evaluated than in their current form.

The main novelty about the above rules is the possibility to use *lazy-aware* constraint solvers, such as the ones presented in section 3.5 and in section 4.2.

Rules for equality constraints

$$\frac{\{x \neq y\} \notin c}{put(c, \mathsf{x}\texttt{=:=}\mathsf{y}) = c \cup \{x = y\}} \quad \text{if x, y variables.}$$

$$\frac{\{x \neq t\} \notin c}{put(c, \mathsf{x}\texttt{=:=}\mathsf{t}) = c \cup \{x = t\}} \quad \text{if x variable.}$$

$$\frac{\mathsf{d_1} \neq \mathsf{d_2}}{put(c, \mathsf{d_1}(\mathsf{t_1}, \ldots, \mathsf{t_n})\texttt{=:=}\mathsf{d_2}(\mathsf{u_1}, \ldots, \mathsf{u_m})) = fail} \quad \text{if } \mathsf{d_1}, \mathsf{d_2} \in DC$$

$$\frac{\mathsf{d_1} = \mathsf{d_2} \quad c_1 = put(c, \mathsf{t_1}\texttt{=:=}\mathsf{u_1}) \ldots c_n = put(c, \mathsf{t_n}\texttt{=:=}\mathsf{u_n})}{put(c, \mathsf{d_1}(\mathsf{t_1}, \ldots, \mathsf{t_n})\texttt{=:=}\mathsf{d_2}(\mathsf{u_1}, \ldots, \mathsf{u_n})) = hclean(\cup_{i=0..n} c_i)} \quad \text{if } \mathsf{d_1}, \mathsf{d_2} \in DC$$

Fig. 4: Herbrand solver rules I

Rules for disequality constraints

$$\frac{\{x = y\} \notin c}{put(c, \mathsf{x}\texttt{=/=}\mathsf{y}) = c \cup \{x \neq y\}} \quad \text{if x, y variables.}$$

$$\frac{\{x = t\} \notin c}{put(c, \mathsf{x}\texttt{=/=}\mathsf{t}) = c \cup \{x\texttt{=/=}t\}} \quad \text{if x variable.}$$

$$\frac{\mathsf{d_1} \neq \mathsf{d_2}}{put(c, \mathsf{d_1}(\mathsf{t_1}, \ldots, \mathsf{t_n})\texttt{=/=}\mathsf{d_2}(\mathsf{u_1}, \ldots, \mathsf{u_m})) = c} \quad \text{if } \mathsf{d_1}, \mathsf{d_2} \in DC$$

$$\frac{\mathsf{d_1} = \mathsf{d_2} \quad c_1 = put(c, \mathsf{t_1}\texttt{=/=}\mathsf{u_1}) \ldots c_n = put(c, \mathsf{t_n}\texttt{=/=}\mathsf{u_n})}{put(c, \mathsf{d_1}(\mathsf{t_1}, \ldots, \mathsf{t_n})\texttt{=/=}\mathsf{d_2}(\mathsf{u_1}, \ldots, \mathsf{u_n})) = \{c_1, \ldots, c_n\}} \quad \text{if } \mathsf{d_1}, \mathsf{d_2} \in DC$$

Fig. 5: Herbrand solver rules II

3.5 A Herbrand domain solver

Once settled our core semantics, we will use a constraint solver over $\mathcal{H}$, with constraint predicates $=, \neq$ to illustrate the usefulness of our semantics as well as to help Curry to regain its previously built in constraint capabilities.

The idea of representing $\mathcal{H}$ as constraint system is already present in [9], although this approach needs the full specification of the constraint system in a white-box fashion.

We could use whatever solver for $\mathcal{H}$ we feel like, but for completeness of the paper we'll specify one – in a style rather different from Curry semantics – and show an example of the full system in action.

Let us consider the Herbrand solver state to be composed of a set of equality and disequality constraints $c = \{c_1, \ldots, c_n\}$.

Then, the predicate *put* – which is just a name for $tell_{\mathcal{H}}$ – takes as arguments a constraint store c and the constraint to evaluate.

Some auxiliary functions are needed, namely:

- *hclean cs*: Returns failure if the constraint store *cs* is inconsistent.

The reader should have noticed that this specification is not completely equivalent to the built in constraint solving in Curry, as Curry implements the optimization of applying a substitution as a side effect of the `=:=` constraint operator.

New rules for laziness

$$\frac{x = \boxed{\mathsf{t}}}{put(c, \mathsf{x\ op\ y}) = eval_{x,1}}$$

$$\frac{y = \boxed{\mathsf{t}}}{put(c, \mathsf{x\ op\ y}) = eval_{y,2}}$$

Fig. 6: $\boxed{\mathsf{t}}$ handling rules

Also, the second `=:=` rule in figure 4 doesn't do occurs check. This can be easily added using *hclean*, but it would mean complete evaluation of `t`.

For the sake of simplicity we haven't yet included the support in the semantics for solvers returning substitutions, however it should be easy, as we should just modify the *tell* type to return a substitution. Then the rules dealing with *tell*, should just apply the solver's proposed substitution to the answer expression.

This constraint solver is also missing the analysis for finite types, as presented in [4]. This is a benefit of these semantics, that we can just enhance the constraint solving side without having to care too much about Curry itself.

Another good effect is that the constraint solver is way clearer than the one which has to take Curry into account.

3.6 An example

As an example of the new hybrid system, consider the program:

```
[] ++ x      = x
(x:xs) ++ y = x : (xs ++ y)

>x ++ y =/= y
```

Then the program develops as:

(i) $\mathsf{Eval}[\![\mathsf{x{+}{+}y=/=y}]\!] \Rightarrow$ (as `x++y` is not in solver's signature, it's read as $\boxed{\mathsf{t}}$).

(ii) $tell(\epsilon, \boxed{\mathsf{x{+}{+}y=/=y}} \neq y) = eval_{\mathsf{x{+}{+}y},1} \Rightarrow$ (tell is invoked as `=/=` is in CP)

(iii) $\mathsf{Eval}[\![\mathsf{x{+}{+}y}]\!] = \{\epsilon; \{x \mapsto []\}[\![\mathsf{y}\}] \cup \{\epsilon; \{x \mapsto (x_1 : x_2)\}[\![\mathsf{x_1 : (x_2{+}{+}y)}\}] \Rightarrow^*$

(iv) First case:
  (a) $tell(\epsilon, y \neq y) = fail \Rightarrow \emptyset$

(v) Second case:
  (a) $put(\epsilon, x_1 : (x_2 \mathsf{{+}{+}} y) \neq y) = \top \Rightarrow$
  (b) $\{\{y \neq x_1 : (x_2 \mathsf{{+}{+}} y)\}; \{x \mapsto (x_1 : x_2)\}[\![Success\}]$

The most important point is the step between point 3 and 4, where *clean* gets applied and both the inconsistent elements of the disjunctive expression get eliminated and the substitutions get applied to the constraints store.

4 General constraint solvers integration

We have shown that the above scheme is suitable for splitting $\mathcal{H}$ to its own constraint solver. What about applying it to general constraint solvers?

Below we list some example solvers and how to integrate them in this scheme.

4.1 Arithmetic solvers

All arithmetic solvers ($\mathcal{FD}$, CLPR) are completely strict in their arguments, so we only need to modify $tell_{\mathcal{FD}}$ – for instance – to return *eval* in every case their argument is not in HNF.

We already have implemented support for CLPR and $\mathcal{FD}$ in Sloth, using the underlying solvers available in Ciao.

As the solvers are completely strict, but they don't directly support tri-valued semantics, we had to write a wrapper for each solver [9]:

```
tell_fd(Constraint, Res) :-
    Constraint ..= [Op, Arg1, Arg2],
    (
        bot(Arg1) →
        Res = bot(1, Arg1)
    ;
        bot(Arg2) →
        Res = bot(2, Arg2)
    ;
        call(Constraint) →
        Res = Success
    ;
        fail
    ).
```

Note that the Prolog solvers carry around the constraint store c as a global variable.

This doesn't preclude the obvious optimization of reducing the arguments to HNF previously to any call to the solvers.

4.2 String equality solver

A better example is the constraint solver for string equality, in the style of Prolog III [1].

Such a constraint solver carries a signature of text strings plus string concatenation ($\bullet$) and the equal $=_S$ constraint operator.

This way, the Curry program:

```
f = "a"
m = loop

g = f • m =_S "ba"
```

can be solved, whereas in a non lazy language the program would loop.

This is due to the fact that previously, telling the constraint would force both `f` and `m` to be evaluated to NF, but in this case, we call the solver with both arguments

[9] actually, it is the same as both are strict.

evaluated, and the solver requests the evaluation of `f`, and this is enough to solve the constraint.

5 Bridges between solvers in Curry

Cooperation among constraint solvers has been a topic which has been treated on several previous works (see, for example [6]). These works are based, at least partially, on sharing variables among solvers (that is, a given variable is known by more than one solver).

This is perfectly possible on some logic languages where a type system does not exist. It is possible, for example, to write constraints such as:

```
X>2.5, X in [2,2.5,3,7,9.8]
```

In the example above it is clear that the type for `X` is numeric but it is not as clear whether it is Int or Float since the first expression seems to hint that `X` has type `Float` while the second one states that `X` belongs to an enumerated type (usually, an integer). Additionally, the list of allowed values for the variable contains both integer and floating point values.

As said before, this can happen in untyped languages, however, this is not the case when we switch to typed functional logic languages. In such functional-logic languages, every term *has* a specific type, so variables definitively belong to either `Int` of `Float`, but no to both.

To remedy this, a new operator called *bridge* has been proposed in [2]. Bridges are a new kind of constraints adopting the shape `x#=y`, where `x` and `y` are different free variables of different types.

A bridge like `x#=y` states that from that point on, variables `x` and `y` will always have *equivalent* values from their respective domains. Think, for example of `x` as an integer variable and `y` as a floating point variable. Then, the values 1 for `x` and 1.0 for `y` are equivalent. However, we should note that domain equivalence is very specific to the solvers. For more details about bridges, we suggest to read the related bibliography.

A example of bridge use in Curry would be:

```
x > 2.5 & x #= fx & fx in [2,2.5,3,7,9.8] where x, fx ↩
   free
```

5.1 How to incorporate bridges

Although we haven't fully developed a semantics for solver cooperation in this framework, our first analysis shows that there is no need to modify the presented semantics at all! This result is surprising, but it comes as a result of encapsulation of the constraint programming side in the semantics.

This way, solver cooperation can be achieved within this semantics just by defining `#=` as a new constraint predicate with its associated constraint store, where bridges are stored in a similar way to other proposals.

Before going on, we should note that due to the typing discipline present in

Curry, it must exist a different bridge operator for every supported scheme cooperation. How can the system infer what bridge to apply is described in section 6.4.

Let's call the *tell* operation for bridges *link*, then link can be just defined as:

$$link\ c\ \texttt{a\#=b} = c \cup \{bridge(a, b)\}$$

5.2 Hooking bridge propagation

Unfortunately, the *link* definition above is not enough, as bridges must *propagate* the constraints to be effective.

This means bridges must be able to access other constraint stores, and modify them. The way we achieve this is by redefining the given *tell* definition (in section 4), so the constraint propagation takes place.

Let's define a new $tell'$, to be used when the system supports cooperation, so for a bridge $\texttt{\#=}_{D,E}$, meaning that it links variables of the domain D with E, and be B the domain of the bridge constraint solver, then the new *tell* would be:

$$tell'\ c\ e = \begin{cases} eval_{e,p} & \text{if } tell_D\ c[D]\ e = eval_{e,p} \\ fail & \text{if } tell_D\ c[D]\ e = fail \\ check(e, D, c) & \text{otherwise} \end{cases}$$

where $e = op_D(e_1, \ldots, e_n)$ and

$$check(e, D, c) = tell_D(c, e) \quad \text{if doesn't exist a } bridge(a, b) \in c[B] \text{ for all variables } a \text{ in } c[D]$$

$$check(e, D, c) = hp(e, c[D], c[E]) \quad \text{for all } bridge(a, b) \in c[B] \text{ and } a \in c[D], b \in c[E]$$

5.3 The cooperation mechanism

The last bit needed is to define the *hp* operator. For this we need a way to:

- Find the projection of the new domain for x over y. That is, find what constraints over y are *equivalent* to the conditions just imposed on x. Let us call *propagate* the function that receives the origin constraint store, the destination one, the newly *told* constraint and returns the relevant projection (i.e. $propagate(S_x, S_y, e)$) will be calculated so as to find a new constraint $C_{S_x \to S_y}$).
- The constraint $C_{S_x \to S_y}$ must be fed into the destination solver S_y: $tell(C_{S_x \to S_y})$.

Then *hp* gets defined as

$$hp(e, c_D, c_E) = \begin{cases} c[D] = tell_D\ c[D]\ propagate(D, E, e, c_D) \\ c[E] = tell_E\ c[E]\ propagate(E, D, e, c_E) \end{cases}$$

Propagation has to be done currently in both ways.

6 Modifications to Curry syntax

In the previous sections we have discussed the operational semantics, without digging into the details of the actual modifications to the language itself.

In fact, for this proposal to be usable at source level, we need to greatly improve some of other language features.

Our preliminary proposal is just to add type classes [13] to Curry and let them resolve the likely ambiguity in constraint predicates.

6.1 Using type classes

The basic operator we want to support is the constrained equality `=:=` operator:

```
class Constrainable a where
   =:= :: a → a → a
```

But, what about algebraic data types? We have suddenly lost the possibility of doing

```
data A = A
> x =:= A
```

as the compiler cannot find an instance of `Constrainable` for A.

There are two possible remedies for this:

- Require a `deriving` declaration for every ADT we want to be constrainable:

  ```
  data A = A
       deriving(Show,Constrainable)
  ```

- Let the compiler automatically derive such an instance for every `data` declaration.

This has the effect[10] of tagging every function where `=:=` is used with the `Constrainable` type constraint:

```
f x y z = x =:= y & y =:= z
> :t f
> f has type Constrainable a ⇒ a → a → a → Success
```

6.2 Constraint flattening

Then, using `Success` as our basic tool for constraint programming feels a little bit unnatural when dealing with functions. Thus the objective of constraint flattening is to allow constraints predicates to be represented as pure functions, by hiding the step of creating a new fresh variable from the user.

Think about the constraint predicate `:+: :: a →a → a → Success`, which is the usual sum constraint operator present in CLPR.

Then, as we have guaranteed that every constraint domain will at least support `=:=` we can implement a more natural plus operator that allows us to sum a variable

[10] We think it is a nice side effect, but we recognize that this "tagging" can be seen as very cumbersome by some people

number of reals, and to get the resulting constraint operator.

```
(+) :: Constrainable a ⇒ a → a → a
a + b = (:+:) a b c &> c where c free
```

This way, we can write the expression:

```
> let a,b free in a + b - 3 + a =:= 3
> Success {2a + b = 6}
```

and the flattening does its job.

6.3 More type classes

Of course, once we have settled the basic class for constrainable data types, we can extend other standard type classes to include support for constraints.

For instance the `Num` type class could look like:

```
class Constrainable a ⇒ Num a where
  (+) :: a → a → a
...
```

We did informally implement this kind of behavior with run-time type detecting, but of course, we believe using type classes for this is a way better approach.

6.4 Representing bridges

This type classes scheme could allow more sophisticated constraints operations, such as representing bridges for constraint solving cooperation:

```
class (Constrainable a, Constrainable b) ⇒ Bridge a b ↩
   where
  =# :: a → b → Success
```

This however is a little bit far fetched, as requires multiparametric type classes[12].

7 Conclusions and future work

We have defined an operational semantics for Curry that allows to incorporate new constraint solvers to a constraint neutral core. This semantics makes explicit the genericity that was suggested by a previous semantics for Curry with disequality constraints over the Herbrand universe.

In order to allow for this modularity we require some functionality to be provided by the constraint solvers, namely the ability to detect and communicate demand information to the core semantics. This simple extension permits a fully lazy behaviour for constraint expressions and a uniform interface when adding new solvers to the core.

We have shown that the semantics is expressive enough to incorporate the constraint extensions currently present in one implementation of Curry (Sloth), namely: equality and disequality constraints over Herbrand terms, linear constraints over rationals, finite domain constraints over naturals and propositional constraints.

In our opinion this is at least a good indication on the practicality of this approach but, of course, a correctness and completeness proof of the semantics is needed. This will be the immediate step following this presentation.

The paper contains also some ideas on the realization of solver cooperation in our framework. These are far more speculative, and it is required to test its practicality by means of representative examples, and to investigate more on the role that type classes can play in this integration.

References

[1] Colmerauer, A., *An introduction to prolog III*, Commun. ACM **33** (1990), pp. 69–90.

[2] Estevez, S., A. J. Fernandez, T. Hortala, M. Rodríguez Artalejo and R. del Vado Vírseda, *A fully sound goal solving calculus for the cooperation of solvers in the cflp scheme*, in: *Proceedings of the 15th Workshop on Functional and (Constraint) Logic Programming (WFLP 2006)*, pp. 235–252.

[3] Gallego Arias, E. J. and J. Mariño, *An overview of the Sloth2005 Curry System: System Description*, in: *WCFLP '05: Proceedings of the 2005 ACM SIGPLAN workshop on Curry and functional logic programming* (2005), pp. 66–69.

[4] Gallego Arias, E. J., J. Mariño and J. M. Rey, *A proposal for disequality constraint in curry*, in: *Proceedings of the 15th Workshop on Functional and (Constraint) Logic Programming (WFLP 2006)*, pp. 269–285.

[5] Hanus, M., S. Antoy, H. Kuchen, F. J. López-Fraguas, W. Lux, J. J. Moreno-Navarro and F. Steiner, "Curry: An Integrated Functional Logic Language," 0.8.2 edition (2006), editor: Michael Hanus.
URL http://www.informatik.uni-kiel.de/~mh/curry/report.html

[6] Hofstedt, P., *Cooperating constraint solvers*, in: *Principles and Practice of Constraint Programming*, 2000, pp. 520–524.
URL citeseer.ist.psu.edu/hofstedt00cooperating.html

[7] Jaffar, J. and M. J. Maher, *Constraint logic programming: A survey*, Journal of Logic Programming **19/20** (1994), pp. 503–581.
URL http://citeseer.ist.psu.edu/jaffar94constraint.html

[8] López-Fraguas, F. J., M. Rodríguez-Artalejo and R. del Vado Vírseda, *A lazy narrowing calculus for declarative constraint programming.*, in: E. Moggi and D. S. Warren, editors, *PPDP* (2004), pp. 43–54.

[9] López-Fraguas, F. J. and J. Sánchez-Hernández, *Disequalities may help to narrow.*, in: M. C. Meo and M. V. Ferro, editors, *APPIA-GULP-PRODE*, 1999, pp. 89–104.

[10] Lux, W., *Adding linear constraints over real numbers to curry*, in: *FLOPS '01: Proceedings of the 5th International Symposium on Functional and Logic Programming* (2001), pp. 185–200.

[11] Mariño, J. and J. M. Rey, *Adding constraints to curry via flat guards*, in: *WCFLP '05: Proceedings of the 2005 ACM SIGPLAN workshop on Curry and functional logic programming* (2005), pp. 14–22.

[12] Simon Peyton Jones, M. J. and E. Meijer, *Type classes: exploring the design space*, in: *Proceedings of the Haskell Workshop 1997*, 1997.
URL http://research.microsoft.com/Users/simonpj/Papers/type-class-design-space/multi.ps.gz

[13] Wadler, P. and S. Blott, *How to make ad-hoc polymorphism less ad-hoc*, in: *Conference Record of the 16th Annual ACM Symposium on Principles of Programming Languages* (1989), pp. 60–76.
URL citeseer.ist.psu.edu/wadler88how.html

a-logic with arrows

Murdoch J. Gabbay
Heriot-Watt University

Michael J. Gabbay
King's College London

Abstract

We present an extension of first-order predicate logic with a novel predicate '$\mathbf{at}\,(t)$' meaning intuitively "this term is a variable symbol". We give simple sequent proof-rules for it, we demonstrate cut-elimination for the resulting logic, and we give a semantics for which the logic is sound and complete.
Because we can now make assertions about what would normally be considered an intensional property of a term (being a variable symbol) we can now express inside the logic, properties of its terms and predicates which would normally be external to the logic. We give axiomatisations in a-logic, including of the lambda-calculus, and discuss what relevance this might have to logic programming.

1 Introduction

a-logic extends classical First-Order Logic (FOL) with a predicate $\mathbf{at}\,s$ such that if s is *not* a variable symbol then $\mathbf{at}\,s$ is contradictory. We read $\mathbf{at}\,a$ as 'a is a variable symbol'.

Let a, b, c, and so on, be variable symbols. A simple inductive definition of substitution on *abstract syntax* without binding is:

$$a[a{:=}b] \equiv b \quad a{\not\equiv}b \Rightarrow b[a{:=}c] \equiv b \quad f(a_1,\ldots,a_n)[a{:=}b] \equiv f(a_1[a{:=}b],\ldots,a_n[a{:=}b])$$

Here f is a term-former and we write $s \equiv t$ for 's and t are identical terms', and $s \not\equiv t$ for 's and t are syntactically different terms'. For the first two clauses it is necessary to identify a and b as variable symbols. Of course substitution is at the heart of many major systems in computer science, including logic and functional- and logic programming.

We find that $\mathbf{at}$ gives just enough power to axiomatise the λ-calculus. We investigate the λ-calculus example in detail but by the end of the paper it should seem very plausible that a-logic has broader applications.

So what is new? Plenty of other systems can express the syntax and reductions of the λ-terms. To our knowledge a-logic is unique in that it has this expressivity

[1] Michael Gabbay gratefully acknowledges the support of the British Academy under the grant PDF/2006/509.

while still being a first-order (that is, β-reduction is not primitive) logic such that: variables of the object theory are variables of the logic *and* λ and application are just binary term-formers taking pairs of terms and giving terms.

We would like to say that again, but we need some (informal) terminology. Say a map is a **shallow embedding** from one system (logic or calculus, say) to another, when variables map to variables (with a **deep embedding** variables map to constants) [2,26]; a map is **compositional** when the interpretation of an expression can be obtained directly by composing the interpretation of its parts in a syntax-directed manner. Say a system is **first-order** when its language of terms does not already contain β-reduction.

Our application of a-logic is unique in that it is to our knowledge the only compositional shallow embedding of the λ-calculus into a first-order logic (see §6.1).

The reader need not care about logic *or* about the λ-calculus. However they are paradigmatic and the basis of logic- and functional programming respectively. It has not previously been possible to directly (in the sense just given) map one to the other. We discuss potential applications and related work in the Conclusions.

Outline of the paper. a-logic is a straightforward classical first-order logic with a Boolean semantics. We define syntax in §2.1 and sequent rules in §2.2. We axiomatise substitution in §3.1, then the untyped λ-calculus in §3.2, and finally we briefly consider other examples. §4 proves cut-elimination. §5 constructs a class of boolean-valued semantics and proves soundness and completeness. §6 builds two of these semantics for the axioms of the λ-calculus.

The 'a' in 'a-logic' refers to the fact that the logic is for making statements about its variable symbols a, b, c. Any resemblance to an indefinite article is coincidental.

2 a-logic

2.1 Terms, directions, predicates

Assume a countably infinite set $a, b, c \in \mathbb{V}$ of variable symbols and some untyped language of abstract syntax trees. Write s, t, for arbitrary terms, built inductively from variable symbols and **term-formers** of fixed arity applied to terms. Write $t[a{:=}t']$ for the usual substitution on terms. Write Vt for variable symbols occurring in t. We may write $a \in t$ for $a \in Vt$. For convenience we may write a variable symbol as s, generally when we are quantifying as in $\forall s.s = s$ (but that quantification is implicit in axioms, which start from §3.1).

We assume three **directions** $\uparrow$ **up**, $\downarrow$ **down**, and $\circlearrowleft$ **up and down**. Where necessary, d will vary over directions. Assume some set of **predicate constant symbols** $p, q, r \ldots \in \mathbb{P}$, each with an arity $\delta = (d_1, \ldots, d_n)$ which is a sentence in $\{\uparrow, \downarrow, \circlearrowleft\}^*$ (thus, a possibly empty list of directions). We write $p : \delta$ for 'p has arity δ'. When δ has length n and some list of terms also has length n, we say that list has **length appropriate to** δ. We assume distinguished predicate constants:

(i) We call $\rightsquigarrow : (\uparrow, \downarrow)$ **aequality** (pronounced 'ayquality').

(ii) We call **at** $: (\downarrow)$ **atom** ('is an atom').

(iii) We call $\bot : ()$ **false** or **contradiction**.

We may write ts as shorthand for a list of terms $t_1, \ldots, t_n$. Write $p(ts)$ for a predicate constant applied to a list of terms of length appropriate to its arity.

Consider $p : (d_1, \ldots, d_n)$ and a list of terms $(t_1, \ldots, t_n)$. When for all i such that $a \in t_i$, it is the case that $d_i \in \{\uparrow, \circlearrowleft\}$, we say a **occurs up in** $p(ts)$ and we write $a{\uparrow}p(ts)$. Similarly when for all i such that $a \in t_i$ we have $d_i \in \{\downarrow, \circlearrowleft\}$, write $a{\downarrow}p(ts)$. When for all i such that $a \in t_i$ we have $d_i = \circlearrowleft$, write $a{\circlearrowleft}p(ts)$.

The intuition of $\rightsquigarrow$ is a reduction relation (like β-reduction, for example). The intuition of $a{\downarrow}p(ts)$ is a 'subject reduction' property, that if $s \rightsquigarrow t$ then $p(ts)[a{:=}s]$ implies $p(ts)[a{:=}t]$. If $a{\uparrow}p(ts)$ and $s \rightsquigarrow t$ then $p(ts)[a{:=}t]$ implies $p(ts)[a{:=}s]$. Finally, if $a{\circlearrowleft}p(ts)$ and $s \rightsquigarrow t$ then $p(ts)[a{:=}s]$ if and only if $p(ts)[a{:=}t]$. More on this in §2.3.

The intuition of $\mathbf{at}\, s$ is that s is a variable symbol. The arity $(\uparrow, \downarrow)$ of $\rightsquigarrow$ is consistent with its intuition. **at** must have arity $(\downarrow)$ to avoid inconsistencies, we see this later.

There is no interaction (yet) between directions and the terms, so $p(ts)$ is always well-formed so long as ts has the right length. **Predicates** are generated by the grammar

$$P ::= p(ts) \mid P \subset P \mid \forall a.P.$$

$\forall a.P$ is binding, nothing else is. Consistent with our convention for terms we write $P \equiv Q$ for 'P and Q are identical predicates up to α-equivalence'.

Write VP for **the variables occurring in** P, defined by: $Vp(t_1, \ldots, t_n) = \bigcup Vt_i$, $V(P \subset Q) = VP \cup VQ$, and $V(\forall b.P) = VP \backslash \{b\}$. We may write $a \in P$ for $a \in VP$, we read it 'a **occurs in** P'. Then $a \notin P$ means 'a does not occur in P'.

We strictly equate predicates up to α-equivalence, we will not *talk about 'occurring free' or 'occurring bound'.*

Write $P[a{:=}s]$ for P with every instance of the variable a replaced by the term s in the usual, capture-avoiding, manner.

2.2 Contexts and judgements

A **(logical) context** Γ, Δ is a set of predicates. We way write $V\{P, Q\}$ for $VP \cup VQ$, and $V\Gamma$ for the natural meaning as a union; we may even mix, as in $V(\Gamma, P, \Gamma')$. We shall write $a \notin \Gamma, P, \Gamma'$ for $a \notin V(\Gamma, P, \Gamma')$, and so on.

A **judgement** is a pair of contexts which we write $\Gamma \vdash \Delta$. When a context is on the right-hand side of a judgement we call it a **cocontext**. The **valid** or **derivable** judgements are inductively defined by:

$$\frac{}{\Gamma, P \vdash P, \Delta}\,(\mathbf{Ax}) \qquad \frac{\Gamma, P \vdash Q, \Delta}{\Gamma \vdash P \subset Q, \Delta}\,(\subset \mathbf{R}) \qquad \frac{\Gamma \vdash P, \Delta \quad \Gamma, Q \vdash \Delta}{\Gamma, P \subset Q \vdash \Delta}\,(\subset \mathbf{L}) \qquad \frac{}{\Gamma, \bot \vdash \Delta}\,(\bot\mathbf{L})$$

$$\frac{\Gamma \vdash P, \Delta \quad [a \notin \Gamma, \Delta]}{\Gamma \vdash \forall a.P, \Delta}\,(\forall \mathbf{R}) \qquad \frac{\Gamma, P[a{:=}t] \vdash \Delta}{\Gamma, \forall a.P \vdash \Delta}\,(\forall \mathbf{L}) \qquad \frac{\Gamma \vdash P, \Delta \quad \Gamma, P \vdash \Delta}{\Gamma \vdash \Delta}\,(\mathbf{Cut})$$

$$\frac{[t \text{ not a variable}]}{\Gamma, \mathbf{at}\, t \vdash \Delta}\,(\mathbf{at}\,\mathbf{L}) \qquad \frac{\Gamma, \mathbf{at}\, a \vdash \Delta \quad [a \notin \Gamma, \Delta]}{\Gamma \vdash \Delta}\,(\mathbf{Fresh}) \qquad \frac{}{\Gamma \vdash t \rightsquigarrow t, \Delta}\,(\rightsquigarrow\mathbf{R})$$

$$\frac{\Gamma, p(ts)[a{:=}s] \vdash \Delta \quad [a{\downarrow}p(ts)]}{\Gamma, s' \rightsquigarrow s, p(ts)[a{:=}s'] \vdash \Delta}\,(\rightsquigarrow\mathbf{L}{\downarrow}) \qquad \frac{\Gamma, p(ts)[a{:=}s'] \vdash \Delta \quad [a{\uparrow}p(ts)]}{\Gamma, s' \rightsquigarrow s, p(ts)[a{:=}s] \vdash \Delta}\,(\rightsquigarrow\mathbf{L}{\uparrow})$$

Here a and b are variables; bound by $\forall$, free otherwise (in particular the a in (**Fresh**) *must* be a variable symbol). Formulae in square brackets are *side-conditions* whose satisfaction can be decided just by examining syntax. (**Fresh**) is not mistyped. It *eliminates* a variable.

Call $\Gamma \vdash \Delta$ a **theorem** when a derivation exists concluding in $\Gamma \vdash \Delta$. Note that Δ may be empty in which case we write $\Gamma \vdash$ and we may call Γ **contradictory** when $\Gamma \vdash$ is derivable.

A subset of the logic is classical predicate logic, so we use standard sugar such as writing $\neg P$ for $P \subset \bot$, $P \vee Q$ for $(\neg P) \subset Q$, $\exists a.P$ for $\neg(\forall a.\neg P)$, $P \wedge Q$ for $\neg(P \subset (\neg Q))$, and in general we shall use other well-known shorthands for classical equivalences.

2.3 Comments on atoms

(**Fresh**) states that atoms exist, and (**at L**) states that we cannot indirectly describe them.

(**at L**) is like (**⊥L**); 'if t is a term then it is not a variable'. $\mathbf{at}\,\langle a, a\rangle \vdash$ is a theorem. (We assume a pair term-former $\langle \text{-}, \text{-}\rangle$.) (**at R**) is like (**⊤L**); 'it is true that we can always generate a fresh variable symbol'. There is no rule $\Gamma \vdash \mathbf{at}\,a, \Delta$. It would break Lemma 4.1 (the substitution lemma) which underlies the essential case of $\forall$ and which formalises the intuition 'variable symbols represent unknown terms'.

We cannot conclude $\mathbf{at}\,a$ just because a *is* a variable symbol! $\mathbf{at}\,a$ in Γ represents a promise that a will never be instantiated to a (non-variable) term. This explains (**at L**) since there the promise has manifestly been broken, and also (**Fresh**) since there a is fresh from Γ and thus immune to any nasty things Γ may do to it (such as contain $a = 0$). We may call a variable symbol a of which we know $\mathbf{at}\,a$ an **atom**.

Define $a{\uparrow}P$, $a{\downarrow}P$, and $a{\circlearrowleft}P$ by:

$$\frac{a{\circlearrowleft}P}{a{\uparrow}P} \quad \frac{a{\circlearrowleft}P}{a{\downarrow}P} \quad \frac{a{\uparrow}P \;\; a{\downarrow}Q}{a{\downarrow}(P{\subset}Q)} \quad \frac{a{\downarrow}P \;\; a{\uparrow}Q}{a{\uparrow}(P{\subset}Q)} \quad \frac{a{\circlearrowleft}P \;\; a{\circlearrowleft}Q}{a{\circlearrowleft}(P{\subset}Q)} \quad \frac{a{\uparrow}P}{a{\uparrow}\forall a.P} \quad \frac{a{\downarrow}P}{a{\downarrow}\forall a.P} \quad \frac{a{\circlearrowleft}P}{a{\circlearrowleft}\forall a.P}$$

It is not hard to prove $\rightsquigarrow$ is transitive and reflexive; $s \rightsquigarrow t,\ t \rightsquigarrow u \vdash s \rightsquigarrow u$ and $\vdash s \rightsquigarrow s$ are derivable. $\rightsquigarrow$ behaves like the transitive reflexive closure of a reduction/rewrite relation. By Theorem 2.1 below if $a{\downarrow}P$ then P satisfies 'subject reduction'; if $P[a{:=}s]$ then $P[a{:=}t]$. But then in $P \subset Q$ (where Q does not mention a) the direction of the 'subject reduction' is reversed, thus we need $a{\uparrow}(P \subset Q)$. Similar issues arise with $\rightsquigarrow$ itself. For example if $(a \rightsquigarrow t)[a{:=}s]$ and $s' \rightsquigarrow s$ then $(a \rightsquigarrow t)[a{:=}s']$. This is why $\rightsquigarrow$ has arity $(\uparrow, \downarrow)$.

Theorem 2.1 *The following two rules are derivable:*

$$\frac{\Gamma, P[a{:=}s] \vdash \Delta \quad (a{\downarrow}P)}{\Gamma, s' \rightsquigarrow s, P[a{:=}s'] \vdash \Delta}\,(\rightsquigarrow\mathbf{L}{\downarrow}) \qquad \frac{\Gamma, P[a{:=}s'] \vdash \Delta \quad (a{\uparrow}P)}{\Gamma, s' \rightsquigarrow s, P[a{:=}s] \vdash \Delta}\,(\rightsquigarrow\mathbf{L}{\uparrow})$$

Proof. It is not hard to check, by induction on P, that directions propagate pointwise but get flipped if they are in negative position (the left-hand side of an implication) and that $a{\circlearrowleft}P$ precisely when $a{\uparrow}P$ and $a{\downarrow}P$. For example, suppose P is

$(Q \subset R)[a{:=}s]$, $(a{\downarrow}Q \subset R)$ and that $\Gamma, (Q \subset R)[a{:=}s] \vdash \Delta$. This can be only if we have $a{\uparrow}Q$ and $a{\downarrow}R$, therefore we can deduce

$$\dfrac{\dfrac{\dfrac{}{\Gamma, Q[a{:=}s'] \vdash Q[a{:=}s'], \Delta}\,(\mathbf{Ax})}{\Gamma, s' \rightsquigarrow s, Q[a{:=}s] \vdash Q[a{:=}s'], \Delta}\,(\mathbf{Ind.Hyp.}) \qquad \dfrac{\dfrac{}{\Gamma, R[a{:=}s] \vdash R[a{:=}s], \Delta}\,(\mathbf{Ax})}{\Gamma, s' \rightsquigarrow s, R[a{:=}s'] \vdash R[a{:=}s], \Delta}\,(\mathbf{Ind.Hyp.})}{\Gamma, s' \rightsquigarrow s, (Q \subset R)[a{:=}s'], Q[a{:=}s] \vdash R[a{:=}s], \Delta}\,(\subset \mathbf{L})}{\Gamma, s' \rightsquigarrow s, (Q \subset R)[a{:=}s'] \vdash (Q \subset R)[a{:=}s], \Delta}\,(\subset \mathbf{R})$$

and then we can derive $\Gamma, P[a{:=}s'] \vdash \Delta$ from our assumption and (**Cut**). □

If we make sure that $a{\circlearrowleft}P$ then $\rightsquigarrow$ behaves just like an equality (at least for that a). We can also encode as $s \rightsquigarrow t \wedge t \rightsquigarrow s$, as we now prove. The standard deduction rules for equality are:

$$\dfrac{}{\Gamma \vdash t = t, \Delta}\,(=\mathbf{R}) \qquad \dfrac{\Gamma, P[a{:=}s'] \vdash \Delta}{\Gamma, s' = s, P[a{:=}s] \vdash \Delta}\,(=\mathbf{L})$$

Theorem 2.2 *Write $s = t$ in a-logic as **shorthand for** $s \rightsquigarrow t \wedge t \rightsquigarrow s$. (=**R**) and (=**L**) are derivable in a-logic.*

Proof. If neither $a{\uparrow}p(ts)$ nor $a{\downarrow}p(ts)$ we rewrite $p(ts)$ with two variables a' and a'', one occurring exclusively up in $p(ts)$, the other occurring exclusively down. It is then easy to verify that with one use each of ($\rightsquigarrow$ **L**$\uparrow$) and ($\rightsquigarrow$ **L**$\downarrow$) we can obtain the effect of (=**L**). (=**R**) is easy. □

So a-logic combines elements of computation, given by $\rightsquigarrow$, of logic, given by the evident logical apparatus, and of something rather strange, given by **at** .

3 Expressivity

3.1 Substitution

We suppose a ternary term-former written $s\langle u{\mapsto}t\rangle$. It is valid syntax whether or not u is a variable symbol (if **at** u is not provable we can prove nothing about it). We assume a binary predicate $\# : (\downarrow, \downarrow)$ **freshness** [2] and a set of axioms given by:

$$a\#s \Leftrightarrow \mathbf{at}\, a \;\wedge\; \forall t.s\langle a{\mapsto}t\rangle \rightsquigarrow s \qquad \mathbf{at}\, a \subset s\langle a{\mapsto}a\rangle \rightsquigarrow s$$
$$\mathbf{at}\, a \subset a\langle a{\mapsto}s\rangle \rightsquigarrow s \qquad \mathbf{at}\, a \wedge b\#s \subset s\langle a{\mapsto}b\rangle\langle b{\mapsto}t\rangle = s\langle a{\mapsto}t\rangle$$
$$\mathbf{at}\, b \wedge a\#b \wedge a\#v \subset s\langle a{\mapsto}u\rangle\langle b{\mapsto}v\rangle = s\langle b{\mapsto}v\rangle\langle a{\mapsto}u\langle b{\mapsto}v\rangle\rangle.$$

(Note that $a\#s$ implies **at** a.) As is standard in axioms, the variables are universally quantified, e.g. the top-right axiom is actually $\forall a, s.(\mathbf{at}\, a \subset s\langle a{\mapsto}a\rangle \rightsquigarrow s)$. **Call this theory** Σ, and for simplicity call the set of axioms Σ as well. [3]

[2] $a\#s$ means a is fresh for the term s in the sense that it is a variable that is new, or 'fresh', to s, i.e. it is not free in s.

[3] Other axiomatisations of substitution exist though we are aware of no authoritative source. Two recent examples are Crabbé [10, p.2] and Salibra [22, p.6]. Crabbé assumes a function symbol which we can identify with #; Salibra gives a more algebraic axiomatisation but the domain *must* be a model of the λ-calculus so he expresses $a\#s$ (a is not free in s) by writing s as $(\lambda x.s)a$. Salibra's and Crabbé's axiomatisations are

The explicit substitution is not identical to substitution on terms. For example $x[y{:=}z] \equiv x$ (as the symbol y does not occur in x to substitute syntactically for z) but $\Gamma \vdash x\langle y{\mapsto}z\rangle = x$ need not be derivable; for example $x = y \vdash x\langle y{\mapsto}z\rangle \rightsquigarrow z$ is easily derivable with the axioms above.

We intend $\#$ to be a notion of 'not free in', and $a\#s$ can be read as 'a is not free in s'. $\Gamma, \mathbf{at}\, a \vdash a\#b$ need not be derivable; for example if $\Gamma \equiv \{a = b\}$.

We can now give an answer to the question "why $\rightsquigarrow$ not $=$?". Assume a context Σ' with the axioms above except the second axiom is $\mathbf{at}\, a \subset a\langle a{\mapsto}s\rangle = s$. Then $\Sigma', \mathbf{at}\, a \vdash a\langle a{\mapsto}a\rangle = a$ is derivable. Using (**Cut**) and (=**L**) also $\Sigma', \mathbf{at}\, a \vdash \mathbf{at}\,(a\langle a{\mapsto}a\rangle)$ is derivable and using (**at L**) and another cut we obtain $\Sigma', \mathbf{at}\, a \vdash$. Finally by (**Fresh**) we derive $\Sigma' \vdash$.

3.2 λ-calculus

Assume the theory of substitution just given. We now construct $\mathcal{L}$ an axiomatisation of the λ-calculus (we also overload $\mathcal{L}$ for the set of axioms below). Fix binary term-formers **application** and **lambda** and write them $s \cdot t$ and $\lambda s.t$. Assume axioms[4]

$$\mathbf{at}\, a \wedge b\#s \subset \lambda a.s = \lambda b.s\langle a{\mapsto}b\rangle \qquad \mathbf{at}\, a \subset (\lambda a.s){\cdot}t \rightsquigarrow s\langle a{\mapsto}t\rangle.$$

Extensionality can be expressed by a third axiom $\mathbf{at}\, b \subset (\lambda b.s){\cdot}b = s$.

This axiomatisation differs from a standard one [23] in that s and t above are *variables*. We have just two (or three with extensionality) axioms, and *not* axiom-schemes for all terms s and t.

Notions of 'model' of the λ-calculus [4] often only give semantics to closed terms. The axioms above invite interpretations of open terms. The most suitable match in the literature for the above are Salibra's Lambda Abstraction Algebras (LAA) [17]; we have not checked whether the axioms above are provably equivalent in a-logic to the axioms of LAA. We do give models in §6.

3.3 Further axioms, including unification and logic-programming

Our semantics will demonstrate that the intuitions above make *some* formal sense. If the reader can suspend disbelief until then, we would like to show some more interesting predicates we can write in a-logic. Assume the theory Σ.

- Introduce a unary predicate symbol $\Box$ with axiom $\Box s \Leftrightarrow \forall a.\mathbf{at}\, a \subset a\#s$. This means '$s$ is closed'.
- Similarly we can express 'b is the only free atom of s' by $\forall a.\mathbf{at}\, a \wedge a \neq b \subset a\#s$, and 'the free atoms of s are precisely b' by $(\neg b\#s) \wedge \forall a.\mathbf{at}\, a \wedge a \neq b \subset a\#s$.
- Assume term-formers for pairs and lists and use standard notations (s,t), **Nil**,

almost equivalent. (The two authors do not cite one another but the interested reader can note that $(\lambda x.s)t$ in Salibra corresponds to $s[a{:=}t]$ in Crabbé, and Crabbé then kindly does the hard work in his Proposition 3.1. We must remove Salibra's axiom β_6, which is β-reduction.)

[4] A slightly weaker alternative is $\forall t.\exists a, t'.\mathbf{at}\, a \wedge \forall s.t'\langle a{\mapsto}s\rangle = t{\cdot}s$ and $\forall t, a.\mathbf{at}\, a \subset \exists t'.\forall s.t\langle a{\mapsto}s\rangle = t'{\cdot}s$, which asserts that a term with the properties of $\lambda a.s$ exists but does not pick a canonical one.

and $hd :: tl$. Introduce unary predicates *isatomicsub* and *issub* with axioms

$$isatomicsub(u) \Leftrightarrow \exists a, b.(u = (a, b) \wedge \mathbf{at}\, a)$$
$$issub(u) \Leftrightarrow u = \mathbf{Nil} \vee \exists hd, tl.(isatomicsub(hd) \wedge issub(tl) \wedge u = hd :: tl).$$

Write σ for terms such that $issub(\sigma)$ is provable. Introduce a binary term-former $applyatomicsub(s, t)$ with axiom $\mathbf{at}\, a \subset applyatomicsub(s, (a, u)) = s\langle a{\mapsto}u\rangle$, and another binary term-former $applysub(s, t)$ with axioms which apply the elements of t as a list. We can express a notion of extensional equality on substitutions by $Eq(\sigma, \sigma') \Leftrightarrow \forall a.applysub(a, \sigma) = applysub(a, \sigma')$ (since a-logic is untyped, we may wish in practise to restrict equality to some subset of all terms). We omit further details, but we hope it is now evident that we can express standard features of the theory of unification such as '$\sigma \geq \sigma'$', 'σ is a (most general) unifier of s and t', 's is a substitution instance of t', and so on.

- The discussion above extends to predicates in a slightly weak sense. We can write $a\#P$ to mean $\forall u.P\langle a{\mapsto}u\rangle \Leftrightarrow P$ where here $P\langle a{\mapsto}u\rangle$ is that predicate obtained by replacing every term s in P by $s\langle a{\mapsto}u\rangle$. We can express $\Box P$ as $\forall a.\mathbf{at}\, a \subset a\#P$, and so on.

The first author was instrumental in developing so-called Nominal techniques, including Nominal Logic [21,13], which have similar applications to a-logic and which have a freshness predicate $\#$. One problem with them is that they are inconsistent with the axiom of choice, so for example Nominal Logic is inconsistent with a unary term-former f such that $(fs)\#s$ always. It seems that we *can* axiomatise such an f in a-logic with Σ, as $\forall s.\exists b.fs \rightsquigarrow b \wedge b\#s$. We cannot explore this in more detail here but we note the use of $\rightsquigarrow$ is what helps avoid contradictions. This might possibly be relevant to the applications for which Nominal Logic was designed, possibly in a weaker theory than Σ designed to axiomatise Nominal Sets [16].

4 Cut-elimination

Lemma 4.1 (Substitution Lemma) *If* $\Gamma \vdash \Delta$ *has a derivation* Π*, then* $\Gamma[a{:=}s] \vdash \Delta[a{:=}s]$ *has a derivation which we write* $\Pi[a{:=}s]$.

Proof. By standard induction on derivations, and not hard. If t is not a variable in $\Gamma, \mathbf{at}\, t \vdash \Delta$ then it is not one in $\Gamma[a{:=}s], \mathbf{at}\, t[a{:=}s] \vdash \Delta[a{:=}s]$ so substitution instances of instances of (**at L**) are still instances of (**at L**). If $\Gamma \vdash \Delta$ is derivable with a derivation concluding in (**Fresh**) for a, then so is $\Gamma[a{:=}s] \vdash \Delta[a{:=}s]$; we just rename a in the derivation to some other a'; this is possible because of the side-condition on (**Fresh**).[5] The rules for $\rightsquigarrow$ may be non-standard, but it is not hard to verify that substitution in a valid instance of a rule is still a valid instance, just as we would do for the usual equality rules. □

Lemma 4.2 (Weakening) *If* $\Gamma \vdash \Delta$ *then* $\Gamma, \Gamma' \vdash \Delta, \Delta'$.

[5] For the reader worried about renaming variable symbols in proofs; we can retain the inductive hypothesis for the renamed derivation using Fraenkel-Mostowski equivariance [16].

Proof. By induction on the derivation of $\Gamma \vdash \Delta$. We may need to rename variables generated by (**Fresh**), or ($\forall$**R**) to avoid clashes with variables in Γ' or Δ'. For example if the derivation ends with (()Ax) then then the weakened conclusion is itself derivable by (**Ax**), if the derivation ends with ($\forall$**R**):

$$\dfrac{\begin{array}{c}\Pi\\ \vdots\\ \Gamma \vdash P, \Delta\end{array}}{\Gamma \vdash \forall a.P, \Delta}\,(\forall\mathbf{R})^{(*)} \qquad \text{...then by the induction hypothesis we have...} \qquad \begin{array}{c}\Pi[a{:=}c]\\ \vdots\\ \Gamma, \Gamma' \vdash P[a{:=}c], \Delta, \Delta'\end{array}$$

where $\Pi[a{:=}c]$ is obtained (using the Substitution Lemma) by replacing a in Π by some atom c that does not occur in Γ, Γ', Δ or Δ' (we know that $\Pi[a{:=}c]$ really is a deduction because a is not free in Γ or Δ). From this we can derive $\Gamma, \Gamma' \vdash \forall a.P, \Delta, \Delta'$ by an application of ($\forall$**R**) (remember from §2.1 that we equate predicates up to α-equivalence).

□

Theorem 4.3 (**Cut**) *is an admissible rule in the system without it.*

Proof. By induction on the depth of the earliest cut, counting from the leaves of the proof down to the conclusion. The proof is standard for first-order logic with equality and uses weakening, and the substitution lemma for the essential case of $\forall$ (in a completely standard way).

(**Fresh**) and (**at L**) have no essential case. Cuts are commuted in the usual way and (**Fresh**) and (**at L**) commutate with the other rules in the cases necessary to bring cuts together to form essential cases. For example

$$\dfrac{\begin{array}{c}\Pi_1\\ \vdots\\ \Gamma \vdash P, \Delta\end{array} \qquad \dfrac{\begin{array}{c}\Pi_2\\ \vdots\\ \mathbf{at}\, a, \Gamma, P \vdash \Delta\end{array}}{\Gamma, P \vdash \Delta}\,(\mathbf{Fresh})}{\Gamma \vdash \Delta}\,(\mathbf{Cut}) \quad \Longrightarrow \quad \dfrac{\dfrac{\begin{array}{c}\Pi_1'\\ \vdots\\ \mathbf{at}\, a, \Gamma \vdash P, \Delta\end{array} \qquad \begin{array}{c}\Pi_2\\ \vdots\\ \mathbf{at}\, a, \Gamma, P \vdash \Delta\end{array}}{\mathbf{at}\, a, \Gamma \vdash \Delta}\,(\mathbf{Cut})}{\Gamma \vdash \Delta}\,(\mathbf{Fresh})$$

Here we use Weakening (proved above) to extend Π_1 to Π_1' by adding $\mathbf{at}\, a$ to the premise of every sequent in Π_1. Since $a \notin \Gamma, \Delta, P$ we can rename all terms in Π_1 so that a does not occur at all in it, and then we can add $\mathbf{at}\, a$ without invalidating any rule applications in Π. The essential cases for $\rightsquigarrow$ are just like those for equality. [6] □

[6] It is worth stating explicitly one of the permutation cases for equality. Suppose $a{\downarrow}p(ts)$ and we have

$$\dfrac{\begin{array}{c}\Pi\\ \vdots\\ \Gamma \vdash p(ts)[a{:=}t'], \Delta\end{array} \qquad \dfrac{\begin{array}{c}\Pi'\\ \vdots\\ \Gamma, p(ts)[a{:=}t] \vdash \Delta\end{array}}{\Gamma, t' \rightsquigarrow t, p(ts)[a{:=}t'] \vdash \Delta}\,(\rightsquigarrow\mathbf{L}{\downarrow})}{\Gamma, t' \rightsquigarrow t \vdash \Delta}\,(\mathbf{Cut})$$

then (by the other permutation rules) we can assume that the left premise of such a (**Cut**) where the Cut formula is of the form $p(ts)$, is actually the conclusion of (**Ax**), and so Γ is $\Gamma', p(ts)[a{:=}t']$ then we may

Recall from Theorem 2.2 that FOL and FOL proofs translate naturally into a-logic, such that $s = t$ maps to $s \rightsquigarrow t \wedge t \rightsquigarrow s$.

Corollary 4.4 *Using the natural translation, a-logic is a conservative extension of FOL.*

Proof. Suppose (the translation of) $\Gamma \vdash \Delta$ is derivable in a-logic but not in FOL.

We consider the derivation rules and reading them from top to bottom we see that only (**Fresh**) can discharge **at** from the left, for a predicate of the form $\mathbf{at}\, a$ for a a variable symbol. No rule can discharge **at** from the right. The only rule which can have $\mathbf{at}\, a$ in its focus is (**Ax**) (*not* ($\mathbf{at}\,\mathbf{L}$), which wants $\mathbf{at}\, t$ for t not a variable). But (**Ax**) introduces **at** into the sequent on the right, and no rule can discharge that. Since we assumed that **at** does not occur in Δ, any uses of (**Fresh**) can be removed and we obtain a valid FOL proof. □

5 A semantics

5.1 The definition

Write $\mathbb{B}$ for some two-element set $\{\mathtt{TT}, \mathtt{FF}\}$ which we interpret as truth values. $\mathbb{B}$ has the usual boolean implication $\subset\colon \mathbb{B} \times \mathbb{B} \to \mathbb{B}$.

Call a function $[\![\text{-}]\!]$ from predicates to $\mathbb{B}$ a **semantics** when:

(i) $[\![\mathbf{false}]\!] = \mathtt{FF}$.

(ii) $[\![P \subset Q]\!] = [\![P]\!] \subset [\![Q]\!]$.

(iii) $[\![\forall a.P]\!] = \mathtt{FF}$ if $[\![P[a{:=}t]]\!] = \mathtt{FF}$ for some t. Otherwise $[\![\forall a.P]\!] = \mathtt{TT}$.

(iv) $[\![\mathbf{at}\, t]\!] = \mathtt{FF}$ if t is not a variable symbol. $[\![\mathbf{at}\, a]\!] = \mathtt{TT}$ for infinitely many variables but *not* necessarily for all.

(v) $s[\![\rightsquigarrow]\!]s = \mathbf{true}$.

(vi) Suppose that $(s[\![\rightsquigarrow]\!]t) = \mathtt{TT}$. Then: if $a{\downarrow}p(ts)$ and $[\![p(ts)[a{:=}s]]\!] = \mathtt{TT}$ then $[\![p(ts)[a{:=}t]]\!] = \mathtt{TT}$, and if $a{\uparrow}p(ts)$ and $[\![p(ts)[a{:=}t]]\!] = \mathtt{TT}$ then $[\![p(ts)[a{:=}s]]\!] = \mathtt{TT}$.

We interpret a judgement $\Gamma \vdash \Delta$ by $[\![\Gamma]\!] \leq [\![\Delta]\!]$, where $\mathtt{FF} \leq \mathtt{FF}$, $\mathtt{FF} \leq \mathtt{TT}$, $\mathtt{TT} \leq \mathtt{TT}$, and if $\Gamma = [G_1, \ldots]$ and $\Delta = [D_1, \ldots]$ then $[\![\Gamma]\!] = [\![G_1]\!] \wedge \ldots \wedge [\![G_m]\!]$ and $[\![\Delta]\!] = [\![D_1]\!] \vee \ldots \vee [\![D_n]\!]$.

This is all standard except that: $[\![\text{-}]\!]$ acts on *all* terms including open terms. We now sketch technical lemmas for Theorem 5.4. Fix some semantics $[\![\text{-}]\!]$.

---

permute this to

$$\dfrac{\dfrac{}{\Gamma', p(ts)[a{:=}t] \vdash p(ts)[a{:=}t], \Delta}\,(\mathbf{Ax}) \qquad \begin{array}{c}\Pi'\\ \vdots\\ \Gamma, p(ts)[a{:=}t] \vdash \Delta\end{array}}{\dfrac{\Gamma', p(ts)[a{:=}t] \vdash \Delta}{\Gamma, t' \rightsquigarrow t \vdash \Delta}\,(\rightsquigarrow\mathbf{L}{\downarrow})}\,(\mathbf{Cut})$$

The other case where $a{\uparrow}p(ts)$ is similar.

The proofs of the next two lemmas are unproblematic, even though we are assigning truth-values directly to open terms (without a valuation):

Lemma 5.1 (($\forall$**L**)) *If* $[\![\Gamma]\!] \wedge [\![P[a{:=}u]]\!] \leq [\![\Delta]\!]$ *then* $[\![\Gamma]\!] \wedge (\forall a.[\![P]\!]) \leq [\![\Delta]\!]$.

Lemma 5.2 (($\forall$**R**)) $[\![\Gamma]\!] \leq [\![P]\!] \vee [\![\Delta]\!]$ *and* $a \notin \Gamma, \Delta$ *imply* $[\![\Gamma]\!] \leq \forall a.[\![P]\!] \vee [\![\Delta]\!]$.

We now introduce the slightly bizzare, but useful, notion of '**free variables of a semantics**'. Suppose a is a variable symbol. Write $a \in [\![\text{-}]\!]$ when $[\![\mathbf{at}\, a]\!] = \texttt{FF}$, otherwise write $a \notin [\![\text{-}]\!]$.

Lemma 5.3 ((**at L**),(**Fresh**)) • $[\![\Gamma]\!] \wedge [\![\mathbf{at}\, u]\!] \leq [\![\Delta]\!]$ *if* u *is not a variable symbol.*
• *If* $[\![\Gamma]\!] \wedge [\![\mathbf{at}\, a]\!] \leq [\![\Delta]\!]$ *and* $a \notin \Gamma, \Delta, [\![\text{-}]\!]$ *then* $[\![\Gamma]\!] \leq [\![\Delta]\!]$.

Proof. The first part is by construction. For the second part, recall that $[\![\mathbf{at}\, a]\!] = \texttt{FF}$ is possible for some variable symbols. Nevertheless, the result follows easily reasoning by cases on whether $[\![\mathbf{at}\, a]\!] = \texttt{FF}$ and $[\![\mathbf{at}\, a]\!] = \texttt{TT}$.

□

Theorem 5.4 (Soundness) *If* $\Gamma \vdash \Delta$ *then* $[\![\Gamma]\!] \leq [\![\Delta]\!]$.

Proof. By induction on depth of derivation. Most cases are immediate from the definitions (we have sketched the reasoning in what we consider the more complex cases in lemmas above). Only rule (**Fresh**) is not straightforward.

Suppose $\Gamma, \mathbf{at}\, a \vdash \Delta$ is derivable where $a \notin \Gamma, \Delta$. We want to extend with (**Fresh**). However, what if $a \in [\![\text{-}]\!]$? We assumed that for infinitely many a' this is *not* the case. Consider a renamed derivation of a renamed conclusion $\Gamma, \mathbf{at}\, a' \vdash \Delta$ where it happens that $a' \notin [\![\text{-}]\!]$ and $a' \notin \Gamma, \Delta$. Then we can proceed using Lemma 5.3. Since a' does not appear in Γ or Δ we are done. □

5.2 Prime theories and completeness

Call a set of predicates $\mathcal{T}$ **inconsistent** when $\mathcal{T} \vdash$ is derivable, otherwise say $\mathcal{T}$ is **consistent**. Say $\mathcal{T}$ is **prime** when:

- $\mathcal{T}$ is consistent and deductively closed.
- If $P \vee Q \in \mathcal{T}$ then $P \in \mathcal{T}$ or $Q \in \mathcal{T}$.
- If $P[a{:=}s] \in \mathcal{T}$ for all s then $\forall a.P \in \mathcal{T}$.
- $\mathbf{at}\, a \in \mathcal{T}$ for infinitely many a.

Lemma 5.5 *If* $\mathcal{T}$ *is prime then for any predicate* P *either* $\mathcal{T} \vdash P$ *or* $\mathcal{T}, P \vdash$. *Also,* $\mathcal{T}, P \vdash$ *if and only if* $\mathcal{T} \vdash \neg P$.

Proof. The logic is classical so $\mathcal{T} \vdash P \vee \neg P$, both parts follow. □

Lemma 5.6 *If* $\mathcal{T}$ *is prime then* $\mathcal{T} \vdash \forall a.P$ *if and only if* $\mathcal{T} \vdash P[a{:=}u]$ *for all* u.

Proof. If $\mathcal{T} \vdash \forall a.P$ and $\mathcal{T} \vdash \neg P[a{:=}u]$ then $\mathcal{T}, P[a{:=}u] \vdash$, so by Lemma 5.5 $\mathcal{T}, \forall a.P \vdash$. Using cut $\mathcal{T} \vdash$ and we contradict consistency. The converse is direct from the definition of being prime. □

Lemma 5.7 *Any finite consistent* Γ *may be extended to a prime theory* $\mathcal{T}$.

Proof. The language is countable so we enumerate its predicates. We proceed along this list inductively extending Γ according to the following steps (from greatest priority to least priority): Suppose not $\Gamma \vdash \mathbf{at}\, a$ and not $\Gamma, \mathbf{at}\, a \vdash$. Then adjoin $\mathbf{at}\, a$ to Γ. Suppose $\Gamma \vdash P \vee Q$ but not $\Gamma \vdash P$ or $\Gamma \vdash Q$. Then adjoin P to Γ. Suppose $\Gamma \vdash \neg\forall a.P$. Then choose some variable b not in Γ and adjoin $\Gamma \vdash \neg P[a{:=}b]$. We continue indefinitely and let $\mathcal{T}$ be the deductive closure of the limit.

$\mathcal{T}$ must be consistent, for the usual reason that any derivation of a contradiction uses a finite part of $\mathcal{T}$ and each finite part is consistent.

If $\Gamma \vdash \mathbf{at}\, a$ then a occurs in Γ. Therefore our algorithm will add $\mathbf{at}\, a$ infinitely often, ensuring that the final condition for being prime is satisfied. □

Theorem 5.8 (Completeness) *If* $[\![\Gamma]\!] \leq [\![\Delta]\!]$ *for all semantics* $[\![\text{-}]\!]$, *then* $\Gamma \vdash \Delta$.

Proof. If Γ is inconsistent the result is trivial. So suppose Γ is consistent.

It suffices to construct one semantics such that $[\![\Gamma]\!] \leq [\![\Delta]\!]$ implies $\Gamma \vdash \Delta$. Build a prime theory $\mathcal{T}$ out of Γ as described above. Define a function $[\![\text{-}]\!]$ by $[\![P]\!] = \mathtt{TT}$ if $P \in \mathcal{T}$ and $[\![P]\!] = \mathtt{FF}$ if $P \notin \mathcal{T}$.

So $\Gamma \vdash \Delta$ if and only if $[\![\Delta]\!] = \mathtt{TT}$. It remains to show that $[\![\text{-}]\!]$ is a semantics. This is easy, we consider only the cases of $\forall$ and $\mathbf{at}$.

For $\forall$, we must verify that $\forall a.P \in \mathcal{T}$ if and only if $P[a{:=}t] \in \mathcal{T}$ for every t. This is Lemma 5.6.

For $\mathbf{at}$, we must verify that $[\![\mathbf{at}\, t]\!] = \mathtt{FF}$ for any t not a variable, and that $[\![\mathbf{at}\, a]\!] = \mathtt{TT}$ for infinitely many variables. This follows by construction $\mathcal{T}$. □

We now see why $[\![\mathbf{at}\, a]\!]$ need not always be $\mathtt{TT}$; a counterexample is any prime theory containing $\Gamma = \{\neg\mathbf{at}\, a\}$.

6 Consistency and nontriviality of the axiomatisation of the λ-calculus

We now exhibit semantics of the theory $\mathcal{L}$ for the untyped λ-calculus from §3.2.

6.1 λ-terms

Let Λ be the set of untyped λ-terms, generated by the grammar $q, r ::= x \mid \lambda x.q \mid qq$. We define a partial map from terms s of $\mathcal{L}$ the a-logic theory of the λ-calculus given in §3.2, to Λ, given by: $\overline{a} \equiv x_a$, $\overline{tt'} \equiv \overline{t}\,\overline{t}'$, $\overline{\lambda a.s} \equiv \lambda x_a.\overline{s}$, and $\overline{s\langle a{\mapsto}t\rangle} \equiv (\lambda x_a.\overline{s})\,\overline{t}$. Here we choose some bijection between variables a in a-logic and variables x_a in the untyped λ-terms.

The map is partial because there are too many terms s; λ is just a binary term-former in a-logic so for example $\lambda s.s$ is a valid term for any s — though the axioms from §3.2 and §3.1 do not allow us to prove much about $\lambda s.s$ unless we also know $\mathbf{at}\, s$. Say s is **proper** when $\overline{s}$ is defined.

$\mathcal{L}$ uses the minimal language of a-logic, with just three predicates: **false**, **at** and $\rightsquigarrow$. We define:

- $[\![\mathbf{false}]\!] = \mathtt{FF}$.
- $[\![\mathbf{at}\, t]\!] = \mathtt{TT}$ if t is a variable symbol (such as a or b), otherwise $[\![\mathbf{at}\, t]\!] = \mathtt{FF}$.

- $[\![s \rightsquigarrow t]\!] = \mathtt{TT}$ when both s and t are proper, and $\overline{s} \to^*_{\alpha\beta} \overline{t}$. Otherwise $[\![s \rightsquigarrow t]\!] = \mathtt{FF}$.
- $[\![P \subset Q]\!] = [\![P]\!] \subset [\![Q]\!]$ and $[\![\forall a.P]\!] = \mathtt{TT}$ if $[\![P[a{:=}t]]\!] = \mathtt{TT}$ for all t, otherwise $[\![\forall a.P]\!] = \mathtt{FF}$.

Technical lemmas for the theorem below are:

Lemma 6.1 • *If $p \to^*_{\alpha\beta} q$ and $p \equiv x$ then $q \equiv x$.*

- *If s, t and $v[a{:=}s]$ are proper, then so is $v[a{:=}t]$.*
- *If furthermore $\overline{s} \to^*_{\alpha\beta} \overline{t}$ then $\overline{v[a{:=}s]} \to^*_{\alpha\beta} \overline{v[a{:=}t]}$.*

Theorem 6.2 *$[\![\text{-}]\!]$ is a semantics, as defined in §5.1.*

Proof. Most properties are easy or direct from definitions. We verify property (6) by induction on P. We consider the two least trivial cases.

Suppose $P \equiv \mathbf{at}\, u$, and $[\![s \rightsquigarrow t]\!] = \mathtt{TT}$. If $[\![\mathbf{at}\,]\!](u[a{:=}s]) = \mathtt{TT}$ then $u[a{:=}s]$ is a variable symbol, so using part 1 of the previous lemma $u[a{:=}t]$ (is equal to $u[a{:=}s]$ and) is a variable symbol.

Suppose $P \equiv u \rightsquigarrow v$ and $a{\downarrow}P$. Suppose $[\![s \rightsquigarrow t]\!] = \mathtt{TT} = [\![u[a{:=}s] \rightsquigarrow u[a{:=}t]]\!]$. Then a occurs only in v if at all, and s, t, u, and $v[a{:=}s]$ are proper. By part 2 of the previous lemma, $v[a{:=}t]$ is also proper and by part 3, $\overline{u} \to^*_{\alpha\beta} \overline{v[a{:=}s]} \to^*_{\alpha\beta} \overline{v[a{:=}t]}$. So $[\![u \rightsquigarrow v[a{:=}t]]\!] = \mathtt{TT}$ as required. Similarly for $a{\uparrow}P$, only then a can occur only in u.

If $P \equiv Q \subset Q'$ the result follows from the inductive hypothesis. □

Theorem 6.3 *If s and t are proper then $\mathcal{L} \vdash s \rightsquigarrow t$ if and only if $[\![s \rightsquigarrow t]\!] = \mathtt{TT}$ (that is, $\overline{s} \to^*_{\alpha\beta} \overline{t}$).*

Proof. Right-to-left is by using axioms of $\mathcal{L}$ to simulate $\alpha\beta$-reduction. Left-to-right is by induction on cut-free derivations. □

As a corollary, $\mathcal{L}$ is consistent, since $x_a \not\to^*_{\alpha\beta} x_b$ so $\mathcal{L} \vdash a \rightsquigarrow b$ is not derivable.

We can also now build our compositional shallow interpretation of the λ-calculus and even prove it sound and complete. Map $p \in \Lambda$ to s in a-logic by inverting $s \mapsto \overline{s}$. That is, $(x_a)' = a$, $(\lambda x_a.p)' = \lambda(x_a)'.p'$ (when we say 'compositionality', we mean it even for λ!), and $(pq)' = p'q'$. By the theorems above $p \to^*_{\alpha\beta} q$ if and only if $[\![p' \rightsquigarrow q']\!] = \mathtt{TT}$.

6.2 λ-terms (again)

It is interesting to consider another semantics of $\mathcal{L}$ which does not interpret variables in the theory by variables in the underlying domain, in fact the underlying domain does not have any.

Let $D \cong D \to D$ be a Scott-style domain model [4]. We define a partial map from terms s of the theory $\mathcal{L}$ (see the previous section) to $\widehat{s} \in D$, for convenience we use $\overline{s}$ as an intermediate step, and we define $\widehat{s}$ to be the standard functional interpretation of $\overline{s}$ when s is proper and $\overline{s}$ is closed, and undefined otherwise. Say s is **closed and proper** when $\widehat{s}$ is defined. We define:

- $[\![\mathbf{false}]\!] = \mathtt{FF}$.
- $[\![\mathbf{at}\, t]\!] = \mathtt{TT}$ if t is a variable symbol (such as a or b), otherwise $[\![\mathbf{at}\, t]\!] = \mathtt{FF}$.

- $[\![s{\rightsquigarrow}t]\!] = \mathtt{TT}$ when both s and t are closed and proper, and $\hat{s} \rightarrow^*_{\alpha\beta} \hat{t}$. Otherwise $[\![s{\rightsquigarrow}t]\!] = \mathtt{FF}$.
- $[\![P \subset Q]\!] = [\![P]\!] \subset [\![Q]\!]$ and $[\![\forall a.P]\!] = \mathtt{TT}$ if $[\![P[a{:=}t]]\!] = \mathtt{TT}$ for all t, otherwise $[\![\forall a.P]\!] = \mathtt{FF}$.

Lemma 6.4 • *If s, t and $v[a{:=}s]$ are closed and proper, then so is $v[a{:=}t]$.*

- *If furthermore $\widehat{s} = \widehat{t}$ then $\widehat{v[a{:=}s]} = \widehat{v[a{:=}t]}$.*

Theorem 6.5 *$[\![\text{-}]\!]$ is a semantics, as defined in §5.1.*

Proof. We omit most cases. Suppose $P \equiv \mathbf{at}\, u$, and $[\![s{\rightsquigarrow}t]\!] = \mathtt{TT}$. So s and t are closed and proper, and in particular are not variable symbols. If $[\![\mathbf{at}\,(u[a{:=}s])]\!] = \mathtt{TT}$ then $u[a{:=}s]$ is a variable symbol, meaning that a does not occur in u. The result is now trivial.

Suppose $P \equiv u \rightsquigarrow v$ and $a{\downarrow}P$. Suppose $[\![s{\rightsquigarrow}t]\!] = \mathtt{TT} = [\![u[a{:=}s]{\rightsquigarrow}u[a{:=}t]]\!]$. Then a occurs only in v if at all, and s, t, u, and $v[a{:=}s]$ are closed and proper. By part 1 of the previous lemma, $v[a{:=}t]$ is also closed and proper and by part 2, $\widehat{u} = \widehat{v[a{:=}s]} = \widehat{v[a{:=}t]}$. So $[\![u{\rightsquigarrow}v[a{:=}t]]\!] = \mathtt{TT}$ as required. Similarly for $a{\uparrow}P$, only then a can occur only in a. □

Theorem 6.6 *If s and t are closed and proper then $\mathcal{L} \vdash s \rightsquigarrow t$ or $\mathcal{L} \vdash t \rightsquigarrow s$ if and only if $[\![s{\rightsquigarrow}t]\!] = \mathtt{TT}$ (that is, $\widehat{s} = \widehat{t}$).*

The proof resembles that of the similar theorem of the last subsection. Note that even though variable symbols do *not* appear in the denotation, **at** still appears to be making assertions about them, and $\mathcal{L} \vdash \exists a.\mathbf{at}\, a$ is still derivable (using (**Fresh**)).

7 Conclusions

a-logic is a classical first-order logic with a two-valued model theory, one of many [5,12]. The twist to the story is that **at** can identify variables, and accordingly truth-values are given also to open predicates, without a valuation interceding.

We think a-logic might provide a logical semantics for *isvar* in PROLOG, as used for example in practical programming in CIAO PROLOG [9]. *isvar* is a *non-logical predicate* which is true when its argument has not yet been unified with a concrete value — that is, when it is a variable. This is used exactly as we have used it in this paper, to master shallow embeddings into CIAO of languages with variables. **at** and *isvar* appear to be compatible but problems arise interpreting PROLOG conjunction, which ceases to be commutative in the presence of *isvar*. This is a known issue which we are attempting to solve by using a modified translation of CIAO conjunction which is outside the scope of this paper.

A previous paper considered a variant of a-logic with equality [14]. There, in order to axiomatise substitution we had to introduce a (in the authors' opinion) *ad hoc* notion which we called 'essentially a term'. In this paper we take a different path and introduce a *directed* equality $\rightsquigarrow$ — the *arrow* in the title of this paper. This solves the issues we encountered in [14], arguably in more elegant way which seems to benefit the resulting theory and axioms. In particular, our principal axiomatisations (of the λ-calculus and of substitution) are improved.

It is relevant how conveniently a-logic can be implemented in a theorem-prover such as Isabelle [20] (we do not know).

$\rightsquigarrow$ gives a-logic a flavour of a 'logical theory of rewriting'.[7] Rewriting itself is traditionally intensional on the syntax of terms (we look at a term's syntax to decide whether to rewrite), or completely abstract (we just take any relation on a set, but the only properties of its elements are the relation so elements are closed, in the sense that instantiation is absent) [3,25]. In some sense **at** provides just enough to give a point of contact between these two worlds and it may be possible to make this idea useful for the theory of rewriting.

Now for our application: how else can the λ-calculus (and friends) be modelled in a formal system and how does a-logic relate to them, if at all?

We can use a higher-order system to begin with (build the system as an extension of a λ-calculus). The λ-calculus cannot recognise a variable symbol as such, so to use it to model the λ-calculus requires deep embedding which model variables by closed terms (in essence, you have to write a Turing machine).

First-order logic also cannot recognise a variable symbol. We can take a two-level hierarchy with object-level variables modelled by constants. We cannot quantify over constants so a hallmark of such approaches is an infinity of axioms, one for each constant (the a-logic axiomatisations look similar, but are finite). This approach is followed for example by Cylindric Algebras [8] (which assume only finitely many variable symbols so as to obtain a finite theory), by Salibra's Lambda Abstraction Algebras [23], and by some treatments of Structural Operational Semantics (SOS) [1], see for example Bernstein's SOS axiomatisations of the π-calculus, the lazy λ-calculus, and CHOCS [6].

We can do away with variables entirely, for example using Boehm trees, Scott domains, or combinators [4]. The translations are only for closed terms (hence, not compositional) and combinators may suffer exponential blow-up [7]. This is a bit off-topic for us, because we are interested not just in a 'naked denotation' but also in the formal system it is constructed in.

We can suppose a two-level hierarchy of variables and meta-variables and assume β-reduction for the former; computation/reasoning is by instantiation of meta-variables following by reduction. One of several examples of this style is Combinatory Reduction Systems (CRS). Note that in CRS meta-variables vary over closed elements — the higher-order elements are used to feed arguments to where they are used. In a-logic unknowns may be substituted for open terms. So for example $a\#b$ states that (whatever b is) it should not depend on a. This is not expressible in a CRS. Being a variable or meta-variable is an intensional property in CRS, whereas in a-logic the difference is expressed by **at** and can be mixed with implication and the rest of the logic. Finally, in CRS β-reduction is built-in. Although we certainly concentrate on substitutions and the λ-calculus in this paper, other theories are possible so a-logic is probably more flexible.

Nominal logic [21] has a notion of atom but it is completely separate from the notion of variable; nominal logic is designed for reasoning *on* syntax — in our

[7] To our surprise, we could find no reference in the literature for a logic-based-on a transitive reflexive not-necessarily-symmetric congruence — that is, for a logic of rewriting. We would be happy to hear of such a reference. Rewriting Logic [18] seems different, informally treating terms as formulae.

terminology, reasoning on a deep embedding. A shallow embedding of (for example) the λ-calculus is not possible. We believe that a-logic is consistent with arbitrary choice. Nominal logic is not [21], so a-logic may have some interesting technical advantages.

a-logic is perhaps closer to Miller and Tiu's logic of Generic Judgements (GJ) [19], where 'being a variable symbol' is managed by scoping for a dedicated quantifier ∇ instead of a dedicated predicate **at** . GJ is significantly different in being higher-order and having a 'definitional' equality [24] suited more to logic-programming than model theory; we suspect it is unsuited for our applications.

Kit Fine's 'Reasoning with arbitrary objects' [11] axiomatises dependence between elements. Perhaps our semantics, which contain variables, may be related.

Note that our semantics is syntax-based; no denotation is given to terms. Technology to do this exists in a recent axiomatic treatment of substitution [15]. Connecting that to a-logic is future work.

References

[1] L. Aceto, W. Fokkink, and C. Verhoef. Structural operational semantics. In *Handbook of Process Algebra*. Elsevier, 1999.
[2] A. Azurat and I.S.W.B. Prasetya. A survey on embedding programming logics in a theorem prover. Technical Report 7, Institute of information and computing sciences, Utrecht University, 2002.
[3] Franz Baader and Tobias Nipkow. *Term rewriting and all that*. Cambridge University Press, Great Britain, 1998.
[4] H. P. Barendregt. *The Lambda Calculus: its Syntax and Semantics (revised ed.)*. North-Holland, 1984.
[5] John Bell and Moshé Machover. *A course in mathematical logic*. North-Holland, 1977.
[6] Karen L. Bernstein. A congruence theorem for structured operational semantics of higher-order languages. In *Logic in Computer Science*, pages 153–164, 1998.
[7] Martin W. Bunder. Some improvements to turner's algorithm for bracket abstraction. *J. Symb. Log.*, 55(2):656–669, 1990.
[8] S. Burris and H. Sankappanavar. *A Course in Universal Algebra*. Springer, 1981.
[9] CIAO. CIAO Prolog. `www.clip.dia.fi.upm.es/Software/Ciao/ciao_html/ciao_toc.html`.
[10] M. Crabbé. On the notion of substitution. *Logic Journal of the IGPL*, 12 n.2:111–124, 2004.
[11] Kit Fine. *Reasoning with Arbitrary Objects*. Blackwell, 1985.
[12] Dov Gabbay and Franz Günthner, editors. *Handbook of philosophical logic*. Number 166 in Synthese Library. D.Reidel Publishing company, 1986.
[13] Murdoch J. Gabbay and J. Cheney. A sequent calculus for nominal logic. In *Proc. 19th IEEE Symposium on Logic in Computer Science (LICS 2004)*, pages 139–148. IEEE Computer Society, 2004.
[14] Murdoch J. Gabbay and Michael J. Gabbay. a-logic. In *We Will Show Them: Essays in Honour of Dov Gabbay*, volume 1. College Publications, 2005.
[15] Murdoch J. Gabbay and Aad Mathijssen. Capture-avoiding substitution as a nominal algebra. In *ICTAC'2006: 3rd Int'l Colloquium on Theoretical Aspects of Computing*, pages 198–212, 2006.
[16] Murdoch J. Gabbay and A. M. Pitts. A new approach to abstract syntax with variable binding. *Formal Aspects of Computing*, 13(3–5):341–363, 2001.
[17] S. Lusin and A. Salibra. The lattice of lambda theories. *Journal of Logic and Computation*, 14 n.3:373–394, 2004.
[18] Narciso Martí-Oliet and José Meseguer. Rewriting logic: roadmap and bibliography. *Theor. Comput. Sci.*, 285(2):121–154, 2002.
[19] Dale Miller and Alwen Tiu. A proof theory for generic judgments: An extended abstract. In *Proc. of LICS 2003*, pages 118–127. IEEE, 2003.
[20] Larry Paulson. *The Isabelle reference manual*. Cambridge University Computer Laboratory, 2001.
[21] A. M. Pitts. Nominal logic, a first order theory of names and binding. *Information and Computation*, 186(2):165–193, 2003.
[22] Antonino Salibra. On the algebraic models of lambda calculus. *Theoretical Computer Science*, 249(1):197–240, 2000.
[23] Antonino Salibra. Lambda calculus: models and theories. In *AMiLP-2003*, number 21 in TWLT Proceedings, pages 39–54, 2003.
[24] Peter Schroeder-Heister. Definitional reflection and the completion. In *Extensions of Logic Programming*, volume 798 of *Springer Lecture Notes in Artificial Intelligence*, pages 333–347, 1993.
[25] Vincent van Oostrom. *Confluence for Abstract and Higher-Order Rewriting*. PhD thesis, Vrije Universiteit, Amsterdam, 1994.
[26] Martin Wildmoser and Tobias Nipkow. Certifying machine code safety: Shallow versus deep embedding. In *TPHOLs*, volume 3223 of *LNCS*, pages 305–320. Springer, 2004.

A Decision Procedure for Non-failure in CLP [1]

Javier Álvez[2] Paqui Lucio[3]

Lenguages y Sistemas Informáticos
Basque Country University
San Sebastian, Spain

Abstract

Non-failure analysis has several applications, especially concerning parallel logic programming and logic program transformation. However, there are few works that are dedicated to study non-failure conditions. In this paper, we present a decision procedure that determines whether a goal is non-failing on some of its variables. We provide intuitive semantics conditions that are based on the Shepherdson's operator `F`, which associate any literal to an scheme of constraints for computing its fail answers. In this way, the problem of checking non-failure is transformed into a problem of constraint satisfiability. We also present an algorithm which allows determining whether a goal cannot fail for a subset of its arguments.

Keywords: constraint solving, decision procedure, failure, constraint logic programming

1 Introduction

In logic programming, there are many works devoted to study termination conditions. However, only a few are dedicated to analyze non-failure. Non-failure analysis has several applications, especially concerning parallel logic programming and logic program transformation. In parallel systems, non-failure information is very useful for granularity control (see [9]) of parallel tasks and, in general, it helps in the control of parallelism in logic programs. Regarding to program transformation, non-failure information can be used not only to guide optimization of programs (e.g. on the basis of delayed non-failing goals), but also to provide aside conditions that guarantee the correctness of transformation rules.

As a toy example, note that the following transformation sequence (where S is any set of clauses)

$$P_1: \quad p(x_1,x_2) \leftarrow q_1(x_1,y),\ q_2(y,x_2) \qquad q_1(x_1,x_2) \leftarrow q_1(x_1,x_2) \qquad S$$

(unfold $q_1(x_1,y)$ in the first clause using the second clause)

$$P_2: \quad p(x_1,x_2) \leftarrow q_1(x_1,y),\ q_2(y,x_2) \qquad q_1(x_1,x_2) \leftarrow q_1(x_1,x_2) \qquad S$$

[1] This work has been partially supported by Spanish Project TIN2004-079250-C03-03.

[2] Email: javier.alvez@ehu.es

[3] Email: paqui.lucio@ehu.es

(fold $q_1(x_1, y)$, $q_2(y, x_2)$ in the first clause of P_2 using the first clause in P_1)

P_3: $p(x_1, x_2) \leftarrow p(x_1, x_2)$ $\quad$ $q_1(x_1, x_2) \leftarrow q_1(x_1, x_2)$ $\quad$ S

is only correct if $q_2(y, x_2)$ is non-failing on x_2.

Here, we present a decision procedure that determines if a goal is non-failing on some of its variables. Our procedure, which is based on equality constraint solving, is theoretically founded on Shepherdson operators (see [13]) and on the decidability of the considered theory. In this paper, we mainly focus on the (denoted by $\mathtt{FET}_{\mathcal{L}}$), which is decidable (see [4]), but our results can be trivially extended to any decidable theory.

To the best of our knowledge, there are only a few approaches, in the literature, to non-failure analysis (see [2,5,10]). However, there are some remarkable differences between these approaches and our proposal. On one hand, the works in [2,10] study subclasses of programs and goals that do not fail and, thus, they are not general decision methods for constraint logic programming. On the other hand, the proposal in [5] is a decision method for the whole class of constraint logic programs, but the proposed algorithm relies on a partition of clause bodies according to a given mode. Besides, the algorithm only decides non-failure of goals regarding all their variables.

Two extensively studied areas that are also related (but not very closely) to the work presented in this paper are non-termination analysis and loop checking. . In the next section, we provide some basic notation and results. In Section 3, we give a semantic characterization of non-failure on the basis of Shepherdson operators. Section 4 is devoted to describe an algorithm for deciding non-failure. Finally, we give some conclusions.

2 Preliminaries

In this section, we give a general review of all the concepts and terminology that are used in the rest of the paper.

Syntactic objects are built from a denumerable set of variables $\mathcal{X}$ and from a first order language $\mathcal{L}$ of predicate and function symbols, denoted by $\mathcal{P}_{\mathcal{L}}$ and $\mathcal{F}_{\mathcal{L}}$, such that equality ($\approx_{/2}$) is one of its predicate symbols. That is, ($\approx_{/2}$) $\in \mathcal{P}_{\mathcal{L}}$.

A is either a variable from $\mathcal{X}$ or a function symbol f of arity n from $\mathcal{F}_{\mathcal{L}}$ applied to n-tuple of terms, where $\mathcal{T}_{\mathcal{L}}(\mathcal{X})$ denotes the set of all possible terms.

An or *( )* is either , or a predicate symbol p of arity n from $\mathcal{P}_{\mathcal{L}}$ applied to n terms. Atoms/literals may be denoted by $L(t_1, \ldots, t_n)$, where an uppercase letter replaces the predicate symbol p. Further, atoms/literals are sometimes represented by uppercase letters. Besides, an atom/literal $L(t_1, \ldots, t_n)$ is said to be *fl* if $t_1, \ldots, t_n$ are pairwise distinct variables.

An *( )* is , or an arbitrary first-order formula such that equality ($\approx_{/2}$) is the only predicate symbol. As usual, first order formulas (hence, constraints) are built using the constants and , the connectives '$\neg$', '$\wedge$', '$\vee$', '$\rightarrow$', '$\leftrightarrow$' and the quantifiers '$\exists$', '$\forall$'. The expressions $\mathtt{Var}(\mathcal{O})$ and $\mathtt{Free}(\mathcal{O})$ respectively denote the variables and the free variables (excluding the quantified ones) occurring in the syntactic object $\mathcal{O}$. Besides, $c^{\exists \setminus \overline{w}}$ and $c^{\forall \setminus \overline{w}}$ are abbreviations for $\exists \overline{v}\ (c)$ and $\forall \overline{v}\ (c)$ respectively, where $\overline{v} = \mathtt{Free}(c) \setminus \overline{w}$.

Given two terms t_1 and t_2, the expression $t_1 \approx t_2$ is called an and $t_1 \not\approx t_2$ a .

The delimiters '{' and '}' denote sets of objects. Often, a '$^-$' is used to abbreviate sets: for example, the set $\{t_1, \ldots, t_n\}$ may be abbreviated by $\overline{t}$. We use some usual set relations, such as '$\in$', '$\subseteq$', '$\cap$' or '$\cup$'. By abuse of notation, tuples of objects are represented as sets and, hence, $\overline{t}$ also denotes the tuple $\langle t_1, \ldots, t_n\rangle$. Set relations are applied to tuples with the obvious meaning. Subscripts are used to denote the components of a tuple and superscripts to enumerate tuples. That is, x_i denotes the i-th component of the tuple $\overline{x}$, whereas $\overline{x}^1, \ldots, \overline{x}^j, \ldots, \overline{x}^n$ is a tuple enumeration where x_i^j denotes the i-th component of the j-th tuple. Concatenation of tuples is represented using the infix operator '$\cdot$': i.e., $\overline{x} \cdot \overline{y}$ denotes the concatenation of the tuples $\overline{x}$ and $\overline{y}$. Given two n-tuples of terms $\overline{t}^1$ and $\overline{t}^2$, the expression $\overline{t}^1 \approx \overline{t}^2$ (resp. $\overline{t}^1 \not\approx \overline{t}^2$) abbreviates the equality constraint $t_1^1 \approx t_1^2 \wedge \ldots \wedge t_1^n \approx t_2^n$ (resp. $t_1^1 \not\approx t_1^2 \vee \ldots \vee t_1^n \not\approx t_2^n$).

A is a mapping from a finite subset of variables $\mathcal{X}$, called , into a subset of $\mathcal{T}_{\mathcal{L}}(\mathcal{X})$, called . It is assumed that any substitution behaves as the identity for the variables outside its domain. Substitutions are denoted by $\{x_1 \leftarrow t_1, \ldots, x_n \leftarrow t_n\}$ (abbreviated by $\{\overline{x} \leftarrow \overline{t}\}$). A substitution σ is said to be a if it is a bijection from $\mathcal{X}$ onto $\mathcal{X}$, and the object $\mathcal{O}\sigma$ is called a of $\mathcal{O}$ if σ is a substitution/renaming. Besides, σ is sometimes interpreted as the conjunction of equations $x_1 \approx t_1 \wedge \ldots \wedge x_n \approx t_n$ (abbreviated by $\overline{x} \approx \overline{t}$), and hence $\neg\sigma$ is interpreted as the disjunction of disequations $x_1 \not\approx t_1 \vee \ldots \vee x_n \not\approx t_n$ (abbreviated by $\overline{x} \not\approx \overline{t}$). Given a set of variables $\overline{x}$, $\sigma_{|\overline{x}}$ denotes the substitution obtained from σ by restricting its domain to $\overline{x}$. The application and composition of substitutions is denoted by juxtaposition, i.e. $x_i\sigma$ serves to denote $\sigma(x_i)$ and $\alpha\beta$ denotes the composition of α and β. Any object $\mathcal{O}$ is said to be than $\mathcal{O}'$ if $\mathcal{O}'$ is an instance of $\mathcal{O}$.

A substitution σ is said to be a of the constraint c if $\mathtt{domain}(\sigma) \subseteq \mathtt{Free}(c)$ and $\mathtt{FET}_{\mathcal{L}} \models c\sigma$. A constraint c is said to be *fi* if it has at least one solution, and the constraint c is said to be - *fi* otherwise. Besides, two constraints c_1 and c_2 are said to be , denoted by $c_1 \equiv c_2$, if for every assignment σ, σ is a solution of c_1 iff σ is a solution of c_2.

The *fi* of a set of objects $\{\mathcal{O}_1, \ldots, \mathcal{O}_n\}$, denoted by $\mathtt{mgu}(\mathcal{O}_1, \ldots, \mathcal{O}_n)$, is an idempotent substitution σ such that $\mathcal{O}_i\sigma = \mathcal{O}_j\sigma$ for every $1 \leq i, j \leq n$ and for any other substitution θ with the same property, $\theta = \alpha\sigma$ holds for some substitution α. If $\mathtt{mgu}(\mathcal{O}_1, \ldots, \mathcal{O}_n)$ does (not) exist, then the objects $\mathcal{O}_1, \ldots, \mathcal{O}_n$ are said to be *(* -*)* *fi* .

A is a finite conjunction of literals $L_1 \wedge \ldots \wedge L_n$ and a constraint c. As usual in logic programming, the symbols ',' and ' $\square$ ' are used as syntactical variants of conjunction. Hence, a goal can be denoted by $L_1, \ldots, L_n \square c$ and abbreviated by $\overline{L} \square c$. Whenever the constraint c is , it is omitted and the goal $\overline{L} \square c$ is simply denoted by $\overline{L}$. A goal $\overline{L} \square c$ is said to be *fl* if every literal $L \in \overline{L}$ is flat and the literals in $\overline{L}$ are pairwise variable disjoint. Given a non-flat goal $\overline{L} \square c$, it is easy to transform it into a flat goal: assuming $\overline{L} = L_1(\overline{t}^1) \wedge \ldots \wedge L_n(\overline{t}^n)$, the corresponding restricted flat goal is $L_1(\overline{z}^1) \wedge \ldots \wedge L_n(\overline{z}^n) \square c \wedge \overline{z}^1 \approx \overline{t}^1 \wedge \ldots \wedge \overline{z}^n \approx \overline{t}^n$, where each $\overline{z}^i$ is a tuple of fresh variables and, thus, $(\overline{z}^i \cap \overline{z}^j) = \emptyset$ for every $1 \leq i \neq j \leq n$.

A is an expression of the form $H \leftarrow \overline{B} \square c$ where H (called) is an atom, $\overline{B} \square c$ (called) is a goal and $\leftarrow$ stands for the reverse implication. A clause $H \leftarrow \overline{B} \square c$ is said to be a if $\overline{B}$ exclusively consists of the literal , and then the clause is also denoted by $H \leftarrow \square c$. For any clause C, $\texttt{head}(C)$ and $\texttt{body}(C)$ respectively denote the head and the body of C. The variables that occur in the clause head of C are said to be . In the same way, the variables that only occur in the clause body are said to be .

A (or simply) is a finite sequence of clauses. Throughout this work, programs are given modulo reordering of the literals in clause bodies and standardization apart is always assumed.

Given a logic program P and any positive literal L, $\texttt{Def}_P[L]$ denotes the sequence of clauses from P such that its clause head unifies with L. We informally say that $\texttt{Def}_P[L]$ is the *fi* L or that $\texttt{Def}_P[L]$ defines L in P. If there is no clause in P such that its head unifies with L, then we assume that $\texttt{Def}_P[L] = \{L \leftarrow \quad\}$.

In the literature, we can find many refutation procedures that refines the original proposal Robinson in [12], based on the resolution inference rule. Next, we adapt the procedure described by Kowalski in [7] (called - in [1]) to constraint logic programming. A goal $(\overline{K},\ L,\ \overline{M} \square c)$ $(\overline{K},\ \overline{N},\ \overline{M} \square c \wedge d \wedge \theta)$, denoted by $(\overline{K},\ L,\ \overline{M} \square c) \Longrightarrow (\overline{K},\ \overline{N},\ \overline{M} \square c \wedge d \wedge \theta)$, if there exists a clause $(L' \leftarrow \overline{N} \square d) \in \texttt{Def}_P[L]$ and a substitution θ such that $\theta = \texttt{mgu}(L, L')$ and the constraint $c \wedge d \wedge \theta$ is satisfiable. In that resolution step, L is said to be the literal. A $(\overline{L}^0 \square c_0)$ $(\overline{L}^n \square c_n)$ is a sequence of n resolution steps $(\overline{L}^0 \square c_0) \Longrightarrow (\overline{L}_1 \square c_1) \Longrightarrow \ldots \Longrightarrow (\overline{L}^{n-1} \square c_{n-1}) \Longrightarrow (\overline{L}^n \square c_n)$. This is shortly denoted by $(\overline{L}^0 \square c_0) \Longrightarrow^* (\overline{L}^n \square c_n)$. The $\overline{L} \square c$ is a tree of nodes labeled with goals such that

- the root node is labeled with $\overline{L} \square c$,
- for each non-leaf node with goal $\overline{N} \square a$, the node has a child with label $\overline{M} \square d$ iff $(\overline{N} \square a) \Longrightarrow (\overline{M} \square d)$ for one arbitrarily selected literal.

A (or simply) of the computation tree T is a leaf with goal $\overline{N} \square a$ such that $\overline{N} =$ and a is a satisfiable constraint. On the contrary, a (or simply) is a leaf such that all its possible children are not goals because their constraint is not satisfiable. A is the root of a (sub-)tree such that all its leaves are failures. A computation tree T is said to be *fi* if all the derivations from the goal in its root node are finite, and T is said to be an *fi* computation tree otherwise. A computation tree T is said to be a - if each derivation from the goal in its root node consists of a finite number of resolution steps.

In ([3]), Clark proposed to complete the definition of predicates. The $p \in \mathcal{P}_{\mathcal{L}}$ is

$$(p(\overline{x}) \ \leftrightarrow \ \bigvee_{k=1}^{m} \exists \overline{z}^k \ (\ \overline{x} = \overline{t}^k \wedge \overline{B}^k \wedge c_k \ ) \ )^{\forall}$$

where $\texttt{Def}_P[p(\overline{x})] \ = \ \{ \ p(\overline{t}^k) \leftarrow \overline{B}^k \square c_k \mid 1 \leq k \leq m \ \}$ and $\overline{z}^k = \texttt{Var}(\overline{t}^k \cdot \overline{B}^k \cdot c_k)$ for each $1 \leq k \leq m$. If $m = 0$, then the completion formula of a predicate p is

$(p(\overline{x}) \leftrightarrow \quad)^{\forall}$. The P, denoted by $\mathtt{Comp}(P)$, consists of the completion formulas of each predicate $p \in \mathcal{P}_{\mathcal{L}}$ and the axioms of $\mathtt{FET}_{\mathcal{L}}$ interpreted in three-valued logic, as proposed in [8].

The - P is defined by (following [11])

$$\mathtt{COMP}[P, \overline{L} \square c] = \{ a \mid \mathtt{Comp}(P) \models_3 (\overline{L} \wedge c \wedge a)^{\forall} \}$$

where $\overline{L} \square c$ is a goal, c and a are general equality constraints and $\models_3$ stands for the three-valued logical consequence relation, as defined in [8].

3 A Characterization Of Non-failure Using Shepherdson Operators

In [13], Clark-Kunen completion semantics is characterized in terms of satisfaction of equality constraints. The so-called $\mathtt{F}$ provides a bottom-up scheme for computing the fail-answers of any literal. In Definition 3.1, we recall the original definition of operator $\mathtt{F}$, adapting it to our notation.

Definition 3.1 Let P be a program and L be a positive literal such that $\mathtt{Def}_P[L] = \{ H_k \leftarrow \overline{B}^k \square c_k \mid 1 \leq k \leq m \}$. The $\mathtt{F}$ is inductively defined as

$$\mathtt{F}_0^P[L] \quad =$$

$$\mathtt{F}_{n+1}^P[L] \quad = \quad \bigwedge_{k=1}^{m} \forall \overline{w}^k \; (\; \mathtt{F}_n^P[\overline{B}^k] \vee \neg c_k \vee \neg \theta_k \;)$$

where each $\overline{w}^k = \mathtt{Var}(H_k \cdot \overline{B}^k \cdot c_k)$ and $\theta_k = \mathtt{mgu}(L, H_k)$. Besides, for all $n \in \mathbb{N}$, the extension to constants and connectives is:

$$\mathtt{F}_n^P[\quad] \; = \qquad \mathtt{F}_n^P[\varphi_1 \wedge \varphi_2] \; = \; \mathtt{F}_n^P[\varphi_1] \vee \mathtt{F}_n^P[\varphi_2]$$

$$\mathtt{F}_n^P[\quad] \; = \qquad \mathtt{F}_n^P[\varphi_1 \vee \varphi_2] \; = \; \mathtt{F}_n^P[\varphi_1] \wedge \mathtt{F}_n^P[\varphi_2] \qquad \square$$

It is easy to see that $\mathtt{F}_n^P[L]$ is a constraint that represents the failures of L that can be derived from P in n resolution steps. In the next example, we show the constraints that are given by the operator $\mathtt{F}$ for a program and a literal.

Example 3.2 Let us consider the following program P:

$$p(x) \leftarrow \square \; x \approx a$$
$$p(s(s(y))) \leftarrow p(y) \; \square$$

The operator $\mathtt{F}$ yields the following constraints (for clarity, some trivial simplifications have been performed):

$$\mathtt{F}_1^P[p(x)] \; = \; (\; x \not\approx a \;) \; \wedge \; \forall y \; (\; x \not\approx s(s(y)) \;)$$

$$\mathtt{F}_2^P[p(x)] \; = \; (\; x \not\approx a \;) \; \wedge \; \forall y \; (\; x \not\approx s(s(y)) \vee \; [\; (\; y \not\approx a \;) \; \wedge \; \forall z \; (\; y \not\approx s(s(z)) \;) \;] \;)$$

$$\mathtt{F}_3^P[p(x)] \; = \; (\; x \not\approx a \;) \; \wedge \; \forall y \; (\; x \not\approx s(s(y)) \vee \; [\; (\; y \not\approx a \;) \; \wedge$$
$$\forall z \; (\; y \not\approx s(s(z)) \; \vee \; [\; (\; z \not\approx a \;) \; \wedge \; \forall w \; (\; z \not\approx s(s(w)) \;) \;] \;) \;] \;) \qquad \square$$

An easy consequence of Definition 3.1 is the following monotonicity property.

Proposition 3.3 (Monotonicity) P, L $n \in \mathbb{N}$

$$\mathtt{FET}_{\mathcal{L}} \models (\mathtt{F}^P_n[L] \rightarrow \mathtt{F}^P_{n+1}[L])^{\forall} \qquad \square$$

The characterization of goal failures under Clark-Kunen semantics given by the operator F of Shepherdson directly follows from Lemma 4.1 in [13] and Theorem 6.3 in [8].

Theorem 3.4 ([6,13]) P $\overline{L} \square c$

$$a \in \mathtt{COMP}[P, \neg(\overline{L} \square c)] \iff n \in \mathbb{N}$$
$$\mathtt{FET}_{\mathcal{L}} \models (\mathtt{F}^P_n[\overline{L}] \vee \neg c \vee \neg a) \qquad \square$$

By means of the above operator F, the non-failure nature of a goal can be easily characterized.

Definition 3.5 Given a program P, a goal $\overline{L} \square c$ is said to be - $\overline{x} \subseteq \mathtt{Var}(\overline{L} \square c)$ iff for all $n \in \mathbb{N}$

$$\mathtt{FET}_{\mathcal{L}} \models \forall \overline{x} \exists \overline{y} \, (\, \neg \mathtt{F}^P_n[\overline{L}] \wedge c \,)$$

where $\overline{y} = \mathtt{Var}(\overline{L} \square c) \setminus \overline{x}$. Otherwise, $\overline{L} \square c$ is said to be $\overline{x}$. $\square$

Note that $\forall \overline{x} \exists \overline{y} \, (\, \neg \mathtt{F}^P_n[\overline{L}] \wedge c \,) = \neg \exists \overline{x} \forall \overline{y} \, (\, \mathtt{F}^P_n[\overline{L}] \vee \neg c \,)$, hence the goal $\overline{L} \square c$ is non-failing on $\overline{x}$ iff the formula $\forall \overline{y} \, (\, \mathtt{F}^P_n[\overline{L}] \vee \neg c)$ is unsatisfiable (in $\mathtt{FET}_{\mathcal{L}}$).

Roughly speaking, if $\overline{L} \square c$ is non-failing on $\overline{x} \subseteq \mathtt{Var}(\overline{L} \square c)$, then for any substitution α of domain $\overline{x}$ there exists some substitution β of domain $\overline{y} = \mathtt{Var}(\overline{L} \square c) \setminus \overline{x}$ such that the goal $(\overline{L} \square c)\alpha\beta$ either succeeds or loops. On the contrary, if $\overline{L} \square c$ is failing on $\overline{x}$, then there exists at least one substitution α of domain $\overline{x}$ such that, for any substitution of domain $\overline{y}$, the goal $(\overline{L} \square c)\alpha\beta$ fails. As a consequence, if $\overline{L} \square c$ is non-failing on $\overline{x}$, then for any substitution θ of domain $\overline{x}$ the goal $(\overline{L} \square c)\theta$ is also non-failing on $\mathtt{Var}(\mathtt{range}(\theta))$. However, the reverse implication does not necessarily hold. That is, a goal $\overline{L} \square c$ could be failing on $\overline{x}$ and $(\overline{L} \square c)\theta$ be non-failing on $\mathtt{Var}(\mathtt{range}(\theta))$ for some substitution θ.

The next two properties can be easily proved:

(i) If a goal $\overline{L} \square c$ is non-failing on $\overline{x}$, then any goal $\overline{N} \square c$ such that $\overline{N}$ is a subset of $\overline{L}$ is non-failing on any subset of $\overline{x}$.

(ii) If a goal $\overline{L} \square c$ is failing on $\overline{x}$, then any goal $\overline{M} \square c$ such that $\overline{M}$ is a superset of $\overline{L}$ is failing on any superset of $\overline{x}$.

4 An Algorithm For Deciding Non-failure

In this section, we provide an algorithm that decides whether a goal is non-failing on some subset of its variables. Given a goal G and a subset $\overline{x}$ of the variables of G, we first construct a finite pre-computation tree, called the of G, which is denoted by $\mathtt{LT}(G)$. On the basis of this tree, we define when a node is free of failures on $\overline{x}$. Then, the goal G is non-failing on $\overline{x}$ if and only if G, which is the root of $\mathtt{LT}(G)$, is free of failures on $\overline{x}$. First, let us introduce some concepts that are

auxiliary for the construction of looping trees. Then, we explain how to construct looping trees and, finally, we prove the correctness and completeness of the decision method.

It is well-known that any constraint c can be transformed into an equivalent disjunction of constraints of some normal form $\theta \wedge d$, where θ is a substitution and d contains disequations. Accordingly, when convenient, we will assume that a constraint c is in some normal form $\theta \wedge d$ where θ is a substitution such that $\mathtt{FET}_{\mathcal{L}} \models (\ c \ \rightarrow \ \theta\ )^{\forall}$ and $\mathtt{FET}_{\mathcal{L}} \models (\ \theta \ \rightarrow \ \sigma\ )^{\forall}$ for all substitution σ such that $\mathtt{FET}_{\mathcal{L}} \models (\ c \ \rightarrow \ \sigma\ )^{\forall}$. The concrete form of d depends on the approach and is not relevant for the work presented here.

From now on, by technical convenience, we assume that every goal in a computation tree is flat. That is, any derivation step has the following form

$$(\overline{K},\ p(\overline{v}),\ \overline{M} \square \sigma \wedge c) \Longrightarrow (\overline{K},\ \overline{N}',\ \overline{M} \square \sigma \wedge \theta_1 \wedge \theta_2 \wedge c \wedge d)$$

where $p(\overline{v})$ is the selected literal, $(L' \leftarrow \overline{N} \square \sigma' \wedge d)$ is the (standardized apart) clause such that $\theta_1 = \mathtt{mgu}(p(\overline{v}), L'\sigma')$, $\overline{N}'$ is the result of substituting a fresh variable for each argument of $\overline{N}$ and θ_2 is $\overline{N}'\theta_2 = \overline{N}\sigma'$.
Note that the domain of θ_1 is $\overline{v}$ and the domain of θ_2 is $\mathtt{Var}(\overline{N}')$. It is easy to see that we can generalize this schema to derivations of several steps: that is,

$$(\overline{L} \square \sigma \wedge c) \Longrightarrow^* (\overline{L}' \square \sigma \wedge \theta_1 \wedge \theta_2 \wedge c \wedge d) \tag{1}$$

where the domain of $\sigma \wedge \theta_1$ are the variables from $\overline{L}$ and other intermediate subgoals and the domain of θ_2 are the fresh variables in $\overline{L}'$. In particular, if (θ_1^1, θ_2^1) and (θ_1^2, θ_2^2) correspond to two derivation steps in this order, but not necessarily consecutive, $w \leftarrow t \in \theta_2^1$ and $w \leftarrow r \in \theta_1^2$, then t and r should be unifiable. Hence, the conjunction of both equations is reduced to $w \leftarrow t\sigma$ where $\sigma = \mathtt{mgu}(t, r)$ and σ is applied to $\mathtt{range}(\theta_2^2)$. We usually abbreviate (1) by

$$(\overline{L} \square c') \Longrightarrow^* (\overline{L}' \square c' \wedge d')$$

where c' and c' respectively stand for $\sigma \wedge c$ and $\theta_1 \wedge \theta_2 \wedge d$.

The crucial point for constructing looping trees is to detect when a derivation for a selected literal should repeat (finitely or infinitely) some resolution steps on that literal. Next, we introduce auxiliary notions for defining the concept of looping goal.

Definition 4.1 Let θ_1, θ_2 be two substitutions of domain $v_1, \ldots, v_n$ and $w_1, \ldots, w_n$, respectively, where $\overline{z} \subseteq \mathtt{Var}(\mathtt{range}(\theta_1))$ and $\overline{z} \subseteq \mathtt{Var}(\mathtt{range}(\theta_2))$. Then, θ_1 is $\overline{z}$-in θ_2 iff $\mathtt{range}(\theta_1)$ and $\mathtt{range}(\theta_2)$ unify and for each $1 \leq i \leq n$:

$$(\mathtt{Var}(v_i\theta_1) \cap \overline{z}) \subseteq (\mathtt{Var}(w_i\theta_2) \cap \overline{z}) \qquad \square$$

Definition 4.2 Let $p(v_1, \ldots, v_n) \in \overline{L}$ and $p(w_1, \ldots, w_n) \in \overline{L}'$ be two literals in a derivation $D = (\overline{L} \square c') \Longrightarrow^* (\overline{L}' \square c' \wedge d')$ where $c' = \sigma \wedge c$ and $d' = \theta_1 \wedge \theta_2 \wedge d$. The pair (v_i, w_i) is d' iff

$$\mathtt{FET}_{\mathcal{L}} \models \forall u((d'\{v_i \leftarrow u\})^{\exists \setminus u} \rightarrow (d'\{w_i \leftarrow u\})^{\exists \setminus u}).$$

It is easy to see that, in particular, the term $v_i\theta_1$ should be more general that the term $w_i\theta_2$. $\square$

Example 4.3 Consider the clause $p(z) \leftarrow p(s(z))$. Any goal $\overline{L}, p(v), \overline{M} \square c$ resolves to $\overline{L}, p(w), \overline{M} \square c, v \approx z, w \approx s(z)$. The pair (v, w) is oriented in $c, v \approx z, w \approx s(z)$ because z is more general than $s(z)$. □

Definition 4.4 Let $\overline{L} \square a$ be a goal such that $\overline{z} = \mathtt{Var}(\overline{L})$ and $p(v_1, \ldots, v_n) \in \overline{N}$ and $p(w_1, \ldots, w_n) \in \overline{N}'$ two literals in a derivation

$$D = (\overline{L} \square a) \Longrightarrow^* (\overline{N} \square c') \Longrightarrow^* (\overline{N}' \square c' \wedge d')$$

where $c' = \sigma \wedge c$ and $d' = \theta_1 \wedge \theta_2 \wedge d$. The literal $p(w_1, \ldots, w_n)$ the literal $p(v_1, \ldots, v_n)$ with respect to $\overline{u} = \mathtt{Var}(\mathtt{range}((\theta_1\theta_2)_{|\overline{z}}))$ iff the following two conditions holds:

(i) $\theta_{1\,|\overline{v}}$ is $\overline{u}$-included in $\theta_{2\,|\overline{w}}$.

(ii) for every $1 \leq i \leq n$, if the pair (v_i, w_i) is not oriented in d', then:

$$\mathtt{FET}_{\mathcal{L}} \models \forall u \, (\, (\, (c' \wedge d')\{w_i \leftarrow u\} \,)^{\exists \backslash u} \rightarrow \, (\, (c' \wedge d')\{v_i \leftarrow u\} \,)^{\exists \backslash u} \,)$$ □

Example 4.5 Consider the clause $p(s(z)) \leftarrow p(z)$. Then, any goal $\overline{L}, p(v), \overline{M} \square$ resolves to $\overline{L}, p(w), \overline{M} \square v \approx s(z), w \approx z$. The pair (v, w) is not oriented in $v \approx s(z), w \approx z$, but $\mathtt{FET}_{\mathcal{L}} \models \forall u((\exists z(u \approx s(z)) \rightarrow \exists z(u = z))$. However, the goal $\overline{L}, p(v), \overline{M} \square v = s(a)$ resolves to $\overline{L}, p(w), \overline{M} \square v \approx s(a), w \approx a$. Then, (v, w) is not oriented in $v \approx s(a), w \approx a$ and $\mathtt{FET}_{\mathcal{L}} \not\models \forall u(u \approx s(a) \rightarrow u \approx a)$.□

Now, we introduce the notion of goal.

Definition 4.6 A flat (sub)goal $\overline{N} \square d$ in a pre-computation tree T is said to be iff every $N_j \in \overline{N}$ has some ancestor $\overline{M} \square c$ such that there exists some $M_k \in \overline{M}$ that satisfies the following conditions:

(i) M_k and N_j unify.

(ii) M_k is the selected literal in one step of the derivation from $\overline{M} \square c$ to $\overline{N} \square d$.

(iii) N_j loops into M_k. □

In order to define looping trees for flat goals, we first define a special class of minimal pre-computation trees. Depending on the literal selection rule, we can obtain different pre-computation trees for the same goal. Our results does not depend on the selection rule, since the constraints of every pre-computation tree for the same goal are logically equivalent. However, in order to provide a deterministic procedure (and for simplicity), we associate to each flat goal a unique looping tree.

Definition 4.7 Given a goal G, the pre-computation tree $\mathtt{T}(G)$ such that

(i) the selection rule is the left-most non-looping literal,

(ii) every leaf in T is either a success, a failure or a looping goal, and

(iii) T is minimal in the sense that no internal node is looping.

The $\mathtt{LT}(G)$ of a goal G results from eliminating in $\mathtt{T}(G)$ all the failure nodes. □

To guarantee that looping trees are finite, every infinite derivation must contain at least one looping goal. For this purpose, we prove that the conditions in Definition 4.6 always are achieved in a finite number of derivation steps.

Proposition 4.8 $\langle G_i \mid i \in \mathbb{N} \rangle$ $p_{/n}$ *fi* $G_i = \overline{L}, p(\overline{v}^i), \overline{M}^i \square c_i \wedge d_i$, $(i,j) \in \mathbb{N}^2$ $1 \leq k \leq n$ (v_k^i, v_k^j) (v_k^j, v_k^i) d_i

Proof. Suppose that for every pair $(i,j) \in \mathbb{N}^2$ there is some k such that neither (v_k^i, v_k^j) nor (v_k^i, v_k^j) is oriented. Then, in particular, $v_k^j\theta_2^j$ and $v_k^i\theta_2^j$ are none more general than the other. As a consequence, the sequence of clauses used in the derivation from G_i to G_j cannot be applied to G_j. Since a program is a finite set of clauses, the derivation must be finite. □

Proposition 4.9 $\langle G_i \mid i \in \mathbb{N} \rangle$ $p_{/n}$ *fi* $\overline{z}$, $G_i = \overline{L}, p(\overline{v}^i), \overline{M}^i \square \sigma_i \wedge \theta_1^i \wedge \theta_2^i \wedge c_i \wedge d_i$, $(i,j) \in \mathbb{N}^2$ $\theta_{1\,|\overline{v}^i}^j$ $\overline{z}$- $\theta_{2\,|\overline{v}^j}^j$

Proof. Suppose that $\overline{v}^i$ is an n-tuple. Let us associate each G_i to the n-tuple $T_i = (\mathtt{Var}(v_1^i\theta_2^i) \cap \overline{z}, \ldots, \mathtt{Var}(v_1^i\theta_2^i) \cap \overline{z})$ which belongs to the finite set $(2^{\overline{z}})^n$. Now, suppose that there is not a pair $(i,j) \in \mathbb{N}^2$ such that $\theta_{1\,|\overline{v}^i}^j$ is $\overline{z}$-included in $\theta_{2\,|\overline{v}^j}^j$. Then, in particular, (T_i, T_j) should be different for any $(i,j) \in \mathbb{N}^2$. That is a contradiction since $(2^{\overline{z}})^n$ is a finite set of n-tuples. □

Lemma 4.10 *fi* □

Proof. Consider any infinite derivation D under the selection rule of the left-most non-looping literal

$$\overline{L}, p_1(v_1^1, \ldots, v_{n_1}^1), \overline{M}^1 \square a_1 \Longrightarrow \ldots \Longrightarrow \overline{L}, p_i(v_1^i, \ldots, v_{n_i}^i), \overline{M}^i \square a_i \Longrightarrow \ldots$$

where, as explained above, each a_i can be decompose into $\sigma_i \wedge \theta_1^i \wedge \theta_2^i \wedge c_i \wedge d_i$. We assume that $\overline{z}$ is the set of variables in the literals in the initial goal of D. Since predicate symbols are finite, it is obvious that there is at least one predicate p such that $\{i \mid p = p_i\}$ is an infinite subset of $\mathbb{N}$. Let us consider any p satisfying that property. By Proposition 4.8, there exists at least one $(i,j) \in \mathbb{N}^2$ such that $p_i = p_j = p$ and, for each k, either (v_k^i, v_k^j) or (v_k^j, v_k^i) is oriented. The derivation steps between G_i and G_j, for one of these pairs $(i,j) \in \mathbb{N}^2$, should be infinitely repeated. Each step of this iteration makes strictly weaker the constraint on the k arguments such that (v_k^j, v_k^i) is oriented, while preserve the oriented property of the k-th arguments such that (v_k^i, v_k^j) is oriented. Hence, in a finite number of steps, namely h, for each k, either (v_k^{h-i+j}, v_k^h) is oriented or the condition (ii) of Definition 4.4 holds. Finally, by Proposition 4.9, the subderivation of D that starts in the h-th step cannot contain an infinite collection of goals such that, for all $i < j$, $\theta_{2\,|\overline{v}^i}^i$ is not $\overline{z}$-included in none of the $\theta_{2\,|\overline{v}^j}^j$. □

Next, we define when a node in a looping tree is free of failures on some subset of its variables.

Definition 4.11 Let be a goal G and $\overline{x} \subseteq \mathtt{Var}(G)$. A node $\overline{M} \square d$ of $\mathtt{LT}(G)$ is said to be (in short, fof) on $\overline{x}$ iff it satisfies one of the following two

conditions:

(i) it is a leaf

(ii) it has a non-empty set of fof children $\{\overline{N}^i \square a_i | 1 \leq i \leq m\}$ such that the constraint $(\neg a_1 \wedge \ldots \wedge \neg a_m)^{\forall \setminus \overline{x}}$ is unsatisfiable in $\mathtt{FET}_{\mathcal{L}}$. □

Next, we illustrate the notion of looping trees by means of two examples.

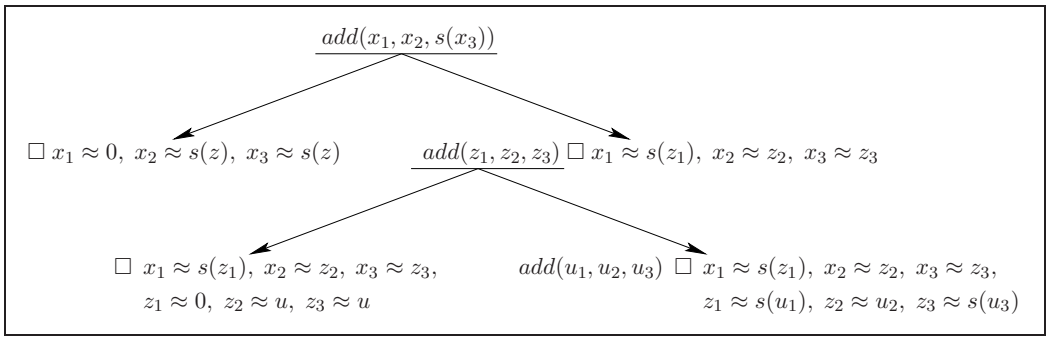


Fig. 1. Looping tree of the goal $add(x_1, x_2, s(x_3))$

Example 4.12 Let us consider the following program P:

$$add(0, z, z) \leftarrow$$
$$add(s(z_1), z_2, s(z_3)) \leftarrow add(z_1, z_2, z_3)$$

The looping tree of the goal $add(x_1, x_2, s(x_3))$ is given in Figure 1. The two left-most leaves are successes, whereas the right-most leaf is a looping goal, since it loops into its ancestor. On one hand,[4] the substitution $\{u_1 \leftarrow u_1',\ u_2 \leftarrow u_2',\ u_3 \leftarrow u_3'\}$ is $\overline{u}'$-included in $\{z_1 \leftarrow s(u_1'),\ z_2 \leftarrow u_2',\ z_3 \leftarrow s(u_3')\}$, where $\overline{u}' = \mathtt{Var}(\mathtt{range}(\{x_1 \leftarrow s(s(u_1')),\ x_2 \leftarrow u_2',\ x_3 \leftarrow s(u_3')\}))$. On the other hand, being b the constraint in that goal, the pair (u_2, z_2) is oriented in b, whereas

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ (\ b\{z_1 \leftarrow u'\}^{\exists \setminus u'}\ \rightarrow\ b\{u_1 \leftarrow u'\}^{\exists \setminus u'}\ )\ \wedge$$
$$(\ b\{z_3 \leftarrow u'\}^{\exists \setminus u'}\ \rightarrow\ b\{u_3 \leftarrow u'\}^{\exists \setminus u'}\ )\ )^{\forall}$$

Note that this goal also loops into the root goal. □

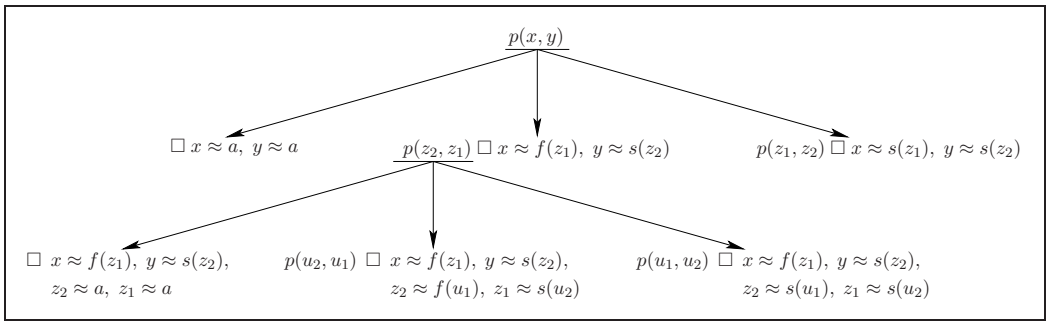


Fig. 2. Looping tree of the goal $p(x, y)$

[4] Note that the constraints in Figure 1 are given in an abbreviated way.

Example 4.13 Let us consider the following program P:

$$p(a,a) \leftarrow$$
$$p(f(z_1), s(z_2)) \leftarrow p(z_2, z_1)$$
$$p(s(z_1), s(z_2)) \leftarrow p(z_1, z_2)$$

The looping tree of the goal $p(x,y)$ is given in Figure 2. Note that the node with goal $p(z_2,z_1) \;\square\; x \approx f(z_1),\ y \approx s(z_2)$ is not a leaf since the substitution $\{z_2 \leftarrow z_2',\ z_1 \leftarrow z_1'\}$ is not $\overline{z}'$-included in $\{x \leftarrow f(z_1'),\ y \leftarrow s(z_2')\}$. □

Since looping trees are finite (see Proposition 4.10) and $\mathtt{FET}_{\mathcal{L}}$ is a decidable theory, we can use looping trees to give a decision method for the non-failure problem of goals on (some of their) variables. More precisely, we are going to show that a goal G is non-failing on some of its variables if and only if G is fof on those variables in the looping tree $\mathtt{LT}(G)$ (whose root is G). For this purpose, we need some auxiliary results about the looping goals in the leaves of $\mathtt{LT}(G)$.

Proposition 4.14 $p(\overline{w}) \;\square\; c \wedge d$ $p(\overline{v}) \;\square\; c$ $(\overline{x}, \overline{x}')$ $\overline{v} \times \overline{w}$ (v_i, w_i) d, $n \in \mathbb{N}$

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ (\mathtt{F}^P_n[p(\overline{w}) \;\square\; c \wedge d])^{\forall \setminus \overline{x}}\ \rightarrow\ (\ \neg c \vee \neg d\ )^{\forall \setminus \overline{x}}\ )^{\forall}$$

Proof. The proof is made by induction on n. The case $n = 0$ trivially holds since the operator $\mathtt{F}$ returns at the 0-iteration. For the inductive case, let $\overline{y} = \overline{v} \setminus \overline{x}$ and assume that for all $n' \leq n$:

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ (\ \forall \overline{y}\ \mathtt{F}^P_{n'}[p(\overline{w}) \;\square\; c \wedge d]\ )\ \rightarrow\ \forall \overline{y}\ (\ \neg c \vee \neg d\ )\ )^{\forall}$$

Since $p(\overline{w}) \;\square\; c \wedge d$ loops into $p(\overline{v}) \;\square\; c$, by Definition 3.1, for some $n' \leq n$

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ \mathtt{F}^P_{n+1}[p(\overline{v}) \;\square\; c]\ \rightarrow\ \forall \overline{z}\ \mathtt{F}^P_{n'}[p(\overline{w}) \;\square\; c \wedge d]\ )^{\forall}$$

where $\overline{z} = \mathtt{Var}(p(\overline{w}) \;\square\; d)$ and also

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ \mathtt{F}^P_{n+1}[p(\overline{w}) \;\square\; c \wedge d]\ \rightarrow\ \forall \overline{z}'\ \mathtt{F}^P_{n'}[p(\overline{w}') \;\square\; c \wedge d \wedge d']\ )^{\forall}$$

where $\overline{z}' = \mathtt{Var}(p(\overline{w}') \;\square\; d')$ and $d' = d\{\overline{z} \leftarrow \overline{z}'\}$. As a consequence of both facts:

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ \forall \overline{y}\ \mathtt{F}^P_{n+1}[p(\overline{w}) \;\square\; c \wedge d]\ \rightarrow\ \forall \overline{y} \cdot \overline{z}'\ \mathtt{F}^P_{n'}[p(\overline{w}') \;\square\; c \wedge d \wedge d']\ )^{\forall}$$

Besides, since $(\overline{x}, \overline{x}')$ are oriented in d, for any tuple of fresh variables $\overline{u}$

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ (\ d\{\overline{x}'' \leftarrow \overline{u}\}\ )^{\exists \setminus \overline{u}}\ \rightarrow\ (\ d\{\overline{x}' \leftarrow \overline{u}\}\ )^{\exists \setminus \overline{u}}\ )^{\forall} \tag{2}$$

where $\overline{x}'' = \overline{x}'\{\overline{z} \leftarrow \overline{z}'\}$. Then, using the induction hypothesis, we have

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ \forall \overline{y}\ \mathtt{F}^P_{n+1}[p(\overline{w}) \;\square\; c \wedge d]\ \rightarrow\ \forall \overline{y} \cdot \overline{z}'\ (\ \neg c \vee \neg d \vee \neg d'\ )\ )^{\forall}$$

and, by (2), the above formula is equivalent to:

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ \forall \overline{y}\ \mathtt{F}^P_{n+1}[p(\overline{w}) \;\square\; c \wedge d]\ \rightarrow\ \forall \overline{y}\ (\ \neg c \vee \neg d\ )\ )^{\forall}$$

□

Proposition 4.15 $p(\overline{w}) \;\square\; c \wedge d$ $p(\overline{v}) \;\square\; c$, $\overline{x}$ $\overline{v}$, $\overline{y} = \overline{v} \setminus \overline{x}$ $(\overline{x}, \overline{x}')$ $\overline{v} \times \overline{w}$ (v_i, w_i) d $n \in \mathbb{N}$,

$$\mathtt{FET}_{\mathcal{L}} \models\ (\ \forall \overline{y}\ \mathtt{F}^P_n[p(\overline{v}) \;\square\; c]\ \rightarrow\ \neg c\ )^{\forall} \tag{3}$$

$$\mathtt{FET}_{\mathcal{L}} \models \ (\ \forall \overline{y} \ \mathtt{F}^P_n[p(\overline{w}) \square c \wedge d] \ \rightarrow \ \forall \overline{y} \ (\ \neg c \vee \neg d \ ) \ )^{\forall}$$

Proof. Since $p(\overline{w}) \square c \wedge d$ is a goal that loops into $p(\overline{v}) \square c$ and $(\overline{x}, \overline{x}')$ are not oriented in d, by condition (ii) in Definition 4.4 we have for any tuple of fresh variables $\overline{u}$:

$$\mathtt{FET}_{\mathcal{L}} \models \ (\ (\ (c \wedge d)\{\overline{x} \leftarrow \overline{u}\} \ )^{\exists \setminus \overline{u}} \ \rightarrow \ (\ (c \wedge d)\{\overline{x}' \leftarrow \overline{u}\} \ )^{\exists \setminus \overline{u}} \ )^{\forall} \tag{4}$$

Besides, for all $n \in \mathbb{N}$:

$$\forall \overline{y} \ \mathtt{F}^P_n[p(\overline{w}) \square c \wedge d] \equiv \forall \overline{y} \ (\ \mathtt{F}^P_n[p(\overline{w}) \square c \wedge \theta] \vee \neg d \ )$$

where $\theta = \mathtt{mgu}(p(\overline{v}), p(\overline{w}))$. Hence, the property (3) is applicable to $\forall \overline{y} \ (\ \mathtt{F}^P_n[p(\overline{w}) \square c \wedge \theta] \ )$ by means of (4). □

Now, the main theorem can be proved

Theorem 4.16 G - $\overline{x}$ *ff* G $\overline{x}$ $\mathtt{LT}(G)$

Proof. For the right-to-left implication, we prove the following stronger result: for every non-leaf goal $\overline{M} \square d$ in $\mathtt{LT}(\overline{M} \square d)$, if $\overline{M} \square d$ is fof on $\overline{x}$ then for all $n \in \mathbb{N}$

$$(\mathtt{F}^P_n[\overline{M}] \vee \neg d)^{\forall \setminus \overline{x}} \tag{5}$$

is unsatisfiable in $\mathtt{FET}_{\mathcal{L}}$ (that is, $\overline{M} \square d$ is non-failing on $\overline{x}$). This fact is proved by induction on n. Since $\overline{M} \square d$ is not a leaf, we assume that $\overline{M} \square d$ has $m > 0$ children $\{ \ \overline{N}^i \square a_i \mid 1 \leq i \leq m \ \}$. Besides, since $\overline{M} \square d$ is fof on $\overline{x}$, by Definition 4.11 we also have that $(\ \neg a_1 \wedge \ldots \wedge a_m \ )^{\forall \setminus \overline{x}}$ is unsatisfiable.

The case $n = 0$ is based on two facts. On one hand, we know that the operator $\mathtt{F}$ returns at the first iteration. On the other hand, each a_i is equivalent to $d \wedge b_i$ for some constraint b_i. Hence, $\bigwedge_{i=1}^m \neg(d \wedge b_i) \equiv \neg d \vee \bigwedge_{i=1}^m b_i$ and $(\neg d)^{\forall \setminus \overline{x}}$ is necessarily unsatisfiable.

Assuming that the hypothesis holds for any $n' \in \mathbb{N}$ smaller than or equal to n, we prove the case $n+1$. Since each $\overline{N}^i \square a_i$ is either a leaf or it is fof on $\overline{x}$, then by Definition 3.1:

$$\mathtt{FET}_{\mathcal{L}} \models \ (\ \mathtt{F}^P_{n+1}[\overline{M} \square d] \ \rightarrow \ \bigwedge_{i=1}^{m} \mathtt{F}^P_n[\overline{N}^i \square a_i] \ )^{\forall}$$

Now, we will show that for each $1 \leq i \leq m$:

$$\mathtt{FET}_{\mathcal{L}} \models \ ((\ \mathtt{F}^P_n[\overline{N}^i \square a_i])^{\forall \setminus \overline{x}} \rightarrow \ (\neg a_i)^{\forall \setminus \overline{x}} \ )^{\forall}$$

If $\overline{N}^i \square a_i$ is a non-leaf that is fof on $\overline{x}$, then, by the induction hypothesis, the left-hand side of the implication is unsatisfiable, hence the above implication trivially holds.

Otherwise, we distinguish two cases. On one hand, if $\overline{N}^i \square a_i$ is a success leaf ($\overline{N}^i =$), then, by Definition 3.1, it is trivial that $\mathtt{F}^P_n[\overline{N}^i] \wedge a_i \equiv \neg a_i$. On the other hand, if $\overline{N}^i \square a_i$ is a looping leaf, then Propositions 4.15 and 4.14 give the required result. Note that the condition (3) in Proposition 4.15 is given by the induction hypothesis, since an unsatisfiable formula implies any constraint.

Therefore, the constraint $(\mathtt{F}^P_{n+1}[\overline{M} \,\Box\, d])^{\forall\setminus\overline{x}}$ implies $(\bigwedge_{i=1}^{m} \mathtt{F}^P_n[\overline{N}^i \,\Box\, a_i]\ )^{\forall\setminus\overline{x}}$. At the same time, this last formula also implies $(\bigwedge_{i=1}^{m} \neg a_i\ )^{\forall\setminus\overline{x}})^{\forall\setminus\overline{x}}$, which is unsatisfiable in $\mathtt{FET}_{\mathcal{L}}$. Hence, $(\mathtt{F}^P_{n+1}[\overline{M} \,\Box\, d])^{\forall\setminus\overline{x}}$ is unsatisfiable in $\mathtt{FET}_{\mathcal{L}}$.

For the left-to-right implication, first note that for any failure leaf $\overline{N} \,\Box\, c$ we know that $F_n(\overline{N} \,\Box\, c) \equiv$ for all $n > 1$. From that, by induction, we can prove that, for every failure node $\overline{N} \,\Box\, c$, there exists some $n \in \mathbb{N}$ such that $F_n(\overline{N} \,\Box\, c) \equiv$. Then, it is easy to prove that, for any non-fof node $\overline{M} \,\Box\, d$, there exists some $n \in \mathbb{N}$ such that

$$\mathtt{F}^P_n[\overline{M} \,\Box\, d]) \equiv \bigwedge_{i=1}^{m} (\neg d_i)$$

where $\{\ \overline{N}^i \,\Box\, d_i \mid 1 \leq i \leq m\ \}$ is the collection of fof (on $\overline{x}$) children of $\overline{M} \,\Box\, d$. Since $\bigwedge_{i=1}^{m} (\neg d_i)^{\forall\setminus\overline{x}}$ is satisfiable, so it is $(\mathtt{F}^P_n[\overline{M} \,\Box\, d])^{\forall\setminus\overline{x}}$. □

Now we show how our decision procedure performs with the two previous examples.

Example 4.15 (Contd.) , *fl* $add(x_1, x_2, s(x_3))$ - *ff* $\overline{x}$ $\mathcal{F}_{\mathcal{L}} = \{0_{/0},\ s_{/1}\ }$

- $add(x_1, x_2, s(x_3))$ - $\{x_3\}$

$$\forall x_1 \cdot x_2 \cdot z\ (\ x_1 \not\approx 0 \vee x_2 \not\approx z \vee x_3 \not\approx s(z)\ ) \wedge$$
$$\forall x_1 \cdot x_2 \cdot \overline{z}\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx z_2 \vee x_3 \not\approx z_3\ )$$

$\forall z(\ x_3 \not\approx s(z)\ ) \wedge \forall z_3\ (\ x_3 \not\approx z_3\ )$, *fi* $\mathtt{FET}_{\mathcal{L}}$,

$$\forall x_1 \cdot x_2 \cdot \overline{z} \cdot u\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx z_2 \vee x_3 \not\approx z_3 \vee$$
$$z_1 \not\approx 0 \vee z_2 \not\approx u \vee z_3 \not\approx u\ ) \wedge$$
$$\forall x_1 \cdot x_2 \cdot \overline{z} \cdot \overline{u}\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx z_2 \vee x_3 \not\approx z_3 \vee$$
$$z_1 \not\approx s(u_1) \vee z_2 \not\approx u_2 \vee z_3 \not\approx s(u_3)\ )$$

$\forall z_3(\ x_3 \not\approx z_3\ ) \wedge \forall u_3\ (\ x_3 \not\approx s(u_3)\ )$, *fi*

- $add(x_1, x_2, s(x_3))$ $\{x_1,\ x_3\}$

$$\forall x_2 \cdot z\ (\ x_1 \not\approx 0 \vee x_2 \not\approx s(z) \vee x_3 \not\approx s(z)\ ) \wedge$$
$$\forall x_2 \cdot \overline{z}\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx z_2 \vee x_3 \not\approx z_3\ )$$

$\forall z\ (\ x_1 \not\approx 0 \vee x_3 \not\approx s(z)\ ) \wedge \forall z_1 \cdot z_3\ (\ x_1 \not\approx s(z_1) \vee x_3 \not\approx z_3\ )$, *fi* ,

$$\forall x_2 \cdot \overline{z} \cdot u\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx z_2 \vee x_3 \not\approx z_3 \vee$$
$$z_1 \not\approx 0 \vee z_2 \not\approx u \vee z_3 \not\approx u\ ) \wedge$$
$$\forall x_2 \cdot \overline{z} \cdot \overline{u}\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx z_2 \vee x_3 \not\approx z_3 \vee$$
$$z_1 \not\approx s(u_1) \vee z_2 \not\approx u_2 \vee z_3 \not\approx s(u_3)\ )$$

$\forall u\ (\ x_1 \not\approx s(0) \vee x_3 \not\approx u\ ) \wedge \forall u_1 \cdot u_3\ (\ x_1 \not\approx s(s(u_1)) \vee x_3 \not\approx s(u_3)\ ),$
fi

, $add(x_1, x_2, s(x_3))$ $\{x_1, x_3\}$,
$add(x_1, x_2, s(x_3))$ $\{x_1, x_2, x_3\}$ □

Example 4.15 (Contd.) $\mathcal{F}_{\mathcal{L}} = \{a_{/0},\ s_{/1}, f_{/1}\}$ -
, $p(x, y)$ $\{x\}$
$p(z_2, z_1) \square x \approx f(z_1),\ y \approx s(z_2)$

$$\forall y \cdot \overline{z}\ (\ x \not\approx f(z_1) \vee y \not\approx s(z_2) \vee z_2 \not\approx a \vee z_1 \not\approx a\ ) \wedge$$
$$\forall y \cdot \overline{z} \cdot \overline{u}\ (\ x \not\approx f(z_1) \vee y \not\approx s(z_2) \vee z_2 \not\approx f(u_1) \vee z_1 \not\approx s(u_2)\ ) \wedge$$
$$\forall y \cdot \overline{z} \cdot \overline{u}\ (\ x \not\approx f(z_1) \vee y \not\approx s(z_2) \vee z_2 \not\approx s(u_1) \vee z_1 \not\approx s(u_2)\ )$$

$(\ x \not\approx f(a)\ ) \wedge \forall u_2\ (\ x \not\approx f(s(u_2))\ ) \wedge \forall u_2\ (\ x \not\approx f(s(u_2))\ ),$
$\exists v\ (\ x \approx f(f(v))\ )$ *fi*
fi $p(x, y)$,
x *fi* $\exists v\ (\ x \approx f(f(v))\ )$ $p(x, y)$ □

Finally, let us show a new example in which some decisions can be taken independently from $\mathcal{F}_{\mathcal{L}}$.

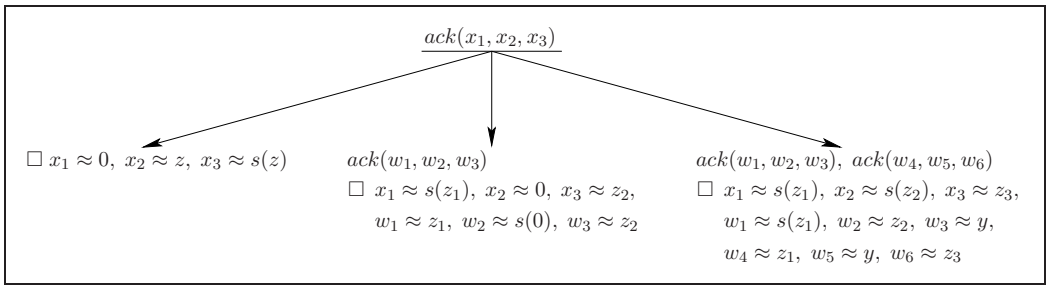


Fig. 3. Looping tree of the goal $ack(x_1, x_2, x_3)$

Example 4.17 Let us consider the following program P:

$ack(0, z, s(z))$
$ack(s(z_1), 0, z_2) \leftarrow ack(z_1, s(0), z_2)$
$ack(s(z_1), s(z_2), z_3) \leftarrow ack(s(z_1), z_2, y),\ ack(z_1, y, z_3)$

The looping tree of the goal $ack(x_1, x_2, x_3)$ is given by Figure 3. The goal $ack(x_1, x_2, x_3)$ is non-failing on x_2 independently from $\mathcal{F}_{\mathcal{L}}$ since the constraint

$$\forall x_1 \cdot x_3 \cdot z\ (\ x_1 \not\approx 0 \vee x_2 \not\approx z \vee x_3 \not\approx s(z)\ ) \wedge$$
$$\forall x_1 \cdot x_3 \cdot \overline{z}\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx 0 \vee x_3 \not\approx z_2\ ) \wedge$$
$$\forall x_1 \cdot x_3 \cdot \overline{z} \cdot y\ (\ x_1 \not\approx s(z_1) \vee x_2 \not\approx s(z_2) \vee x_3 \not\approx z_3\ )$$

is equivalent to $\forall z\ (\ x_2 \not\approx z\ ) \wedge (\ x_2 \not\approx 0\ ) \wedge \forall z_2\ (\ x_2 \not\approx s(z_2)\ )$, which is unsatisfiable. □

5 Conclusions

We have given a decision procedure for non-failure that is based on constraint solving and that does not require any additional information about the program. Moreover, in some cases, our algorithm decides independently from $\mathcal{L}$ (see Example 4.17).

In this paper, for simplicity, we have concentrated on the case of the $\mathtt{FET}_{\mathcal{L}}$ or Herbrand domain. However, our procedure and results are directly applicable to any other domain with a decidable theory. Besides, since $\mathtt{FET}_{\mathcal{L}}$ can be axiomatized for many-sorted languages, our decision algorithm also works for typed programs.

We plan to extend the presented decision procedure to constraint normal logic programs.

References

[1] Apt, K. R. and M. H. van Emden, *Contributions to the theory of logic programming*, J. ACM **29** (1982), pp. 841–862.

[2] Bossi, A. and N. Cocco, *Programs without failures*, in: N. E. Fuchs, editor, *Logic Programming Synthesis and Transformation, 7th International Workshop, LOPSTR'97, Leuven, Belgium, July 10-12, 1997, Proceedings*, Lecture Notes in Computer Science **1463** (1997), pp. 28–48.

[3] Clark, K. L., *Negation as failure*, in: H. Gallaire and J. Minker, editors, *Logic and Data Bases* (1978), pp. 293–322.

[4] Comon, H. and P. Lescanne, *Equational problems and disunification*, J. Symb. Comput. **7** (1989), pp. 371–425.

[5] Debray, S. K., P. López-García and M. V. Hermenegildo, *Non-failure analysis for logic programs*, in: L. Naish, editor, *Logic Programming, Proceedings of the Fourteenth International Conference on Logic Programming, July 8-11, 1997, Leuven, Belgium.* (1997), pp. 48–62.

[6] Fitting, M., *A kripke-kleene semantics for logic programs*, J. Log. Program. **2** (1985), pp. 295–312.

[7] Kowalski, R. A., *Predicate logic as programming language*, in: *IFIP Congress*, 1974, pp. 569–574.

[8] Kunen, K., *Negation in logic programming*, J. Log. Program. **4** (1987), pp. 289–308.

[9] Lopez, P., M. V. Hermenegildo and S. K. Debray, *A methodology for granularity-based control of parallelism in logic programs*, Journal of Symbolic Computation **21** (1996), pp. 715–734.

[10] Pedreschi, D. and S. Ruggieri, *On logic programs that do not fail*, Electr. Notes Theor. Comput. Sci. **30** (1999).

[11] Pettorossi, A. and M. Proietti, *Transformation of logic programs*, in: D. M. Gabbayand, C. J. Hogger and J. A. Robinson, editors, *Handbook of Logic in Artificial Intelligence and Logic Programming, Vol. 5*, Oxford University Press, 1998 pp. 697–787.

[12] Robinson, J. A., *A machine-oriented logic based on the resolution principle*, J. ACM **12** (1965), pp. 23–41.

[13] Shepherdson, J. C., *Language and equality theory in logic programming*, Technical Report PM-91-02, University of Bristol (1991).

Call-By-Undetermined-Value in the Pattern Matching Calculus

Towards Equational Reasoning for Functional-Logic Programming

Wolfram Kahl[⋆,⋆⋆]

Department of Computing and Software, McMaster University

Abstract

The Pattern Matching Calculus was originally shown to faithfully embed Haskell pattern matching into a confluent rewriting system.
We show an extension that captures the "lazy call-time choice" of modern non-strict functional-logic programmign languages and thus opens up new perspectives on equational reasoning about functional-logic programs.

Keywords: functional-logic programming, pattern matching calculus, equational reasoning, call-by-undetermined-value

1 Introduction

Functional-logic programming, by its commonly accepted definitions, strives to combine advantages of functional programming with those of logic programming, and populates a design space of varying vicinity not only to each of the two paradigms, but also of varying biases within those paradigms; for a survey see [7].

The functional-logic programming language Curry [8,9] (and similarly the language $\mathcal{TOY}$ [3]) consciously cultivates proximity to the non-strict purely functional programming language Haskell [13]: The concrete syntax of Curry is very close to that of Haskell; data constructor and function application are by default non-strict in the arguments, and side effects are encapsulated in the *IO* monad.

In this paper, I consciously take the view of a Haskell programmer looking at Curry, and trying to see how far the analogy really can be made to carry. The key

⋆ Email:kahl@cas.mcmaster.ca

⋆⋆This research has been supported by an NSERC Discovery Grant

aspect I am going to concentrate on is *equational reasoning*, which, for a Haskell programmer, is a standard tool of software development.

One of the features Curry can be seen as adding to functional programming is *choice*: One can write "overlapping definitions" like the following:

$$\mathit{coin} = 0$$
$$\mathit{coin} = 1$$

With this, the evaluation of the expression $\mathit{coin}+\mathit{coin}$ produces the sequence $\langle 0, 1, 1, 2\rangle$ of "solutions".

As a functional programmer, since I not only want to flip a two-sided coin twice, but also roll a six-sided *cube* twice, I will naturally tend to abstract the pattern behind the expression $\mathit{coin} + \mathit{coin}$, with the following definition:

$$\mathit{twice}\ x = x + x$$

I would then expect that, by equational reasoning, I have:

$$\mathit{twice}\ \mathit{coin} = \mathit{coin} + \mathit{coin}$$

However, evaluating *twice coin* gives me $\langle 0, 2\rangle$ as the sequence of solutions. Haskell programmers straying onto the Curry mailing list are frequently upset about this apparent violation of basic equational reasoning.

A closer look at Haskell reveals that it is also not exactly haven for equational reasoning — consider the following Haskell definition (Haskell uses ":", read "cons", for list construction from head and tail):

$$\mathit{isEmptyList}\ (x : \mathit{xs}) = \mathit{False}$$
$$\mathit{isEmptyList}\ \mathit{ys} = \mathit{True}$$

The second line of this Haskell function definition, although it looks like an equation, is not an equation valid for arbitrary *ys*, but a rule that is only used if the left-hand side of the first rule gives rise to a mismatch — here, the only value of *ys* for which this is the case is the empty list $[\,]$.

Function definitions with patterns as arguments on the left-hand sides are typical of modern functional programming languages. In non-strict languages like Haskell, Clean, or Miranda, the operational semantics of pattern matching is quite complex; usually it is formulated as the *functional rewriting strategy*, which is a priority rewriting strategy with a quite involved definition, see for example [14], or the definition of the semantics of case expressions in [13].

In [11], I introduced the first members of the "PMC" family of "pattern matching calculi", with $\mathsf{PMC}_{\oslash}$ capturing the semantics of Haskell pattern matching in a confluent rewriting system (with a machine-checked confluence proof in Isabelle), thus laying a novel foundation for equational reasoning about Haskell programs.

The main technical innovation that enabled PMC to achieve this is the separation into two syntactic categories, with generalised case alternatives, called *matchings*, equipped with their own reduction rules sufficiently separated from the reductions

of *expressions* to be able to capture the precise treatment of failure, commitment to alternatives, and limited backtracking in the functional rewriting strategy.

In the current paper I strive to demonstrate that a different kind of distinction allows to bridge the gap between equational reasoning in functional programming and the behaviour of "lazy choice" and "logical variables" in functional-logic programming languages.

In the resulting system, for which I do not yet propose a programming language syntax, we can use addition to define both "throwing *twice*" and "*dup*licating" of choices like *coin*s:

$$\begin{aligned} \mathit{coin} &= \{\!|\, \uparrow 0 \uparrow \mid \uparrow 1 \uparrow \,|\!\} \\ \mathit{twice} &= \{\!|\, \underline{x} \mapsto \uparrow x + x \uparrow \,|\!\} \\ \mathit{dup} &= \{\!|\, !x \mapsto \uparrow x + x \uparrow \,|\!\} \end{aligned}$$

With these definitions,

- *twice coin* reduces to $\{\!|\, \uparrow 0 \uparrow \mid \uparrow 1 \uparrow \mid \uparrow 1 \uparrow \mid \uparrow 2 \uparrow \,|\!\}$, and
- *dup coin* reduces to $\{\!|\, \uparrow 0 \uparrow \mid \uparrow 2 \uparrow \,|\!\}$,

making — hopefully — both the functional programmer and the functional-logic programmer happy.

In the next section I will take a semantic view on variable binding mechanisms in functional programming versus functional-logic programming, motivating the extension of PMC with *call-by-undeternined-value* (CBUV) variable bindings. Section 3 then presents the syntax of $\mathsf{PMC}_!$ and explains informally its general semantics. The rules (or equations) that do not deal with CBUV variable binding, and which are mostly inherited from previous versions of PMC, are presented in Sect. 4; those that involve CBUV variables are presented and explained in Sect. 5.

2 Semantic Considerations

Influenced by the tradition of logic programming, the semantics of functional-logic programming languages has so far normally been defined in terms of substitutions that represent "solutions for goals", see for example [6], or in terms of more operational instantiations thereof, for example the "values in modified heap" of [1].

In this paper, I will strive to follow the point-of-view of functional programming, which is essentially one of expression reduction, where "values" are a sub-class of expressions.

In strict functional programming languages, like the languages of the ML family, a value is a ground constructor term, and an expression of non-functional type evaluates to an element of the flat cpo (complete partial order) consisting of a bottom element $\bot$ representing non-termination (or run-time errors) and finite ground constructor terms.

In non-strict functional languages, constructors are non-strict ("CONS should not evaluate its arguments" [5]), giving rise to, in general, non-flat cpo's of potentially partially defined values, where each constructor argument is either $\bot$ or again a potentially partially defined value.

Functional-logic programming in Curry and $\mathcal{TOY}$ uses non-strict constructors in general, and introduces the additional twist that a single expression can evaluate to a *choice* at any level, i.e., the result of expression evaluation could be $\bot$, or it could be a choice of $\bot$ or constructor applications, and each argument of these constructors could again be such a choice.

However, functional-logic programming languages define the behaviour of logic variables and of call-time choice in terms of ground constructor terms, i.e., not involving choice, and, at least in some contexts, also not involving $\bot$.

The reason why *twice coin* in Curry evaluates to $\langle 0, 2\rangle$ is that the definition $\mathit{twice}\ x = x + x$ is applied [1] to $\mathit{coin} = 0\ ?\ 1$ using *call-time choice*.

Simply put, this can be understood as meaning that *coin* is evaluated to a semantic element involving choice, from which the two ground constructor terms ("values") 0 and 1 are extracted, and the function *twice* is applied to each of these.

In some contexts in the FLP literature, for example in [1], a "value" is defined, as in FP, to be a strong head normal form (SHNF), and therefore could be a constructor with choices as arguments — it is important to note that the call-time choice described above does not just choose an SHNF, but (at least conceptionally) chooses a single choice-free ground constructor term. Therefore, we will from now on use the word *value* only for single choice-free ground constructor terms.

So far, this appears to suggest a call-by-value approach for implementation: *First* evaluate *coin* to a choice of values, *then* apply *twice* to all resulting values. But, as mentioned above, constructors and functions in Curry and $\mathcal{TOY}$ are non-strict. For example, with the definition

$$\mathit{two}\ x = [x, x]$$

Curry evaluates $\mathit{length}\ (\mathit{two}\ \bot)$ to the single solution 2, avoiding to evaluate $\bot$ via the use of lazy evaluation, just like most Haskell implementations.

In the same way, evaluating *two coin* will return the outermost list constructor ":" in $[x, x] = x : (x : [\,])$ before the first occurrence of x forces a choice of *coin*. Since the two occurrences of x are shared, the second occurrence will use the same choice, and backtracking to the second choice will, thanks to book-keeping information, backtrack past both occurrences of x in a single step, so that *two coin* altogether evaluates to $\langle [0, 0], [1, 1]\rangle$.

Since the words "lazy" and "delayed" both have implementation connotations, I will not use them to characterise this calling convention, but use the term *call-by-undetermined-value* (CBUV) instead. The rationale for this is that this is essentially call-by-value, in that x is considered as bound to a *value* each time it is used, but this value needs not be determined at call-time.

In contrast, a *call-by-name* (CBN) variable is considered as bound to an *expression* each time it is used, and this is how the $\mathsf{PMC}_!$ function $\mathit{twice} = \{\!|\, \underline{x} \mapsto \lceil x + x \rceil \,|\!\}$, in the application *twice coin*, binds x to the expression *coin*, and then continues evaluation with the same result as *coin* + *coin*.

In the remainder of this paper I will outline the pattern matching calculus $\mathsf{PMC}_!$

[1] We follow a part of the FLP literature in using "?" as a pseudo-Curry infix operator for choice.

which essentially results from adding CBUV variables to $\mathsf{PMC}_{\oslash}$, and relaxing the rules for the PMC choice construct "|" so that it can be used for functional-logic programming.

Logic variables do not require a separate extension; as shown in [2,4], they can be replaced by (type-indexed) "generators"; for example, for the inductive type of natural numbers

$$\mathsf{data}\ \mathit{Nat} = \mathit{Zero} \mid \mathit{Succ}\ \mathit{Nat}$$

the corresponding generator, which I will write $\top_{\mathsf{Nat}}$ in PMC, is defined using recursive choice:

$$\begin{aligned}\mathit{topNat} &= \mathit{Zero}\\ \mathit{topNat} &= \mathit{Succ}\ \mathit{topNat}\end{aligned}$$

3 $\mathsf{PMC}_!$ Syntax

$\mathsf{PMC}_!$ has two major syntactic categories, *expressions* and *matchings*, defined by mutual recursion.

When considering the analogy to functional programs, only PMC *expressions* correspond to expressions of functional languages.

Matchings can be seen as a generalisation of (groups of) case alternatives. Matchings can expose patterns to be matched against arguments; we say such matchings are *waiting for argument supply*, and give them function types. Complete case expressions correspond to expressions formed from matchings that already have an argument supplied to their outermost patterns. Argument supply to patterns is separated from performing pattern matching itself; depending on the outcomes of the involved pattern matchings, saturated matchings can *succeed* and then *return* an expression, or they can *fail*.

Patterns form a separate, auxiliary syntactic category that will be used to construct pattern matchings.

Throughout this paper, we will use the following conventions for meta-level variables:

- α, α_i, β, ... are types; τ, τ_i are constructed types — these will occur only in the typing rules in this section.
- v, v_i, w_i, x, x_i, y, y_i are variables; c, d are constructors.
- p, p_1, p_2, ..., q are patterns; m, m_1, m_2, ... are matchings,
- a, b, e, e_1, e_2, ..., f are expressions,
- i, k, n are natural numbers,

Patterns are built from variables and constructor applications. Variables are bound by occurring in a pattern p inside the pattern matching construct $p \Rightarrow m$; in $\mathsf{PMC}_!$, a pattern can contain a variable x in two different ways:

- "$!x$" designates x as a *call-by-undetermined-value* (CBUV) variable;

- "$\underline{x}$" designates x as a *call-by-name* (CBN) variable, and corresponds to just "x" in previous versions of PMC.

$$\begin{array}{lll} \mathsf{Pat} ::= \underline{\mathsf{Var}} & & \text{CBN variable} \\ \quad \mid\ !\,\mathsf{Var} & & \text{CBUV variable} \\ \quad \mid\ \mathsf{Constr}(\mathsf{Pat},\ldots,\mathsf{Pat}) & & \text{constructor pattern} \end{array}$$

All variables occurring in a pattern are *free* in that pattern; for every pattern $p : \mathsf{Pat}$, we denote its set of free variables by $\mathsf{FV}(p)$. In the following, we silently restrict *all* patterns to be *linear*, i.e., not to contain more than one occurrence of any variable. The pattern typing rules are shown in Fig. 1.

$$\vdash_{\mathsf{P}} \underline{v} : \mathsf{type}\ v \qquad \frac{c \in \mathsf{Constr}_{\alpha_1\times\cdots\times\alpha_n\to\tau} \qquad \vdash_{\mathsf{P}} p_1 : \alpha_1 \quad \cdots \quad \vdash_{\mathsf{P}} p_n : \alpha_n}{\vdash_{\mathsf{P}} c(p_1,\ldots,p_n) : \tau}$$

Fig. 1. Pattern typing rules

Expressions are the syntactic category that embodies the term construction aspects:

$$\begin{array}{lll} \mathsf{Expr} ::= \mathsf{Var} & & \text{variable} \\ \quad \mid\ \mathsf{Constr}(\mathsf{Expr},\ldots,\mathsf{Expr}) & & \text{constructor application} \\ \quad \mid\ \mathsf{Expr}\ \mathsf{Expr} & & \text{function application} \\ \quad \mid\ \{\!|\ \mathsf{Match}\ |\!\} & & \text{(result) extraction} \\ \quad \mid\ \mathsf{fix}_{\mathsf{Type}} & & \text{fixed-point combinator} \\ \quad \mid\ \oslash_{\mathsf{Type}} & & \text{empty expression} \end{array}$$

Besides variables, constructor application and function application, we also have the following special kinds of expressions: Every matching m gives rise to the *(result) extraction* $\{\!|\, m \,|\!\}$. If the type of matching m is a function type, then $\{\!|\, m \,|\!\}$ extracts a function from m. If m is not a pattern matching again, then it can either succeed or fail; if it succeeds, then $\{\!|\, m \,|\!\}$ extracts the value(s) "returned" by m; otherwise, $\{\!|\, m \,|\!\}$ extracts "nothing", which can also be expressed as the expression $\oslash$, which is henceforth called the *empty expression*.

None of the expression constructors binds any variables; we overload the $\mathsf{FV}(_)$ notation and use it to denote the set of free variables $\mathsf{FV}(e)$ for an expression $e : \mathsf{Expr}$. Expressions are typed according to the rules in Fig. 2.

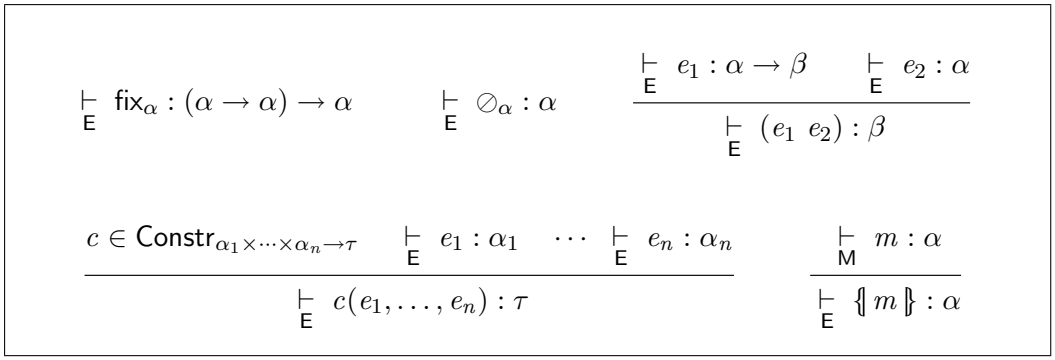


Fig. 2. Expression typing rules

Matchings are the syntactic category that embodies the pattern analysis aspects:

Match ::=	⌈Expr⌉	lifting
|	Type	failure
| Pat	⇛ Match	pattern matching
| Expr	▷ Match	argument supply
| Match	❙ Match	alternative

- For an expression e : Expr, the *lifting* or *expression embedding* ⌈Expr⌉ can be seen as the matching that always succeeds and attempts to lift the result e into the enclosing expression, so we propose to read it "*lift e*".
- Failure is the matching that always fails.
- The *pattern matching* $p \Rrightarrow m$ waits for supply of one argument more than m; this pattern matching can be understood as succeeding on instances of the (linear) pattern p : Pat and then continuing to behave as the resulting instance of the matching m : Match. It roughly corresponds to a single case alternative in languages with case expressions, or to pattern-binding λ-abstractions.
- *argument supply* $a \triangleright m$ is the matching-level incarnation of function application, with the argument on the left and the matching it is supplied to on the right. It saturates the first argument m is waiting for. "$a \triangleright m$" can be read "*a into m*" or "*a feeds m*". The inclusion of argument supply into the calculus is an important source of flexibility. By separating the aspects of traversing the boundary between expressions and matchings, and matching patterns against the right arguments, the design of the reduction system is made more modular.
- the *alternative* $m_1 \mid m_2$ combines the possible matching results of m_1 and m_2 in some way that can usefully be understood as "alternative". In the instances corresponding to functional programming [11], it is understood sequentially; in $\mathsf{PMC}_!$, this is just the usual choice of functional-logic programming.

The typing rules for matchings are again straight-forward, and are shown in Fig. 3.

Pattern matching $p \Rrightarrow m$ binds all variables occurring in p, so $\mathsf{FV}(p \Rrightarrow m) = \mathsf{FV}(m) - \mathsf{FV}(p)$, letting $\mathsf{FV}(m)$ denote the set of free variables of a matching m. Pattern matching is the only variable binder in this calculus — taking this into

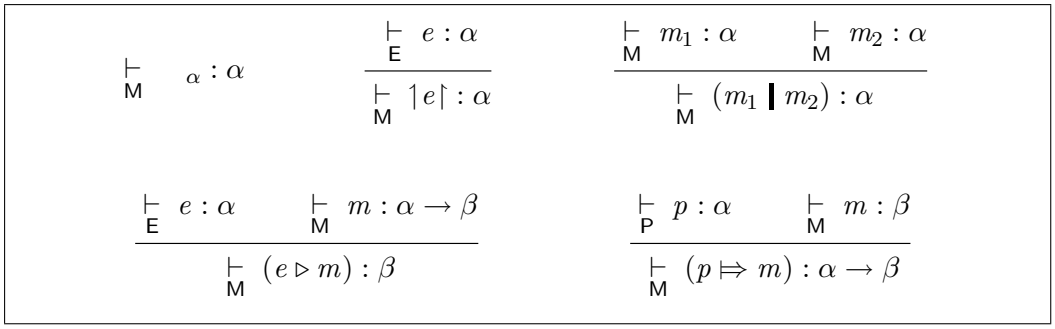


Fig. 3. Matching typing rules

account, the definitions of free variables, bound variables, and substitution are as usual. Note that there are no "matching variables"; variables can only occur as patterns or as expressions.

Also note that the distinction between call-by-name variables and call-by-undetermined-value variables happens not at the variable level, but as part of the pattern syntax, that is, at the binding site of variables. The restriction to linear patterns exludes patterns like $c(\,!x, \underline{x})$, and occurrences of the same variable (e.g., "x") in different patterns (e.g., in "$c(\,!x) \Mapsto d(\underline{x}) \Mapsto x$") are dealt with using the usual scope rules.

Strictly speaking we only have "variables bound at CBN primitive patterns" and "variables bound at CBUV primitive patterns", but we will still use the more convenient terms "CBN variables" and "CBUV variables" to refer to these, respectively, although syntactically we have only one kind of variable. (Different organisations would, of course, be possible, but we found the one chosen here to stay reasonably close to the principle of least surprise.)

We will omit the parentheses in matchings of the shape $a \triangleright (p \Mapsto m)$ since there is only one way to parse $a \triangleright p \Mapsto m$ in PMC. We will also consider $\mid$ to be of lower precedence than $\triangleright$ and $\Mapsto$.

4 Core PMC Rules

We now present the core PMC rules not concerned with call-by-undetermined-value, nor with FLP-specific aspects of alternatives. Except where explicitly mentioned, these all are part of the original pattern matching calculi proposed in [11] for functional programming, and also form the basis of the generic monadic PMC semantics presented in [12].

Formally, these rules together define two *redex reduction* relations:

$$\underset{\mathsf{E}}{\longrightarrow} : \mathsf{Expr} \leftrightarrow \mathsf{Expr} \qquad \text{for expressions, and}$$

$$\underset{\mathsf{M}}{\longrightarrow} : \mathsf{Match} \leftrightarrow \mathsf{Match} \qquad \text{for matchings.}$$

4.1 Failure and Returning

Failure is the left and right unit for $\mid$ — right unit is not necessary for FP, but consistent with it.

$$(1) \qquad \mid m \;\xrightarrow[\mathsf{M}]{}\; m \qquad\qquad m \mid \;\xrightarrow[\mathsf{M}]{}\; m$$

Matching abstractions built from expression matchings are equivalent to the contained expression:

$$(2) \qquad \{\!|\, \lceil e \rceil \,|\!\} \;\xrightarrow[\mathsf{E}]{}\; e$$

Reduction of the converse situation $\lceil \{\!|\, m \,|\!\} \rceil$ is not included, since a function composed from alternatives in general cannot be re-decomposed into those alternatives; see also Sect. 5.6.

A matching abstraction where all alternatives fail can be understood as representing an ill-defined case — this is reduced to the "empty expression"; in FLP, also a matching that returns the empty expression is not distinguishable from a matching returning nothing:

$$(3) \qquad \{\!|\;\;|\!\} \;\xrightarrow[\mathsf{E}]{}\; \oslash \qquad\qquad \lceil \oslash \rceil \;\xrightarrow[\mathsf{M}]{}$$

4.2 Application and Argument Supply

Function application and argument supply correspond to each other via lifting and extraction:

$$(4) \qquad \{\!|\, m \,|\!\}\; a \;\xrightarrow[\mathsf{E}]{}\; \{\!|\, a \triangleright m \,|\!\} \qquad\qquad a \triangleright \lceil e \rceil \;\xrightarrow[\mathsf{M}]{}\; \lceil e\;\, a \rceil$$

Both are strict in the function:

$$(5) \qquad \oslash\; e \;\xrightarrow[\mathsf{E}]{}\; \oslash \qquad\qquad e \triangleright \;\xrightarrow[\mathsf{M}]{}$$

Argument supply distributes into alternatives:

$$(6) \qquad e \triangleright (m_1 \mid m_2) \;\xrightarrow[\mathsf{M}]{}\; (e \triangleright m_1) \mid (e \triangleright m_2)$$

Argument supply also advances over saturated pattern matchings:

$$(7) \qquad a \triangleright b \triangleright p \Rrightarrow m \;\xrightarrow[\mathsf{M}]{}\; b \triangleright p \Rrightarrow a \triangleright m \qquad \text{if } \mathsf{FV}(p) \cap \mathsf{FV}(a) = \{\}$$

(The proviso can always be achieved via variable renaming.)

4.3 Pattern Matching

Everything matches a call-by-name variable pattern; this matching gives rise to substitution, corresponding to β-reduction:

$$(8) \qquad a \triangleright \underline{v} \Rrightarrow m \;\xrightarrow[\mathsf{M}]{}\; m[v \backslash a]$$

Deeper constructor patterns can always be decomposed — this is the fundamental rule for understanding delay of constructor arguments; note that this rule introduces new *call-by-name* variables:

$$(9) \qquad c(p_1, \ldots, p_n) \Rightarrow m \quad \underset{\mathsf{M}}{\longrightarrow} \quad c(\underline{v_1}, \ldots, \underline{v_n}) \Rightarrow v_1 \triangleright p_1 \Rightarrow \cdots \Rightarrow v_n \triangleright p_n \Rightarrow m$$

Matching constructors match, and the proviso in the following rule can always be ensured via α-conversion (for this rule to make sense, linearity of patterns is important):

$$(10) \qquad c(e_1, \ldots, e_n) \triangleright c(p_1, \ldots, p_n) \Rightarrow m \quad \underset{\mathsf{M}}{\longrightarrow} \quad e_1 \triangleright p_1 \Rightarrow \cdots e_n \triangleright p_n \Rightarrow m \quad \text{if } \mathsf{FV}(c(e_1, \ldots, e_n)) \cap \mathsf{FV}(c(p_1, \ldots, p_n)) = \{\}$$

Matching of different constructors fails:

$$(11) \qquad d(e_1, \ldots, e_k) \triangleright c(p_1, \ldots, p_n) \Rightarrow m \quad \underset{\mathsf{M}}{\longrightarrow} \quad \text{if } c \neq d \text{ or } k \neq n$$

For the case where an empty expression, representing "no alternatives" as an expression, is matched against a constructor pattern, the result can only be "no alternatives" on the matching side:

$$(12) \qquad \oslash \triangleright c(p_1, \ldots, p_n) \Rightarrow m \quad \underset{\mathsf{M}}{\longrightarrow}$$

In a certain sense analogoulsy to (12), supply to a constructor pattern also distributes into alternatives:

$$(13) \qquad \{\!|\, m_1 \mid m_2 \,|\!\} \triangleright c(\mathbf{p}) \Rightarrow m \quad \underset{\mathsf{M}}{\longrightarrow} \quad (\{\!|\, m_1 \,|\!\} \triangleright c(\mathbf{p}) \Rightarrow m) \mid (\{\!|\, m_2 \,|\!\} \triangleright c(\mathbf{p}) \Rightarrow m)$$

If p is a constructor pattern, then matching some argument a against p might fail to terminate, giving rise to $a \triangleright p \Rightarrow \quad = \bot$ in such cases. In deterministic functional programming, is used to activate alternatives which would not be activated otherwise, so $\bot$ can never be replaced by . In FLP, it makes sense to assume that if a solution exists, it will eventually be found (with an appropriate search strategy), so $\bot$ can be considered as semantically equivalent to failure, and we can introduce the following additional rule:

$$(14) \qquad p \Rightarrow \quad \underset{\mathsf{M}}{\longrightarrow}$$

4.4 Recursion

The fixed-point combinator reduces via the fixed-point equation:

$$(15) \qquad \mathsf{fix}\ e \quad \underset{\mathsf{E}}{\longrightarrow} \quad e\ (\mathsf{fix}\ e)$$

4.5 Core PMC *Examples*

For example, the standard recursive definition of list concatenation

$$\begin{aligned} &\mathit{conc}\ [\,]\ \mathit{ys} = \mathit{ys} \\ &\mathit{conc}\ (x : \mathit{xs})\ \mathit{ys} = x : \mathit{conc}\ \mathit{xs}\ \mathit{ys} \end{aligned}$$

turns into the following explicit PMC definition:

$$\mathit{conc} = \mathsf{fix}\ \{\!|\, \underline{f} \Mapsto ([] \Mapsto \underline{\mathit{ys}} \Mapsto \uparrow \mathit{ys} \uparrow \mid \underline{x} : \underline{\mathit{xs}} \Mapsto \underline{\mathit{ys}} \Mapsto \uparrow x : f\ \ \mathit{xs}\ \ \mathit{ys} \uparrow)\ |\!\}$$

This is overlap-free, and the application

$$\mathsf{fix}\ \{\!|\, \underline{f} \Mapsto ([] \Mapsto \underline{\mathit{ys}} \Mapsto \uparrow \mathit{ys} \uparrow \mid \underline{x} : \underline{\mathit{xs}} \Mapsto \underline{\mathit{ys}} \Mapsto \uparrow x : f\ \ \mathit{xs}\ \ \mathit{ys} \uparrow)\ |\!\}\ (1 : 2 : [])\ (3 : 4 : [])$$

reduces (in 43 $\mathsf{PMC}_{\mathrm{core}}$ steps) to $1 : 2 : 3 : 4 : []$.

In a functional programming language, one could make use of the implicit prioritisation of definition lines, and define concatenation in the following way:

$$\begin{aligned} &\mathit{concF}\ (x : \mathit{xs})\ \mathit{ys} = x : \mathit{concF}\ \mathit{xs}\ \mathit{ys} \\ &\mathit{concF}\ _ \qquad\ \ \mathit{ys} = \mathit{ys} \end{aligned}$$

The corresponding application

$$\mathsf{fix}\ \{\!|\, \underline{f} \Mapsto (\underline{x} : \underline{\mathit{xs}} \Mapsto \underline{\mathit{ys}} \Mapsto \uparrow x : f\ \ \mathit{xs}\ \ \mathit{ys} \uparrow \mid \underline{\ } \Mapsto \underline{\mathit{ys}} \Mapsto \uparrow \mathit{ys} \uparrow)\ |\!\}\ (1 : 2 : [])\ (3 : 4 : [])$$

reduces (in 39 $\mathsf{PMC}_{\mathrm{core}}$ steps) to

$$\{\!|\, \uparrow 1 : \{\!|\, \uparrow 2 : 3 : 4 : [] \uparrow \mid \uparrow 3 : 4 : [] \uparrow\ |\!\} \uparrow \mid \uparrow 3 : 4 : [] \uparrow\ |\!\}\ ,$$

which is a tree structure, expanding to $\langle [1,2,3,4], [1,3,4], [3,4] \rangle$, which is exactly the list of solutions that Curry produces for $\mathit{concF}\ [1,2]\ [3,4]$. In the pattern matching calculi proposed in [11] originally for (deterministic) functional programming, the additional rule

$$\uparrow e \uparrow \mid m \quad \underset{\mathsf{M}}{\longrightarrow} \quad \uparrow e \uparrow \tag{16}$$

would give preference to the first alternatives and reduce this to $1 : 2 : 3 : 4 : []$, again. (In [12], it was noted that at least in some contexts, the more restricted version

$$\uparrow c(e_q, \ldots, e_n) \uparrow \mid m \quad \underset{\mathsf{M}}{\longrightarrow} \quad \uparrow c(e_q, \ldots, e_n) \uparrow \tag{17}$$

is necessary.)

5 Characterising CBUV Variables

As for call-by-value, the simple β-rule also is not generally valid for call-by-undetermined-value. We first present a set of rules that do allow instantiation of CBUV variables,

and then another set that allows propagation of CBUV variable matchings to move them closer to instantiation opportunities.

5.1 Undetermined Supply

In strict call-by-value, we would have $\bot \triangleright\, !v \Mapsto m = \bot$. But since the pattern $!v$ produces a call-by-*undetermined*-value binding, and no value will arise from $\oslash$, this is one case where β-reduction is valid for CBUV variables:

$$\oslash \triangleright\, !v \Mapsto m \quad \underset{\mathsf{M}}{\longrightarrow} \quad m[v \backslash \oslash] \tag{18}$$

Since $\bot$ is not a problem for bindings to CBUV variables, the β-rule is also valid for them if there is no occurrence of the bound variable::

$$a \triangleright\, !v \Mapsto m \quad \underset{\mathsf{M}}{\longrightarrow} \quad m \qquad \text{if } v \notin \mathsf{FV}(m) \tag{19}$$

For the identity function, it does not make any difference whether it is formulated using CBN or CBUV binding:

$$a \triangleright\, !v \Mapsto \lceil v \rceil \quad \underset{\mathsf{M}}{\longrightarrow} \quad \lceil a \rceil \tag{20}$$

5.2 Value Supply

An expression (in a context of variable binders) is a *value* iff it is of one of the following shapes:

- a CBUV variable (i.e., a variable bound at a $!_$ binder),
- a function extracted from a pattern abstraction $\{\!|\, p \Mapsto m \,|\!\}$,
- a constructor expression $c(v_1, \ldots, v_n)$ where the contructor arguments are values again.

The following three rules are special instances of β-reduction for value supply to CBUV variable abstraction.

The first case is where one CBUV variable v is matched against another. This rule would be easier to formulate if CBUV variables were syntactically different from CBN variables; then we could just write:

$$v \triangleright\, !u \Mapsto m \quad \underset{\mathsf{M}}{\longrightarrow} \quad m[u \backslash v] \qquad \text{if } v \text{ is a CBUV variable.}$$

Since we distinguish the two kinds only by their binding sites, we have formalise this using pattern and matching contexts[2] $P[_]$ and $M[_]$, with the restriction that no binder inside $M[_]$ binds v inside the hole:

$$P[\,!v] \Mapsto M[v \triangleright\, !u \Mapsto m] \quad \underset{\mathsf{M}}{\longrightarrow} \quad P[\,!v] \Mapsto M[m[u \backslash v]] \tag{21}$$

[2] A pattern context here is a pattern that instead of exactly one occurrence of a sub-pattern has a hole, and a matching context is a matching that instead of exactly one occurrence of a sub-matching has a hole. Because of our restriction that patterns be linear, writing "$P[\,!v]$" implies that $v \notin \mathsf{FV}(P[_])$.

Functions constructed from pattern abstraction are considered as values, and therefore can also be substituted via another instance of the β-rule:

$$\{\!|\, p \Rrightarrow m_0 \,|\!\} \triangleright\, !v \Rrightarrow m \quad \underset{\mathsf{M}}{\longrightarrow} \quad m[v \backslash \{\!|\, p \Rrightarrow m_0 \,|\!\}] \qquad \text{if } v \notin \mathsf{FV}(p \Rrightarrow m_0) \tag{22}$$

Supplying a constructor expression to a CBUV variable can only supply value components, i.e., choices contained in constructor arguments must not be substituted in. We therefore "delay" substitution of the constructor arguments by matching them against new CBUV variables:

$$\begin{aligned} & c(e_1, \ldots, e_n) \triangleright\, !v \Rrightarrow m \\ & \qquad \underset{\mathsf{M}}{\longrightarrow} \quad c(e_1, \ldots, e_n) \triangleright c(\,!v_1, \ldots, \,!v_n) \Rrightarrow m[v \backslash c(v_1, \ldots, v_n)] \end{aligned} \tag{23}$$

5.3 Narrowing

The narrowing rule instantiates a local variable with the top level of a pattern it is matched against, but only if this happens immediately inside the local variable's binder:

$$\begin{aligned} & !v \Rrightarrow v \triangleright c(p_1, \ldots, p_k) \Rrightarrow m \\ & \qquad \underset{\mathsf{M}}{\longrightarrow} \quad c(\,!v_1, \ldots, \,!v_k) \Rrightarrow (v \triangleright c(p_1, \ldots, p_k) \Rrightarrow m)[v \backslash c(v_1, \ldots, v_k)] \\ & \qquad\qquad\qquad\qquad \text{if } \{v_1, \ldots, v_k\} \cap (\mathsf{FV}(p) \cup \mathsf{FV}(m)) = \{\} \end{aligned} \tag{24}$$

This is a binding to v "the other way around", and in a certain sense justifies the CBV character of logical variable.

This rule as currently stated only allows narrowing immediately inside the CBUV binder; the CBUV binding propagation rules in Sect. 5.5 have the purpose of producing such situations.

If we were to extend narrowing to supply of CBUV variables farther away from their binder, then we still would have to involve the binder for being able to substitute into all occurrences, and we would also have to be careful that no choices come in-between:

$$\begin{aligned} & P[\,!v] \Rrightarrow M[v \triangleright c(p_1, \ldots, p_k) \Rrightarrow m] \\ & \qquad \underset{\mathsf{M}}{\longrightarrow} \quad P[c(\,!v_1, \ldots, \,!v_k)] \Rrightarrow (M[(v \triangleright c(p_1, \ldots, p_k) \Rrightarrow m)])[v \backslash c(v_1, \ldots, v_k)] \\ & \qquad\qquad\qquad\qquad \text{if } \{v_1, \ldots, v_k\} \cap (\mathsf{FV}(p) \cup \mathsf{FV}(m)) = \{\} \\ & \qquad\qquad\qquad\qquad \text{and if the hole in } M[_] \text{ is not a subterm of a choice construct.} \end{aligned}$$

A different approach would have a "local" narrowing rule that substitutes only into the narrowed subterm, and not into other occurrences of the narrowed variable:

$v \triangleright c(p_1, \ldots, p_n) \Rightarrow m$

$\quad \xrightarrow[\mathsf{M}]{} \quad v \triangleright c(\,!v_1, \ldots, \,!v_n) \Rightarrow v_1 \triangleright p_1 \Rightarrow \cdots \Rightarrow v_n \triangleright p_n \Rightarrow m[v \mapsto c(v_1, \ldots, v_n)]$

if v is a CBUV variable

For use in a reduction system, this does of course have to be limited to cases where not all p_i are CBUV binders already. Also, it does by itself not fully replace rule (24), but requires addition of the following clean-up rule for the narrowed variable itself:

$$!v \Rightarrow v \triangleright p \Rightarrow m \quad \xrightarrow[\mathsf{M}]{} \quad p \Rightarrow m \qquad \text{if } p \text{ has no CBN bindings, and } v \notin \mathsf{FV}(p \Rightarrow m)$$

To obtain redexes for this latter rule, we still need the propagation machinery of Sect. 5.5, and it is currently not clear to me whether this approach really has advantages over directly using rule (24).

5.4 CBUV and Choices

Supply to a CBUV variable should distribute over alternative:

$$\{\!|\, m_1 \,|\, m_2 \,|\!\}\triangleright !v \Rightarrow m = (\{\!|\, m_1 \,|\!\}\triangleright !v \Rightarrow m) \,|\, (\{\!|\, m_2 \,|\!\}\triangleright !v \Rightarrow m) \tag{25}$$

I state this here as a semantic equation instead of a reduction rule, since using this in a reduction system destroys first-order sharing, and apparently would require second-order sharing mechanisms to prevent divergence.

Together with (19), this rule implies idempotence of $|$, and together with (28) it also implies at least a restricted form of commutativity. Associativity is natural and unproblematic to add even to the original PMC systems, so for $\mathsf{PMC}_!$ we end up having to consider alternatives as producing *sets* of results.

5.5 CBUV Binding Propagation

In matchings of the shape $a \triangleright !v \Rightarrow m$, if the argument a does not make any of the instantiation rules above applicable, then one needs to try to produce a redex for the narrowing rule (24). In this section we list a number of rules that can be used for that purpose, by "moving $a \triangleright !v \Rightarrow$ into m".

As long as we don't have $\lceil\{\!|\, m \,|\!\}\rceil = m$, we at least need to let CBUV variable supply move in:

$$a \triangleright !v \Rightarrow \lceil\{\!|\, m \,|\!\}\rceil \quad \xrightarrow[\mathsf{M}]{} \quad \lceil\{\!|\, a \triangleright !v \Rightarrow m \,|\!\}\rceil \tag{26}$$

If a CBV variable occurs only in *one* constructor argument, it can move into this argument together with its supplied argument:

$$a \triangleright !v \Rightarrow \lceil c(e_1, \ldots, e_n)\rceil \quad \xrightarrow[\mathsf{M}]{} \quad \lceil c(e_1, \ldots, \{\!|\, a \triangleright v \Rightarrow \lceil e_k \rceil \,|\!\}, \ldots, e_n)\rceil \tag{27}$$

where e_k is the only e_i with $v \in \mathsf{FV}(e_i)$. This is justified since constructors are non-strict.

We naturally expect CBUV pattern bindings to distribute over alternative:

$$a \triangleright\, !v \Rightarrow (m_3 \mid m_4) \quad \xrightarrow[\mathsf{M}]{} \quad (a \triangleright\, !v \Rightarrow m_3) \mid (a \triangleright\, !v \Rightarrow m_4) \tag{28}$$

With distributivity of supply over alternative (6), this would also be implied by the following more general rule:

$$!v \Rightarrow (m_3 \mid m_4) \quad \xrightarrow[\mathsf{M}]{} \quad (!v \Rightarrow m_3) \mid (!v \Rightarrow m_4)$$

This rule, however, would lead to non-confluence together with (22) and (25), and would also affect the discussion in Sect. 5.6.

A CBUV binding can move into an extraction in the argument of supply to a function that does not contain the bound variable:

$$a \triangleright\, !v \Rightarrow \{\!|\, b \,|\!\} \triangleright m \quad \xrightarrow[\mathsf{M}]{} \quad \{\!|\, a \triangleright\, !v \Rightarrow b \,|\!\} \triangleright m \qquad \text{if } v \notin \mathsf{FV}(m) \tag{29}$$

The converse of (7) can be used to move the binding over supply of an argument that does not contain the bound variable.

A CBUV binding can also propagate over a pattern that does not re-bind the variable (if it does rebind it, then (19) eliminates the CBUV binding):

$$a \triangleright\, !v \Rightarrow p \Rightarrow m \quad \xrightarrow[\mathsf{M}]{} \quad p \Rightarrow a \triangleright\, !v \Rightarrow m \quad \text{if } v \notin \mathsf{FV}(p) \text{ and } \mathsf{FV}(p) \cap \mathsf{FV}(a) = \{\} \tag{30}$$

5.6 Choice and Eta-Expansion

The latest version of the Curry implementation PAKCS [10] changed the way choice definitions at function types are implemented; in a recent message on the Curry mailing list, López Fraguas proposed the following example to demonstrate the difference:

```
g x = 0
h x = 1
fcoin = g
fcoin = h   -- fcoin is a 'functional coin'
fadd f g x = (f x) + (g x)
fdouble f = fadd f f
```

In $\mathsf{PMC}_!$ we can define:

$$\begin{aligned} \mathit{fdouble} &= \{\!|\, !f \Rightarrow \lfloor \{\!|\, !f \Rightarrow !g \Rightarrow !x \Rightarrow \lfloor \mathit{natAdd}\ (f\ x)\ (g\ x) \rceil \,|\!\}\ f\ f \rceil \,|\!\} \\ \mathit{fcoin} &= \{\!|\, !x \Rightarrow \lfloor \mathit{Zero} \rceil \mid !x \Rightarrow \lfloor \mathit{Succ}\ \mathit{Zero} \rceil \,|\!\} \end{aligned}$$

The expression *fdouble fcoin Zero* corresponds to the Curry version without η-expansion and evaluates (in 70 steps) to $\{\!|\lceil Zero\rceil \mid \lceil Succ\ (Succ\ Zero)\rceil |\!\}$, corresponding to $\langle 0, 2\rangle$, just like the latest PAKCS version.

η-expanding *fcoin* in Curry would produce one of the following two equivalent definitions:

$$\begin{aligned}&\mathit{fcoin'}\ x = 0\\&\mathit{fcoin'}\ x = 1\\&\mathit{fcoin''}\ x = \mathit{coin}\end{aligned}$$

In $\mathsf{PMC}_!$, these are slightly different, but also equivalent:

$$\begin{aligned}\mathit{fcoin'} &= \{\!|\ !x \Rightarrow (\lceil Zero\rceil \mid \lceil Succ\ Zero\rceil)\ |\!\}\\ \mathit{fcoin''} &= \{\!|\ !x \Rightarrow \lceil \mathit{coin}\rceil\ |\!\}\end{aligned}$$

The expressions *fdouble fcoin′ Zero* and *fdouble fcoin″ Zero* now both evaluate to $\{\!|\lceil\{\!|\lceil Zero\rceil \mid \lceil Succ\ Zero\rceil|\!\}\rceil \mid \lceil Succ\ \{\!|\lceil Zero\rceil \mid \lceil Succ\ Zero\rceil|\!\}\rceil|\!\}$, which expands to $\langle 0, 1, 1, 2\rangle$, which is exactly what the previous version of PAKCS produced for *fdouble fcoin* 0.

6 Conclusion and Outlook

Starting from the equational formalisation of Haskell pattern matching in $\mathsf{PMC}_\oslash$, I have shown that the "call-by-undetermined-value" variable binding convention of Curry can be equationally captured in the slightly extended system $\mathsf{PMC}_!$, thus opening up functional-logic programming to more systematic equational reasoning than has been possible so far.

Although the precise set of rules is still work in progress, the examples presented above should convey the impression that reasoning about functional-logic programs does not have to consider substitutions as results, and can instead adopt the equational reasoning style more common in functional programming.

Although I presented most equations as directed rewriting rules, I indicated that expansion of alternatives cannot be used for implementations without constraints, and that propagation of CBUV variable pattern bindings involved the converse of a rule that is useful in other contexts, so there is still significant work to be done before a rule set permitting a normalising strategy has been identified, and a confluence proof performed.

An experimental implementation of the presented rules (strongly typed, in Haskell) produces promising results, but does not yet reduce all test cases with a single orientation of the rules. (Using a parallel reduction strategy, it does reduce some examples to normal form (including all "solutions") that diverge in PAKCS.)

However, this equational approach is not really intended to provide a competing implementation paradigm for functional-logic programming — the fact that its view of the semantics involves *sets* of results instead of sequences (or multi-sets) is a strong indication that it is better suited as a basis for implementation-independent high-level reasoning about functional-logic programs, and for guiding the eventual

formulation of a denotational semantics in a flavour similar to those for functional programming languages.

Acknowledgement

I would like to thank the reviewers for their constructive suggestions.

References

[1] Albert, E., M. Hanus, F. Huch, J. Oliver and G. Vidal, *Operational semantics for declarative multi-paradigm languages*, Journal of Symbolic Computation **40** (2005), pp. 795–829.
URL http://www.informatik.uni-kiel.de/~mh/publications/papers/JSC05.html

[2] Antoy, S. and M. Hanus, *Overlapping rules and logic variables in functional logic programs*, in: *Proceedings of the International Conference on Logic Programming (ICLP 2006)* (2006), pp. 87–101.
URL http://www.informatik.uni-kiel.de/~mh/publications/papers/ICLP06.html

[3] Caballero, R., J. Sánchez et al., *$\mathcal{TOY}$: A multiparadigm declarative language, version 2.2.3*, Technical report, Universidad Complutense Madrid (2006), http://toy.sourceforge.net.

[4] de Dios Castro, J. and F. López Fraguas, *Extra variables can be eliminated from functional logic programs*, ENTCS (2007), p. 16 pages, special issue with extended selected papers from PROLE 2006.

[5] Friedman, D. P. and D. S. Wise, *CONS should not evaluate its arguments*, in: S. Michaelson and R. Milner, editors, *Third International Colloquium on Automata, Languages and Programming, ICALP '76* (1976), pp. 257–284.

[6] González-Moreno, J., M. Hortalá-González, F. López-Fraguas and M. Rodríguez-Artalejo, *An approach to declarative programming based on a rewriting logic*, The Journal of Logic Programming **41** (1999), pp. 47–87.
URL http://www.ingentaconnect.com/content/els/07431066/1999/00000040/00000001/art10029

[7] Hanus, M., *The integration of functions into logic programming: From theory to practice*, J. Logic Programming **19,20** (1994), pp. 583–628.

[8] Hanus, M., *A unified computation model for functional and logic programming*, in: *Proc. 24st ACM Symposium on Principles of Programming Languages (POPL'97)*, 1997, pp. 80–93.
URL http://www-i2.informatik.rwth-aachen.de/~hanus/publications/papers/POPL97.html

[9] Hanus, M. et al., "Curry — An Integrated Functional Logic Language, Version 0.8.2," (2006), http://www.informatik.uni-kiel.de/~curry/report.html.
URL http://www.informatik.uni-kiel.de/~curry/report.html

[10] Hanus, M. et al., "PAKCS 1.8.0, The Portland Aachen Kiel Curry System," (2007), http://www.informatik.uni-kiel.de/~pakcs/.

[11] Kahl, W., *Basic pattern matching calculi: A fresh view on matching failure*, in: Y. Kameyama and P. Stuckey, editors, *Functional and Logic Programming, Proceedings of FLOPS 2004*, LNCS **2998** (2004), pp. 276–290.
URL http://link.springer.de/link/service/series/0558/tocs/t2998.htm

[12] Kahl, W., J. Carette and X. Ji, *Bimonadic semantics for basic pattern matching calculi*, in: T. Uustalu, editor, *Mathematics of Program Construction, MPC 2006*, LNCS **4014** (2006), pp. 253–273.
URL http://link.springer.de/link/service/series/0558/tocs/t4014.htm

[13] Peyton Jones, S. et al., "The Revised Haskell 98 Report," Cambridge Univ. Press, 2003, also on http://haskell.org/.
URL http://titles.cambridge.org/catalogue.asp?isbn=0521826144

[14] Plasmeijer, R. and M. van Eekelen, "Functional Programming and Parallel Graph Rewriting," International Computer Science Series, Addison-Wesley, 1993.

Declarative Debugging Meets the World

Wolfgang Lux[1]

University of Münster

Abstract

Declarative debugging has been proposed as a suitable technique for diagnosing errors. It is particularly attractive for declarative programming languages, whose operational semantics differs substantially from their declarative semantics. Declarative debuggers are usually described and implemented by means of a program transformation. However, this transformation does not mix well with monadic I/O, which is used by lazy functional and functional-logic languages. Therefore, declarative debuggers for such languages either do not support debugging of programs involving stateful computations at all, or require special support from the compiler and its runtime system. In this paper, we present a modified program transformation that blends nicely with monadic I/O and therefore covers the whole language without additional support from the target implementation.

Keywords: declarative debugging, monadic I/O

1 Introduction

Declarative debugging has been proposed as a suitable technique for diagnosing errors on the basis of the intended meaning of a program. Since it abstracts from the operational behavior of programs, it is particularly attractive for debugging programs written in declarative programming languages. Understanding the operational behavior of programs implemented in such languages can become quite difficult due to lazy evaluation, concurrency, and non-deterministic search and thus distract from finding the actual causes of an error.

The key idea of declarative debugging is to build a computation tree (also called an evaluation dependency tree) representing the computations performed by an erroneous program. The exact form of the computation tree depends on the kinds of errors that are to be diagnosed. Each node of a computation tree represents the outcome of a computation step in the program and is connected to the nodes representing the subcomputations that were performed in order to produce the result of the computation.

If a program does not compute the expected result or produces unexpected results, its computation tree contains one or more erroneous nodes, whose result

[1] Email: wlux@uni-muenster.de

does not match the semantics intended by the programmer. Some of these erroneous nodes are just the result of other erroneous computations, while other erroneous nodes represent genuine programming errors. These *buggy nodes* are distinguished by the fact that they do not have any erroneous children, i.e., their represented computation step computes a wrong result from correct inputs.

Since a debugging session starts with an observation that the program does not work as expected, i.e., the root of the computation tree is an erroneous node, and all nodes of the computation tree are connected with that root, it is possible to find a buggy node by a top-down navigation of the computation tree. To that end, an *oracle* – usually the user – is asked questions about the computation steps recorded in the computation tree and whether they agree with the program's intended semantics or not.

A difficult aspect for implementing declarative debuggers for lazy functional and functional-logic languages like Haskell [8], Curry [4], and $\mathcal{TOY}$ [5] is the integration of input and output. This difficulty is twofold. For one, the program's own interaction with the external world must be kept separate from the interaction of the debugger with the oracle. Second, input and output is encapsulated in the `IO` monad, which has an implicit higher order nature and is not directly eligible to a program transformation. For that reason, present declarative debuggers for these languages either do not cover I/O at all or use a substitute `IO` monad that must be mapped back to the real `IO` monad by means of an additional language primitive.

In this paper, we present a modification to the standard program transformation approach that avoids these shortcomings by introducing a special case for `IO` functions. Using the fact that `IO` is an abstract type and elements of that type cannot be inspected directly by user defined functions, it becomes sufficient to define appropriate adaptors for the primitive `IO` functions. For almost all of these adaptor functions, their definition can be derived mechanically from the semantic model of the `IO` type. Only for a few distinguished primitives one needs to provide predefined implementations. Nevertheless, no additional non-standard primitives are needed for their definition. Thus, while using the semantic model of the type `IO` in order to derive the adaptors, their concrete implementation does not rely on any particular implementation technique of I/O actions in the target implementation.

The rest of this paper is structured as follows. In the next section, we briefly review the standard program transformation approach for detecting wrong answers in functional and functional-logic programming languages. The third section describes monadic I/O and the problems it poses for the program transformation approach. In section 4, we introduce our modified program transformation that mixes well with monadic I/O. Finally, the two last sections present related work and conclude.

2 Declarative Debugging

In this section we introduce declarative debugging of wrong answers for lazy functional and functional-logic languages informally. See [1,3] for a complete theoretical treatment. For our presentation, we will use the multi-paradigm declarative language Curry [4]. However, we do not rely on any specific features of Curry so the examples are equally valid (after minor syntactic changes) for $\mathcal{TOY}$ and – except

```
reverse ::  [a] → [a]
reverse []      = []
reverse (x:xs)  = append xs [x]

append ::  [a] → [a] → [a]
append []      ys = ys
append (x:xs)  ys = x :  append xs ys
```

Fig. 1. A buggy reverse function

where noted otherwise – for the functional language Haskell [8].

2.1 Computation Trees

The debugging process is started with an initial symptom detected while executing a goal. Two different kinds of symptoms are possible, corresponding to different kinds of bugs.

- A *positive symptom* is an unexpected answer obtained for the goal. This is called a *wrong answer*.
- A *negative symptom* is an expected answer that is missing from the multiset of results computed for the goal. This is a *missing answer*.

Often, missing answers and wrong answers occur together. For instance, consider the buggy program from Fig. 1. For this program, the goal `reverse [1,2,3]` yields the result `[2,3,1]`, which is a wrong answer. The expected result `[3,2,1]` is not computed, so there is a missing answer too.

In order to detect wrong answers, the oracle must know the intended meaning $\mathcal{I}$ of the program. As proved in [1], it is sufficient to consider $\mathcal{I}$ as a set of *basic facts* of the form $f\,t_1 \ldots t_n \to t$, where $t_1, \ldots, t_n, t$ are constructor terms. A basic fact $f\,t_1 \ldots t_n \to t$ means that function f produces the result t when applied to the arguments $t_1, \ldots, t_n$. The oracle is asked only questions whether a basic fact is in $\mathcal{I}$ or not. This ensures that the questions asked are as simple as possible and is achieved by replacing nested function applications by their results obtained during the computation. Applications that were not evaluated by the program are denoted by the special constructor term $\bot$, which is represented by the symbol `_` in the debugger's questions.

The *computation trees* used by the debugger will have basic facts at their nodes. Each node has an associated program rule, the program rule used at the corresponding computation step. Thus, the debugger will point out the program rule associated with a buggy node as an incorrect program rule. The children of a node correspond to the subcomputations carried out while evaluating the guard and right hand side of the program rule. The soundness and completeness results in [1] ensure that, given a wrong answer, an incorrect program rule is detected by the debugger.

2.2 Program Transformation

Several strategies have been proposed to create and navigate computation trees. a well-known approach widely employed in Logic Programming uses meta-interpreters to re-execute the goal during the debugging phase. Thus, the computation tree is not computed explicitly, and both wrong an missing answers are easily handled. This idea been extended in the case of NUE-Prolog to functional-logic languages [7].

However, this solution is not available to languages that do not provide built-in meta-instructions, as Haskell or Curry. To the best of our knowledge, only declarative debuggers for *wrong answers* have been developed for functional and functional-logic languages. The reason is twofold: first, the computation trees necessary for detecting missing answers are much more complicated. Second, often wrong and missing answers occur simultaneously, as in the case of our buggy `reverse` example. In these cases, it is enough to find out the reason for the wrong answer to get rid of both errors. This is also the case of our debugger.

Two different techniques have been proposed in related papers for producing a computation tree associated with a wrong answer (see [6] for a comparison):

(i) Modify the implementation of the abstract machine to produce the computation tree during the goal's evaluation.

(ii) Transform the source program P into a new program P' in which all functions return the same result as in P, but paired with their corresponding computation tree.

We have adopted the latter approach because of its greater flexibility and portability. Computation trees are represented in transformed programs by elements of the type

```
data CTree = Void | Node Rule [Term] Term [CTree]
```

where `Rule` and `Term` are types suitable for identifying program rules and representing constructor terms, respectively. The `Term` type must be capable of representing unevaluated expressions. However, it is sufficient to use the same representation for all unevaluated expressions. For instance,

```
data Term = CAp String [Term] | Var String | Suspend
```

would be a suitable (untyped) representation, with `CAp` $c\,[t_1, \ldots, t_n]$ representing an application of constructor c to the argument terms $t_1, \ldots, t_n$, `Var` x an unbound logical variable x, and `Suspend` an unevaluated expression.

`Void` nodes in computation trees are used for transformed functions which cannot be the cause of an error (*trusted functions*). A `Node` f `[`$t_1, \ldots, t_n$`]` t *cts* represents a basic fact $f\,t_1 \ldots t_n \to t$ with subcomputations represented by the elements of the list *cts*. In order to transform – possibly unevaluated – expressions into terms, we use an impure function `dval :: a` $\to$ `Term` that converts its argument without further evaluation. The result of `dval` e thus depends on the order of evaluation. However, since `dval` is applied only to (sub)expressions of the original program and the computation tree is traversed only after the final result of the program has been computed, this evaluation order dependence does not affect the debugger.

The program transformation transforms each n-ary function $f :: \tau_1 \to \ldots \to$

$\tau_n \to \tau$ of the program into a transformed function

$$f' :: \tau'_1 \to \ldots \to \tau'_n \to (\tau', \texttt{CTree})$$

where the transformation of types $'$ is defined by the following rules:

$$\begin{aligned} \alpha' &= \alpha \\ (T\,\overline{\tau_n})' &= T\,\overline{\tau'_n} \\ (\mu \to \nu)' &= \mu' \to (\nu', \texttt{CTree}) \end{aligned} \tag{1}$$

Here, α is type variable, T is a type constructor of arity n, and μ and ν are types. Note that the transformation does not presume that functions are η-expanded, i.e., the result type τ of a function may contain arrows. However, the transformed types of η-expanded and η-reduced functions differ. For instance, the `append` function from Fig. 1 is transformed into a function

```
append' ::  [a] → [a] → ([a],CTree)
```

whereas for the following, somewhat unusual definition of `append`

```
append ::  [a] → [a] → [a]
append []      = \ys → ys
append (x:xs)  = \ys → x :  append xs ys
```

the transformed function would have type

```
append' ::  [a] → ([a] → ([a],CTree), CTree)
```

By the transformation (1), only partial applications lacking a single argument have the correct signature to be used as arguments of higher-order functions. Therefore, auxiliary functions are introduced by the program transformation in order to adapt partial applications with two or more missing arguments to their expected types. We will not consider these auxiliary functions further in this paper. The type transformation $'$ is also applied to the argument types of all data constructors. However, the left hand sides of the corresponding type declarations are unaffected by the transformation.

The source code of a function is transformed by adding a computation tree to the result and flattening nested applications into simple applications using auxiliary local declarations. The details of the transformation can be found in [3]. As an example, the second equation of the `append` function from Fig. 1 is transformed into (an equivalent of) the following declaration.

```
append' (x:xs) ys = (r,t)
  where  (r1,t1) = append' xs ys
         r = x :  r1
         t = Node "append"  [dval (x:xs), dval ys] (dval r)
                            (clean [(r1,t1)])
```

For simplicity, we have assumed that rules are represented by their names. The

`clean` function (`clean :: [(Term,CTree)] → [CTree]`) prunes the list of sub-computation trees by removing all trees which are associated with an unevaluated expression. This is necessary in order to ensure that computation trees remain finite in the presence of potentially infinite computations, thereby making an efficient depth-first traversal of computation trees in the debugger feasible.

Trivial wrappers are introduced as transformed functions for primitives, e.g., the arithmetic operations on integer numbers. These wrappers simply pair the result of the primitive with a `Void` node. For instance, the following auxiliary function is provided for integer addition

```
add_int' x y = (x + y,Void)
```

3 Debugging Interactive Programs

Monadic I/O has been introduced by Wadler in a series of papers [12,13], and since then has become the de-facto standard for introducing stateful computations, and in particular input and output operations, into pure functional and functional-logic languages with a non-strict semantics.

Programs interacting with the external world have type `IO` t, which can be considered an abbreviation for

$$\texttt{IO}\ t = \mathit{World} \to (t, \mathit{World}) \tag{2}$$

where *World* is a representation of the state manipulated by the program. However, the type `IO` is abstract in order to ensure a disciplined use of the state parameter. In particular, this prevents user programs from accidentally duplicating the state. In the following, we will use Eq. (2) as a semantic model of type `IO`. However, this does not imply that implementations are restricted to implement type `IO` as a function of that particular type.

The two fundamental operations for type `IO` are

```
return :: a → IO a

(>>=) :: IO a → (a → IO b) → IO b
```

An expression `return` e lifts the result of expression e into the `IO` monad. An expression m `>>=` f first performs the action associated with the expression m and then the action associated with the expression $f\,x$, where x is the result delivered by the first action. Using our semantic model (2) for type `IO`, these functions can be defined as follows

```
return x = \world → (x,world)

m >>= f = \world → let (x,world') = m world in f x world'
```
(3)

Notice that `return` does not change the state and how `(>>=)` carefully ensures a single threaded use of the state.

The interaction with the external world is implemented by primitive operations. For instance, Haskell and Curry programs can use the primitive functions `getLine :: IO String` and `putStrLn :: String → IO ()`, which implement actions that read a line from the standard input and write a line to the standard

output, respectively.

Monadic I/O presents two challenges for a declarative debugger. First, one must carefully separate the debugged program's interaction with the external world and the debugger's interaction with the oracle. Second, one must provide appropriate computation trees for the primitive `IO` functions. The first of these problems can be solved easily by executing the program to completion before starting the debugger's navigation of the computation tree. The second problem is more difficult.

As noted above, for each primitive function the standard program transformation introduces an auxiliary function, which pairs the result of the primitive with a `Void` node. However, this approach is valid only for first order primitives and not for functions like `(>>=)`. This is witnessed by `(>>=)`'s type after applying the transformation (1): [2]

```
(>>=') ::  IO a → (a → (IO b,CTree)) → (IO b,CTree)
```

In a functional language, there is no safe way to implement a function with that type. In a functional-logic language like Curry one might consider using logical variables in order to implement an auxiliary function for `(>>=)`. In a spirit similar to our approach to extend the program transformation approach to cover encapsulated search [2], one might come up with the following definition.

```
m >>=' f =
  ((m >>= \x → let (m',t') = f x in t=:=t' &> m'), t)
  where t free
```

Here, the expression e_1 `=:=` e_2 denotes a strict equality on data terms between the two expressions e_1 and e_2 instantiating unbound variables in the two expression as necessary. The predefined operator `(&>)` implements a restricted identity such that c `&>` e is equivalent to e if the constraint c is satisfied, and fails otherwise.

Unfortunately, it turns out that the above definition for `(>>=')` does not work in practice because the free variable `t` is instantiated too early by the equality constraint, namely before the monadic action `m'` has been executed at all. Thus, if the result of the subexpression `f x` is computed by an application of `(>>=')`, the variable `t'` is an unbound variable itself. In the end, this may lead to either an instantiation error or an unintended non-deterministic search in the `clean` function.

4 An Improved Program Transformation

The discussion in the previous section shows that the standard transformation cannot be used directly for programs using monadic I/O. In order to extend our transformation to monadic I/O, we will now have a more careful look at the program transformation.

If `IO` were not an abstract type, but defined as in Eq. (2), we could apply our standard transformation rules. In that case, the type `IO` τ would be equivalent to

[2] The operator symbol `(>>=')` is not legal in neither Haskell nor Curry. Nevertheless, we use this symbol in this paper in order to emphasize the relation between the `(>>=)` primitive and its auxiliary function.

World → (τ, *World*) and would be transformed into

$$\mathit{World} \to ((\tau', \mathit{World}), \texttt{CTree}).$$

Obviously, this type is not an instance of `IO`, which is the source of our difficulties with monadic I/O. However, there is an (almost) isomorphic type, which is an instance of `IO`, namely

$$\mathit{World} \to ((\tau', \texttt{CTree}), \mathit{World}) = \texttt{IO}\ (\tau', \texttt{CTree}).$$

Thus, with the help of the additional flip of tuple elements, there appears to be a transformation that takes the type `IO` τ into `IO` (τ′, `CTree`). This observation suggests a refinement of the type transformation (1), adding this relation as a special case.

$$\begin{aligned} \alpha' &= \alpha \\ (\texttt{IO}\,\tau)' &= \texttt{IO}\ (\tau', \texttt{CTree}) \\ (T\,\overline{\tau_n})' &= T\,\overline{\tau_n'} \qquad\qquad (T \neq \texttt{IO}) \\ (\mu \to \nu)' &= \mu' \to (\nu', \texttt{CTree}) \end{aligned} \tag{4}$$

Since the type `IO` is abstract and therefore user defined functions cannot inspect elements of that type, the source code transformation for them is unaffected by the special treatment of type `IO`. In conjunction with the fact that the types ((τ, `CTree`), *World*) and ((τ, *World*), `CTree`) are equivalent as far as the debugger is concerned, this also ensures that the soundness and completeness results from [1] are valid for our modified transformation, too, provided that we can define suitable auxiliary functions for the primitive I/O operations.

4.1 Return

Things are very simple for `return`. If `return` were a user defined function as in (3), it would be transformed into

```
return' x w = ((x,w), Void)
```

Flipping the tuple elements in the result, this becomes

```
return' x w = ((x,Void), w)
```

which is equivalent to `return' x = return (x,Void)`. However, as we noted in Sect. 2, η-reduction also changes the type of the transformed function and we end up with the following definition for our auxiliary function.

```
return' ::  a → (IO (a, CTree), CTree)

return' x = (return (x, Void), Void)
```

Notice how the presence of two computation trees in the auxiliary function nicely reflects the double nature of the `IO` monad as pure computations that deliver another, stateful computation.

4.2 Sequential Execution

In order to provide an auxiliary function for the `(>>=)` primitive, we also consider its implementation as a user defined function first. Its definition from (3) would be transformed into an equation

```
(>>=') m f w = ((r,w2), t)
  where  ((x,w1),t1)  = m w
         (m',t2)      = f x
         ((r,w2),t3)  = m' w2
         t = Node  "(>>=)" [dval m,dval f,dval w] (dval (r,w2))
                   [t1,t2,t3]
```

By transforming all expressions with type $((\tau, \mathit{World}), \texttt{CTree})$ into expressions of type $((\tau, \texttt{CTree}), \mathit{World})$, this definition becomes

```
(>>=') m f w = ((r, t),w2)
  where  ((x,t1),w1)  = m w
         (m',t2)      = f x
         ((r,t3),w2)  = m' w2
         t = Node  "(>>=)" [dval m,dval f,dval w] (dval (r,w2))
                   [t1,t2,t3]
```

It is straight forward to derive the following auxiliary function for `(>>=')` from this definition.

```
(>>=') ::  IO (a,CTree) → (a → (IO (b,CTree), CTree))
       → (IO (b,CTree), CTree)
m >>=' f =
  ((  m >>=                  \(x,t1) →
      let (m',t2) = f x in
      m' >>=                 \(r,t3) →
      let t = Node "(>>=)" [dval m,dval f] (dval r) [t1,t2,t3]),
    Void)
```

4.3 Other Primitives

Almost all other primitive I/O operations have a type of the form $\tau_1 \to \ldots \tau_n \to$ `IO` τ, where none of the τ_i nor τ is an arrow or `IO` type. According to our modified type transformation (4), we must provide an auxiliary function with type $\tau_1 \to \ldots \tau_n \to (\texttt{IO}\ (\tau, \texttt{CTree}), \texttt{CTree})$. The signature of these I/O operations indicates

that operations are atomic with respect to the semantics of the host language. Therefore, both computation trees supplied by the auxiliary function can be only `Void` nodes. Thus, we introduce an auxiliary function

$$f' \, x_1 \ldots x_n = ((f \, x_1 \ldots x_n \texttt{ >>= } \backslash x \to (x, \texttt{Void})), \texttt{Void})$$

for each n-ary primitive with an `IO` result and no `IO` or function arguments. Note that the Haskell foreign function interface, which is also adopted by the Münster Curry compiler, allows defining foreign functions with an `IO` result, but does not allow `IO` or function arguments, so the above rule can be applied to such foreign declarations as well.

Things are not so simple for the remaining primitives that have `IO` arguments or arguments with function types. Fortunately, there are only few of them [3], which makes it feasible to provide a list of predefined wrappers together with the debugger. Given their semantics using the semantic model of `IO` from Eq. (2), it is straight forward to define appropriate auxiliary functions in the same way as we did for the `return` and `(>>=)` primitives.

An interesting case is the function `unsafePerformIO`. While its use is generally not recommended, it turns out to be useful in order to embed stateful computations into pure code, provided that one can prove that the resulting code does not break the declarative reading of the program. For instance, the purely functional `Array` library of the Münster Curry compiler is implemented on top of mutable arrays defined in the `IO` monad via a few safe uses of `unsafePerformIO`.

Using the semantic model (2), `unsafePerformIO` could be defined as follows

```
unsafePerformIO m = let (x,w') = m w in x
```

where `w` is a global constant that acts as substitute for the state of the external world while executing the monadic action. Applying the standard transformation to this definition and converting $((\tau, \mathit{World}), \texttt{CTree})$ expressions into $((\tau, \texttt{CTree}), \mathit{World})$, we arrive at the following definition of an auxiliary function for `unsafePerformIO`.

```
unsafePerformIO' m = (x,t)

  where  ((x,t),w') = m w
```

Yet, this is nothing other than `unsafePerformIO` itself.

5 Related Work

Declarative debugging was introduced by Shapiro as a technique to diagnose errors in Prolog programs in [11]. This technique was later generalized into a generic debugging scheme by Naish [6].

The standard program transformation for functional-logic programs, on which our transformation is based, was introduced and proven sound in [1,3]. In [2] it was shown that the transformation can be applied to Curry programs using encapsulated search with the help of a suitable auxiliary function.

[3] In case of the Münster Curry compiler, the primitives which require special treatment are `(>>=)`, `catch`, `unsafePerformIO`, `fixIO`, and `encapsulate`.

Buddha [9] is a declarative debugger for Haskell and is based on a program transformation similar to ours [10]. I/O is handled by introducing a substitute monad `MIO`, which features an explicit $World \to (\tau, World)$ definition and thus makes `IO` functions eligible to program transformation. However, this approach requires the introduction of a new primitive function `unsafeSyncIO` in order to embed the substitute in the real `IO` monad.

6 Conclusion

In this paper we have presented a modified program transformation approach for declarative debugging that mixes well with monadic I/O. Using a modified transformation rule for type `IO`, we avoid the need to disclose the implementation of type `IO` and do not need to resort to controversial functions like `unsafePerformIO` and `seq`. In conjunction with our previous work on declarative debugging and encapsulated search this now gives a full account to declarative debugging of Curry programs.

A prototype implementation of the debugger is part of the Münster Curry compiler [4] and can be used for experimenting with the debugger.

The present implementation of the debugger executes the whole program to completion before entering the navigation phase. While easy to understand from the perspective of the user and straight forward to implement, this approach suffers from the high memory demands of the transformed programs, which essentially keep a trace of the whole program's execution in memory. We plan to investigate improving the debugger in this respect by writing our partial traces of the program's execution to disk and eventually interleaving the navigation phase with the program's execution. The difficult aspect here is to reconcile the two conflicting demands to write out program traces early in order to reduce memory usage on one hand and to provide sufficient information for the user to answer the debugger's questions on the other hand.

References

[1] Caballero, R., F. J. López-Fraguas and M. Rodríguez-Artalejo, *Theoretical foundations for the declarative debugging of lazy functional logic programs*, in: H. Kuchen and K. Ueda, editors, *Proc. FLOPS 2001*, LNCS 2024 (2001), pp. 170–184.

[2] Caballero, R. and W. Lux, *Declarative debugging for encapsulated search* **76** (2002).
URL http://www.elsevier.com/locate/entcs/volume76.html

[3] Caballero, R. and M. Rodríguez-Artalejo, *A declarative debugging system for lazy functional logic programs*, Electronic Notes in Theoretical Computer Science **64** (2002).

[4] Hanus (ed.), M., *Curry: An integrated functional logic language. (version 0.8.2)*, `http://www.informatik.uni-kiel.de/~mh/curry/report.html` (2006).
URL http://www.informatik.uni-kiel.de/~mh/curry/report.html

[5] López-Fraguas, F. J. and J. Sánchez-Hernández, $\mathcal{TOY}$ *a multiparadigm declarative system*, in: M. R. Paliath Narendran, editor, *Proc. RTA-99*, LNCS 1631 (1999), pp. 244–247.

[6] Naish, L., *A declarative debugging scheme*, Journal of Functional and Logic Programming **3** (1997).

[7] Naish, L. and T. Barbour, *A declarative debugger for a logical-functional language*, in: G. Forsyth and M. Ali, editors, *Eight International Conference on Industrial and Engineering Applications of Artificial Intelligence and Expert Systems - Invited and additional papers*, 1995, pp. 91–99.

[4] http://danae.uni-muenster.de/\char'~lux/curry

[8] Peyton Jones, S. L., editor, "Haskell 98 Language and Libraries The Revised Report," Cambridge University Press, 2003.
URL http://haskell.org/onlinereport/

[9] Pope, B., *Declarative debugging with Buddha*, in: V. Vene and T. Uustalu, editors, *Advanced Functional Programming, 5th International School, AFP 2004*, LNCS **3622** (2005), pp. 273–308.

[10] Pope, B. and L. Naish, *Practical aspects of declarative debugging in Haskell-98*, in: *Fifth ACM SIGPLAN Conference on Principles and Practice of Declarative Programming*, 2003, pp. 230–240.

[11] Shapiro, E. Y., *Algorithmic program debugging*, in: *Proc. POPL'82* (1982), pp. 299–308.

[12] Wadler, P., *Comprehending monads*, Mathematical Structures in Computer Science **2** (1992), pp. 461–493.

[13] Wadler, P., *The essence of functional programming*, in: *Proc. POPL'92*, 1992, pp. 1–14.

A New Proposal for Debugging Datalog Programs[1]

R. Caballero[a,2] Y. García-Ruiz[a,3] F. Sáenz-Pérez[b,4]

[a] *Departamento de Sistemas Informáticos y Computación*
Universidad Complutense de Madrid
Madrid, Spain

[b] *Departamento de Ingeniería del Software e Inteligencia Artificial*
Universidad Complutense de Madrid
Madrid, Spain

Abstract

In this paper, we propose to apply declarative debugging to Datalog programs. Our approach relies on program semantics rather than on the computation mechanism. The debugging process starts when the user detects an unexpected answer. By asking questions about the intended semantics, the debugger looks for incorrect program relations. While usual declarative debuggers for other languages are based on computation trees, we show that graphs are more convenient structures for representing Datalog computations. The theoretical framework is complemented by the implementation of a debugger for the deductive database system DES, a publicly available open-source project.

Keywords: Declarative Debugging, Datalog Programs.

1 Introduction

The declarative programming paradigm is targeted to raise the semantic level of programs, therefore isolating them from the computation model. Thus, programmers are intended to focus on a higher semantic level rather than on the level corresponding to the underlying computation procedures.

Deductive database languages such as Datalog [9], which inherit the declarative nature of the Logic Programming language Prolog [15], increase the gap between the program semantics and the computations because the computation model of Datalog is much more intricate than that of Prolog. Prolog computation model is based on the SLD resolution principle [7], which deals with SLD computation trees, whereas Datalog computation model is based on a number of proposals, ranging

[1] This work has been funded by the projects TIN2005-09207-C03-03 and S-0505/TIC/0407.

[2] Email: rafa@sip.ucm.es

[3] Email: ygarciar@fdi.ucm.es

[4] Email: fernan@sip.ucm.es

from interpreters [17] to compilation to Prolog using magic sets [3]. The first proposal needs some sort of fixpoint computations to solve queries, which is not the computation model the user might have in mind. The second proposal makes things even worse, in the sense that programs are transformed into another one which will be executed with SLD resolution. This semantic gap between program semantics and program execution makes debugging Datalog programs a hard task if one tries to use existing tools for debugging in a quite different level the user thinks about (for instance, using a trace debugger in the level of the transformed program).

Our approach to debug Datalog programs is anchored to the semantic level, which is a natural requirement every user imposes to development systems. We propose a novel way of applying *declarative debugging*, also called Algorithmic Debugging (a term first coined in the logic programming field by E.H. Shapiro [12]) to Datalog programs, allowing to debug queries and diagnose *missing* (an expected tuple is not computed) as well as *wrong* (a given computed tuple should not be computed) answers with the same tool.

What a Datalog programmer would find useful is to catch program rules or relations which are responsible for a mismatch between the intended semantics of a query and its actual computed semantics. Our system, by means of a question-answering procedure which starts when the user detects an unexpected answer for some query, looks for those errors pointing to the program fragment responsible for the incorrectness. For this procedure, we propose to use *computation graphs* as a novel data structure for declarative debugging of Datalog programs. We find that these graphs are more convenient for modeling program computations, instead of computation trees, which have been typically used in declarative debuggers for other languages (e.g., Prolog [12], Java [5] and $\mathcal{T}oy$ [4]).

With this aim we have implemented a working prototype for DES [11], a Datalog system publicly available as an open-source project which was released in 2004, and it has been mainly used in several universities for teaching deductive databases since then. The current version with debugging capabilities can be downloaded and tested on almost any platform.

The few existing proposals for debugging Datalog programs are usually based on "imperative" debugging, that try to follow the computation model to find bugs. These proposals are mainly based on forests of proof trees [2,18,14], which makes debugging a trace-based task not so amenable to users. To our knowledge, the very first work on this setting is due to [10], but a variant of SLD resolution is used by the user to look for program errors, therefore imposing to traverse at least as many trees as particular answers are obtained for any query. In a database framework, where the answer can contain many individual values, this makes the task of debugging quite cumbersome. In our setting, we deal with the set of values for a query as a single entity, therefore reducing the complexity of the debugging task.

This paper is organized as follows. Section 2 introduces some definitions. Our proposal of computation graph is defined in 3. Section 4 discusses how the computation graph can be used for debugging. Section 5 presents the debugger tool integrated in the system DES. Finally, Section 6 summarizes conclusions from this work and points out some future work.

2 Datalog Preliminaries

Definitions for Datalog mainly come from the field of Logic Programming. In this section, we only introduce the concepts which are needed in our setting, referring the reader to [7] for a more general presentation of Logic Programming.

We consider (recursive) Datalog programs with stratified negation [1,17], i.e., normal logic programs without function symbols. Stratification is imposed to ensure a clear semantics when negation is involved, and function symbols are not allowed in order to guarantee termination of computations, a natural requirement with respect to a database user.

A *term* is either a variable or a constant symbol. An *atom* is $p(t_1, \ldots, t_n)$, where p is an n-ary predicate symbol and t_i are terms, $1 \leq i \leq n$, which can also be written as $p(\bar{t}_n)$. A *literal* is either an atom or a negated atom. A *positive* literal is an atom, and a *negative* literal is a negated atom. A negated atom is syntactically constructed as $not(A)$, where A is an atom. The atom contained in a literal L will be denoted as *atom(L)*. A *rule* R is an expression of the form $A : -L_1, \ldots, L_n$, where A is an atom and L_i are literals. All of the variables in a rule are assumed to be universally quantified. Concerning the rule R, A is referred to as the *head* of R, $L_1, \ldots, L_n$ as its *body*, and L_i as *subgoals*. Commas in bodies stand for conjunctions. A *fact* is a rule with empty body and *ground* head. The symbol $: -$ is usually dropped in this case. A *Datalog program* is a finite set of Datalog facts and rules. In order to fit with database notation, the term *relation* is used *in lieu* of *predicate*. A database *relation* is, therefore, a set of rules with the same predicate symbol and arity. If a relation is only defined with facts, it is called an *extensional-database* relation (EDB), whereas it is otherwise called an *intensional-database* relation (IDB). A *query* (term preferred in a deductive database context) or *goal* (term preferred in a logic programming context) is a literal (i.e., an atom or a negated atom) which can be solved by a Datalog system with respect to a given program. Analogous to literals, we say that a *positive* query is an atom, and a *negative* query is a negated atom. In contrast to facts, queries may contain variables.

Substitutions are defined as usual in logic programming. *Subst* denotes the set of all the substitutions. We also assume the existence of a composition operation between substitutions defined in the usual way and fulfilling the property $(s\theta)\sigma = s(\sigma \cdot \theta)$ for all $\sigma, \theta \in Subst$. Two formulae φ, φ' are *variants* if $\varphi = \varphi'\theta$ with θ a renaming. We use the notation *fresh(φ)* to represent a renaming of the formula φ which replaces all its variables by new variables.

Datalog programs resemble Prolog programs as the program (adapted from [19]) in Figure 1 suggests, which will be used as a running example for the rest of the paper. In Datalog programs, variables start with uppercase letters whereas constants start with lowercase letters (e.g., `X` and `nil`, respectively, in the example). Predicate symbols start with lowercase letters (e.g., `between`). As a matter of notation for runnable programs under the system DES, we use a true type font for programs. Code remarks start with `%` and apply up to the end of line.

The example program is intended to compute the parity of a given base relation `br(X)`, i.e., it can determine whether the number of elements in the relation (cardinality) is even or odd by means of the relations `br_is_even`, and `br_is_odd`,

```
% Pairs of non-consecutive elements in br
between(X,Z) :-       br(X), br(Y), br(Z), X<Y, Y<Z.

% Consecutive elements in the sequence, starting at nil
next(X,Y) :-          br(X), br(Y), X<Y, not(between(X,Y)).
next(nil,X) :-        br(X), not(has_preceding(X)).

% Values having preceding values in the sequence
has_preceding(X) :-   br(X), br(Y), Y>X. % error: it should be Y<X

% Values in an even position of the sequence, including nil
even(nil).
even(Y) :-            odd(X), next(X,Y).

% Values in an odd position of the sequence
odd(Y) :-             even(X), next(X,Y).

% Succeeds if the cardinality of the sequence is even
br_is_even :-         even(X), not(next(X,Y)).

% Succeeds if the cardinality of the sequence is odd
br_is_odd :-          odd(X), not(next(X,Y)).

% Sequence
br(a).
br(b).
```

Fig. 1. Example of Datalog program

respectively. The relation `next` defines an ascending chain of elements in `br` based on their textual ordering, where the first link of the chain connects the distinguished node `nil` to the first element in `br`. The relations `even` and `odd` define the even, resp. odd, elements in the chain. Finally the relation `has_preceding` defines the elements in `br` such that there are previous elements to a given one (the first element in the chain has no preceding elements). The rule defining this relation includes an intended error (fourth rule in the example) which will be used in forthcoming sections to show how it is catched by the declarative debugger. The symbol $<$ denotes a built-in relation checking if some element is less than another w.r.t. the predefined term ordering. Observe that relations `br_is_even`, and `br_is_odd` are not *range restricted* because variable Y occurs only in a negative literal, and that therefore the program does not fulfill the usual *safety conditions* [17]. In fact, our setting does not enforces the use of safe programs. Only the stratification requirement is needed for the correctness of the debugging technique.

The semantics (i.e., the "meaning") of a Datalog program can be given by either the model-theoretic, proof-theoretic or fixpoint semantics. We focus on the model-theoretic semantics. In particular, we consider *Herbrand interpretations* and *Herbrand models*, i.e., Herbrand interpretations that make every Herbrand instance of the program rules logically true formulae.

Given a Herbrand model $\mathcal{M}$, we define the meaning of a query Q in the context of a program P as the set:

$$Q_{\mathcal{M}} = \{Q\theta \in \mathcal{M}\}$$

with $\theta \in Subst$. We call $Q_{\mathcal{M}}$ the *answer* for Q w.r.t. P.

In Logic Programming languages such as Prolog, the least Herbrand model cannot be actually computed, because programs in general are non-terminating. Also the use of *negation as failure* contributes to the lack of completeness of Prolog computations. However, due to the use of stratified programs in Datalog, the existence of a so-called *supported model* $\mathcal{M}$ is ensured [1]. $\mathcal{M}$ is a minimal Herbrand model of program P that can be actually computed by a Datalog system. Thus, we assume that our Datalog system implementation yields the value $Q_{\mathcal{M}}$ for any query Q.

Additionally, we use the term *intended interpretation* represented by $\mathcal{I}$, to denote the model the user has in mind for the program. The *intended* answer for a query Q is accordingly defined as:

$$Q_{\mathcal{I}} = \{Q\theta \in \mathcal{I}\}$$

Then, the user can focus on queries and compare the intended interpretation to the minimal Herbrand model actually computed. We therefore speak of *validity* of a computed query w.r.t. the intended model of a program when:

$$Q_{\mathcal{M}} = Q_{\mathcal{I}}$$

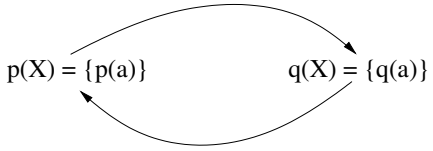
If given a program P we find some query Q s.t. $Q_{\mathcal{M}} \neq Q_{\mathcal{I}}$, we have that P is an *incorrect program*, which must include one or more *incorrectly defined relations*.

3 Computation Graphs

In this section, we define a suitable structure for representing Datalog computations. Usually in logic programming languages such as Prolog, the computations are represented through some tree structure such as the SLD-tree [7]. In the case of Datalog, we claim that a tree is not a convenient structure due to the different treatment of recursive programs. For instance consider the program:

```
p(a).
r(b).
p(X) :- q(X),r(X).
q(X) :- p(X).
```

In Prolog, the SLD-tree for the goal `p(X)` will contain an infinite branch, representing a non-terminating computation. However in Datalog the same goal is terminating and returns the finite answer: {`p(a)`} because the computation mechanism detects the repetition of the subgoal `p(x)` and avoids the infinite loop. Thus, our computation structure must represent finitely these situations, which can be achieved by using a graph:



The graph contains the two subgoals occurred during the computation together with their respective answers. It also indicates that `p(X)` and `q(X)` are mutually dependent. We will call such graph the *computation graph* for the goal w.r.t. the

program. Observe that this graph is different from the *predicate dependency graph* [19] of the Datalog program, which show the connections between the relations from a static point of view. In the example, the dependence graph will include a vertex for the relation `r` connected to the vertex for `p`. However, our computation graph depends on the initial goal and the subgoals that occur during the computation and hence does not include any vertex for `r`.

The computation graph (CG in short) is a directed graph which only has one connected component, and that can contain cycles in the case of queries involving recursive predicates.

Each vertex of the CG contains all the information necessary for detecting its validity w.r.t. the intended model of the program. Hence the information stored at each vertex of the CG is the following:

- The query Q.
- The answer for the query.

Now we describe formally how the debugger builds a CG.

Definition 3.1 (**Computation Graph**) Given a Datalog program P and a query Q the computation graph of Q w.r.t. P is defined as the value *cg(P,Q)* defined as follows:

cg(P,Q)

Input:

- P: a stratified Datalog program.
- Q: a program query. It must be either of the form $p(\bar{a}_n)$ or $not(p(\bar{a}_n))$, with $p(\bar{a}_n)$ an atom and p a relation defined in P.

Output: a directed graph (V, E), where V is a set of vertices of the form $(Q = Q_{\mathcal{M}})$ and $E = \{(u, v) | u, v \in V\}$ is a set of edges.

Steps:

1 The first vertex v of the graph is associated to the initial query Q and is defined as $v := (p(\bar{a}_n) = \Pi)$, with $\Pi := p(\bar{a}_n)_{\mathcal{M}}$. The set Π can be obtained directly using the system to obtain the answer for the query $p(\bar{a}_n)$.

2 $V := \{v\}$, $E := \emptyset$.

3 Let A be an auxiliary set containing the vertices that must be unfolded in order to build the graph. Initially $A := \{v\}$.

4 While $A \neq \emptyset$ do:
   i) Select any vertex u in A. $A := A \setminus \{u\}$. The vertex u must be of the form $(q(\bar{b}_k) = \Pi_q)$ for some q, Π_q, $\bar{b}_k$.
   ii) Consider all the rules (disregarding facts) defining q: $R_{q_1}, \ldots, R_{q_s}$.
   For each R_{q_i}, $1 \leq i \leq s$:
   a) N := *newVertices*($q(\bar{b}_k)$,*fresh*(R_{q_i}))
   b) For each new vertex $(Q = Q_{\mathcal{M}}) \in N$ check whether exists already a vertex $(Q' = Q'_{\mathcal{M}}) \in V$ such that Q and Q' are variants. There are two possibilities:

- There exists such $(Q' = Q'_{\mathcal{M}})$. Then, $E := E \cup \{(u, (Q' = Q'_{\mathcal{M}}))\}$. That is, if the vertex already exists we simply add a new edge from u.
- Otherwise, $V := V \cup \{(Q = Q_{\mathcal{M}})\}$, $A := A \cup \{(Q = Q_{\mathcal{M}})\}$, and $E := E \cup \{(u, (Q = Q_{\mathcal{M}}))\}$.

newVertices(A,R)

Input:

- A: An atom of the form $q(\bar{b}_k)$.
- R: A program rule of the form $q_i(\bar{t}_n) :- L_1, \ldots, L_m$.

Output: a set S of vertices.

Steps:

1 If $\theta := m.g.u.(q(\bar{b}_k), q_i(\bar{t}_n))$ does not exist return $\emptyset$.

2 Otherwise, for each literal L_j in the right-hand side of rule R consider the next two possibilities:

i) $j = 1$. Then $v_1 := (atom(L_1)\theta = \Pi_1)$, where $\Pi_1 := (atom(L_1)\theta)_{\mathcal{M}}$. $S := \{v_1\}$. v_1 is the vertex associated with the first literal L_1 .

ii) If $j > 1$, let $\delta_1, \ldots, \delta_r$ be all the substitutions such that:

$$L_h\theta\delta_1 \in \mathcal{M}, \ldots, L_h\theta\delta_r \in \mathcal{M} \ , \ h = 1, \ldots, j-1$$

This means that the first $j-1$ literals have succeeded for each substitution $\theta\delta_1, \ldots, \theta\delta_r$. Then the following new r vertices associated with the literal L_j are defined

$$v_{j_1} := (\ atom(L_j)\theta\delta_1 = \Pi_{j_1}) \qquad ... \qquad v_{j_r} := (\ atom(L_j)\theta\delta_r = \Pi_{j_r})$$

with $\Pi_{j_1} := (atom(L_j)\theta\delta_1)_{\mathcal{M}}, \ \ldots, \ \Pi_{j_r} := (atom(L_j)\theta\delta_r)_{\mathcal{M}}$.

Finally set $S := S \cup \{v_{j_1}, \ldots, v_{j_r}\}$, stating that we have created r new nodes.

End of Definition

Observe that if a relation p is only defined by facts (i.e., $s = 0$ at step 4.*ii*) of the *cg* algorithm), the CG only contains one vertex, and has no edges. A CG of the query `br_is_even` w.r.t the program in Figure 1 is presented in Figure 2 (ignore the bounded and colored vertices at the moment). For instance the vertex `even(X) = { even(nil) }` includes the answer `even(nil)` for the query `even(X)`. The relation `even` is defined through one fact and one rule. The fact does not appear in the figure (they do not create new subgoals), and the rule yields two new vertices, one for each literal in its right-hand side. The first one, namely `odd(X)` , includes an answer with one result. This result has an associated substitution $\delta_1 = \{X \mapsto b\}$. The second vertex corresponds to `next(X,Y)`δ_1, which produces an empty answer. Notice that there is no edge from vertex `next(b,X) = { }` to any vertex corresponding to the relation `between`, although `between` occurs in the rhs of the first rule defining `next`. The reason is that there is not exist a substitution δ such that

$$br(b)\delta \in \mathcal{M} \ \wedge \ \ br(Y)\delta \in \mathcal{M} \ \wedge \ \ (b < Y)\delta \in \mathcal{M}$$

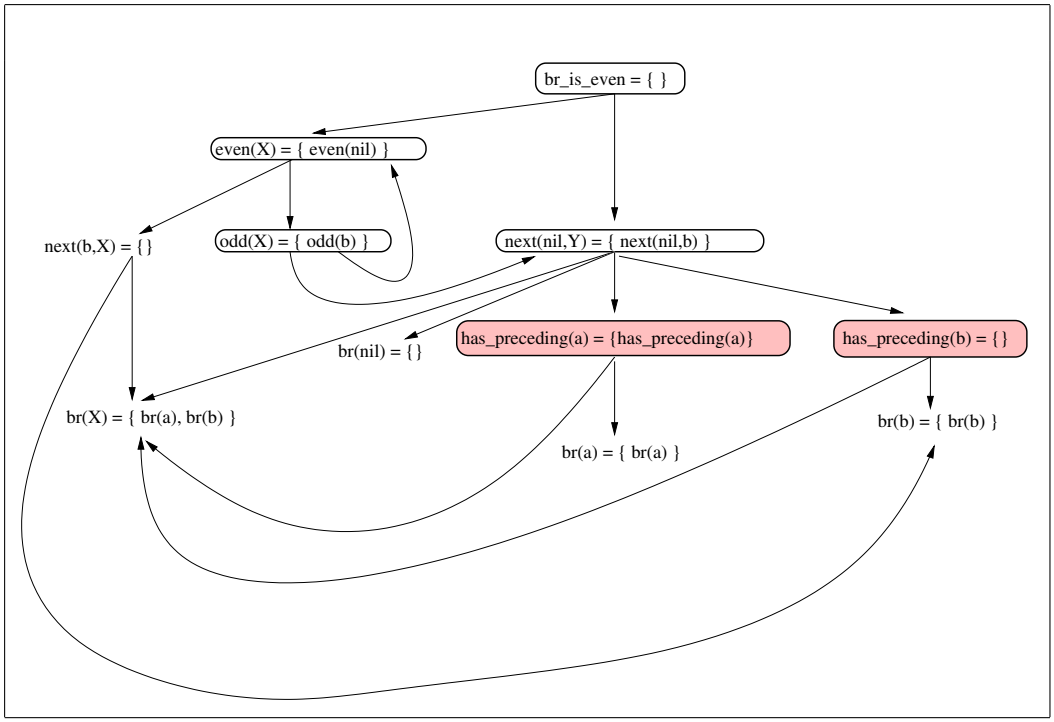


Fig. 2. Example of Computation Graph

i.e. the first three literals for this clause do not succeed for any substitution δ. Also observe that there is no vertex corresponding to the symbol $<$ in the graph, because it is a primitive built-in relation and it is hence assumed correct by the debugger.

4 Declarative Debugging with CG's

In this section, we show how the computation graph is used by the debugger in order to detect incorrectly defined relations.

The debugging process starts when the user finds some unexpected answer for a query, i.e., some *initial symptom*. For instance, in the case of the program in Figure 1, the user expects that the answer for the query `br_is_even` should be {`br_is_even`}, because the relation `br` contains two elements: `a` and `b`. However, the answer returned by the system is { }, which means that the corresponding goal was unsuccessful. Therefore, the user will start the debugger. The debugger proceeds by following the stages:

(i) First the system ensures that the computation for the goal is up to date. Then, it generates a suitable *computation graph* that represents the computation. In the case of our running example and the query `br_is_even`, the CG is displayed in Figure 2. This first phase is automatically performed by the tool.

(ii) Second, the CG obtained in the previous phase is traversed asking to the user about the validity of some vertices looking for a *buggy* vertex. A vertex is called buggy when it is non-valid but all its immediate descendants are valid. A buggy vertex always corresponds to an incorrectly defined relation, which is pointed out by the debugger as the cause of the error.

Therefore, the debugger will ask questions to the user about the validity of certain vertices of the CG w.r.t. to the intended interpretation $\mathcal{I}$ of the program P. For instance, the intended interpretation of the program of the Figure 2 is:

$$\mathcal{I} = \{ \quad \textit{br_is_even, even(nil), even(b), odd(a), next(nil,a), next(a,b),}$$
$$\textit{has_preceding(b), br(a), br(b)} \ \}$$

The debugger assumes that the user knows whether an instance of a query is in $\mathcal{I}$, i.e., that the user can determine the answer for any query. For instance, a possible question could be *Is odd(X)={odd(b)} valid?* The question must be understood as *Is {odd(b)} the expected answer for the query odd(X)?*. The answer will be `no` because the expected answer for a query *odd(X)* is {*odd(a)*}, because *a* is the only element of the sequence *a,b* that is in an odd position.

We can distinguish two reasons for detecting that a computed answer $Q_{\mathcal{M}}$ for a query Q is incorrect:

(i) There exists $\sigma \in \mathit{Subst}$ s.t. $Q\sigma \in \mathcal{M}$ but $Q\sigma \notin \mathcal{I}$. Then $Q_{\mathcal{M}}$ is called a *wrong answer*. For instance, the answer {*has_preceding(a)*} for the query *has_preceding(a)* in the CG of Figure 2 contains a wrong answer, because *a* has no preceding value in the intended interpretation.

(ii) There exists $\sigma \in \mathit{Subst}$ s.t. $Q\sigma \in \mathcal{I}$ but $Q\sigma \notin \mathcal{M}$. Then $Q_{\mathcal{M}}$ is called a *missing answer*. For instance, the vertex *even(X)*={*even(nil)*} in the Figure 2 contains a missing answer because for $\sigma = \{X \mapsto b\}$ we have $even(X)\sigma = even(b)$, and $even(b) \in \mathcal{I}$ but $even(b) \notin \mathcal{M}$.

Observe that the two errors can exist at the same time: $(odd(X) = \{odd(b)\}$ is both a wrong answer ($odd(b)$ should not be in the answer) and a missing answer ($odd(a)$ should be in the answer). Declarative debuggers usually require the user to distinguish both types of errors in order to initiate the debugging process. These types of errors can require even different types of different computation structures. An advantage of our approach is that this distinction is not needed, and that the same structure, the CG is valid for both types of errors.

In fact, our approach does not match the general scheme proposed by L. Naish in [8] for declarative debugging, because it is based on a graph rather than on a tree. However, some of the basic results are still valid. For instance the *correctness* of the technique:

Theorem 4.1 *Any buggy vertex in a CG corresponds to an incorrectly defined relation.*

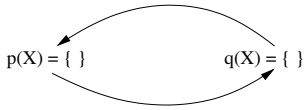
The informal reasoning behind this result is easy: since a buggy vertex is an incorrect vertex, this means that it contains an incorrect answer for its associated query. Also, in the graph construction algorithm, it can be checked that the immediate descendants of the vertex are the subqueries whose result were needed to produce such incorrect answers. But, if the subqueries returned correct values, the error must come from the relation itself, which is therefore incorrectly defined.

In the CG of the Figure 2 the incorrect vertices are surrounded by a box, while the two buggy vertices are contained in a colored box. Both buggy vertices correspond to the relation `has_preceding`, which corresponds to the only incorrectly

defined relation of the program of Figure 1.

Another nice property of the general scheme based on computation trees is completeness: every computation tree with an incorrect node contains a buggy node. Unfortunately, this result does not hold in our setting. The reason is directly related to Datalog semantics. Consider for instance the program:

```
p(X) :- q(X).
q(X) :- p(X).
```



The CG, displayed at the right of the program will contain two vertices, both displaying the empty answer. Imagine also that either p or q is an incorrectly defined relation, because the user forgot to include in the program either the fact *p(a)* or *q(a)*. In either of theses cases $\mathcal{I} = \{p(a), q(a)\}$. Then, we will have a CG with two incorrect vertices and with no buggy vertex. In this case, our debugger will not point out any relation, but a *set of connected relations* as the cause of the error. Fortunately, this situation that leads to less informative diagnosis, is not usual in common Datalog programs.

5 Implementation

The Datalog Educational System (DES) is an open-source free Prolog-based implementation of a basic deductive database with stratified negation with Datalog as a query language. The system is implemented on top of Prolog and can be used from several Prolog interpreters (Ciao Prolog, GNU Prolog, SICStus Prolog, and SWI Prolog) running on several operating systems (OSs). Moreover, executables for several OSs (Windows 98 and later, and SunOS/Solaris) are also provided. It was aimed to have a simple, interactive, multiplatform, and affordable system for students and researchers. DES 1.3.0 is the current release, which enjoys full recursive evaluation with memoization techniques and stratified negation. DES is implemented with the seminar ideas found in [6,16], that deal with termination issues of Prolog programs. These ideas have been already used in the deductive database community. Our implementation uses the concept of *extension table* for achieving a top-down driven bottom-up approach. In its current form, it can be seen as an extension of the work in [6] in the sense that, in addition, we deal with negation and undefined (although incomplete) information (cfr. [11] for further details about undefinedness). Once a query is computed, the extension table holds the computed meaning of the program restricted to the query, i.e., only the meanings of needed relations for computing the meaning of the query are computed.

We have implemented a debugger tool in the DES system based on the ideas presented in previous sections. Next, we describe some of its features.

As we have seen in Section 4, the debugging process consists of two phases. During the first phase, the tool builds a CG for the initial query Q w.r.t. the program P. This phase, in turn, can be divided into two parts:

(i) The debugger uses the system DES in order to produce the *extension table* for Q w.r.t. P.

(ii) The CG is built following the description given in Section 3. The answers

Π at each vertex are obtained from the extension table. This corresponds to assuming that the system computes the supported Herbrand model $\mathcal{M}$. As we explained in Section 1, this is possible due to the requirements of stratification imposed to our Datalog programs.

The second phase consists of traversing the CG in order to find either a buggy vertex or a set of related incorrect vertices. The vertex associated to the initial query Q is marked automatically as *non-valid* by the debugger. The rest of the vertices are marked initially as *unknown*. In order to minimize the number of questions asked by a declarative debugger, several traversing strategies have been studied [4,13]. However, these strategies are only valid for declarative debuggers based on trees and not on graphs and new strategies are still to be investigated for this structure. Nevertheless, the currently implemented strategy already contains some ideas of how to minimize the number of questions in a CG:

- It firstly asks about the validity of vertices that are not part of a cycle, in order to find a buggy vertex if it exists. Only when this is no longer possible the vertices that are part of cycles are visited.
- Each time the user indicates that a vertex ($Q = \Pi$) is valid, i.e., the validity of the answer Π for the subquery Q is ensured, the tool changes to *valid* all the vertices with associated queries of the form $Q\theta$, with $\theta \in Subst$.
- Each time the user indicates that a vertex ($Q = \Pi$) is non-valid, the tool changes to *non-valid* all the vertices with associated queries Q', with $Q = Q'\theta$, $\theta \in Subst$.

The two last items help to reduce the number of questions deducing automatically the validity/non-validity of some vertices from the validity/non-validity of others. For instance, in Figure 2, the validity of the vertices containing *br(a)*={*br(a)*} and *br(nil)*={} can be deduced automatically from the validity of the vertex *br(X)*= {*br(b),br(a)* }. The soundness of these deductions is established by the following proposition:

Proposition 5.1 *Let Q be a query, let $Q_{\mathcal{I}}$ be the answer of Q w.r.t. the intended interpretation $\mathcal{I}$, and let θ be a ground substitution, $\theta \in Subst$. Then:*

(i) *If $Q = Q_{\mathcal{I}}$ is valid, then $Q\theta = Q\theta_{\mathcal{I}}$ is valid.*

(ii) *If $Q = Q'\theta$ for some Q' and $Q = Q_{\mathcal{I}}$ is non-valid, then $Q' = Q'_{\mathcal{I}}$ is non-valid.*

Proof.

(i) If $Q = Q_{\mathcal{I}}$ is valid, then for all $\sigma \in Subst$, $Q\sigma \in \mathcal{I} \iff Q\sigma \in \mathcal{M}$. Thus, for all σ', $(Q\theta)\sigma' \in \mathcal{I} \iff (Q\theta)\sigma' \in \mathcal{M}$ simply considering $\sigma = (\sigma' \cdot \theta)$.

(ii) If $Q = Q'\theta$ for some Q' and $Q = Q_{\mathcal{I}}$ is non-valid, then there exists $\sigma \in Subst$ s.t. one of the two possibilities hold:
   (a) $Q\sigma \in \mathcal{M}$ but $Q\sigma \notin \mathcal{I}$.
   (b) $Q\sigma \in \mathcal{I}$ but $Q\sigma \notin \mathcal{M}$.
   Hence, $Q_{\mathcal{M}}$ is either a wrong or a missing answer. By defining $\sigma' = (\sigma \cdot \theta)$ we have that $Q'\sigma'$ is also a wrong or missing answer.

□

A debugger session for the query `br_is_even` of our running example:

```
DES> /debug br_is_even
Debugger started ...
Is br(b) = {br(b)}  valid(v)/non-valid(n) [v]? v
Is has_preceding(b) = {}  valid(v)/non-valid(n) [v]? n
Is br(X) = {br(b),br(a)}  valid(v)/non-valid(n) [v]? v

! Error in relation:  has_preceding/1
! Witness query: has_preceding(b) = { }
```

In this particular case, only three questions are necessary to find out that the relation *has_preceding* is incorrectly defined.

6 Conclusions and Future Work

In the previous sections we have presented a framework for the declarative debugging of Datalog programs. The proposed technique finds incorrect relation definitions in Datalog programs by comparing the results of the computations to the intended interpretation of each relation, which is assumed to be known by the user. Thus, our technique relies on the program semantics for debugging, disregarding the implementation issues. In Datalog this is not only and advantage; it is almost a necessity. The Datalog computations are based on operational features or program transformations that make the execution very difficult to follow and understand using the normal trace facilities included in logic languages such as Prolog.

We have also defined a suitable structure for representing the computations, the computation graphs. This represents a novelty w.r.t. the traditional presentation of declarative debugging, which is based on trees rather than on graphs. Nevertheless, declarative debugging using computation graphs lacks some of the nice properties of usual declarative debugging such as completeness. We have shown that indeed in some Datalog programs is not possible to point out a single relation as the cause of an unexpected computation result, and that in those programs the debugger can only detect sets of mutually dependent relations as possible error sources, meaning that one or more of these relations are incorrectly defined. However, these situations are not common in practice.

It is important to emphasize that the debugger can be used for diagnosing errors starting either from a wrong or from a missing answer. Since Datalog programs are terminating, we can claim that the presented technique covers *all* the possible errors that produce unexpected answers in Datalog programs. This makes a difference w.r.t. other declarative debuggers that are limited to a particular kind of errors (i.e., only missing or only wrong answers). The ideas have been implemented in a working prototype included as part of the Datalog system DES.

As future work we plan to represent graphically the computation graph. This will help the user to find the error more easily, inspecting the graph and choosing freely the more convenient vertices to start the debugging process. Another improvement can be obtained by allowing the user to provide more informative answers. For instance, if the debugger knows that an answer is not only non-valid but *wrong*, i.e., it contains an unexpected atom, it can use this information to skip some questions; in particular, the questions involving children with empty answers, which are always

valid w.r.t. wrong answers.

References

[1] Apt, K. R., H. A. Blair and A. Walker, *Towards a theory of declarative knowledge* (1988), pp. 89–148.

[2] Arora, T., R. Ramakrishnan, W. G. Roth, P. Seshadri and D. Srivastava, *Explaining program execution in deductive systems*, in: *Deductive and Object-Oriented Databases*, 1993, pp. 101–119.

[3] Beeri, C. and R. Ramakrishnan, *On the power of magic*, 1987, pp. 269–284.

[4] Caballero, R., *A declarative debugger of incorrect answers for constraint functional-logic programs*, in: *WCFLP '05: Proceedings of the 2005 ACM SIGPLAN workshop on Curry and functional logic programming* (2005), pp. 8–13.

[5] Caballero, R., C. Hermanns and H. Kuchen, *Algorithmic Debugging of Java Programs*, Electronics Notes in Theorical Computer Science (2007), in Press.

[6] Dietrich, S. W., *Extension tables: Memo relations in logic programming.*, in: *SLP*, 1987, pp. 264–272.

[7] Lloyd, J., "Foundations of Logic Programming," Springer Verlag, 1984.

[8] Naish, L., *A Declarative Debugging Scheme*, Journal of Functional and Logic Programming **3** (1997).

[9] Ramakrishnan, R. and J. Ullman, *A survey of research on Deductive Databases*, The Journal of Logic Programming **23** (1993), pp. 125–149.

[10] Russo, F. and M. Sancassani, *A declarative debugging environment for Datalog*, in: *Proceedings of the First Russian Conference on Logic Programming* (1992), pp. 433–441.

[11] Sáenz-Pérez, F., *Datalog Educational System. User's Manual*, Technical Report 139-04, Facultad de Informática, Universidad Complutense de Madrid (2004), `http://des.sourceforge.net/`.

[12] Shapiro, E., "Algorithmic Program Debugging," ACM Distiguished Dissertation, MIT Press, 1982.

[13] Silva, J., *A Comparative Study of Algorithmic Debugging Strategies*, in: *Proc. of International Symposium on Logic-based Program Synthesis and Transformation LOPSTR 2006*, 2007, pp. 134–140.

[14] Specht, G., *Generating explanation trees even for negations in deductive database systems*, in: *Proceedings of the 5th Workshop on Logic Programming Environments*, Vancouver, Canada, 1993.

[15] Sterling, L. and E. Shapiro, "The art of Prolog: advanced programming techniques," MIT Press, Cambridge, MA, USA, 1986.

[16] Tamaki, H. and T. Sato, *Old resolution with tabulation*, in: *Proceedings on Third international conference on logic programming* (1986), pp. 84–98.

[17] Ullman, J., "Database and Knowledge-Base Systems Vols. I (Classical Database Systems) and II (The New Technologies)," Computer Science Press, 1995.

[18] Wieland, C., *Two explanation facilities for the deductive database management system DeDEx.*, in: H. Kangassalo, editor, *ER* (1990), pp. 189–203.

[19] Zaniolo, C., S. Ceri, C. Faloutsos, R. T. Snodgrass, V. S. Subrahmanian and R. Zicari, "Advanced Database Systems," Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1997.

Inferring static non-monotonically sized types through testing

Ron van Kesteren, Olha Shkaravska, Marko van Eekelen[1,2]

Institute for Computing and Information Sciences
Radboud University Nijmegen

Abstract

We propose a size analysis procedure that combines testing and type checking to automatically obtain static output-on-input size dependencies for first-order functions. Attention is restricted to functions for which the size of the result is strictly polynomial, not necessarily monotonic, in the sizes of the arguments. To infer a size dependency, the procedure generates hypotheses for increasing degrees of polynomials. For each degree, a polynomial is defined by a finite number of points. Based on interpolation theory, in this paper we establish an upper bound on the number of test runs and a correct choice of test data that guarantees that all polynomials representing sizes of output lists can be found. The resulting hypothesis is then checked using an existing type checking procedure. The procedure is not tied to the current sized type checker. The sized type of a function will be inferred if it exists and if it is accepted by the sized type checker. For terminating functions, our sized type inference procedure is complete with respect to type checking.

Keywords: Memory complexity analysis, type checking, testing

1 Introduction

Embedded systems or server applications often have limited resources available. Therefore, it can be important to know in advance how much time or memory a computation is going to take, for instance to determine how much memory should at least be put in a system to enable all desired operations. Such decisions can only reliably be based on formally verified upper bounds of the resource consumption. However, an advanced detailed analysis of these bounds requires knowledge of the sizes of the data structures used throughout the program [ESvK+07]. Trivially, the time it takes to iterate over a list depends on the size of that list. In this paper we focus on the task of automatically deriving the exact output-on-input size dependencies of functions. The ratio behind exactness is explained later in this section. A possible relaxation of it is considered in Section 5. Size dependencies

[1] Email:R.vanKesteren@cs.ru.nl, O.Shkaravska@cs.ru.nl, M.vanEekelen@cs.ru.nl

[2] This research is sponsored by the Netherlands Organisation for Scientific Research (NWO), project Amortized Heap Space Usage Analysis (AHA), grantnr. 612.063.511.

can be represented in function . We focus on shapely functions, where shapely means that the size relations are exactly polynomial (not necessarily monotonic). As an example, consider the function that computes the Cartesian product of two lists. It generates all pairs of elements, one taken from the first list, the other from the second.

```
pairs x []       =  []
pairs x (y:ys)   =  [x,y]:pairs x ys

cprod []     ys  =  []
cprod (x:xs) ys  =  pairs x ys ++ cprod xs ys
```

The size of a list is its number of nodes (its length). Given lists of size 3 and 2, the output is a list of size $3 * 2 = 6$ whose elements are pairs, i.e., lists of size 2.

```
cprod [1,2,3] [4,5] = [[1,4],[1,5],[2,4],[2,5],[3,4],[3,5]]
```

The of the `cprod` function expresses the relation between its argument and result sizes. When two input lists have size s_1 and s_2 respectively, the output is a list of lists, with an outer list of size $s_1 * s_2$ and inner lists all of size 2.

$$\texttt{cprod} : \texttt{[Int]}^{s_1} \longrightarrow \texttt{[Int]}^{s_2} \longrightarrow \texttt{[[Int]}^{2}\texttt{]}^{s_1 * s_2}$$

In general, all lists at the input side, before the arrow, have an associated size variable. After the arrow, at the output side, all lists have an associated polynomial that determines the size of the output list. These polynomials are defined in terms of the input size variables. The current presentation is limited to a language over lists for reasons of simplicity; sized types are straightforwardly generalized to general data structures and other programming languages. Recently, we have developed a sized type checking algorithm to formally verify polynomially sized types (section 2) [SvKvE07a]. Given a sized type, the algorithm automatically checks if the function definition satisfies that type. Unfortunately, inferring such types is a lot more challenging than type checking and the type system approach does not straightforwardly extend (section 2.3). Therefore, we have suggested an alternative method of inferring sized types [SvKvE07a]. This paper develops this method into a practical type inference procedure. The method is based on the observation that it is relatively easy to generate hypotheses for an size dependency by testing. Exactness (or strictness) of sizes makes it possible to place sizes of run-time tests exactly on the dependency graph. Because a polynomial of a given degree is determined by a finite number of values, its coefficients can be computed from the output sizes of run-time tests (figure 1). If the size expression is indeed a polynomial of that degree, it can be only polynomial. This theory is used to create a practical

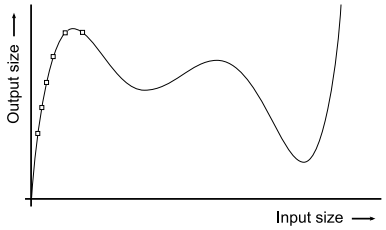


Fig. 1. A fifth degree polynomial is determined completely by any six of its points.

procedure that yields hypotheses for sized types (section 3). Combining hypothesis generation and type checking yields an procedure that can infer the sized type of a function (section 4). The procedure generates hypotheses for an increasing degree. For each degree, hypotheses for all polynomial size expressions in the output type are determined. The resulting sized type is checked using the sized type checking procedure. Thus:

(i) Infer the underlying type (without sizes) using standard type inference;

(ii) Annotate the underlying type with size variables;

(iii) Assume the degree of the polynomial;

(iv) For every output size: determine which tests are needed, do the required series of test runs and compute the polynomial coefficients based on the test results;

(v) Annotate the type with the size expressions found;

(vi) Check the annotated type;

(vii) If checking fails, repeat from step 4 assuming a higher degree.

Indeed, for terminating programs the procedure is only guaranteed to find the sized type if it exists. In practice, an upper limit on the degree can be used as a stopping criterium. Note that the procedure can also be applied with any other type checker for polynomially sized types. The main contribution of this paper is developing the method suggested in [SvKvE07a] into a practical sized type inference procedure. Specifically, this means dealing with cases where the function definition only - *fi* the output size polynomial: when the output type is a nested list and the output value is the empty list, there is no information on the sizes of the inner lists, like, for instance, in the case $[[\texttt{Int}]^{?}]^{0}$. We adopted the results from interpolation theory [CL87], on existence and uniqueness of Lagrange interpolations, to define sets of test data that determine output polynomials in the right way.

2 Sized type checking

Essentially, our approach to sized type inference for shapely functions is based on reducing inference to sized type checking. This section briefly describes the existing strict size-aware type system for a functional language and accompanying type checking procedure [SvKvE07a] that we use in the inference procedure. This also motivates our approach to type inference.

The zero-order types we consider are integers, sized lists of integers, sized lists of sized lists, etc. A strict list of length n is a list exactly of length n (not of some length up to n, as, for instance in sized types of Pareto [Par98]). For lists of lists the element lists have to be of the same size and in fact it would be more precise to speak about matrix-like structures. For instance, the type $[[\texttt{Int}]^{3}]^{2}$ is given to a list which two elements are both lists of exactly three integers, such as

```
[[2,5,3], [7,1,6]].
```

$$\tau ::= \texttt{Int} \mid \alpha \mid \texttt{[}\tau\texttt{]}^{p} \qquad \alpha \in$$

Here p denotes a , i.e. a polynomial in size variables.

$$::= \mathbb{N} \mid s \mid + \mid - \mid * \qquad s \in$$

For instance, type $\texttt{[}\alpha\texttt{]}^{4}$ represents a list containing four elements of some type α and $\texttt{[Int]}^{(s_1-s_2)^2}$ represents a list of integers of size $(s_1 - s_2)^2$ where s_1 and s_2 are size variables. Size expressions are subject to the standard associativity, commutativity and distributivity laws for addition and multiplication. Types with negative sizes have no meaning. Because the current system does not support Currying, first-order types are functions from tuples of zero-order types to zero-order types.

$$\tau^{f} ::= \tau_1 \ \ldots \ \tau_n \rightarrow \tau_{n+1}$$

For example, the type of `cprod`, $\texttt{[Int]}^{s_1} \rightarrow \texttt{[Int]}^{s_2} \rightarrow \texttt{[[Int]}^{2}\texttt{]}^{s_1*s_2}$ is a first-order type. In well-formed first-order types, the argument types are annotated only by size variables and the result type is annotated by size expressions in these variables. Type and size variables occurring in the result type should also occur in at least one of the argument types. Thus, the type of `cprod` is a well-formed type, whereas $\texttt{[}\alpha\texttt{]}^{s_1+s_2} \rightarrow \texttt{[}\alpha\texttt{]}^{2*s_1}$ is not, because the argument is annotated by a size expression that is not a variable.

Previously, we have developed a sound size-aware type system and a type checking procedure for a first-order functional language with call-by-value semantics [SvKvE07a]. The language supports lists and integers and standard constructs for pattern matching, if-then-else branching, and let-binding. The typing rules follow the intuition on how sizes are created and changed during evaluation. The construction of a list gives a list that is one element longer than its tail. The `then` and `else` parts of the if-statement are required to yield the same size. The same goes for the `nil` and `cons` branch of pattern matching, but that rule also takes into account that the matched list is known to be empty in the `nil` branch: when matching a list of size s, if the `cons` branch has size $s*4$, the `nil` branch can have size $0*4 = 0$ because, there, $s = 0$. In the formal rules, a context Γ is a mapping from zero-order program variables to zero-order types, a signature Σ is a mapping from function names to first-order types, and D is a set of Diophantine equations that keeps track of which lists are empty. A typing judgment is a relation of the form $D;\ \Gamma \ \vdash_{\Sigma} e : \tau$ which means that if the free program variables of the expression e have the types defined by Γ, and the functions called have the types defined by Σ, and the size constraints D are satisfied, then e will be evaluated to a value of type τ, if it terminates. For

example:

$$\frac{D \vdash \quad = \quad' + 1}{D;\ \Gamma,\ \ :\tau,\quad :[\tau]^{p'} \ \vdash_\Sigma \mathsf{cons}(\ ,\ ):[\tau]^{p}}\ \textsc{Cons}$$

$$\frac{\Gamma(\ ) = \mathtt{Int} \qquad D;\ \Gamma \vdash_\Sigma e_t : \tau \qquad D;\ \Gamma \vdash_\Sigma e_f : \tau}{D;\ \Gamma \vdash_\Sigma \mathsf{if}\ x\ \mathsf{then}\ e_t\ \mathsf{else}\ e_f : \tau}\ \textsc{If}$$

$$\frac{\begin{array}{c} = 0,\ D;\ \Gamma,\ x:[\tau']^{p} \vdash_\Sigma e_{\mathsf{nil}} : \tau \\ \ ,\quad \notin \quad (\Gamma) \qquad D;\ \Gamma,\ \ :\tau',\ x:[\tau']^{p},\quad :[\tau']^{p-1} \vdash_\Sigma e_{\mathsf{cons}} : \tau \end{array}}{\begin{array}{l} D;\Gamma, x:[\tau']^{p} \vdash_\Sigma \mathsf{match}\ x\ \mathsf{with} \mid \mathsf{nil} \Rightarrow e_{\mathsf{nil}} \qquad\qquad :\tau \\ \qquad\qquad\qquad\qquad\quad \mid \mathsf{cons}(\ ,\ ) \Rightarrow e_{\mathsf{cons}} \end{array}}\ \textsc{Match}$$

Sized type checking eventually amounts to checking entailments of the form $D \vdash p = p'$, which means that $p = p'$ is derivable from D in the axiomatics of the ring of integers. Because p and p' are known polynomials of universally quantified size variables, comparing them is straightforward. For instance, for the `cprod` function we obtain $s_1 = 0 \vdash s_1 * s_2 = 0$ (in the `nil` branch) and $\vdash s_1 * s_2 = s_2 + (s_1 - 1) * s_2$ (in the `cons` branch). A syntactical condition that prohibits let-bindings before pattern matching was shown to be necessary and sufficient to make type checking decidable for this system [SvKvE07a].

Type inference in this type system is not straightforward. Applying the typing rules to types with unknown size expressions leads to sets of, non-linear equations. For an example we refer to the technical report [SvKvE07b]. Testing reduces inference to solving linear equations. The testing approach presented in this paper does not use the type system directly. Hypotheses for types are constructed based only on the observed behavior of the function. This avoids solving non-linear systems of equations. To validate the hypotheses we use the existing type checking algorithm (in practice, any type checker can be used). This ensures that, for terminating programs, type inference is complete with respect to the type checker.

3 Generating size hypotheses

This section develops a procedure (a "semi-algorithm") that uses run-time tests to automatically obtain a hypothesis for an output size polynomial, given its maximum degree. This hypothesis is correct if the output size is in fact a polynomial of the same or lower degree. In section 4, this is combined with the type checker from section 2 to obtain a sized type inference procedure. The essence of the problem is giving the conditions under which a set of data points has a unique polynomial interpolation and constructing an algorithm to find points satisfying these conditions. This is not trivial since for the case of nested lists we have no information on the inner list when the outer list is empty.

Looking at the sizes of the arguments and results of some tests of the `cprod` function gives the impression that the size of the outer list in the output is always the product of the sizes of the arguments. More specifically, if $p_1(s_1, s_2)$ is the size of the outer list given arguments of size s_1 and s_2, tests yielding $p_1(1,3) = 3$, $p_1(4,6) = 24$, and $p_1(3,5) = 15$ may be interpolated to $p_1(s_1, s_2) = s_1 * s_2$. Such a hypothesis can also be derived automatically by fitting a polynomial to the size data. We are looking for the polynomial that best approaches the data, i.e., the Lagrange interpolation. The Lagrange interpolation is unique under some conditions on the data, which are explored in polynomial interpolation theory [CL87,Lor92]. If the true size expression is polynomial and the degree of the unique Lagrange interpolation is high enough, the interpolating polynomial coincides with the true size expression. We seek a condition under which the interpolation is unique. In the well-known univariate case this is simple. A polynomial $p(x)$ of degree m with coefficients $a_1, \ldots, a_{m+1}$ can be written as follows:

$$a_1 \; + \; a_2\, x \; + \; \ldots \; + \; a_{m+1}\, x^m = \; p(x)$$

The values of the polynomial function in any $m+1$ points determine a system of linear equations w.r.t. the polynomial coefficients. More specifically, given the set $\big(x_i, p(x_i)\big)$ of pairs of numbers, where $1 \leq i \leq m+1$, and coefficients $a_1, \ \ldots \ , a_{m+1}$, the set of equations can be represented in the following matrix form, where only the a_i are unknown:

$$\begin{pmatrix} 1 & x_1 & \cdots & x_1^{m-1} & x_1^m \\ 1 & x_2 & \cdots & x_2^{m-1} & x_2^m \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_m & \cdots & x_m^{m-1} & x_m^m \\ 1 & x_{m+1} & \cdots & x_{m+1}^{m-1} & x_{m+1}^m \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \\ a_{m+1} \end{pmatrix} = \begin{pmatrix} p(x_1) \\ p(x_2) \\ \vdots \\ p(x_m) \\ p(x_{m+1}) \end{pmatrix}$$

The determinant of the left matrix, contains the measurement points, is called the Vandermonde determinant. For pairwise different points $x_1, \ldots, \ x_{m+1}$ it is non-zero. This means that, as long as the output size is measured for $m+1$ different input sizes, there exists a unique solution for the system of equations and, thus, a unique interpolating polynomial. The conditions under which there exists a unique polynomial that interpolates data are not so trivial. A polynomial of degree m and dimension n (the number of variables) has $N_m^n = \binom{m+n}{n}$ coefficients. The condition under which a set of data uniquely determines a polynomial interpolation is stated as a condition on a set of $W = \{\bar{w}_i : i = 1, \ldots, N_m^n\}$, the input sizes for which a measurement is done, such that for every set of associated measurement data $\{f_i : i = 1, \ldots, N_m^n\}$, there is a unique polynomial $p(\bar{w}) = \Sigma_{0 \leq |j| \leq m} a_j \bar{w}^j$ with total degree m which interpolates the given data at the nodes [CL87]. That is, $p(\bar{w}_i) = f_i$, where $1 \leq i \leq N_m^n$. Here $\bar{w}^j = w_1^{j_1} \ldots w_n^{j_n}$, $|j| = j_1 + \ldots + j_n$ is the usual multivariate notation. In the next subsections, node configurations that

satisfy this condition are defined, starting with bivariate polynomials and ending with the general case.

For a two-dimensional polynomial of degree m, the condition on the nodes that guarantees a unique polynomial interpolation is as follows. In the input space, there are $m+1$ lines, each containing $m+1, \ldots, 1$ of the nodes, respectively, and the nodes do not lie on the intersections of the lines. Such a configuration is depicted for parallel lines in figure 2a. This corresponds to the **NCA** configuration studied, for instance, by Chui [CL87].

Definition 3.1 [Two-dimensional node configuration] There exist lines in the input space, $\gamma_1, \ldots, \gamma_{m+1}$, such that $m+1$ nodes of W lie on γ_{m+1}, m nodes of W lie on $\gamma_m \setminus \gamma_{m+1}$, ..., and 1 node of W lies on $\gamma_1 \setminus (\gamma_2 \cup \ldots \cup \gamma_{m+1})$.

Assuming the function terminates on all inputs, such points can be found algorithmically, at least for outermost lists, using a triangle of points on parallel lines (figure 2b). An example of the two dimensional case is the `cprod` function from the introduction. Standard type inference and annotating gives the following type:

$$\texttt{cprod} : \texttt{[}\alpha\texttt{]}^{s_1} \longrightarrow \texttt{[}\alpha\texttt{]}^{s_2} \longrightarrow \texttt{[[}\alpha\texttt{]}^{p_2(s_1,s_2)}\texttt{]}^{p_1(s_1,s_2)}$$

We derive that $p_1(s_1, s_2) = s_1 * s_2$ assuming p_1 is a quadratic polynomial:

$$p_1(s_1, s_2) = a_{0,0} + a_{0,1}s_1 + a_{1,0}s_2 + a_{1,1}s_1s_2 + a_{0,2}s_1^2 + a_{2,0}s_2^2$$

Running the function at the six nodes from figure 2b gives the following results:

s_1	s_2	`x`	`y`	`cprod x y`	$p_1(s_1, s_2)$	$p_2(s_1, s_2)$
0	0	`[]`	`[]`	`[]`	0	–
1	0	`[0]`	`[]`	`[]`	0	–
0	1	`[]`	`[0]`	`[]`	0	–
1	1	`[0]`	`[1]`	`[[0, 1]]`	1	2
2	1	`[0,1]`	`[2]`	`[[0,2],[1,2]]`	2	2
1	2	`[0]`	`[1,2]`	`[[0,1],[0,2]]`	2	2

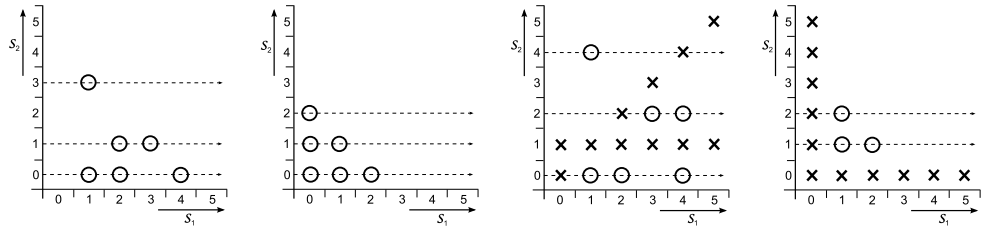


Fig. 2. (a) A node configuration that has a unique two-dimensional polynomial interpolation (b) A more systematic node configuration that has a unique two-dimensional polynomial interpolation (c) Undefined points complicate finding a node configuration (d) Undefined measurements for the pairs in the output of `cprod`.

This defines the following linear system of equations for the coefficients of p_1:

$$\begin{aligned}
a_{0,0} &= 0 \\
a_{0,0} + a_{0,1} + a_{0,2} &= 0 \\
a_{0,0} + a_{1,0} + a_{2,0} &= 0 \\
a_{0,0} + a_{0,1} + a_{1,0} + a_{0,2} + a_{1,1} + a_{2,0} &= 1 \\
a_{0,0} + 2a_{0,1} + a_{1,0} + 4a_{0,2} + 2a_{1,1} + a_{2,0} &= 2 \\
a_{0,0} + a_{0,1} + 2a_{1,0} + a_{0,2} + 2a_{1,1} + 4a_{2,0} &= 2
\end{aligned}$$

The unique solution is $a_{1,1} = 1$ with the rest of the coefficients zero. Thus, we obtain the correct $p_1(s_1, s_2)$ equal to $s_1 * s_2$. This procedure is relatively straightforward. However, there is a problem in repeating it for p_2. There are cases in which nodes have no corresponding output size (the dashes in the table). `cprod` only defines p_2, because the size of the inner lists can only be determined when there is at least one such a list. Thus, the outer list may not be empty. As can be seen in figure 2d, for `cprod` this is always the case when one of the two input lists is empty. In the next section, we show that, despite this, it is still possible to always find enough measurements and give an upper bound on the number of nodes that have to be searched.

fi

From the example in the previous section, it is clear that care should be taken when searching for hypotheses for output types with nested lists. In general, for $[\ldots[\alpha]^{p_k}\ldots]^{p_1}$ we will not find a value for p_j at a node if one of the outer polynomials, p_1 to p_{j-1}, is zero at that node. Thus, the nodes where p_1 to p_{j-1} are zero should be excluded from the testing process. Here, we show that, despite this, it is always possible to find enough nodes so that it becomes possible to construct an algorithm to find them. First note that we do not consider nested lists with the size of the outer list a constant zero, like $[[\tau]^{q}]^{0}$, because it is not a principal type. Also, remember that we are searching parallel lines $p(x, i)$ for the node configuration. Then, for any non-zero polynomial there is a finite number of lines $y = i$, which we will call , where $p(x, i) = 0$ (see lemma 3.2).

Lemma 3.2 $p(x, y)$ m 0 m $y = i,$ $p(x, i) = 0$

Proof. Suppose there are more than m root lines. Then, it is easy to pick $1, \ldots, m+1$ nodes on $m + 1$ root lines. With these nodes, at which $p(x, y) = 0$, the system of linear equations for the coefficients of p will have the zero-solution, that is, all the coefficients of p will be zeros. This contradicts the assumption that p is not constant 0. □

Because of this property, diagonal search can always find as many nodes (x, y) as desired, such that $p(x, y) \neq 0$ (see figure 2c, where roots are marked with crosses). In fact, without requiring diagonal search, we can give a limit on the number of parallel lines $y = i$ and nodes on them that have to be searched at most. Essentially, we just try to find the triangle shape (as in figure 2b) while skipping all crosses. First, we show that for a nested list type $[[\alpha]^q]^p$ with bivariate polynomial sizes q and p, only the nodes in $[0, \ldots, m_1 + m_2] \times [0, \ldots, m_1 + m_2]$ have to be searched to determine q, where m_1 and m_2 are the degrees of p and q respectively. Say one needs to find coefficients of an output type $[[\alpha]^q]^p$, and let $n = 2$ be the amount of variables, m_1 be the degree of $p(x, y)$ and m_2 be the degree of $q(x, y)$. One looks for test points for q that determine a unique polynomial interpolation at places where $p(x, y) \neq 0$. We restrict ourselves to lines γ parallel to the x-axis and we look for $(m_2 + 1)(m_2 + 2)/2$ data points satisfying the condition from definition 1.

Lemma 3.3 $q(x, y)$ $p(x, y) \neq 0,$ *ffi* $y = 0, \ldots, y = m_1 + m_2$ $[0, \ldots, m_1 + m_2] \times [0, \ldots, m_1 + m_2]$

Proof. For the configuration it is sufficient to have m_2+1 lines with at least m_2+1 points where $p(x, y) \neq 0$. Due to lemma 3.2 there are at most m_1 lines $y = i$ such that $p(x, i) = 0$, so at least $m_2 + 1$ are not root lines for p. The polynomial $p(x, j)$, with $y = j$ not a root line, has at most degree m_1, thus $y = j$ contains at most m_1 nodes (x, j), such that $p(x, j) = 0$. Otherwise, it would have been constant zero, and thus a root line. Hence, this leaves at least $m_2 + 1$ points on these lines for which p is not zero. □

This straightforwardly generalizes to all nested types with polynomials in two variables, say $[\ldots[\alpha]^{p_k}\ldots]^{p_1}$. If we want to derive the coefficients of p_i, searching the square of input values $[0, \ldots, \Sigma_{i=1}^{k} m_i] \times [0, \ldots, \Sigma_{i=1}^{k} m_i]$ suffices, where m_i is the degree of p_i. Each p_i has at most m_i root lines, so there are at most $\Sigma_{j=1}^{i-1} m_j$ root lines. Also, each of the p_i can have at most m_i zeros on a non root line. Hence, since the length of the search interval is $\Sigma_{j=1}^{k} m_j + 1$, there are always $m_i + 1$ values known. For `cprod` there are two size expressions to derive, p_1 for the outer list and p_2 for the inner lists. Deriving that $p_1(s_1, s_2) = s_1 * s_2$ is no problem. Because p_1 has roots for $s_1 = 0$ and for $s_2 = 0$, these nodes should be skipped when measuring p_2 (see figure 2d).

The generalization of the condition on nodes for a unique polynomial interpolation to polynomials in n variables, is a straightforward inductive generalization of the two-dimensional case. In a hyperspace there have to be hyperplanes, on each of which nodes lie that satisfy the condition in the $n-1$ dimensional case. A hyperplane K_j^n may be viewed as a set in which test points for a polynomial of $n-1$ variable of the degree j lie. There must be $N_j^{n-1} = N_j^n - N_{j-1}^n$ such points. The condition on the nodes is defined by:

Definition 3.4 [n-dimensional node configuration] The **NCA** configuration for n variables (n-dimensional space) is defined inductively on n [CL87]. Let $\{x_1, \ldots, x_{N_m^n}\}$ be a set of distinct points in $\mathcal{R}^n$ such that there exist $m+1$ hyperplanes K_j^n, $0 \leq j \leq m$ with

$$\begin{array}{l} x_{N_{m-1}^n+1}, \ldots, x_{N_m^n} \in K_m^n \\ x_{N_{j-1}^n+1}, \ldots, x_{N_j^n} \in K_j^n \setminus \{K_{j+1}^n \cup \ldots \cup K_m^n\}, \text{for } 0 \leq j \leq m-1 \end{array}$$

and each of set of points $x_{N_{j-1}^n+1}, \ldots, x_{N_j^n}$, $0 \leq j \leq n$, considered as points in $\mathcal{R}^{n-1}$ satisfies **NCA** in $\mathcal{R}^{n-1}$.

Thus, similarly to lines in a square in the two dimensional case, parallel hyperplanes in a hyperspace have to be searched. Using a reasoning similar to the two-dimensional case one can show that it is always sufficient to search a hypercube with sides $[0, \ldots, \Sigma_{i=1}^k m_i]$. The proof is also straightforwardly generalized.

4 Automatically inferring sized types

The type checking procedure from section 2 and the size hypothesis generation from section 3 are combined into a type increasing degree. The procedure is semi-decidable: it only terminates when the function is well-typable in the type system of the type checker used. A prototype for this procedure is under construction.

For any shapely program, the underlying type (the type without size annotations) can be derived by a standard type inference algorithm [Mil78]. After straightforwardly annotating input sizes with size variables and output sizes with size expression variables, we have for example

$$\texttt{cprod} : [\alpha]^{s_1} \rightarrow [\alpha]^{s_2} \rightarrow [[\alpha]^{p_2(s_1,s_2)}]^{p_1(s_1,s_2)}$$

To derive the size expressions on the right hand side we use the following procedure. First, the maximum degree of the occurring size expressions is assumed, starting with zero. Then, a hypothesis is generated for each size expression. This is done from the outside in, because of the problems with partially definedness noted in section 3.3. After hypotheses have been obtained for all size expressions they are added to the type and this hypothesis type is checked using the type checking algorithm. If it is accepted, the type is returned. If not, the procedure is repeated for a higher degree. Figure 3 shows the procedure in pseudo-code. Note that if

Function: TryIncreasingDegrees

Input: the function definition

Output: the sized type of that function

```
TryIncreasingDegrees(   ,   ) =
  let      = InferUnderlyingType( )
           = AnnotateWithSizeVariables(    )
      vs   = GetOutputSizeVariables(    )
           = GetSizedType(   ,   ,     ,   , [ ])
  in  if (CheckSizedType(     ,   )) then
      else TryIncreasingDegrees(   + ,   )
```

Function: GetSizedType
Input: a degree, the function definition with its annotated type, the variables to derive and the polynomials already derived
Output: the sized type of that function if the degree is high enough

```
GetSizedType(   ,   ,     , [ ],    ) =
  AnnotateWithSizeExpressions(    ,    )
GetSizedType(   ,   ,     ,   :   ,    ) =
  let      = GetNodeConf(   ,     ,    )
           = RunTests( ,     )
           = DerivePolynomial(   ,   ,     ,     )
  in  GetSizedType(   ,   ,     ,   ,   :   )
```

Fig. 3. The weak type inference procedure in pseudo-code

the assumed degree is lower than the true degree, the derived polynomials may be wrong. In that case, also the places where the size function is undefined cannot be determined correctly. It might happen that the node configuration includes points where the size expression is undefined so the test results do not provide enough information to uniquely infer the polynomial. In that case, by convention, the zero polynomial is returned. If a type is rejected, this can mean two things. First, the assumed degree was too low and one of the size expressions has a higher degree. That is why the procedure continues for a higher degree. Another possibility is that one of the size expressions is not a polynomial (the function definition is not shapely) or that the type cannot be checked due to incompleteness. In that case the procedure will not terminate. Fortunately, in practice a suitable stopping criterium may be known. If the function is well-typable, the procedure will eventually find

the correct sized type and terminate.

Based on the results of Section 3 we define a on the number of tests in the following way. Let $m =$ $\{m_1, \ldots, m_k\}$. The theoretical maximum is the upper bound $(1 + km)^n$. In practice, as a rule, the number of tests needed to define all the output polynomials for a given function is significantly closer to the theoretical lower bound defined by $\binom{m+n}{n}$ which is the number of coefficients of the polynomial with n variables and degree m. The minimum number of tests will be enough when there are no empty nested lists and the set of test data is - i.e. corresponds to an **NCA** configuration. In the tests for the `append` example this was e.g. the case. The type of `append` does not contain nested lists and the testing procedure uses a diagonal search (within subroutine GetNodeConf) which happens to yield the right configuration for this example. In practice, the number of tests is usually closer to the minimum than to the maximum since the case of nested non-empty lists occurs more frequently than the empty case. So, only a few extra tests are needed. Furthermore, the diagonalisation usually only requires a few extra test points to achieve an **NCA** configuration. The algorithm is illustrated by four functions: `cprod` (Cartesian product), `append` (standard list concatenation), `competition` (generates a competition in which every team plays a home and away match against every other team), and `sqdiff` (illustration of non-monotonicity).

```
competition  xs       =  randomize_order (competition' xs [])
competition' []  ys   =  []
competition' x:xs ys  =  pairs x xs ++ competition' xs x:ys

sqdiff []   ys        =  cprod ys ys
sqdiff x    []        =  cprod xs xs
sqdiff x:xs y:ys      =  sqdiff xs ys
```

Table 1 gives for each function all hypotheses generated for each needed iteration.

5 Discussion and Future Work

The procedure currently has three apparent limitations. First, the procedure has two possible sources of non-termination. Second, it only works for exact sizes and not for upper bounds. Third, it is developed for a first-order functional language with lists as the only supported data structures. Here, these issues are discussed and improvements are suggested.

Because the procedure uses run-time tests, it does not terminate when one of these tests does not terminate. In practice, however, this is not an important problem, because the analysis will typically be run on a stable product where non-termination should be rare. Just in case, a termination analysis can be done first or the procedure may be adapted to start looking for replacement tests if evaluation of a test takes too long and non-termination is suspected. In general, this problem is very related to the active research field of test-case construction. The second source of nontermination

function	m	nr. of tests	type suggested	type checker
`cprod`	0	1 (1)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [[\alpha]^2]^0$	reject
(n = 2, k = 2)	1	8 (9)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [[\alpha]^2]^{s_1+s_2-1}$	reject
	2	14 (25)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [[\alpha]^2]^{s_1*s_2}$	accept
`append`	0	1 (1)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [\alpha]^0$	reject
(n = 2, k = 1)	1	3 (4)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [\alpha]^{s_1+s_2}$	accept
`competition`	0	1 (1)	$[\alpha]^{s_1} \to [[\alpha]^0]^0$	reject
(n = 1, k = 2)	1	3 (3)	$[\alpha]^{s_1} \to [[\alpha]^0]^{s_1}$	reject
	2	5 (5)	$[\alpha]^{s_1} \to [[\alpha]^2]^{s_1^2-s_1}$	accept
`sqdiff`	0	1 (1)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [[\alpha]^0]^0$	reject
(n = 2, k = 2)	1	3 (9)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [[\alpha]^0]^0$	reject
	2	8 (25)	$[\alpha]^{s_1} \to [\alpha]^{s_2} \to [[\alpha]^2]^{(s_1-s_2)^2}$	accept

Table 1
Type construction for four functions (n is the number of input variables, k the number of output polynomials). Both the actual number of tests and the theoretical maximum (between parentheses) are given.

is the iteration over increasing degrees of polynomials. If none of the generated types is accepted by the checker, either because the function definition is not shapely or due to incompleteness, the procedure in principle does not stop. In practice, often an upper bound can be put on the degree because only size expressions of low degree are desired. We do not expect that the procedure may be easily adopted for . All functions must be well-typed. We must be able to infer types for each of them. If one of the functions does not terminate during inference-via-testing then the procedure for the main function gets stuck. For a strict language this coincides with an incorrect, non-terminating program. For a lazy language the main function will not necessarily be non-terminating. One of the possible ways to continue testing in such cases is to involve .

-

The current hypothesis generation procedure relies on the limitation to shapely programs; output sizes need to be exactly polynomial in the input size. In practice many programs are not shapely, but still have a polynomial upper bound. Consider inserting an element in a set. This increases the set size by one only if the element

was not in it. Its actual upper bound is:

$$\texttt{insert} : [\alpha]^s \to \alpha \to [\alpha]^{s+1}$$

To extend our approach to such upper bounds, we have begun studying program transformations that transform an unshapely function into a shapely function with the strict size dependency corresponding to an upper bound of the size dependency of the original function. For instance, the `insert` function would be transformed into a shapely function that always inserts the element. We believe that in many practical cases the testing approach combined with program transformations will succeed in providing good upper bounds.

In this paper, we presented the procedure for a simple functional language over lists. We plan to extend and implement the procedure for an existing language with more general data structures. Good candidates are XML transformation languages [Wad00,Fri06] because such transformations are very likely to be shapely. For these applications, the general type inference procedure will stay the same. The only requirement is that a type checker for such a language is developed.

6 Conclusion

We have developed a procedure to infer static non-monotonically sized types through interpolating data from run-time tests. The dynamically generated types are only accepted after checking them by a formal type checking algorithm. So, the types are static: the size expressions hold for every possible future run of the program. Our key idea was the use of a dynamic testing procedure to generate hypotheses for the sized types. This replaces an otherwise infeasible to define formal type inference procedure and essentially reduces type inference to type checking. As a consequence, type inference is complete with respect to type checking.

Some interesting initial work on inferring size relations within the output of XML transformations has been done by Su and Wassermann [SW04]. Although this work does not yield output-on-input dependencies, it is able to infer size relations within the output type, for instance if two branches have the same number of elements. Hermann and Lengauer presented a size analysis for functional programs over nested lists [HL01]. However, they do not solve recurrence equations in their size expressions, as this is not important for their goal of program parallelization. Other work on size analysis has been restricted to monotonic dependencies. Research by Pareto has yielded an algorithm to automatically check linear sized types where size expression are upper bounds [Par98]. Inspired by this work Chin and Khoo [CK01] devised a fixed-point method for inferring linear lower and upper bounds. Construction of non-linear upper bounds using a traditional type system approach has been presented by Hammond and Vasconcellos [VK04], but this work leaves recurrence equations unsolved and is limited to monotonic dependencies. Debray and Lin

[DL93] reduce inference of monotonic size-relations for logical programs to solving difference equations and rely on external solving as well. We will study their generic size functions in order to extend our method. The work on quasi-interpretations by Amadio [Ama03] also requires monotonic dependencies.

References

[Ama03] Amadio, R.: "Max-plus quasi-interpretations"; Typed Lambda Calculi and Applications, Lecture Notes in Computer Science 2701 (2003), 31–45.

[CK01] Chin, Wei-Ngan Chin and Khoo, Siau-Cheng: "Calculating Sized Types"; Higher-Order and Symbolic Computation, 14(2-3)(2001), 261–300.

[CL87] Chui, C. and Lai, H. C.: "Vandermonde determinant and Lagrange interpolation in R^s"; Nonlinear and convex analysis (1987), 23–35.

[DL93] Debray, Saumya K. and Lin, Nai-Wei: "Cost analysis of logic programs"; ACM Transactions on Programming Languages and Systems, 15(5)(1993), 826–875.

[ESvK+07] van Eekelen, M., Shkaravska, O. and van Kesteren, R. and Jacobs, B and Poll, E. and Smetsers, S.: "Aha: Amortized heap space usage analysis"; Proceedings of the Eighth Symposium on Trends in Functional Programming, TFP 2007, XVI–1–16. Submitted for selected papers.

[Fri06] Frisch, A.: "Ocaml + XDuce"; Proceedings of the eleventh ACM SIGPLAN international conference on Functional programming (2006), 192–200.

[HL01] Herrmann, C. A. and Lengauer, C.: "A transformational approach which combines size inference and program optimization"; Semantics, Applications, and Implementation of Program Generation (SAIG'01), Lecture Notes in Computer Science 2196(2001), 199–218.

[Lor92] Lorenz, R. A.: "Multivariate Birkhoff Interpolation"; Lecture Notes in Math. 1516(1992)

[Mil78] Milner, R.: "A theory of type polymorphism in programming"; Journal of Computer and System Sciences, 17(3)(1978), 348–375.

[Par98] Pareto, R.: "Sized Types. Dissertation for the Licentiate Degree in Computing Science"; Chalmers University of Technology, Göteborg (1998)

[SvKvE07a] Shkaravska, O., van Kesteren, R. and van Eekelen, M.: "Polynomial size analysis of first-order functions"; to appear in Proceedings of International Conference on Typed Lambda Calculi and Applications (2007)

[SvKvE07b] Shkaravska, O., van Kesteren, R. and van Eekelen, M.: "Polynomial size analysis of first-order functions"; Technical Report ICIS-R07004, Radboud University Nijmegen (2007)

[SW04] Su, Z. and Wassermann, C.: "Type-based inference of size relationships for xml transformations"; Technical Report U CD//CSE-2004-8, UC Davis (2004)

[VK04] Vasconcelos, P. B. and Hammond, K.: "Inferring cost equations for recursive, polymorphic and higher-order functional programs"; Lecture Notes in Computer Science 3145(2004), 86–101.

[Wad00] Wadler Ph.: "A formal semantics of patterns in XSLT and XPath"; Markup Lang., 2(2)(2000), 183–202.

Computational Origami Construction as Constraint Solving and Rewriting

Tetsuo Ida, Mircea Marin, Hidekazu Takahashi, Fadoua Ghourabi[1,2]

Department of Computer Science, University of Tsukuba, Tsukuba 305-8357, Japan

Abstract

Computational origami is the computer assisted study of origami geometry. An origami is constructed by a finite sequence of fold steps, each consisting in folding along a fold line or unfolding. We base the fold methods on a formal system called Huzita's axiom system, and show how folding an origami can be formulated by a conditional rewrite system. A rewriting sequence of origami structures is seen as an abstraction of origami construction. We also explain how the basic concepts of constraint and functional and logic programming are related to this computational construction. Our approach is not only useful for computational construction of an origami, but leads to automated theorem proving of the correctness of the origami construction.

Keywords: computational origami, constraint solving, rewriting, functional logic programming, automated theorem proving

1 Introduction

We are interested in programming support for computational origami, the computer assisted study of origami. An origami is constructed by a finite sequence of fold steps, each consisting in folding along a fold line or unfolding. We use several admitted fold methods to find a fold line and then we fold an origami along the fold line.

As Euclidean postulates are the basis of Euclidean geometry, origami can be based on a formal system. An axiom system called Huzita's axiom system has been studied deeply, and hence we base our study on origami on Huzita's axiom system [3]. We will formulate Huzita's axiom system first in the language of the first-order predicate logic and then as a conditional rewrite system. A rewriting sequence of origami structures is seen as an abstraction of the origami construction.

[1] Emails:{ida, mmarin, hidekazu, ghourabi}@score.cs.tsukuba.ac.jp

[2] This research is supported by the JSPS Grants-in-Aid for Scientific Research (B) No. 17300004 and for Exploratory Research No. 19650001.

In so doing, we will be able to turn the declarative statements about the origami foldability to computing statements, i.e. program.

We use an origami construction of Morley's triangle as an illustrative example of our study. We also explain how the basic concepts of constraint and rewrite-based programming are related to this computational construction. Our approach is not only useful for computational construction of origami, but leads to automated theorem proving of correctness of the construction.

The rest of the paper is organized as follows. Section 2 gives the basic notions and notations used by us in modeling origami. In Sect. 3, we formalize Huzita's axioms. Then in Sect. 4, we formalize the operation fold. In Sect. 5, we give the construction of Morley's triangle. Section 6 describes the current programming and computational capabilities of our computational origami environment *Eos*, and indicates desirable new features that we plan to add. In Sect. 7, we summarize our work and indicate a direction of further research.

2 Basics of origami modeling

We will briefly give the notions about the geometric objects of our study and notations used in this paper. More thorough descriptions of geometric notions are treated in standard textbooks [8,9].

A point is the basic object we use without definition. We denote points by $P_1, \ldots, P_n$. By $\underline{P_{i,j}}$, where $i \leqslant j$, we denote the sequence $P_i, ..., P_j$ of points. When $i = 1$, we omit i and write $\underline{P_j}$.

By $\langle \underline{P_n} \rangle$, we denote several geometric objects that are the ingredients of origami. A ray, i.e. a directed line segment, is represented by a structure $\langle P_1, P_2 \rangle$, where P_1 and P_2 are the points that the line passes through. A ray is also denoted by $\overrightarrow{P_1P_2}$.

An $n (n \geqslant 3)$-gon, i.e. a simple n-edge polygon (polygon consisting of n edges none of which intersect), is represented by a structure $\langle P_1, \ldots, P_n \rangle$ made of a sequence of points $P_1, \ldots, P_n$, with the following property: $\langle P_1, \ldots, P_n \rangle$ and its cyclic permutation $\langle P_{1+k}, \ldots, P_{n+k} \rangle$ for arbitrary $k \geqslant 1$ are the same [3].

Definition 2.1 (Face) *A face is a convex n-gon with orientation. It is represented by* face$[\langle \underline{P_n} \rangle]$. *A face* $f =$ face$[\langle \underline{P_n} \rangle]$ *is up if the vertices* $P_1, \ldots, P_n$ *are arranged counter-clockwise, otherwise down.*

Definition 2.2 (Face division) *Let* f *be* face$[\langle \underline{P_n} \rangle]$. *Suppose that points* X *and* Y *lie on the ray* $\overrightarrow{P_iP_{i+1}}$ *and* $\overrightarrow{P_jP_{j+1}}$, *respectively, where* $i < j$. *The division of the face* f *by the ray* $\overrightarrow{XY}$ *is given by* $\delta_{\overrightarrow{XY}}(f)$ *defined as follows:*

Case 1: $\overrightarrow{XY}$ *does not overlap with one of the edges of* f

$$\delta_{\overrightarrow{XY}}(f) = \langle f_1, f_2 \rangle$$

where $f_1 =$ face$[$simplify$(\langle Y, X, \underline{P_{i+1,j}} \rangle)]$ *and* $f_2 =$ face$[$simplify$(\langle X, Y, \underline{P_{j+1,n+i}} \rangle)]$. *The function* simplify *is used here to make the n-gon a simple polygon. Note that the n-gons* $\langle Y, X, \underline{P_{i+1,j}} \rangle$ *and/or* $\langle X, Y, \underline{P_{j+1,n+i}} \rangle$ *are not simply if the points* X

[3] The index addition is performed in modulo the number of elements of the sequence.

and/or Y are the vertices of the n-gon.

Case 2: *$\overrightarrow{XY}$ overlaps with one of the edges of f*

$$\delta_{\overrightarrow{XY}}(f) = \langle f, \langle\!\langle X, Y \rangle\!\rangle \rangle \text{ or } \langle \langle\!\langle Y, X \rangle\!\rangle, f \rangle .$$

The output of δ_r is a pair consisting of geometric objects f_1 and f_2, and f_1 is to the right of r and f_2 to the left of r.

The face division is illustrated in Fig. 1.

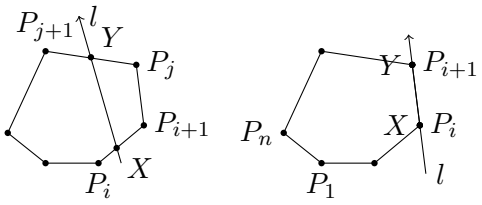


Fig. 1. Face division.

Definition 2.3 (Face rotation) *For a point P, $\rho_r^\theta(P)$ is the rotation of P by an angle θ along a ray r. ρ_r^θ is extended to a face. Namely, if $f = \text{face}[\langle\!\langle P_1, \ldots, P_n \rangle\!\rangle]$ then $\rho_r^\theta(f) = \text{face}[\langle\!\langle \rho_r^\theta(P_1), \ldots, \rho_r^\theta(P_n) \rangle\!\rangle]$.*

In origami folds, faces are divided before rotation. When face f is divided into f_1 and f_2, i.e. $\delta_r(f) = \langle f_1, f_2 \rangle$, then we rotate f_1 along r by applying ρ_r^θ to f_1.

Definition 2.4 (Abstract Origami) *An abstract origami is a structure $(\Pi, \succ, \mathcal{A})$, where Π is the set of faces, $\succ$ is an overlay relation over Π and $\mathcal{A}$ is an adjacency mapping from Π to 2^Π. We call the abstract origami simply an origami.*

For the overlay and adjacency mapping, refer to [5]. We do not treat those relations in this paper. We use $\mathcal{O}$ to denote a set of origami's. When we make a fold, we specify which faces we will fold. The set of faces that we are interested in folding is called the set of the *faces of concern*.

Definition 2.5 (Single-step fold) *Let $O_1, O_2 \in \mathcal{O}$ where $O_1 = (\Pi_1, \succ_1, A_1)$. An origami single-step fold from O_1 to O_2 is defined as a relation:*

$$O_1 \looparrowright_{\mathcal{F},r,\theta} O_2$$

where $\mathcal{F} \subseteq \Pi_1$ is the set of faces of concern, r is a fold ray, and θ is the angle of rotation.

$\mathcal{V}(O)$ denotes the set of points in O. When the number of divided faces is k, and r is $\overrightarrow{XY}$, we have:

$$\mathcal{V}(O_2) = \mathcal{V}(O_1) \cup \{\underline{X_k}, \underline{Y_k}\},$$

where the points $\underline{X_k}, \underline{Y_k}$ are created in this fold [4] .

Definition 2.6 (Origami construction) *Let $O_1, \ldots, O_n \in \mathcal{O}$. A fold sequence*

$$O_1 \looparrowright_{\mathcal{F}_1,r_1,\theta_1} O_2 \looparrowright_{\mathcal{F}_2,r_2,\theta_2} \cdots \looparrowright_{\mathcal{F}_{n-1},r_{n-1},\theta_{n-1}} O_n$$

[4] We let $X_1 = X$ and $Y_1 = Y$.

is called an origami construction *from* O_1 *to* O_n. *We may also write* $O_1 \rightsquigarrow^* O_n$.

3 Huzita's axioms of origami construction

3.1 Huzita's axioms in brief

Huzita proposed the following fold methods:

(O1) We can make a fold along a line that passes through given points P and Q.

(O2) We can make a fold along the line that superposes given points P and Q.

(O3) We can make a fold along a line that superposes two given lines m and n.

(O4) We can make a fold along a line that is perpendicular to a given line m, and passes through a given point P.

(O5) We can make a fold along a line that passes through a given point Q and superposes a given P on a given line m.

(O6) We can make a fold along a line that superposes a given P on a given line m and a given point Q on a given line n

Remarks

(a) In the case of (O3), there exists one or two fold lines.

(b) In the cases of (O5) and (O6), there may be multiple fold lines or none. It is decidable whether a fold line satisfying the properties exists or not in all the cases.

When we say that we are given n points, those n points are distinct and they are all on the origami. Likewise, when we are given a line, we assume the existence of the two distinct points on the origami that are on the line.

3.2 Huzita's axioms in the first-order language

Let P and Q denote points, and l, m and n denote lines. Corresponding to the statements (O1) $\sim$ (O6) in the previous subsection, we have the following formulas.

(A1) $\forall P, Q\, \exists l\ \mathrm{OnLine}(P, l) \wedge \mathrm{OnLine}(Q, l)$.

(A2) $\forall P, Q\, \exists l\ \mathrm{reflection}(P, l) = Q$.

(A3) $\forall m, n\, \exists l\ \forall P\, \mathrm{OnLine}(P, m) \Rightarrow \mathrm{OnLine}(\mathrm{reflection}(P, l), n)$.

(A4) $\forall P, m\, \exists l\, \forall Q\, \mathrm{OnLine}(P, l) \wedge (\mathrm{OnLine}(Q, m) \Rightarrow \mathrm{OnLine}(\mathrm{reflection}(Q, l), m))$.

(A5) $\forall P, Q, m\, \exists l\ \mathrm{OnLine}(Q, l) \wedge \mathrm{OnLine}(\mathrm{reflection}(P, l), m)$.

(A6) $\forall P, Q, m, n\, \exists l\ \mathrm{OnLine}(\mathrm{reflection}(P, l), m) \wedge \mathrm{OnLine}(\mathrm{reflection}(Q, l), n)$.

All the predicates and functions have to be given precise meaning in the proper semantic domain. A rigorous treatment of the semantics of Huzita's axioms is explained in [2]. We give informal meaning to those symbols based on the geometrical intuition, as we go along.

We first fix the domain of interpretation to be the domain of algebraic numbers. We will see shortly that if we represent a line as a term $\mathrm{line}(a, b, c)$ that passes

through two given points defining the line, we are always able to find the values of a, b and c in formulas (A1) $\sim$ (A6). However, the values may be complex numbers. In such cases, we can not give geometric meaning in the Euclidean plane. Remark in subsection 3.1 is intended to purport this observation.

Formula (A1) states that for any points P and Q, there exists a line l such that P is on l and Q is on l. In formula (A2), the predicate reflection$(P, l) = Q$ states that the reflection of a point P in a line l is the point Q.

3.3 Huzita's axioms in rewrite rules

We next consider computation, since our first objective is the construction of an origami. To reason about the computation implicit in the Huzita's axioms, the description in the first-order predicate logic is not ideal. When we prove the correctness of an existentially quantified formula, often we are not only interested in the validity of the formula, but in the substitution made to the existential variables. Obtaining the values instantiated in the existential variables is often the intended result of the proof.

A rewrite rule realizes clearly this intention, if we are more interested in computation rather than declarative statements. Let us consider Axiom (O1) to begin with. The basic idea in transcribing the first-order formula to a rewrite rule is to define a new relation $\mathcal{R}(P, Q, l)$, which is **true** if $\mathrm{Online}(P, l) \wedge \mathrm{OnLine}(Q, l)$ holds. Namely, we state

$$\forall P, Q, l\,(\mathcal{R}(P, Q, l) \Leftarrow \mathrm{Online}(P, l) \wedge \mathrm{OnLine}(Q, l)).$$

This means that for any combinations of values of P, Q and l, if $\mathrm{Online}(P, l) \wedge \mathrm{OnLine}(Q, l)$ holds, then $\mathcal{R}(P, Q, l)$ holds. In particular if for any combinations of values of P, Q and l for which formula (A1) holds, then $\mathcal{R}(P, Q, l)$ holds. This can be written as a 3-CTRS (type 3 Conditional Term Rewrite System)

$$\mathrm{foldTh}(P, Q) \to l \Leftarrow \mathrm{Online}(P, l) \wedge \mathrm{OnLine}(Q, l).$$

Here, we have introduced a new function symbol foldTh, and replaced the relation $\mathcal{R}(P, Q, l)$ by the rewrite relation $\mathrm{foldTh}(P, Q) \to l$.

We then want to define OnLine as a rewrite rule. At this point we need to commit ourselves to a certain representation of points and lines. In this paper, we use the Cartesian coordinate system to represent points and a linear equation to represent a line. We use constructor symbols point and line to represent them in term structures. Unless we have a facility of object abstraction as in an object oriented language, we need to write explicitly our representation of objects, i.e. terms in our conditional term rewriting language.

$$\mathrm{OnLine}(\mathrm{point}(x, y), \mathrm{line}(a, b, c)) \to \mathbf{true} \Leftarrow ax + by + c = 0$$

To be more rigorous, we need a constraint on the coefficients a, b, and c to make the equation $ax + by + c = 0$ always represent a line. This constraint is stated in the predicate Coeff(line(a, b, c)). One possible definition of Coeff(line(a, b, c)) is the

following:

$$(-1+b)b = 0 \wedge (-1+a)(-1+b) = 0.$$

A slightly complicated re-reasoning of the above rewrite rule leads to the following rewrite rule.

$$\text{foldTh}(P,Q) \to l \Leftarrow \text{Online}(P,l) \wedge \text{OnLine}(Q,l) \wedge \text{Coeff}(l).$$

Axiom (O2) is easily transcribed to the following rewrite rule:

$$\text{foldBr}(P,Q) \to l \Leftarrow \text{reflection}(P,l) = Q \wedge \text{Coeff}(l), \tag{1}$$

where $\text{reflection}(P,l)$ is written as a rewrite rule:

$$\text{reflection}(\text{point}(x,y), \text{line}(a,b,c)) \to \text{point}\left(\frac{-a^2x + b^2x - 2a(c+by)}{a^2+b^2}, \frac{-2b(c+ax) + a^2y - b^2y}{a^2+b^2}\right).$$

Since Axiom (O3) is already a formula of implication, transcribing Axiom (O3) needs a bit of algebraic manipulation. Let P, l, m, n be $\text{point}(x,y)$, $\text{line}(a,b,c)$, $\text{line}(a_1,b_1,c_1)$, $\text{line}(a_2,b_2,c_2)$, respectively. Then the algebraic interpretation of the right-hand side (of $\Rightarrow$) in formula (A3) is

$$c_2 + \frac{b_2(-2b(c+ax) + a^2y - b^2y)}{a^2+b^2} + \frac{a_2(-a^2x + b^2x - 2a(c+by))}{a^2+b^2} = 0,$$

which is equivalent to the formula $a'x + b'y + c' = 0$, where

$$\begin{aligned} a' &= -a^2a_2 + a_2b^2 - 2abb_2 \\ b' &= -2aa_2b + a^2b_2 - b^2b_2 \\ c' &= -2aa_2c - 2bb_2c + a^2c_2 + b^2c_2 \end{aligned}$$

Let l' be the line $\text{line}(a',b',c')$. Then it is easy to see that

$$(a',b',c') = -k \cdot (a_1,b_1,c_1) \text{ for some } k.$$

From the above formula, we can obtain the following rewrite rules:

$$\begin{aligned} &\text{foldBrLine}(\text{line}(a_1,b_1,c_1), \text{line}(a_2,b_2,c_2)) \to l \Leftarrow l = \text{line}(a,b,c) \\ &\quad \wedge -a^2a_2 + a_2b^2 - 2abb_2 + ka_1 = 0 \wedge -2aa_2b + a^2b_2 - b^2b_2 + kb_1 = 0 \\ &\quad \wedge -2aa_2c - 2bb_2c + a^2c_2 + b^2c_2 + kc_1 = 0 \wedge \text{Coeff}(l). \end{aligned}$$

Note that the algebraic expressions in the above formula are square-root free. In general, solving the conditional part of this rewrite rule yields two solutions for a, b, c and k, which shows that there are two fold lines that superpose lines m and n.

Similarly, we can obtain the following rewrite rules that correspond to Axioms (O4) $\sim$ (O6), respectively.

$$\text{foldPerTh}(P, m) \to l \Leftarrow \text{OnLine}(P, l) \wedge (\text{OnLine}(Q, m) \Rightarrow \text{OnLine}(\text{reflection}(Q, l), m)) \wedge \text{Coeff}(l).$$

$$\text{foldThBr}(P, Q, m) \to l \Leftarrow \text{OnLine}(Q, l) \wedge \text{OnLine}(\text{reflection}(P, l), m) \wedge \text{Coeff}(l).$$

$$\text{foldBrBr}(P, Q, m, n) \to l \Leftarrow \text{OnLine}(\text{reflection}(P, l), m) \wedge \text{OnLine}(\text{reflection}(Q, l), n) \wedge \text{Coeff}(l).$$

Note that the conditional part of the rewrite rules will become the conjunction of equations when the rules are applied. In order to perform rewriting of the left-hand side to the righthand side of the rewrite rules, we need to solve the system of equations. This can be done either statically or dynamically. By *statically*, we mean that we can transform the rewrite rule to a new rewrite rule that does not require constraint solving at run time. This will be detailed in the next subsection.

The conditional rewrite system that describes Huzita's axiom system can rewrite the same ground term in more than one way. This behavior indicates that some single-step folds can be performed in more than one way. It can be shown that every ground term can be transformed in at most 3 ways. When the rewriting is non-deterministic, it is desirable to allow the user to view the alternatives and to choose one to proceed.

3.4 Transformation of rewrite rules

In this subsection we will show an example of static transformation of rewrite rules. Rewriting of the term $\text{foldBr}(P, Q)$ by the rewrite rule (1) for Axiom (O2) requires constraint solving of the set of equations

$$\begin{aligned}\{&(-1+a)(-1+b) = 0, (-1+b)b = 0,\\ &\frac{-2b\,(c+ax_1) + a^2y_1 - b^2y_1}{a^2+b^2} = y_2,\\ &\frac{-a^2x_1 + b^2x_1 - 2a\,(c+by_1)}{a^2+b^2} = x_2\}\end{aligned}$$

for a, b, and c. The solution of the above system is

$$\left\{a \to 1, b \to 0, c \to \frac{-x_1 - x_2}{2}\right\}$$

if $y_1 = y_2$, and

$$\left\{a \to \frac{x_1 - x_2}{y_1 - y_2}, b \to 1, c \to \frac{-x_1^2 + x_2^2 - y_1^2 + y_2^2}{2\,(y_1 - y_2)}\right\}$$

if $y_1 \neq y_2$. Therefore, the rewrite rule (1) is reduced to

$$\text{foldBr}(\text{point}(x_1, y), \text{point}(x_2, y)) \to \text{line}(1, 0, \frac{-x_1 - x_2}{2})$$

$$\text{foldBr}(\text{point}(x_1, y_1), \text{point}(x_2, y_2)) \to \text{line}(\frac{x_1 - x_2}{y_1 - y_2}, 1, \frac{-x_1^2 + x_2^2 - y_1^2 + y_2^2}{2\,(y_1 - y_2)})$$

If rewriting is performed in a symbolic computation environment such as of *Mathematica* [10], the above transformation can be done semi-automatically. So on one hand it is possible to reduce the computing cost at run time by static transformation. We need not solve the same constraints repeatedly.

On the other hand, for theorem proving, we need to maintain the set of equations symbolically. Origami construction is interactive. As our experiences with *Eos* [6] show, constraint solving is not prohibitively expensive, even when we use ordinary desktop computers. Therefore solving constraints dynamically is also feasible.

4 Formalizing Fold

Now let us consider formalization of the essential part of origami, i.e. fold. Each fold requires the following parameters: the set $\mathcal{F}$ of faces of concern, the rotation angle θ (either π or $-\pi$ in this paper) and the axiom to apply together with appropriate lines and points given to the axiom.

A fold operation can be decomposed into seven computational steps:

(F-1) Choose a fold method.

(F-2) Obtain a fold line l according to the fold method.

(F-3) Specify the set $\mathcal{F}$ of the faces of concern by the fold and the ray r obtained from l.

(F-4) Compute the set $\mathcal{G}$ of all the faces that are affected by the fold.

(F-5) Divide the faces by l and classify all the obtained faces into "to be moved" or "to be non-moved".

(F-6) Rotate the "to be moved" faces along r.

(F-7) Compute the overlay relation and adjacency mapping.

In this paper we consider an origami construction where each fold is followed by an unfold, so that we always make a fold on a square piece of origami paper. This is the case of the construction of a Morley's triangle. The modeling of the folds in general cases requires the analysis of overlay relation and adjacency mapping on the faces at step (F-4). Then the faces are divided and rotated at steps (F-5) and (F-6). These steps need preparation consisting of classifying origami faces [5]. Finally, at step (F-7) the overlay relation and the adjacency mapping are computed using the relations computed in the previous origami construction step. The resulting origami forms layers of faces and exhibits an artistic shape.

5 Construction of Morley's triangle

We will show how to formalize the origami construction of Morley's triangle [1]. Given a triangle, Morley's triangle is the triangle inside the given triangle, formed by the three intersections of neighboring trisectors of the angles of the given triangle. The Morley's triangle is always equilateral. This is an interesting example of a construction that can be realized with only a piece of origami paper by following simple constructive steps [4]. Since angle trisection cannot be realized by means of a compass and a ruler, it is a surprising result, which shows that an origami

construction exceeds the capabilities of the Euclidean construction by the ruler and compass.

5.1 Overview of the construction

Figure 2 illustrates the construction of a Morley's triangle. We need 28 fold steps to obtain Morley's triangle. Morley's triangle is clearly marked in thick solid line (red if colored) in O_{29}. Suppose we are given an initial origami ABCD with a point E. The thin solid line segments (light brown if colored) EA and EB in Fig. 2 are shown for the convenience of readers, and are not part of the construction. Let this origami be O_1. The construction proceeds by trisecting the internal angles of $\triangle$ABE. The points B_1, O_1 and Q_1 are the intersections of the neighboring trisectors. Morley's theorem states that $\triangle \mathrm{B}_1\mathrm{O}_1\mathrm{Q}_1$ is equilateral.

The construction is described as follows:

$$O_1 \overset{}{\rightarrowtail}_{\{\mathrm{ABCD}\},\overrightarrow{GF}} O_2 \overset{}{\rightarrowtail}_{\{\mathrm{FGAB}\},\overrightarrow{FG}} O_3 \overset{}{\rightarrowtail}_{\{\mathrm{FGAB}\},\overrightarrow{IH}} O_4 \overset{}{\rightarrowtail}_{\{\mathrm{HIAB}\},\overrightarrow{HI}}$$

$$O_5 \overset{}{\rightarrowtail}_{\{\mathrm{GFCD}\},\overrightarrow{OJ}} O_6 \overset{}{\rightarrowtail}_{\{\mathrm{MODG}\},\overrightarrow{JO}} O_7 \overset{}{\rightarrowtail}_{\{\mathrm{JBHK}\},\overrightarrow{AQ}} O_8 \overset{}{\rightarrowtail}_{\{\mathrm{AJS}\},\overrightarrow{QA}}$$

$$O_9 \overset{}{\rightarrowtail}_{\{\mathrm{AJS}\},\overrightarrow{AP}} O_{10} \overset{}{\rightarrowtail}_{\{\mathrm{ASK}\},\overrightarrow{PA}} O_{11} \overset{}{\rightarrowtail} \cdots O_{29}$$

Here we omit θ since θ is always π. $\mathcal{F}$ is specified by the set of the names of the faces of concern. The name of face$[\langle P_i, \ldots, P_j \rangle]$ is $P_i \cdots P_j$.

5.2 Trisection of an angle

The steps from O_1 to O_{11} is the construction of the trisector of $\angle$EAB. The rest is for constructing the other two trisectors. Let us see in more detail how the computation proceeds. The rewrite rules applied in the steps from O_1 to O_{11} is foldBr, unfold, foldBr, unfold, foldBrBr, unfold, foldTh, unfold, foldTh and unfold. During the construction, the system solved the constraints numerically, and at the same time it saved the constraints in symbolic forms. Therefore, at the step 11 we can generate the set $\mathcal{C}$ of the equations, and can prove that we indeed constructed the trisector. The set $\mathcal{C}$ is given in Fig. 3. We abuse the notation of a set of polynomials and identify it with the system of equations.

The conclusion of the proposition that we want to have must then be formulated. For the proof based on the Gröbner bases, we need the statement about the trisection given in terms of polynomials. We use the square of sin of concerned

angles. Then the conclusion is formulated as follows:

$$\begin{aligned}
\sin^2 \angle \mathrm{QAB} &= \frac{y_3^2}{x_3^2 + y_3^2}, \\
\sin^2 \angle \mathrm{PAQ} &= \frac{(-x_4 y_3 + x_3 y_4)^2}{\left(x_3^2 + y_3^2\right)\left(x_4^2 + y_4^2\right)}, \\
\sin^2 \angle \mathrm{EAP} &= \frac{(x_4 - u y_4)^2}{(1 + u^2)\left(x_4^2 + y_4^2\right)}, \text{ and} \\
\sin^2 \angle \mathrm{QAB} &= \sin^2 \angle \mathrm{PAQ} = \sin^2 \angle \mathrm{EAP}
\end{aligned}$$

From the above, we obtain $\mathcal{D}$ given in Fig. 4. We want to prove

$$\mathcal{C} \Rightarrow \mathcal{D}. \tag{2}$$

Let $\mathcal{C} = \{c_1, \ldots, c_n\}$ and $\mathcal{D} = \{d_1, \ldots, d_m\}$. The proof of proposition (2) is by contradiction using the theory of Gröbner bases. We compute the Gröbner basis of the set of polynomials

$$\{c_1, \ldots, c_n, (\kappa_1 d_1 - 1) \cdots (\kappa_m d_m - 1)\} \tag{3}$$

where $\kappa_1, \ldots, \kappa_m$ are newly introduced slack variables.

We consider the set of the polynomials (3) as the system of the equations. The non-existence of its solution implies the correctness of the proposition. Namely, if the reduced Gröbner basis of (3) is $\{1\}$, the proposition is proved.

Eos has an interface to assist computational origamists to try the proofs. To compute the reduced Gröbner basis of (3), it took 0.17 seconds for the proof of the trisector of $\angle$EAB, 0.28 seconds for $\angle$ABE and 4.78 seconds for $\angle$BEA on a Pentium 1.2 G Hz laptop computer equipped with 1 GB memory running under Windows XP.

After trisecting the angles and proving the correctness of the construction of the trisectors, we will go on to prove that the triangle $\triangle \mathrm{B_1O_1Q_1}$ is equilateral. For a fixed numerical value of the x-coordinate of E, we are able to obtain the proof that the triangle is equilateral. It took 5.1 seconds on the same computer. For arbitrary u of the x-coordinate of E, it could not compute the reduced Gröbner basis over the coefficient domain of rational functions. Only the careful analysis of the polynomials and the choice of appropriate ordering of monomials led to successful computation. It took 16 hours of computation by the computing server, however.

6 Programming language support for origami

We have seen the model of computation for computational origami. The model comes with a formal language to reason about the model. However, the formal language is not sufficient for end-users, i.e. origamists, to explore the possibilities of computational origami. The computational origami environment *Eos* developed by us assists the user to perform origami construction and reason about geometric properties. The programming and computational capabilities described in this

paper are an abstraction of what have been implemented so far. We need more programming support for computational origami.

In this section we briefly discuss what we already have and some desiderata for further development. *Eos* provides primitives for the following operations:

- Simulation of the origami construction: The realization of the single-step fold $O_1 \hookrightarrow_{\mathcal{F},r,\theta} O_2$ is achieved by one of the Huzita's axiom, together with appropriate parameters and the extra information needed in the formalization of the fold. For example, the fold using Axiom (O1) is realized by a call `fold[A, Through -> {P, Q}]` and the fold using Axiom (O2) is realized by `fold[P, Q]`. In the latter case, point P is brought to point Q. In this way, the origamists do not have to know the faces of concern. In the former case the origamists must give point A as an extra parameter, such that the system can identify the face of concern. The system processes the input specification, and computes $\mathcal{G}$ and r, and the data structure O_2 for the origami produced by the single-step fold.
- Visualization of the origami O_2 produced by single-step folds $O_1 \hookrightarrow_{\mathcal{F},r,\theta} O_2$.
- Maintaining an algebraic representation of the origami construction, that can be used for proving geometric theorems about origami.
- Providing a set of functions that enable origamists to compute and reason about the geometric objects used in origami.

The implementation of these capabilities requires support for conditional term rewriting, capabilities of symbolic computation such as Gröbner basis computation and manipulation of polynomials, and graphics processing. *Eos* is implemented in *Mathematica* [10], since it is based on higher-order term rewriting with functionalities that we described above.

As our experiences grow with many examples of origami's, more functionalities have become needed. From programming point of view, typing and object orientation are highly desirable. However, typing of geometrical objects is non-trivial since the type may change as the shape of an object gets degenerated; for instance a polygon becomes a line when the number of vertices become 2. Features proposed by Liang and Wang [7] would have to be taken into account. For computational origami the desired functionalities are also for producing high-quality art pieces of origami. We expect to have better human friendly interface which can be obtained by integrating off-the-shelf components.

7 Conclusion

We have shown how the origami construction can be modeled by a sequence of fold steps. At each fold step, an origami structure is transformed to a new structure. The fold line along which the fold is made is computed by a 3-CTRS. The rewrite rules of 3-CTRS are derived from Huzita's axiom system. The application of the rewrite rules requires constraints solving in order to satisfy the conditions of the rewrite rules.

The numerical solutions are used to simulate the construction of origami, and to visualize the origami shapes at each step. The accumulated constraints are used for proving the geometric properties of the constructed origami.

We argued for the usefulness of the modeling of the construction as a rewrite sequence, and discussed the language features that are essential in the geometrical modeling of origami.

The formalization of origami is an important step towards the study of reverse problems of origami (e.g., finding a shape from the fully unfolded origami with creases) and the study of foldability beyond the power of Huzita's origami axiom system.

References

[1] A. Bogomolny. Morley's Miracle. 1996. http://www.cut-the-knot.org/triangle/Morley, ©1996-2005.

[2] F. Ghourabi, T. Ida, H. Takahashi, M. Marin, and A. Kasem. Logical and Algebraic View of Huzita's Origami Axioms with Applications to Computational Origami. In *Proceedings of the 22nd ACM Symposium on Applied Computing*, pages 767–772. ACM Press, March 2007.

[3] H. Huzita. Axiomatic Development of Origami Geometry. In H. Huzita, editor, *Proceedings of the First International Meeting of Origami Science and Technology*, pages 143–158, 1989.

[4] T. Ida, M. Marin, and H. Takahashi. Computational Origami of a Morley's Triangle. In M. Kohlhase, editor, *Proceedings of 4th International Conference on Mathematical Knowledge Management (MKM 2005)*, volume 3863 of *LNCS*, page 267. Springer.

[5] T. Ida, H. Takahashi, M. Marin, and F. Ghourabi. Modelling Origami for Computational Construction and Beyond. In *The 2007 International Conference on Computational Science and Its Applications (ICCSA 2007)*, Lecture Notes in Computer Sciences, August 2007. (To appear).

[6] T. Ida, H. Takahashi, M. Marin, A. Kasem, and F. Ghourabi. Computational Origami System Eos. In *Proceedings of 4th International Conference on Origami, Science, Mathematics and Education (4OSME)*, page 69, 2006.

[7] T. Liang and D. Wang. Towards a Geometric-Object-Oriented Language. In *Automated Deduction in Geometry*, volume 3763 of *Lecture Notes in Computer Science*, pages 130–155, 2006.

[8] F. P. Preparata and M. I. Shamos. *Computational Geometry: An Introduction*. Texts and Monographs in Computer Science. Springer-Verlag, 1985.

[9] N. J. Wildberger. *Divine Proportions*. Wild Egg Pty Ltd, 2005.

[10] S. Wolfram. *The Mathematica Book*. Wolfram Media, 5th edition, 2003.

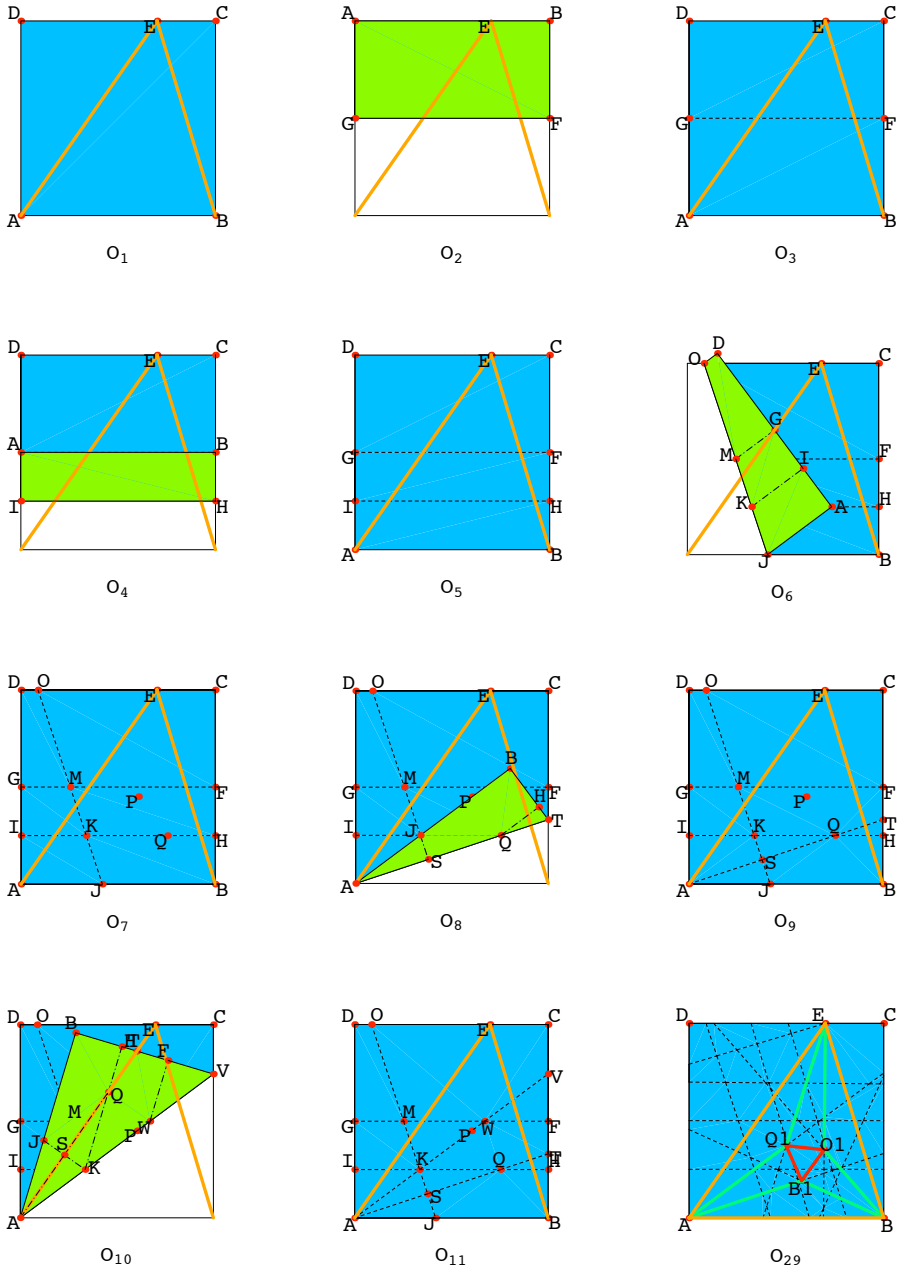


Fig. 2. Origami construction of a Morley's triangle

$$\{(-1+b_1)b_1, (-1+a_1)(-1+b_1), c_1, b_1+c_1+a_1u,$$
$$(-1+b_2)b_2, (-1+a_2)(-1+b_2), a_2, \frac{1}{2}(b_2+2c_2),$$
$$x_1, c_2+a_2x_1+b_2y_1, (-1+b_3)b_3, (-1+a_3)(-1+b_3), -b_3x_1+a_3y_1,$$
$$\frac{1}{2}(2c_3+a_3x_1+b_3y_1), x_2, c_3+a_3x_2+b_3y_2, (-1+a_4)(-1+b_4), (-1+b_4)b_4,$$
$$c_1+a_4^2c_1-2a_1c_4-2a_1a_4c_4+2a_1b_4c_4-2b_1b_4c_4-a_1x_1-$$
$$a_1a_4^2x_1+2b_1x_1-2a_4b_1x_1+2a_1b_4x_1-2b_1b_4x_1+$$
$$2a_1y_1-2a_1a_4y_1+b_1y_1+a_4^2b_1y_1-2a_1b_4y_1-2b_1b_4y_1,$$
$$(-1+b_4)(c_1-2a_1c_4-a_1x_1+b_1y_1),$$
$$c_3+a_4^2c_3-2a_3c_4-2a_3a_4c_4+2a_3b_4c_4-2b_3b_4c_4, (-1+b_4)(c_3-2a_3c_4),$$
$$-a_3a_4c_1c_4-b_3c_1c_4+a_1a_4c_3c_4+b_1c_3c_4-2a_3b_1c_4^2+$$
$$2a_1b_3c_4^2+2a_3b_1b_4c_4^2-2a_1b_3b_4c_4^2-a_1c_3x_1+$$
$$a_4b_1c_3x_1+2a_1a_3c_4x_1+a_1a_3a_4c_4x_1-2a_3b_1c_4x_1+$$
$$a_1b_3c_4x_1-2a_1a_3b_4c_4x_1+2a_3b_1b_4c_4x_1+a_1a_4c_3y_1+$$
$$b_1c_3y_1-2a_1a_3c_4y_1-2a_3b_1c_4y_1-a_3a_4b_1c_4y_1-$$
$$b_1b_3c_4y_1+2a_1a_3b_4c_4y_1+2a_3b_1b_4c_4y_1, b_4x_6-a_4y_6,$$
$$c_4+\frac{a_4x_6}{2}+\frac{b_4y_6}{2}, b_4(-x_2+x_5)+a_4(y_2-y_5),$$
$$c_4+\frac{1}{2}a_4(x_2+x_5)+\frac{1}{2}b_4(y_2+y_5), x_4-x_5, y_4-y_5, x_3-x_6, y_3-y_6\}$$

We have A(0, 0), B(1, 0), C(1, 1), D(0,1), E(u, 1), G(x_1, y_1), I(x_2, y_2), P(x_4, y_4) and Q(x_3, y_3). Lines AE, GF, IH and OJ are represented by line(a_1, b_1, c_1), line(a_2, b_2, c_2), line(a_3, b_3, c_3) and line(a_4, b_4, c_4), respectively.

Fig. 3. Premise equations for the proof of the trisection of ∠EAB

$$\{y_4(2x_3x_4y_3-x_3^2y_4+y_3^2y_4),$$
$$-2(1+u^2)x_3x_4y_3y_4+x_3^2(-x_4^2+2ux_4y_4+y_4^2)+$$
$$uy_3^2(2x_4y_4+u(x_4^2-y_4^2))\}$$

Fig. 4. Conclusion equations for the proof of the trisection of ∠EAB

Phil: A Lazy Implementation of a Language for Approximate Filtering of XML Documents

M. Baggi D. Ballis

Dip. Matematica e Informatica, Via delle Scienze 206, 33100 Udine, Italy.
Email: {baggi,demis}@dimi.uniud.it.

Abstract

In this paper, we introduce a system, written in Haskell, for filtering information from XML data. Essentially, the system implements a simple declarative language which allows one to extract relevant data as well as to exclude useless and misleading contents from an XML document by matching patterns against XML documents.
The matching mechanism employs a cost-based pattern transformation algorithm which searches for patterns in an approximate way (i.e. modulo renaming, insertion, and deletion of XML items) and ranks the results w.r.t. their cost. In order to improve efficiency, the implementation uses sophisticated indexing techniques and exploits laziness to automatically avoid the construction of unnecessary data structures.
We analyzed both the expressiveness of our filtering language and the performance of the system using the well known XMark benchmark suite.

Keywords: XML filtering and query languages, approximate pattern matching, tree embedding problem.

1 Introduction

The adoption of XML[12] as a widely accepted standard for data representation and exchange has led to a rapid growth in the amount of XML data available over the internet. Nowadays, large-scale XML repositories are constantly browsed, queried and modified by internet users, who typically retrieve a lot of information which is not always possible to absorb in a pleasant and/or understandable fashion. In order to tame the inherent complexity of such a massive amount of data, a lot of research effort has been invested by computer scientists in the last years giving rise to a proliferation of decision-support systems to manage and explore XML repositories.

Arguably, a growing attention has been dedicated to query and filtering languages as means to efficiently extract all and only the relevant information from huge data collections. As a matter of fact, information frequently appears obscure or difficult to interpret; moreover, most of the time, just a small percentage of the whole amount of the data received is considered interesting by the user. Therefore, query and filtering systems represent a valid way to obtain those contents which best fit user's needs. The World Wide Web Consortium has defined XQuery[14] and XPath[13] as standard languages to consult and filter information in XML doc-

uments, nonetheless a plethora of alternative and worthwhile proposals have been developed independently, e.g. [3,7,5]. Basically, they all work by *exactly* matching a given pattern (or path expression) representing the information to be searched for against an XML document. Hence, recognized pattern instances are delivered to the user.

Although the languages mentioned above are very advantageous in many applications, they may be of limited use when dealing with data filtering in a pure information retrieval context, since (i) they require the user to be aware of the complete XML document structure, (ii) results that are not fully matched are not delivered, (iii) there is no result ranking. Therefore, in this context, a more flexible matching mechanism which can manage the lack as well as the vagueness of the information is necessary. Such an *approximate* behaviour is not typically implemented in the standard query languages, and actually only few works address this issue.

For instance, the PIX[9] system is a phrase matching system tailored to XML for searching a given phrase into an XML document. It implements a rough approximate matching method which basically allows to ignore some tags included in the document, while no deletion and renaming of XML items are permitted.

A more flexible approach is followed in ApproxXQL[11,10], which is an approximate query language which provides a more sophisticated approximate matching mechanism. It is based on a cost-based query transformation algorithm which allows to rename, insert and delete XML items in order to find the best match between a pattern and a given XML document. However, this language is still rather simple and does not offer the full expressive power of modern query languages.

Our contribution. The original contribution of this paper is twofold.

(i) We present a novel declarative language for approximate filtering of XML documents, which allows the user to easily select the desired information (*positive filtering*) as well as to remove noisy, spurious data (*negative filtering*) from a given Web page. Our language is easy to use and thus can be employed even by those users who are typically not used to express themselves using formal methodologies, since no special expertise is required. Basically, in our approach, XML documents and filtering queries are encoded as tree-shaped terms of a suitable term algebra, then an approximate tree embedding algorithm is employed to execute filtering queries on XML documents to recognize the information that the user wants to select or to strike out. Our approach is inspired by ApproXQL and extends it in several ways.
- ApproXQL allows to define only ground patterns, while our language provides "logical" variables, which can be used to extract parts of the document on which we can perform further tests.
- We add regular expressions and built-in functions to model conditional filtering rules with the aim of refining the approximate search engine.
- Nested filtering queries are allowed, while ApproXQL manages only flat queries.
- ApproXQL does not support negative filtering, while our language does. This feature allows to introduce the expressive power of negation in the language. As a matter of fact, within our framework, we can easily formulate rules to

answer queries of the form: "Which people don't have a homepage?" Besides, the approach followed in this paper improves our previous filtering framework [1] which formalizes an exact tree embedding algorithm for filtering XML documents.

(ii) The filtering language has been implemented in the prototypical system Phil using a lazy functional language (Haskell). Through a thorough experimental evaluation, we will show the convenience of such a lazy implementation and the usefulness of the undertaken approach. In particular, we will illustrate how laziness automatically avoids the construction of unneeded data structures and allows to directly obtain a simple and efficient implementation of the filtering language.

Plan of the paper. The rest of the paper is structured as follows. In Section 2, we formalize our filtering language, while Section 3 illustrates how the filtering problem can be reduced to a tree matching problem. In Section 4, we outline our approximate pattern matching algorithm. Section 5 provides some experiments and a qualitative as well as quantitative evaluation of the implementation of our methodology with a particular focus on the benefits offered by lazy functional languages in this application domain. Finally, Section 6 concludes.

2 The Filtering Language

The filtering language we describe is a declarative, pattern-based language in which we can specify filtering rules as (possibly conditional) patterns. A filtering rule matches an XML document if the pattern is somehow "embedded" into the XML document and fulfills the desired relationships and conditions. Basically, a filtering rule can be formalized by means of the following syntax:

```
{count} <filterop> <pat> in <XML doc> where <cond> (<mode>)
```

which informally says that

(i) a pattern `pat` is searched in a document `XML doc`;

(ii) only detected instances of `pat` which satisfy the given condition `cond` are either extracted (*positive filtering*) or removed (*negative filtering*) from `XML doc` according to the value of the filtering mode `mode`. A filtering mode is a label belonging to the set {`P`,`N`}. Positive filtering rules are identified by means of the filtering mode `P`, while the negative one are denoted by `N`. Whenever a filtering rule does not specify a filtering mode, it has to be considered a positive filtering rule.

(iii) `count` is an optional operator which allows to count the number of pattern instances detected in the given XML document.

Moreover, several filtering operators `filterop` have been formulated to support approximate as well as exact matching mechanisms. Finally, note that, when no condition is specified, the `where` part of a filtering rule can be omitted.

In the remainder of this section, we present the syntax of each component of a filtering rule providing a brief explanation of the basic constructs of the language.

Filtering operators. We provide four filtering operators which can model both exact and approximative filtering w.r.t. a universal as well as existential semantics.

- `filterOneBest` is an operator that allows one to search for the best approximate match between a given pattern and an XML document. Approximate matching has to be intended modulo renaming, insertion and deletion of pattern items. More precisely, when no exact match is found, either some tag items of the filtering pattern may be renamed or new elements may be inserted/removed in order to find an approximate match. Any insertion/deletion/renaming operation has a fixed cost. `filterOneBest` returns the match with lower cost.

  Informally speaking, given a pattern, we try to generate a result for every position in the document where there is a tag that matches the pattern root. `filterOneBest` then selects the position referring to the document subpart that better matches the pattern. When there is more than one best approximate match, the operator will only deliver the first one it discovered.
- `filterAllBest` returns all the best approximate matches found, i.e. all the matches of minimum cost.
- `filterOneExact` is an operator that exactly matches a specified pattern against an XML document. In case that more than one exact match is detected, this operator will only deliver the first one it discovered.

  "Exactly" means that the labels and the structure of the pattern are preserved and precisely recognized inside the XML document. In other words, no renaming, insertion and deletion of pattern items are allowed to "adapt" the pattern to the given document.
- `filterAllExact` returns all the exact matches found.

Patterns. Patterns of filtering rules are used to describe the information we want to detect inside a given XML document. A pattern is built by composing the following syntactical elements.

- *Variables* (we assume to have a countable infinite set of variables {`X,Y,...`}).
- *Text selectors*, that is strings of plain text surrounded by single quotes (e.g. `'Dear friend'`). Text selectors will be matched against the textual part of the XML document.
- *Tag selectors* represent XML tags and are denoted by strings of characters (e.g. `author`, `book`,...). Tag selectors can be followed by the *occurrence* operator `[i]`, where `i`$\in \mathbb{N} \cup \{\text{last}\}$. Given a sequence of terms (i.e., XML documents) all rooted by a tag `t`, `t[i]` selects the `i`-th term. The keyword `last` is used to select the last element of the sequence (e.g. `book[1]`, `book[last]`).

  Moreover, tag selectors can be used together with the *synonymity* operator "`$`", which enables the flexible matching of tag selectors. More precisely, given a tag selector `t`, `$t` allows to match `t` against any synonym of `t` which has been defined by the user.
- The containment operator is represented by brackets "`()`" and it is used in combination with tag selectors and boolean operators to define boolean-connected structured patterns. Given a tag selector `t` and $\texttt{pat}_1, \ldots, \texttt{pat}_n$ patterns, the following syntactical expressions are legal patterns:

- `t(pat`$_1, \ldots,$ `pat`$_n$`)`. The comma "`,`" separator represents the logical conjunctive operator and allows to build conjunctions of patterns. For instance, the pattern `book(title(X),author(Y))` searches for all the books containing both a title `X` *and* an author `Y`.
- `t(pat`$_1|\ldots|$`pat`$_n$`)`. The "`|`" separator allows to model boolean disjunctions of patterns. For example, `book(title(X) | author(Y))` selects all the `book` instances containing a title `X` *or* an author `Y`.
- `t(pat`$_1?\ldots?$`pat`$_n$`)`. The separator "`?`" formalizes the boolean *xor* operator. One may use this operator to obtain the evidence of the existence of exactly one of the patterns in the list.

Operators "`,`" , "`|`" and "`?`" are called *inner boolean operators*. Note that brackets "`()`" are also used to specify precedence in boolean-connected patterns as shown in the following example:

```
pubs(report(author(X),year('2007'))|article(author(Y),year('2007')))
```

- Several patterns can be connected together at the root level by means of the outer boolean operators ("`and`", "`or`", "`xor`"). Given patterns `pat`$_1, \ldots,$ `pat`$_n$, the syntactical expression `pat`$_1$*op* $\ldots$ *op* `pat`$_n$, where *op* $\in \{$`and`, `or`, `xor`$\}$ is still a legal pattern.

  Although, outer and inner boolean operators behave very similarly, there is a subtle difference between them. Roughly speaking, when an inner boolean operator is used, it always refers to a parent node explicitly. For example, in the pattern `h(f(X),g(Y))`, the parent node of operator "`,`" is the tag selector `h`, so the patterns `f(X)` and `g(Y)` are somehow connected to the parent node `h`. When an outer operator is used, the parent node is not specified, and the boolean-connected patterns are executed independently.

XML documents and nested filtering rules. Filtering rules work on XML documents. There are three ways to supply an XML document to a filtering rule:

- directly giving the XML code. For instance, `filterOneBest a(X) in <a>b</a>`.
- giving the name of a file containing the XML data. In this case, the keyword `file` must precede the file name to be loaded. For instance,

  ```
  filterOneExact a(X) in file 'test.xml'.
  ```

- The execution of a filtering rule generates an XML document. Thus, the outcome of a filtering rule may be employed to feed another filtering rule. Or, equivalently, the result of an inner rule becomes the source document for an outer rule. Our language supports nested filtering rules with an arbitrary level of nesting. As an example, consider

  ```
  filterOneBest a(X) in
   (filterAllBest b(a(X),c()) in file 'test.xml' where X match [fg]*)
  ```

Conditions. The condition is an optional part of the filtering rule, which can be employed to further refine the search of a given pattern inside an XML document. Formally, a condition is a sequence `c`$_1, $`c`$_2, \ldots,$ `c`$_n$, where each `c`$_i$ can be

- a membership test of the form `X match RegExp`, where `X` is a variable occurring

in the pattern and `RegExp` is a regular expression [1] .

- an equation `s=t`, where `s` and `t` are terms built over a set of primitive operators and the set of variables occurring in the pattern. Note that terms may be non linear, that is, they may contain multiple occurrences of the same variable. Our language supports a number of built-in operators to deal with strings and numbers (arithmetic operators, string concatenation, equality over numbers and strings, *etc.*).

Counting the results. By executing a filtering rule on an XML document `doc`, we generate a new XML document containing one or more instances of a given pattern that are embedded into `doc`. However, we might be interested only in the number of embeddings found (e.g. we want to know the number of books written by an author). To model this feature, our language is equipped with the *counting* operator `count` which takes a filtering rule f and a maximum cost c as arguments. The result of applying the `count` operator to f and c is the number of embeddings found by executing f whose cost does not exceed the value c.

Some examples. The following examples formalize, within our framework, three queries (namely, query **Q3**, query **Q5**, and query **Q17**) of the XMark benchmark set [4], which is typically used to evaluate XML filtering and query languages. More precisely,

Q3: Return the IDs of all open auctions whose current increase is at least twice as high as the initial increase.

```
filterAllExact id(Z) in
 (filterAllBest $site($open_auctions(open_auction(id(Z),
                                     bidder[1](increase(X)),
                                     bidder[last](increase(Y)))))
      in file 'auction.xml' where 2*X <= Y (P))
```

Q5: How many sold items cost more than 40?

```
count 0 (filterAllExact price(X) in
   (filterAllBest site($closed_auctions(closed_auction(price(X))))
              in file 'auction.xml' where X >= 40 (P)))
```

Q17: Which people don't have a homepage?

```
filterAllExact site(people(person(name(X)))) in (
   filterAllBest site(people(person(name(X),homepage(Y))))
   in file 'auction.xml'(N))
```

Note that we introduced some occurrences of the "`$`" operator to explicity allow the flexible matching of some tag selectors.

3 Filtering is a Tree Embedding Problem

Filtering can be treated as a matching problem over trees. In fact, filtering rule patterns and XML document can be straightforwardly encoded into tree-shaped terms of a suitable term algebra.

[1] Regular languages are represented by means of the usual Unix-like regular expressions syntax [8].

On the one hand, a pattern can be interpreted as a tree in the following way:

- each variable and text selector is mapped to a leaf node;
- each tag selector and boolean operator is mapped to an inner node;
- the containment operator is interpreted as the standard tree parent-child relation.

The tree representing the pattern is called *pattern tree*. Figure 1(a) illustrates the tree encoding of the pattern `h(f('a',X)|g(Y?m('b')))`.

On the other hand, XML documents are provided with a tree-like structure in which plain text elements are mapped to leaf nodes, while tag elements define the inner structure of the tree. Note that XML tag attributes can be considered as common tagged elements, and hence translated in the same way. Precisely, the following piece of XML <tag att$_1$="val$_1$" ... att$_n$="val$_n$"> ... </tag> can be first translated into <tag><att$_1$>val$_1$</att$_1$>... <att$_n$>val$_n$</att$_n$> ...</tag>, and next encoded into a tree as described above. The tree representing the XML document is also called *data tree*.

By interpreting patterns and documents as trees, executing a filtering rule boils down to finding one (or all) the matches (i.e., embeddings) of a given pattern tree into a data tree. The recognized subtrees are then either selected or removed from the data tree according to the filtering mode of the rule. Therefore, our filtering mechanism is inspired by and slightly modifies the *unordered path inclusion problem*[6]. Roughly speaking, the unordered path inclusion of a tree T_1 in a tree T_2 is defined as an *injective* function from T_1 to T_2 that preserves labels of the nodes and parent-child relationships (i.e. the structure), but not the order of the siblings. Equivalently, it can be considered as a particular instance of the Kruskal's embedding relation[2]. We think that ignoring the order of siblings is favorable or even necessary for filtering XML data, because the ordering of the XML items may not be known to the user.

Following [11], our methodology discards the injectivity property, which is required in the path inclusion problem. Although this can imply a possible loss of precision of the computed results, the efficiency of the matching method is greatly improved, since the computed non-injective embeddings encompasses several injective embeddings at the same time. Moreover, while the unordered path inclusion problem typically searches for *exact* answers (in a sense that the labels and the

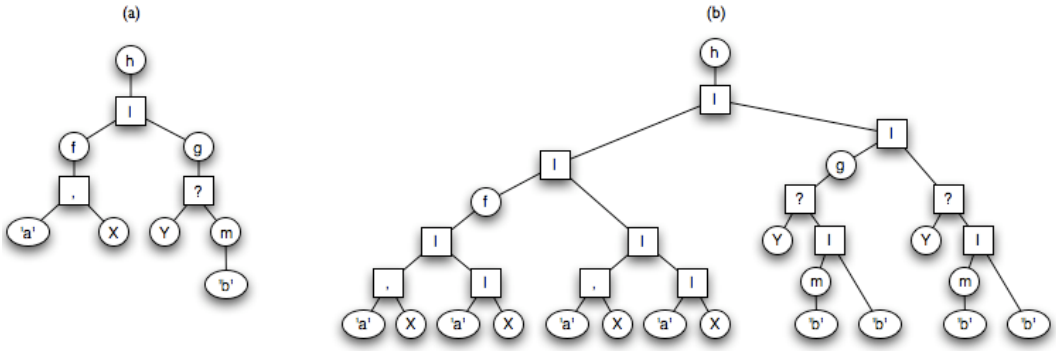


Figure 1. Tree encodings of a filtering rule pattern.

structure of the pattern tree are precisely embedded in the data tree), our goal is to find a match even when no exact instances of the pattern tree can be recognized inside the data tree.

To find such *approximate* results, we use pattern transformations, which minimally modify the original pattern tree and adapt it to the data tree with the aim of finding the best match which now might be not precise.

A pattern transformation consists of a sequence of basic transformations. Each basic transformation has a cost which is represented by a natural number. The total cost of a sequence of basic transformations is assigned to the matching result of the transformed pattern tree against the data tree and used to rank the result by increasing cost. We consider the following three types of basic pattern transformations.

Renaming. A Label l of a pattern tree inner node can be renamed with a new label l' provided that l' is a *synonym* of l. The synonymity relation might be explicitly provided by the user or automatically computed by querying an ontology. Renaming is enabled only for tag selectors to which the synonymity operator "$" is applied.

Deletion. A pattern tree node (corresponding to either a tag selector or a text selector) can be deleted, whenever it is not the pattern tree root.

Insertion. A new tree node (corresponding to a tag selector) can be inserted into the pattern tree. However, it is not allowed to add a new root or new leaf nodes. In particular, leaf nodes cannot be inserted, because they represent user-dependent data which cannot be automatically inferred by the algorithm.

In the next section, we describe a pattern-transformation algorithm which implements the strategy mentioned above.

4 An Approximate Tree Matching Algorithm

We start describing a basic procedure for approximate tree matching for ground patterns. Next, we will add all the other components of a filtering rule (variables, conditions,...). The core algorithm is a slightly modified version of the one proposed by Schlieder in [10] which not only finds a single best match, but also allows to find all the best matches of a pattern tree w.r.t. a data tree.

From a theoretical point of view, to evaluate a pattern tree $\mathcal{P}$ against a data tree $\mathcal{D}$, we can follow these steps:

1 Derive from $\mathcal{P}$, every pattern tree $\mathcal{P}'$ which is obtained by applying a sequence of basic transformations to $\mathcal{P}$ (i.e. we compute the *pattern transformation closure* w.r.t. the basic transformations) and compute the corresponding total cost $c_{\mathcal{P}'}$.

2 Find all the exact matches of $\mathcal{P}'$ against $\mathcal{D}$, for each $\mathcal{P}'$.

3 Group the matches found into embedding sets, where an embedding set is a set of matches which refer to the same subtree of the data tree.

4 From each embedding set, choose either one match or all the matches with lower cost $c_{\mathcal{P}'}$ according to the filtering operator applied.

5 Rank the selected matches according to their costs.

Obviously, the brute-force generation of the pattern transformation closure is not feasible, since it would lead to an infinite set of transformed patterns. Nonetheless, in the following, we will show a possible way to solve the problem, which exploits smart representations of data and pattern trees. Basically, all possible node deletions will be encoded in a single pattern tree, while renamings and insertions will be encoded into an index modeling the data tree.

4.1 Data Tree Encoding

As shown in Section 3, an XML document can be represented by means of a data tree in which leaf nodes represent plain text items and inner nodes represent XML tags. In the following, given a node u of a data tree $\mathcal{D}$ the *position* (or *preorder number*) of u in $\mathcal{D}$ is a natural number $n \geq 1$ assigned to u by a preorder traversal of $\mathcal{D}$. The position of the root node is 1.

In order to construct an approximate match of a pattern tree w.r.t. a data tree, some nodes may need to be inserted into the pattern tree or simply renamed. To avoid the explicit insertions of nodes into a pattern tree, we use a special encoding of the data tree which measures the insertion distance between two nodes in a data tree. Formally, given two nodes u and v of a data tree $\mathcal{D}$ such that u is an ancestor of v, the *insertion distance* between u and v is the sum of the insertion costs of all nodes along the path from u to v (excluding u and v). Moreover, information regarding label renaming of pattern tree nodes is formalized by providing an extensional representation of the synonymity relation (i.e. any node label is decorated with the list of its possible synonyms) along with the associated renaming cost.

The encoding is based on an indexing technique that is inspired by the *partial index* data structure which has been originally introduced in [11] and then successfully employed in the language ApproXQL[10]. The data structure is as follows.

Given a data tree $\mathcal{D}$, for each node u appearing in $\mathcal{D}$, we define a *posting* as a tuple containing the following information:

- pre(u) is the position of u in $\mathcal{D}$;
- dist(u) is the sum of the insertion costs of all ancestors of u in $\mathcal{D}$, which is computed by means of the insertion distance;
- bound(u) is the position of the rightmost leaf of the subtree of $\mathcal{D}$ rooted at u;
- rencost(u) represents the renaming cost of the pattern tree node that matches u,
- embcost(u) stores the cost of embedding a pattern subtree into the subtree of $\mathcal{D}$ rooted at u. The value is zero if u is the match of a pattern tree leaf.
- embtree(u) stores the pattern subtree embedded into the subtree of $\mathcal{D}$ rooted at u whose cost is embcost(u).

The first three fields are fixed and depend on the considered data tree, while the last three will change during the execution of the algorithm.

We now define a data tree *index* [2] which contains an entry for each label occur-

[2] For the sake of efficiency, the implementation indeed uses two indexes to encode the data tree: the former to store the leaf nodes of the data tree (i.e. the plain text elements) and the latter to store the inner nodes

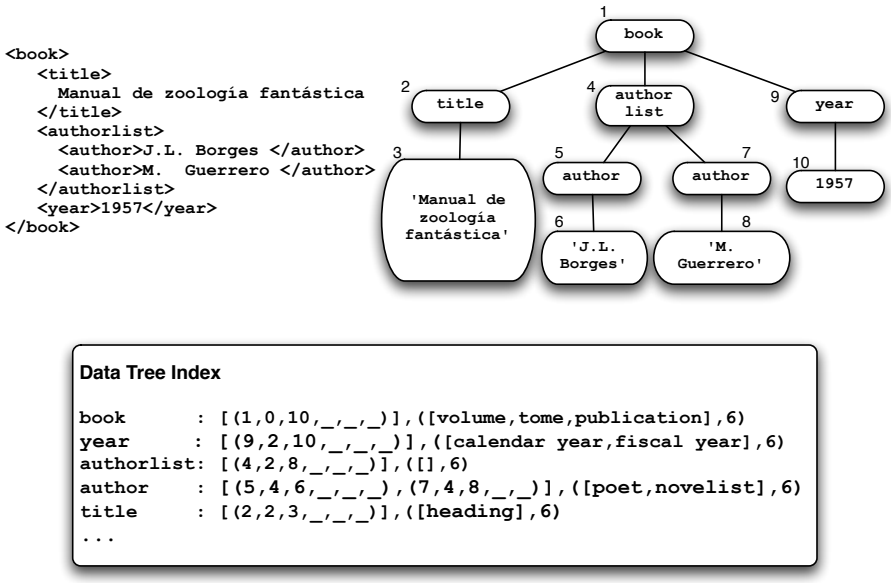


Figure 2. Data tree and data tree index for an XML document

ring in $\mathcal{D}$. An entry for a label l contains

- a list of all postings referring to nodes in $\mathcal{D}$ with label l. Such a list of postings is sorted by ascending preorder numbers.
- a pair containing a list of all synonyms of label l and the associated renaming cost;

Figure 2 illustrates the data tree $\mathcal{D}_{book}$ and the corresponding data tree index for a piece of XML. For the sake of readability, we omitted the postings regarding the leaf nodes of $\mathcal{D}_{book}$. Moreover, we assumed an insertion cost equal to 2 and a renaming cost equal to 6.

4.2 Expanded Pattern Tree

The expanded representation of a pattern tree allows one to explicitly encode all possible node deletions of a pattern tree node. More precisely, all permitted deletions of inner pattern tree nodes are represented by transforming the original pattern tree in the following way.

Every inner pattern tree node (except the root) w which represents a tag selector or a "," operator (i.e. boolean conjunction) [3] is replaced by a fresh binary "|"-labelled node w_{or}. The left child of w_{or} refers to w, while the right child represents the fact that the pattern tree node is removed from the pattern tree. Basically, the fresh "|"-labelled (i.e. boolean disjunction) nodes inserted play the role of *choice points* in which the algorithm is called to decide whether to delete a node. Figure 1(b) shows the expanded version of the pattern tree depicted in Figure 1(a). Observe

(i.e. the XML tags).

[3] Actually, in our system, we implemented an equivalent, optimized version of the pattern tree expansion which only replaces tag selector nodes, while boolean conjunctive nodes are implicitly managed by the pattern matching mechanism.

that we do not have to directly manage leaf deletions, which are hardcoded in the algorithm. Since leaves cannot be renamed, whenever a leaf node n of a pattern tree does not match any leaf node of the data tree, the pattern tree is automatically transformed by deleting n. Besides, the corresponding deletion cost is computed.

Now for every node w of the expanded pattern tree, we define a function delcost(w) which computes the total cost of deleting the node w and all its inner descendants. The returned value is greater than zero only for those right children of "|"-labelled node representing the deletion of an inner node.

In the following, sometimes an expanded pattern tree is simply called pattern tree.

4.3 Evaluating an unconditional, positive, ground filtering rule

For the sake of simplicity, we first describe how the methodology works on an unconditional, positive, ground filtering rule, and then we will describe how to implement the other language features. Basically, the problem amounts to finding one or all best approximate matches (i.e. also called embeddings) of a ground pattern against an XML document. We assume to have already generated both the expanded pattern tree $\mathcal{P}$ and the data tree index $\mathcal{D}$ representing the data tree.

The evaluation algorithm is based on the dynamic programming principle. The embedding cost of a subtree of the expanded pattern tree $\mathcal{P}$ rooted at a node w is calculated from (i) the embedding costs of the subtrees rooted at the children of w; (ii) the insertion distance between the match of w and the matches of the children of w.

All matches of pattern tree node labels against data tree node labels are stored in posting lists. To find the best approximate embedding of $\mathcal{P}$ in $\mathcal{D}$, the algorithm uses operations on postings. Two types of operations are needed.

(i) Given a posting list of potential ancestors and a posting list of potential descendants, the algorithm must find all ancestor-descendant pairs with the smallest embedding cost (*vertical axis*). Such an operation can be performed basically using some information stored in the posting list. Given a posting list S, by S_p, we denote the p-th posting of S. Moreover, $S_{[i,j]}$ represents the sublist of S containing the postings from position i to position j. If U is a posting representing a potential ancestor node u and R is a posting list including n potential descendants of u, recalling that the postings are ordered by preorder numbers, all the n descendants $v_1, \ldots, v_n$ of the node u must reside in the interval $R_{[j,j+n]}$ of the posting list R, since $\text{pre}(u) < \text{pre}(v_i) \wedge \text{bound}(u) \geq \text{pre}(v_i)$. When no deletions are allowed, the smallest embedding cost of u w.r.t. the descendants in R is thus calculated using the following formula

$$\begin{aligned} \text{embcost}(u) = \min\{\text{embcost}(R_k) + \text{dist}(R_k) \mid j \leq k < j+n\} - \\ \text{dist}(u) + \text{rencost}(u) + c_{ins} \end{aligned} \quad (1)$$

[4]

where c_{ins} is the insertion cost of a node. Whenever we also allow deletions,

[4] Given a data tree node v, which is represented by the posting R_k, embcost(R_k) (resp., dist(R_k)) stands for embcost(v) (resp., dist(v)).

the smallest embedding cost become the minimum between delcost(u) and the value computed by the formula (1).

(ii) Given two posting lists that represent the embeddings of two distinct children of a pattern tree node w, the algorithm must find all pairs that belong to the same data node (*horizontal axis*).

In this case, if w is connected to its children through a boolean conjunction, then the sum of the embedding costs must be calculated, whereas w is connected to its children via a boolean disjunction (i.e. "|" or "?" operator), then we must select the embedding with the cheapest cost.

When evaluating a ground filtering rule, the algorithm visits the pattern tree nodes in depth-first order. During the traversal, it fetches from the data tree index the posting lists belonging to the labels of the visited nodes. Arrived at the leftmost leaf, it joins the posting belonging to this leaf with the posting belonging to the parent node (*vertical axis*) and then proceeds the visit. If a node u has two or more children, then the cheapest combination of matches belonging to u's children is chosen and stored in the posting list generated for u (*horizontal axis*).

The posting lists returned by the algorithm contains the postings representing the transformed pattern that best matches the data tree along with the embedding cost.

4.4 Evaluating a generic filtering rule

As shown in Section 2, a filtering rule is a quite complex object which may contain non ground patterns and filtering conditions. Moreover, according to the filtering operator and the filtering mode chosen, it can be employed for approximate/exact matching and to implement positive as well as negative filtering. In the following, we briefly discuss how to adapt the algorithm presented in the previous section to cope with such features.

Nonground patterns and filtering conditions. Patterns may contain variables. Therefore, substitutions that bind such variables to subparts of the data tree must be computed during the matching process to find the desired embeddings.

We extend the matching algorithm to deal with variables in the following way. Given a nonground pattern tree $\mathcal{P}$, we consider the pattern $\mathcal{P}'$ which is obtained from $\mathcal{P}$ by removing all the variables in $\mathcal{P}$. For each node labelled with a variable we removed, we record its position into a list. Since $\mathcal{P}'$ is ground, we can apply the previous tree matching algorithm to find the embeddings of $\mathcal{P}'$ into the data tree. Moreover, as we saved the positions of the variables appearing in $\mathcal{P}$, we can compute the embedding substitutions by analyzing the matched subtrees in the data tree and hence selecting those parts which correspond to the variable positions. It can happen that a variable cannot be bound to a subtree, e.g. the algorithm computes an embedding for $\mathcal{P}'$ which requires some node deletions involving an ancestor of a variable node. In this case, such an embedding is simply discarded.

By using this approach, we can thus produce all the possible embedding substitutions for a nonground pattern $\mathcal{P}$ by simply analyzing all the embeddings of $\mathcal{P}'$ in the data tree. We can then apply such substitutions to filtering conditions to generate instantiated conditions and subsequently check their satisfiability.

Approximate and Exact Filtering. The algorithm we described is mainly used for approximate filtering. Nevertheless it can be employed for exact matching. We just have to look for matches with a null embedding cost, since no insertion, deletion, or renaming of nodes is allowed in this case. Therefore, to optimize the search one can think to ignore the basic pattern transformations. Unfortunately, the insertions are implicitly defined in the matching algorithm, and hence we cannot avoid them. However, deletions and renamings can be disabled in the following way: (i) to avoid deletion of nodes, we simply use the original pattern tree instead of using its expanded representation; (ii) to avoid renaming of nodes, the synonymity relation encoded in the data tree index is ignored.

Positive and Negative Filtering. As shown in Section 2, our language defines syntax constructs for negative filtering which incorporate the expressive power of negation into our formalism. To enforce such a behaviour, we just execute the negative filtering rule as it was a positive one. We then remove all the embeddings found from the data tree returning the desired filtered outcome. In order to exaclty remove all and only the desired information, we allow only to "filter out" exact matches. In other words, any negative filtering is always an exact filtering.

5 A Lazy Implementation: an Experimental Evaluation

The proposed filtering language has been implemented in the Phil system, which is freely available at `http://www.dimi.uniud.it/demis/#software`. The implementation has been written in the lazy functional language Haskell with the aim of showing how *laziness* can be particularly fruitful in developing such kind of applications. Lazy functional (and functional logic) languages allow one to somehow "minimize" the amount of information needed to be processed in order to evaluate expressions.

As we have seen in Section 4, our methodology employes a quite sophisticated data structure whose whole generation may be very time-expensive. Therefore, exploiting laziness in this context amounts to saying that only the portion of the data tree index which is strictly necessary to evaluate a filtering rule is generated with a consequent gain in the overall system performance. Our claim is supported by the following experiment. We have evaluated a given filtering rule on XML documents of increasing sizes in two distinct ways.

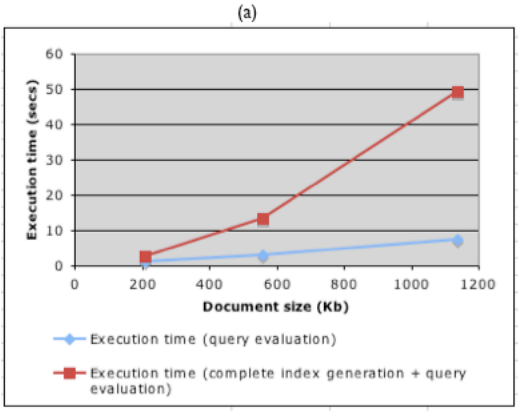
(i) We have forced the whole data tree index generation before executing the rule.

(ii) We have just "lazily" executed the given filtering rule letting Haskell to produce the (portion of) the data structure which is needed to process the rule.

The results of our experiment are depicted in Figure 3(a) and clearly show how laziness speeds up the query evaluation by automatically avoiding the construction of unnecessary data structure (e.g. for a of 1Mb XML document, the evaluation of the filtering rule in Case 2 is $\sim$7 times faster than the rule evaluation of Case 1).

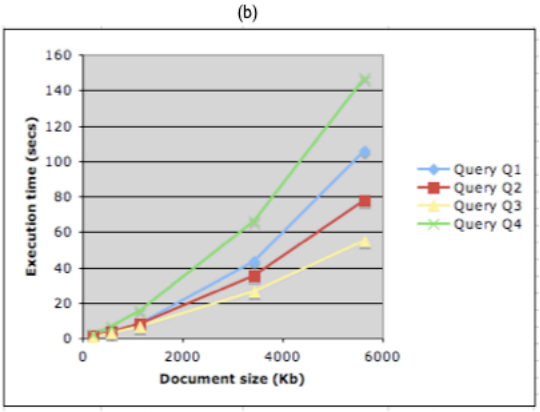
Qualitative and Quantitative Analysis. In order to evaluate the usefulness of our approach in a realistic scenario, we have benchmarked our system using the XMark benchmark suite[4]. The suite offers a set of 20 queries, each of which is intended to challenge a particular primitive of the filtering engine, along with the

(a)



Document size (Kb)	Ex. time: case 1 (s)	Ex. time: case 2 (s)
208	2.459	0.930
560	13.227	3.001
1136	49.212	7.492

(b)



Document size (Kb)	Ex. time Q1	Ex. time Q2	Ex. Time Q3	Ex. time Q4
208	0.985	1.267	1.101	2.183
560	3.046	3.941	3.158	6.812
1136	7.781	8.428	6.647	15.196
3424	43.053	35.248	26.674	65.821
5640	105.358	77.766	54.801	146.839

Figure 3. Experimental evaluation of the Phil System

XML documents generator xmlgen which can be used to produce the synthetic data on which running the experiments. By means of our formalism, we are able to express 17 queries out of 20. The remaining 3 queries cannot be formalized, since they involve document transformation and computational capabilities which are out of the scope of a simple filtering language (e.g. lexicographic ordering (query n. 19), currency conversion (query n. 18), and output formatting (query n. 10)).

From a purely quantitative point of view, we tested the system on a Macbook Intel Core 2 Duo 2Ghz equipped with 2Gb of RAM memory. We defined 4 filtering rules encompassing all the language features. Specifically, rule $Q1$ models a positive nested filtering rule with regular expressions, rule $Q2$ is a positive nested filtering rule which includes applications of the occurrence operator and arithmetic tests, rule $Q3$ employs the counting operator, and finally rule $Q4$ is an example of negative filtering. All the considered rules contain one or more occurrences of the synonymity operator.

Figure 3(b) shows the results obtained by executing the 4 filtering rules to 5 different, randomly generated XML documents which have been synthesized by xmlgen data generator. We tuned the generator in order to yield XML documents whose size ranges from 208Kb to almost 6Mb. Execution times of each filtering rule are computed as the average of three filtering rule's runs. The preliminary results are quite encouraging even on experiments that exceed the toy size: all the rules are evaluated in less than 3 minutes on a repository whose size is almost 6Mb. Moreover, it is a matter of few seconds obtaining an answer on a 1Mb XML document. Finally, note that negative filtering behaves worse than positive filtering. This is mainly due to the fact that, in the current implementation, negative filtering has to traverse the entire data tree (which may easily consist of more than 100000 nodes) in order to get rid of the detected patterns. Therefore, when dealing with large XML documents, the overhead due to the data tree traversal might be considerably high.

6 Conclusion

The growing complexity of the World Wide Web demands for tools which are able to tame the so-called information overload. To this respect, filtering and query languages allow one to extract relevant and meaningful information within the enormous amount of data available on the Web. In this paper, we firstly presented a declarative XML filtering language which has several advantages w.r.t. other approaches. It is inspired by the approximate pattern-based query language ApproXQL and extends it by introducing a number of new syntax constructs which provide a much more expressive framework (e.g. negative filtering, pattern variables, nested queries, conditional filtering, etc.). Secondly, we implemented the language in the prototype Phil using the lazy functional language Haskell pointing out the inborn benefits of laziness by means of a thorough experimental evaluation.

Finally, let us conclude by mentioning some directions for future work. We are currently formalizing a denotational semantics of the language which precisely models the behaviour of the language constructs. Moreover, we are trying to combine the filtering language with an ontology reasoner which simplifies and automatizes the retrieval of the synonymity relation by querying a given (possibly remote) ontology.

References

[1] D. Ballis and D. Romero. Filtering of XML Documents. In *Proc. of 2nd Int'l Workshop on Automated Specification and Verification of Web Systems (WWV'06), Paphos (Cyprus)*. IEEE Computer Society Press, 2006. To appear.

[2] M. Bezem. *TeReSe, Term Rewriting Systems*, chapter Mathematical background (Appendix A). Cambridge University Press, 2003.

[3] F. Bry and S. Schaffert. The XML Query Language Xcerpt: Design Principles, Examples, and Semantics. Technical report, 2002. Available at: `http://www.xcerpt.org`.

[4] Centrum voor Wiskunde en Informatica. XMark – an XML Benchmark Project, 2001. Available at: `http://monetdb.cwi.nl/xml/`.

[5] A. Cortesi, A. Dovier, E. Quintarelli, and L. Tanca. Operational and Abstract Semantics of a Graphical Query Language. *Theoretical Computer Science*, 275:521–560, 2002.

[6] P. Kilpeläinen. Tree Matching Problems with Applications to Structured Text Databases. Ph.d. thesis, University of Helsinki (Finland), 1992.

[7] W. May. XPath-Logic and XPathLog: A Logic-Programming Style XML Data Manipulation Language. *Theory and Practice of Logic Programming*, 2004.

[8] The Open Group. Unix Regular Expressions. Available at: `http://www.opengroup.org/onlinepubs/7908799/xbd/re.html`.

[9] S.Amer-Yahia, M. F. Fernández, D. Srivastava, and Y. Xu. Phrase matching in xml. In *Proc. of 29th International Conference on Very Large Data Bases (VLDB'03)*, pages 177–188, 2003.

[10] T. Schlieder. ApproXQL:Design and Implementation of an Approximate Pattern Matching Language for XML. Technical Report B 01-02, Freie Universität Berlin, 2001.

[11] T. Schlieder and H. Meuss. Querying and Ranking XML documents. *Journal of the American Society for Information Science and Technology JASIST*, 53(6):489–503, 2002.

[12] World Wide Web Consortium (W3C). Extensible Markup Language (XML) 1.0, second edition, 1999. Available at: `http://www.w3.org`.

[13] World Wide Web Consortium (W3C). XML Path Language (XPath), 1999. Available at: `http://www.w3.org`.

[14] World Wide Web Consortium (W3C). XQuery: A Query Language for XML, 2001. Available at: `http://www.w3.org`.

Using tccp for the Specification and Verification of Communication Protocols[1]

Alexei Lescaylle[a,2] Alicia Villanueva[a,3]

[a] *Dept. de Sistemas Informáticos y Computación*
Technical University of Valencia
Valencia, Spain

Abstract

The automatic analysis of cryptographic protocols by using formal methods on concurrent languages is a subject widely treated in the literature. From its beginning in the decade of the 70s, the field has been gaining maturity and consolidation. Some declarative approaches based on CLP, Haskell or Maude have been proposed for the specification and analysis of communication protocols. In this paper, we propose the Timed Concurrent Constraint Language (tccp in short) as the specification language for protocols. tccp is a declarative concurrent programming language which allows one to intuitively model concurrent and reactive systems. Cryptographic protocols can be specified in a very compact way thanks to certain features of the language such as the non-deterministic behavior and concurrency. We show how to specify the basic actions that are usually performed during a protocol run. Thereafter, we show how to specify the participants in the protocol. Finally, we specify the intruder model by means of a hostile environment in which principals run the protocol. The model considered for the intruder is the popular model of Dolev-Yao. We use along the paper the Needham-Schroeder public key authentication protocol to illustrate the approach. The basic actions and the intruder implementation can be reused for the specification of many different protocols.

Keywords: Communication Protocols, Timed Concurrent Constraint Language, Automatic Verification.

1 Introduction

Cryptographic (communication) protocols are protocols that use cryptography primitives to authenticate principals over a network. The network is usually assumed to be a hostile environment controlled by an intruder who can read, modify and delete messages sent by *honest* principals. Session establishment protocols aim to establish a secure connection among two or more principals in such a way that each principal is sure to be communicating to the principal he intends to. Dishonest participants are able to modify the protocol behavior, for example making a principal believe to be communicating with a principal when actually is communicating with a different one. The huge cost of designing, analyzing or verifying protocols does not depend

[1] This work has been partially supported by the EU (FEDER) and the Spanish MEC under grants TIN2004-7943-C04, HA2006-0007, and Valencian Government under Grant GV06/285

[2] Email: alescaylle@dsic.upv.es

[3] Email: villanue@dsic.upv.es

only on the protocol specification, but also on the environment in which the protocol is run. Formal methods have been shown to be effective tools when specifying, analyzing and verifying protocols thanks to their capability to detect *non intuitive* attacks, impossible to find otherwise. In that context, declarative techniques have also come up as a very convenient approach.

Logic programming languages have been applied to the problem of specification and analysis of communication protocols in different ways. For example, [19] was a pioneer work, where a reachability analysis was implemented in Prolog. In [1], a logic programming language was used to specify in a declarative way knowledge, protocol rules and intruder capabilities. Prolog was again used in [7] for the specification of protocol abstractions for proving the absence of attacks. Other declarative languages have also been shown suitable for the protocol specification and analysis problems. In [15], the specification language Maude was shown as an alternative approach. Later, in [11], an approach based on multiset-rewriting was presented. Another work based on multiset-rewriting was presented and implemented in OCAML [12,13]. Haskell has also been considered for the specification of communication protocols: in [5] we find a comparison (both qualitative and comparative analysis is done) between the specification of protocols in Maude and in Haskell.

The above mentioned literature shows how a formal concurrent language allows the designer to write unambiguous, clear, and concise specification. In this paper, we propose the Timed Concurrent Constraint Language (tccp in short, [9]) as the specification language. Its declarative and concurrent nature, together to some additional features such as the non-determinism, makes this language suitable for our purpose. On the one hand, the non-determinism behavior of tccp allows one to obtain compact specifications for systems. On the other hand, the agent-based model provides an intuitive way to specify principals. Finally, the notion of time allows us to intuitively model the interaction among principals, and to use the LTL logic to check temporal properties over protocol specifications.

Synchronous languages such as ESTEREL [6] or LUSTRE [18] have also been shown to be suitable for the specification and analysis of protocols. For example, in [23] the language ESTEREL is used to analyze the TMN protocol [24]. There, principals of the protocol (initiator, responder, server and intruder) are specified, and some security properties are verified.

We propose the tccp language for the specification of communication protocols due to its nature. tccp shares some concurrency features with synchronous languages. Moreover, it shares the formal nature and non-determinism with logic-based approaches. We intend to take advantage of both views, and use the formal verification tools defined for tccp ([2,3,17]) in order to verify protocols' properties.

The main contributions of this work are discussed in the following. We first show how we can specify the basic actions that principals can perform. Afterwards, we show how to model the participants of a protocol. We illustrate our proposal by using the Needham-Schroeder public key authentication protocol [20]. We use a general notation that can be used to specify other protocols, for example protocols in which messages can be partially encrypted, etc. Regarding the intruder, let us mention that we do not model the intruder in an explicit way, but we follow the approach of specifying a (hostile) environment which controls the messages

flaw. In other words, the Dolev-Yao intruder [16] is implicitly specified by a non-deterministic choice among (correct or incorrect) *actions* of the environment, such as losing a message, modifying a message, or even modifying the destination principal of a message. The specification of the basic actions as well as the specification of the intruder are defined in such a way that they can be reused for the specification of different protocols.

The remainder of this paper is organized as follow. In Section 2 we show an introduction to cryptographic protocols, and in particular to the Needham-Schroeder public key authentication protocol that will be our running example in the rest of the paper. An overview of tccp and an extension which makes specifications clearer is given in Section 3. Sections 4 and 5 present the method to model principals and the Dolev-Yao intruder. Section 6 is devoted to the verification of the protocol and discusses some related work. Finally, Section 7 concludes and discusses future work.

2 Cryptographic Protocols

A protocol can be seen as a recipe that enables the connection and data exchange between two or more entities through messages. We usually call these entities *principals*. In its simplest form, a protocol can be defined as the rules governing the syntax, semantics and synchronization of communication. Protocols are executed to achieve some specific goal. For instance, a typical goal is to establish a *secure* channel that principals would use later for exchanging confidential data. The principals can be users, hosts or processes [14] usually playing the role of initiator, responder or server. A protocol describes how messages must be sent from one principal to another, and how these principals react when receiving a message. Messages consist of atoms (principals' names, nonces, etc.) and, to guarantee confidentiality, they may be encrypted with a given key that only the target principal can decrypt. A nonce is a value (ideally) randomly generated by a principal which is usually sent to other principals in order to ensure the freshness of messages. As occurs typically in the literature, we assume that nonces cannot be guessed by intruders.

Cryptographic protocols are an important subclass of protocols where (perfect [4]) cryptographic techniques are used. In the following, we illustrate the notation that is normally used to represent a protocol.

- A and B are principals (Alice and Bob in the literature).
- K_{AP} and K_{AS} denote A's public and secret keys, respectively.
- K_{BP} and K_{BS} denote B's public and secret keys, respectively.
- N_A and N_B are nonces generated by A and B, respectively.
- $A \rightarrow B : M$ means that A sends to B the message M.

We show in Figure 1 the specification of the Needham-Schroeder public key authentication protocol [20] based on public-key cryptography without using trusted key servers. Some implicit assumptions are commonly made:

- any principal knows the public key of the principals involved in the protocol, and

[4] Perfect cryptography assumes that one cannot decrypt an encrypted message without the decryption key.

- only the owner of a secret key knows his secret key.

The second assumption implies that intruders cannot decrypt messages sent to other principals. Note that assumptions can be the source of many problems in the design and verification of protocols since make more difficult to find subtle problems unless an automatic verification tool were used.

1. $A \rightarrow B : \{A, N_A\}K_{BP}$
2. $B \rightarrow A : \{N_A, N_B\}K_{AP}$
3. $A \rightarrow B : \{N_B\}K_{BP}$

Fig. 1. Needham-Schroeder public key authentication protocol

As we can observe, this protocol establishes the order and contents of messages between two principals, A and B, that play the role of *initiator* and *responder*, respectively. Once the protocol has been completed, both principals are convinced about the identity of their partners. From that time instant, they can share critical information. In other words, the protocol ensures A to be communicating with B, and vice-versa. Let us describe each of the three steps of the protocol:

(i) A sends to B a message encrypted with B's public key (K_{BP}). The message contains A's name, and a fresh nonce generated by A (N_A). B decrypts the message by using his private key K_{BS}, thus B *knows* the content of the message.

(ii) B sends to A, encrypted with the public key of A, a message which contains the received nonce N_A, and a newly generated nonce N_B. A decrypts the message by using her secret key K_{AS}. Since the message contains the nonce N_A, A can deduce that the message has recently been sent by B.

(iii) A sends the nonce N_B to B, encrypted with B's public key K_{BP}. Then, B can infer that he is communicating to A since he had previously sent to A such nonce.

We use an asynchronous model for the specification of messages exchange. This means that each step in the protocol is modeled as the execution of two asynchronous actions: a principal sending a message, and a second principal receiving the message.

3 Introduction to the tccp Language

The Timed Concurrent Constraint Language (tccp) is a concurrent declarative language defined in [9] as an extension of the Concurrent Constraint Programming paradigm (ccp) [22]. In the ccp paradigm, the notion of *store as valuation* is replaced by the notion of *store as constraint*. The computational model is based on a global store where constraints are accumulated, and on a set of agents that interact with the store. Intuitively, the execution of a tccp program evolves by asking and telling information to the store. It is a simple but powerful model in which partial information can easily be handled. The model is parametric w.r.t. a cylindric constraint system $\mathcal{C}$ defined as follows:

Definition 3.1 [Constraint System [9]] Let $\langle \mathcal{C}, \leq, \sqcup, \mathit{true}, \mathit{false} \rangle$ be a complete algebraic lattice where $\sqcup$ is the lub operation, and *true*, *false* are the least and the

greatest elements of $\mathcal{C}$, respectively. Assume that *Var* is a denumerable set of variables, and for each $x \in Var$, there exists a function $\exists_x : \mathcal{C} \to \mathcal{C}$ such that, for each $u, v \in \mathcal{C}$:

1. $\exists_x u \leq u$
2. $u \leq v$ then $\exists_x u \leq \exists_x v$
3. $\exists_x(u \sqcup \exists_x v) = \exists_x u \sqcup \exists_x v$
4. $\exists_x(\exists_y u) = \exists_y(\exists_x u)$

Then, $\langle \mathcal{C}, \leq, \sqcup, true, false, Var, \exists \rangle$ is a *cylindric constraint system.*

Like in [3], we use the entailment relation $\vdash$ instead of its inverse relation $\leq$. Formally, given $u, v \in \mathcal{C}, u \leq v \iff v \vdash u$. The interested reader can find in [9,22] more details about the constraint system.

In tccp, it is possible to model behaviors typical from reactive or embedded systems. In these systems, the absence of information can cause the execution of an action. tccp introduces the conditional agent (now c then A else A) in order to model such behaviors. Let us briefly recall the syntax of the language:

$$A ::= \mathsf{stop} \mid \mathsf{tell}(c) \mid \textstyle\sum_{i=1}^{n} \mathsf{ask}(c_i) \rightarrow A_i \mid \mathsf{now}\, c\, \mathsf{then}\, A\, \mathsf{else}\, A \mid A||A \mid \exists x\, A \mid \mathsf{p}(\overline{x})$$

where c, c_i are *finite constraints* (i.e., atomic propositions) of $\mathcal{C}$. A tccp program P is an object of the form $D.A$, where D is a set of procedure declarations of the form $\mathsf{p}(\overline{x}) :- A$, and A is an agent. Intuitively, the stop agent finishes the execution of the program. $\mathsf{tell}(c)$ adds the constraint c to the store, but c will be available only in the subsequent time instant. The choice agent $\sum_{i=1}^{n} \mathsf{ask}(c_i) \rightarrow A_i$ checks whether the store satisfies the guards, and non-deterministically executes (in the following time instant) one of the agents A_i, provided its guard c_i were satisfied. In case no condition c_i is entailed, the choice agent *suspends*. The conditional agent (now c then $A1$ else $A2$) executes agent $A1$ if the store satisfies c, otherwise executes $A2$. $A1||A2$ executes the two agents $A1$ and $A2$ in parallel (the concurrent model used is *maximal parallelism*). The $\exists x\ A$ agent is used to define variables local to the process A. Finally, $\mathsf{p}(\overline{x})$ is the procedure call agent.

The notion of time is introduced by defining a global clock which synchronizes all agents. In the semantics of tccp, the only agents that consume time are the tell, choice, and procedure call agents. The store in the original tccp model can be seen as a blackboard where information is continuously written and never canceled. Stores grow monotonically, thus it is not possible to change the value of a given variable. In order to model the evolution of variable values along the time, we use *streams*: A stream allows one to handle imperative-style variables in the same way as logical lists work for concurrent logic languages. We write $X = [Y|Z]$ for denoting a stream X recording (i) the current value Y of the (imperative-style) variable, and (ii) the stream Z of future values of such variable.

The tccp model presents the problem that we cannot retrieve in a simple way the order in which the information has been added to the store, or the information added at a specific time instant. In [3], a new computational model was proposed in which a new notion of store, called *structured store*, was defined. This new

computational model allows us to recover the order in which the information is added during a computation. In this paper, we consider that framework instead of the original one since, as we will show later, it makes simpler some tasks such as the nonce generation.

A structured store consists of a timed sequence of stores where each store only contains the information added at a given time instant. Let us recall the definition:

Definition 3.2 [Structured Store [3]] A *structured store* is an infinite indexed sequence of stores where the ith component of the sequence st is denoted as st_i, and it represents the store at time i.

The new model makes possible to retrieve from the store in a simple way the current value of a stream, i.e., the last value added to the stream:

Definition 3.3 [Current value [3]] Given a stream S, a structured store st, and $t \in \mathbb{N}$. Then, A is the value of S in st at instant t, denoted by $st \models_t S = [A|As]$ or $S \doteq_t A$, iff $\exists m > 0$ such that:

$$st \models_t \exists A_1 \cdots \exists A_{m-1} \exists As \text{ such that } S = [A_1, \cdots, A_{m-1}, A|As] \text{ and } st \not\models_t \exists A' \exists As'.As = [A'|As']$$

Under these conditions, the length of stream S in the store st at time t is m, denoted by the term $len(S, st, t) = m$.

Following the notation described in [3], $st \vdash_t c$ is used to check if the information stored up to the tth element of st entails c. Moreover, $st \sqcup_t \{c\}$ is used to add the constraint c to the tth element of st.

In Figure 2 we show the transition relation $\longrightarrow$ of the operational semantics where $\longrightarrow \in (A \times \textsc{Store} \times \mathbb{N})^2$, A is a set of tccp agents, and $\mathbb{N}$ is the domain of natural numbers. In the figure, the symbol $\not\longrightarrow$ is used to indicate that it is not possible to evolve by using relation $\longrightarrow$, i.e., the execution suspends.

R1	$\langle \mathsf{tell}(c), st\rangle_t \longrightarrow \langle \mathsf{stop}, st \sqcup_{t+1} c\rangle_{t+1}$	
R2	$\langle \sum_{i=0}^{n} \mathsf{ask}(c_i) \rightarrow A_i, st\rangle_t \longrightarrow \langle A_j, st\rangle_{t+1}$	if $0 \leq j \leq n$ and $st \vdash_t c_j$
R3	$\dfrac{\langle A, st\rangle_t \longrightarrow \langle A', st'\rangle_{t+1}}{\langle \mathsf{now}\, c\, \mathsf{then}\, A\, \mathsf{else}\, B, st\rangle_t \longrightarrow \langle A', st'\rangle_{t+1}}$	if $st \vdash_t c$
R4	$\dfrac{\langle B, st\rangle_t \longrightarrow \langle B', st'\rangle_{t+1}}{\langle \mathsf{now}\, c\, \mathsf{then}\, A\, \mathsf{else}\, B, st\rangle_t \longrightarrow \langle B', st'\rangle_{t+1}}$	if $st \not\vdash_t c$
R5	$\dfrac{\langle A, st\rangle_t \not\longrightarrow}{\langle \mathsf{now}\, c\, \mathsf{then}\, A\, \mathsf{else}\, B, st\rangle_t \longrightarrow \langle A, st\rangle_{t+1}}$	if $st \vdash_t c$
R6	$\dfrac{\langle B, st\rangle_t \not\longrightarrow}{\langle \mathsf{now}\, c\, \mathsf{then}\, A\, \mathsf{else}\, B, st\rangle_t \longrightarrow \langle B, st\rangle_{t+1}}$	if $st \not\vdash_t c$
R7	$\dfrac{\langle A, st\rangle_t \longrightarrow \langle A', st'\rangle_{t+1},\ \langle B, st\rangle_t \longrightarrow \langle B', st''\rangle_{t+1}}{\langle A||B, st\rangle_t \longrightarrow \langle A'||B', st' \sqcup st''\rangle_{t+1}}$	
R8	$\dfrac{\langle A, st\rangle_t \longrightarrow \langle A', st'\rangle_{t+1},\ \langle B, st\rangle_t \not\longrightarrow}{\langle A||B, st\rangle_t \longrightarrow \langle A'||B, st'\rangle_{t+1}}$	
R9	$\dfrac{\langle A, st_1 \sqcup \exists x\, st_2\rangle_t \longrightarrow \langle A', st'\rangle_{t+1}}{\langle \exists^{st_1} x\, A, st_2\rangle_t \longrightarrow \langle \exists^{st'} x\, A', st_2 \sqcup \exists x\, st'\rangle_{t+1}}$	
R10	$\langle p(x), st\rangle_t \longrightarrow \langle A, st\rangle_{t+1}$	if $p(x) :- A \in D$

Fig. 2. Structured operational semantics.

Given a tccp program P, an agent A_0, and an initial structured store $st^0 = st_0^0 \cdot true^\omega$ where st_0^0 represents the first component of st^0, the *timed operational semantics* of P w.r.t. the initial configuration $\langle A_0, st^0\rangle$ is defined as

$$\mathcal{O}_T(P)[\langle A_0, st^0\rangle] = \{st = st_0^0 \cdot st_1^1 \cdot \ldots | \langle A_i, st^i\rangle_i \longrightarrow \langle A_{i+1}, st^{i+1}\rangle_{i+1} \text{ for } i \geq 0\}$$

*3.1 The **ask-tell** agent*

During the protocol specification process, an important issue is the generation of nonces that can be done by following different approaches. For instance, many approaches generate the new nonce by applying a (more or less complicated) function to the last generated nonce. In this paper, nonces are generated by recovering from a stream in the store the last generated nonce. Then, a new value is computed and stored. In tccp, this process must be done in two steps, so we define the ask-tell agent which mechanizes and makes clearer that task.

The new ask-tell agent instantiates a given variable with the current value of a given stream. In order to motivate the definition of this agent, let us first show some examples that illustrate the different behavior of some tccp agents. Let A and B be generic agents of the language:

(i) The agent now $X = 5$ then A else B executes A if the store satisfies the constraint $X = 5$; Otherwise B is executed. This agent **does not** force the binding of variable X to the value 5, i.e., the store does not change.

(ii) The agent ask $(X = 5) \rightarrow A$, which is a non-deterministic choice in which only one possible branch has been defined, executes (at the following time instant) the agent A provided the store satisfies $X = 5$; Otherwise the agent suspends. Again, this agent **does not** modify the store.

(iii) The agent tell$(X = 5)$ adds the constraint $X = 5$ to the store. The added information is available at the following time instant. This agent **may modify** the store by instantiating the variable X to 5.

As one can observe, consults do never add information to the store. They must be combined with tell agents in order to add any kind of information to the store. The ask-tell agent consults the store and updates it by instantiating a fresh variable.

Definition 3.4 Let S be a stream and A a fresh variable (i.e., a non-instantiated variable). The ask-tell(S, A) agent recovers in A the current value of stream S. Moreover, it instantiates A to such value, adding the corresponding information to the store. The new information (the value of A) will be available at the following time instant, similarly to the tell agent.

Figure 3 shows the operational semantic for the new agent where st is a structured store.

R11	$\langle \text{ask-tell}(S, A), st\rangle_t \longrightarrow \langle \text{stop}, st \sqcup_{t+1} A = Q\rangle_{t+1}$	if $S \doteq Q$, $free(A)$, and $len(S, st, t) > 0$

Fig. 3. Operational semantics for the ask-tell agent

4 Protocol Specification using tccp

This section presents how to encode protocols in tccp. We use the Needham-Schroeder protocol as a running example. The system consists of a set of principals modeled by procedure declarations, and a controller which models the interaction among principals. The controller can be seen as a synchronization process which actually models both, the *honest behavior* of principals, and the *intruder behavior*. We assume perfect cryptographic primitives. In particular, we assume that each principal can encrypt a message with the public key of any principal, and one principal can decrypt a message only if it has the corresponding secret key. Our notation allows one to specify protocols which use symmetric and shared keys instead of pairs of public and secret keys.

4.1 Facts and procedure declarations

In the following, we first show the set of terms that are handled by processes, i.e., terms that agents add/consult to the store. We also show some procedure declarations that implement basic actions executed by principals during a protocol run. For example, the generation of nonces, the decryption of a message, or the process of sending messages. All these definitions can be reused for the specification of protocols different from the Needham-Schroeder protocol.

`a(X).` The term `a/1` identifies the principal `X` as a participant of the protocol.

`kp(X).` The term `kp/1` identifies the public key of principal `X`. This information allows us to indicate which key has been used to encrypt a message.

`ks(X).` The term `ks/1` identifies the secret key of principal `X`. This information allows one to check whether a given principal can decrypt a message.

`sk(X,Y).` The term `sk/2` specifies a key which is shared by two principals: `X` and `Y`. Only these two principals know the shared key.

`enc(K,MsgItem).` The term `enc/2` represents a list of data elements `MsgItem` encrypted with the key `K`. `K` can be a public key `kp(X)`, or a shared key `sk(X,Y)`.

`msg(MsgCnt).` The term `msg/1` represents a message containing the list of items `MsgCnt` (nonces, encrypted messages, principal names, etc.). Note that each element in the list may be encrypted with a different key, or non encrypted at all.

`nonce(X,NonceSt).` The term `nonce/2` stores in the stream `NonceSt` the nonces generated by principal `X`.

`know(X,Info).` The term `know/2` stores the fact that principal `X` knows `Info`. `Info` can be instantiated to terms such as keys, names of principals, messages, etc.

`deliver(R,X,Y,M,P).` The term `deliver/5`, when present in the store, represents the fact that principal `R` has sent the message `M` to `Y`, (maybe) impersonatig `X`. Moreover, if the variable `P` is instantiated to `ok`, then the message has already been processed (traveled the network); Otherwise, it must still be delivered.

Let us now show some basic actions that are commonly part of protocols. An important action that is commonly part of protocol specifications is the generation of nonces. The generation process applied in this paper uses the value of the last

generated nonce to calculate the new nonce.

```
generator(X,N) :- ∃ S,St(tell(nonce(X,S)) ||
                         tell(S=[_|St]) ||
                         ask(true)→ask-tell(S,Q) ||
                                    ask(true)→tell(N is Q+1) ||
                                              ∃ Aux(tell(St=[N|Aux]))).
```

The predicate `generator/2` generates and stores a new nonce `N`. The first tell agent instantiates `S` to the stream containing the nonces generated by the principal `X`. Then, the new ask-tell agent instantiates `Q` to the current value of such stream. Since the information is only available in the following time instant, we force the execution to let pass one time instant by executing an `ask(true)` agent. At that point, the new nonce `N` can be generated and the stream of nonces of `X` is consistently updated.

The predicate `dec/2` models the process of a principal decrypting a message. The message consists of a list of elements that are recursively processed. These elements may be encrypted, thus the principal `X` will know the contents of the message only if knows the appropriate encryption key.

```
dec(X,Msg):-
   ∃ E,T(tell(Msg=[E|T])) ||
         ask(true) →
            now (E=enc(_,_)) then
                 ∃ K,L(tell(E=enc(K,L))||
                        ask(true) →
                           now((K=kp(X) ∧ know(X,ks(X))) ∨
                                K=sk(X,_) ∨ K=sk(_,X))
                           then tell(know(X,L))
                           else tell(know(X,E)))
            else tell(know(X,E)) ||
         now (T≠[]) then dec(X,Y,T) else stop).
```

Finally, the predicate `send/3`, models the action when Principal `S` sends a message `Msg` to principal `R` :

```
send(S,R,Msg) :- now (a(S) ∧ a(R) ∧ knows(S,kp(R))) then
                     ∃ Proc tell(deliver(S,S,R,Msg,Proc)).
```

A condition for the action to be taken is that both, `S` and `R` participate in the protocol. The process tells to the store that the message `Msg` must be delivered to principal `R`. It also stores the fact that `S` is the sender. The environment would decide whether the message is actually delivered or not. The asynchronous model for the communication becomes clear at this point. Note that `send` predicates are invoked by honest principals, thus they always impersonate themselves.

4.2 The participants

Once the basic terms and the basic actions have been modeled, let us show how the two participants in the protocol are modeled. Whereas the above described actions can be reused for modeling other protocols, the code for principals should

be redefined for each case.

Next we show the behavior of the protocol initiator in the Needham-Schroeder example, i.e., the A principal in Figure 1.

```
initiator(X,Y) :- tell(a(X)) ||
 ∃ N_A (now (know(X,kp(Y))) then
            (generator(X,N_A) ||
             ask(inst(N_A))→send(X,Y,[enc(kp(Y),[X,N_A])]) ||
             ask(know(X,[N_A,_])) →
                    ∃ I_1(tell(know(X,[N_A,I_1])) ||
                         ask(true)→send(X,Y,[enc(kp(Y),[I_1])]))))
        else stop).
```

First of all, the initiator stores that she is a participant of the protocol. Then, she checks whether she knows the public key of `Y`. In that case, sends to `Y` a message with her name and a new generated nonce.[5] In parallel, she waits until knowing a message containing the generated nonce. The process won't proceed until knowing (receiving) such message. Once received, the initiator sends the confirmation message to `Y`.

The model for the responder principal (principal B in Figure 1) is defined as:

```
responder(X,Y) :- tell(a(X)) ||
   ∃ N_B(now(know(X,kp(Y))) then
           ask(know(X,[Y,_]))→
              ∃ R_1(tell(know(X,[Y,R_1]))) ||
                   generator(X,N_B) ||
                   ask(inst(N_B))→ send(X,Y,[enc(kp(Y),[R_1,N_B])]) ||
        ask(know(X,[N_B])) → stop).
```

The responder waits until knowing (receiving) the initial message of the protocol. Once received, he generates a new nonce and sends the corresponding message to the principal `Y`. Finally, he waits for the last message of the protocol.

Up to this point, we have modeled the basic actions that are commonly performed during a protocol execution, and the principals participating in the Needham-Schroeder protocol. In the following section we show how the environment is defined to model the Dolev-Yao intruder.

5 Modeling the Intruder

The design of protocols turns out problematic even assuming perfect cryptography. The problem is mainly due to the fact that principals communicate over a network controlled by an intruder who can intercept, analyze, and modify messages, being thus able to carry out malevolent actions. These capabilities correspond to the Dolev-Yao attacker [16]. A man-in-the-middle attack (assuming the Dolev-Yao model) was detected for the Needham-Schroeder public key authentication protocol. We show the attack in Figure 4 in order to illustrate which kind of actions an intruder is able to do. I is, in principle, another participant of the protocol, just as

[5] We assume the constraint system can check whether a variable is non-free: `inst(N_A)`.

A or B, thus any principal could initiate a protocol run to communicate with him.

1. $A \longrightarrow I : \{A, N_A\}K_{IP}$
2. $I_A \longrightarrow B : \{A, N_A\}K_{BP}$
3. $B \longrightarrow I_A : \{N_A, N_B\}K_{AP}$
4. $I \longrightarrow A : \{N_A, N_B\}K_{AP}$
5. $A \longrightarrow I : \{N_B\}K_{IP}$
6. $I_A \longrightarrow B : \{N_B\}K_{BP}$

Fig. 4. Lowe's attack on Needham-Schroeder public key authentication protocol

In the attack, A initiates a protocol run with I, who (impersonating A) starts a second run of the protocol with B. In other words, the intruder asks B to initiate a communication session saying that he is A. At the end of the attack, B thinks he is communicating to A, which is false. We use the notation I_X to denote that the intruder I is playing the role of the principal X, and I for denoting the intruder acting as himself. We assume that the intruder I, as any other principal, has a key pair K_{IP} and K_{IS} corresponding to his public and secret key, respectively.

The sequence of steps in the attack is described below:

(i) A initiates a protocol run to communicate with the principal I.

(ii) I_A initiates a second protocol run to communicate with B playing the role of A. Therefore, the intruder I_A sends to B the nonce N_A just received from A.

(iii) B assumes that the message has been sent by A, thus it responds to I_A by sending the nonces N_B and N_A encrypted with the public key of A. Note that the intruder cannot decrypt the received message since it has been encrypted by using the key of A.

(iv) I sends to A the message just received from B.

(v) A, once received the message from I, responds to it by sending the nonce just received. Note that A assumes is communicating to I, thus the message will be encrypted by using the public key of I. This means that I will be able to recover the nonce N_B.

(vi) I, playing the role of A, completes the protocol by sending the nonce N_B to B, thus B believes that A has correctly established a session with him.

As we have said before, we model the intruder by implicitly implementing it in the environment. This means that the environment decides whether a message reaches its destination, whether it is modified, or even whether protocol runs are mixed. In Figure 5 we show the intruder role in `tccp`. Note that, when the environment receives a message, the message can non-deterministically be forwarded to the destination principal (modeling the correct behavior), be canceled (ignored), be sent to a different principal (without modification), or some principal can impersonate another one changing the sender information stored in the `deliver` term.

Let us describe the actions the environment can do. We have labeled the code in Figure 5 to make clearer the description. First of all, it must be said that the environment is defined as a cycle where, during each iteration, one of the specified actions labeled 1 to 11 is non-deterministically executed. The actions 2 to 11 correspond to the environment starting a protocol run (actually activating a prin-

cipal playing a specific role in the protocol). Note how the honest participants can interact with dishonest ones as `Z` in this example.

The first possible choice (labeled 1) models the case when someone has sent a message that must be delivered. The environment detects that there is a message to process, so non-deterministically decides what to do with such message. First of all, the environment could do nothing ($1a$), modeling the case when the message is lost (ignored). $1b$ models the case when the message is correctly delivered, thus the destination participant decrypts the contents of the message. The third and fifth options represent the case when the intruder impersonates the first participant, whereas in the rest of the options, the intruder impersonates the second participant. The environment processes each message only once. The last argument in the `deliver` term controls such restriction.

```
    environment(X,Y,Z) :- ∃N,M,Ms,Proc,Proc',R(
1       (ask (deliver(_,_,_,_,_)) →
             tell(deliver(R,X,Y,M,Proc)) ||
             tell(M=msg(Ms)) ||
             ask(true) →
                now (Proc/=ok) then
1a                ask(true) → stop ||
                                tell(Proc = ok) +
1b                ask(true) → dec(Y,Ms)
                                tell(Proc = ok) +
1c                ask(a(Z) ∧ Z≠Y ∧ Z≠X) →
                      tell(a(Z)) ||
                      dec(Z,Ms) ||
                      now(know(Z,[X,N])) then
                          ask(true)→tell(deliver(Z,X,Y,msg(kp(Y),[X,N])),Proc')) ||
                                tell(Proc = ok)
                      else stop +
1d                ask (a(Z) ∧ Z≠Y ∧ Z≠X) →
                      tell(a(Z)) ||
                      dec(Z,Ms) ||
                      now(know(Z,[Y,N])) then
                          ask(true)→tell(deliver(Z,Y,X,msg(kp(X),[Y,N])),Proc')) ||
                                tell(Proc = ok)
                      else stop +
1e                ask(a(Z) ∧ Z≠Y ∧ Z≠X) →
                      tell(a(Z)) ||
                      dec(Z,Ms) ||
                      now(know(Z,[N])) then
                          ask(true)→tell(deliver(Z,X,Y,msg(kp(Y),[N])),Proc')) ||
                                tell(Proc = ok)
                      else stop +
1f                ask (a(Z) ∧ Z≠Y ∧ Z≠X) →
                      tell(a(Z)) ||
                      dec(Z,Ms) ||
                      now(know(Z,[N])) then
                          ask(true)→tell(deliver(Z,Y,X,msg(kp(X),[N])),Proc')) ||
                                tell(Proc = ok)
                      else stop +
                else stop
         +
2, 3     ask(true) → initiator(X,Y) + ask(true)→ responder(Y,X) +
4, 5     ask(true) → initiator(Z,X) + ask(true)→ responder(X,Z) +
6, 7     ask(true) → initiator(Z,Y) + ask(true)→ responder(Y,Z) +
8, 9     ask(true) → initiator(X,Z) + ask(true)→ responder(Z,X) +
10, 11   ask(true) → initiator(Y,Z) + ask(true)→ responder(Z,Y)) ||
        environment(X,Y,Z).
```

Fig. 5. Procedure modeling the attacks that an intruder carries out in the hostile environment

We have shown here the fragment of the intruder that can affect the protocol run (the form of messages determines the fragment). To conclude, we show the initialization of the system:

```
run :- tell(know(alice,kp(alice))) || tell(know(alice,kp(bob))) ||
       tell(know(alice,kp(intruder))) || tell(know(bob,kp(alice))) ||
       tell(know(bob,kp(bob))) || tell(know(bob,kp(intruder))) ||
       tell(know(intruder,kp(alice))) || tell(know(intruder,kp(bob))) ||
       tell(know(intruder,kp(intruder))) || tell(know(alice,ks(alice))) ||
       tell(know(bob,ks(bob))) || tell(know(intruder,ks(intruder))) ||
       environment(alice,bob,intruder).
```

6 Verification of the specified protocol

In this section we show how it is possible to verify properties for the protocol. Recall that we want to use the model checker defined for the tccp language. The most important point, independently from the verification or validation technique used, is to identify which properties we must check, and how can we specify them. Note that depending on the protocol considered, some properties have no sense, whereas others are the ones characterizing the well-behavior of the system.

In our case, an interesting property is whether, nonces are always known by at most two principals. The situation when more than two principals know a given nonce characterizes a protocol run where a man-in-the-middle attack has happened. If we try to check the property by using a model checker and it is not satisfied, then the model checker shows a counterexample, i.e., a system execution where a nonce is known by too many people.

We use the linear temporal logic (LTL) presented in [8] for dealing with tccp programs in order to specify the property. Then, we use the tccp model checker in [17] to verify the property. In the LTL logic, a formula must be satisfied by all the possible executions in order to be valid. We can use the classical temporal operators *always* $\Box\phi$, *eventually* $\Diamond\phi$, *until* $\phi\,\mathcal{U}\,\psi$, *next* $\bigcirc\phi$, as well as an existential quantificator over variables and classical logic operators.

Let us assume that we can check whether a principal knows a given nonce by using the predicate `know-nce/2` (we can avoid this assumption by expanding the predicate). `know-nce(X,N)` is true whenever the store satisfies the following formula: `know(X,[N])` $\vee$ `know(X,[N,_])` $\vee$ `know(X,[_,N])`. Then, the following temporal formula must be satisfied by the system:

$\neg\Diamond$(∃ `N(know-nce(alice,N)` $\wedge$ `know-nce(bob,N)` $\wedge$ `know-nce(intruder,N)))`

The problem of this protocol comes up when a participant uses a nonce generated by a different participant in order to start a new protocol run. The intruder takes advantage making the responder to construct a message that he later will use in order to ask to the actual owner of the nonce.

Note that to specify a good property, i.e., a property that characterizes the well or bad behavior of a system, the user must know the goal of the protocol. Otherwise he could be proving properties that have no sense in a real environment.

We could also specify more sophisticated properties, for example to make explicit the amount of time that a principal waits for the completion of the protocol by another principal. For example, we could check whether the second confirmation comes up no later than 10 time units from the addition of the first one to the store. This kind of properties are especially interesting in the protocol verification context. A traditional LTL logic is not expressive enough to model such properties, thus we need a real-time logic as the one presented in [3].

In the introduction of this paper, we have already mentioned some related work where other declarative languages and different logics have been proposed for the specification of protocols in order to take advantage of the available analysis tools. Some of these languages share some good feature with tccp, but only tccp joins three of these features. First of all, it is a concurrent language in whose semantics time is modeled. Therefore, the specification of interactive or reactive systems is very intuitive. Moreover, it handles constrains, thus providing the capability to reason about partial information. CLP is a constraint based language that was proposed as an alternative. Constraints provide an important feature: to deal with partial information, and in particular with infinite-state systems. The idea of the CLP-based approaches is to transform the verification problem into a CLP model, and then to check the satisfiability of the generated model. In this paper, we use the constraint system to handle information and specify properties in a more compact way. We do not transform the verification process since we model the system and later we specify the property and use the model checker. Another difference between the two languages is that tccp is an agent-based model with time, which simplifies the specification of reactive systems.

7 Conclusions and Future Work

We have shown how tccp is a suitable specification language for communication protocols. The language was defined as a simple model to specify reactive and embedded systems. Features such as concurrency or non-determinism natural in tccp marry perfectly these systems. In this paper, we have seen that these characteristics are also useful when specifying communication protocols. We have defined a compact and intuitive translation from the informal specification of protocols to the formal one (in tccp). To improve the compactness and clarity of models, we have defined a new agent for tccp. We have also shown that many of the components in the defined model can be reused for the specification of other protocols. In this paper we have chosen the Needham-Schroeder protocol as a running example due to its simplicity and wide use in the literature but we can also model other protocols such as the Otway-Rees protocol [21].

The design of communication protocols sometimes seems an easy, even trivial task: usually protocols are composed by few simple steps. However, nowadays everyone know that, mainly due to the (hostile and powerful) nature of the environment in which a protocol is executed, formal methods are essential to guarantee the correctness and well behavior of these systems. By specifying protocols in tccp, we can use its model checker to verify, not only LTL properties, but also real-time temporal properties.

We plan to extend this work in several ways. First of all, we plan to compare our approach to others previously defined. We aim to compare, not only the clarity and compactness in the protocol specification, but also the kind of properties we are able to check. We also plan to move to a more specialized verification method in order to optimize the search algorithm, thus the verification time cost. Finally, we plan to study the impact of the nonce generation method, first by improving its implementation (maybe using the functional extension of tccp in [4]), and then modeling

the capability of guessing nonces by the intruder (thus removing the assumption of secure nonces).

References

[1] L.C. Aello and F. Massacci. Verifying security protocols as planning in logic programming. *Transactions on Computational Logic*, 2(4):542–580, 2001.

[2] M. Alpuente, M.M. Gallardo, E. Pimentel, and A. Villanueva. A Semantic Framework for the Abstract Model Checking of tccp Programs. *Theoretical Computer Science*, 346(1):58–95, 2005.

[3] M. Alpuente, M.M. Gallardo, E. Pimentel, and A. Villanueva. Verifying Real-Time Properties of tccp Programs. *Journal of Universal Computer Science*, 12(11):1551–1573, 2006.

[4] M. Alpuente, B. Gramlich, and A. Villanueva. A Framework for Timed Concurrent Constraint Programming with External Functions. *Electronic Notes in Theoretical Computer Science*, to appear, 2007.

[5] D. Basin and G. Denker. Maude versus Haskell: an Experimental Comparison in Security Protocol Analysis. *Electronic Notes in Theoretical Computer Science*, 36, 2001.

[6] G. Berry. *Proof, language, and interaction: essays in honour of Robin Milner*, chapter The foundations of Esterel, pages 425–454. MIT Press, Cambridge, MA, USA, 2000. ISBN 0-262-16188-5.

[7] B. Blanchet. An Efficient Cryptographic Protocol Verified Based on Prolog Rules. In S. Schneider, editor, *Proceedings of the 14th IEEE Computer Security Foundations Workshop*, 2001.

[8] F. de Boer, M. Gabbrielli, and M.C. Meo. A Temporal Logic for reasoning about Timed Concurrent Constraint Programs. In *Proceedings of the 8th International Symposium on Temporal Representation and Reasoning (TIME'01)*, page 227. IEEE Computer Society, 2001.

[9] F. S. de Boer, M. Gabbrielli, and M. C. Meo. A Timed Concurrent Constraint Language. *Information and Computation*, 161(1):45–83, 2000.

[10] M. Bozzano and G. Delzanno. Automated Verification of Secrecy Properties for Linear Logic Specifications of Cryptographic Protocols. *Journal of Symbolic Computation*, 38(5):1375–1415, 2004.

[11] I. Cervesato, N. Durgin, P. Lincoln, J. Mitchell, and A. Scedrov. A meta-otation for protocol analysis. In *Proceedings of the 12th Computer Security Foundations Workshop*, pages 55–69. IEEE Computer Society Press, 1999.

[12] Y. Chevalier, F. Jacquemard, M. Rusinowitch, M. Turuani, and L. Vigneron. CASRUL web site. `http://www.loria.fr/cassis/protheo/softwares/casrul`.

[13] Y. Chevalier and L. Vigneron. A tool for lazy verification of security protocols. 2001.

[14] J. A. Clark and J. L. Jacob. A survey of authentication protocol literature. Technical report, Defence Evaluation Research Agency, 1997.

[15] G. Denker, J. Meseguer, and C. Talcott. Protocol specification and analysis in Maude. In N. Heintze and J. Wing, editors, *Proceedings of Workshop on Formal Methods and Security Protocols*, 1998.

[16] D. Dolev and A. C. Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, March 1983.

[17] M. Falaschi and A. Villanueva. Automatic Verification of Timed Concurrent Constraint Programs. *Theory and Practice of Logic Programming*, 6(3):265–300, 2006.

[18] N. Halbwachs, P. Caspi, R. Raymond, and D Pilaud. The synchronous data-flow programming language LUSTRE. *Proceedings of the IEEE*, 79(9):1305–1320, 1991.

[19] C. Meadows. The NRL protocol analyzer: An overview. *Journal of Logic Programming*, 26(2):113–131, 1996.

[20] R. Needham and M. Schroeder. Using Encryption for Authentification in Large Networks of Computers. *Communications of the ACM*, 21(12):993–999, December 1978.

[21] D. Otway and O. Rees. Efficient and timely mutual authentication. *Operating Systems Review*, 21(1):8–10, 1987.

[22] V. A. Saraswat. *Concurrent Constraint Programming Languages*. The MIT Press, Cambridge, MA, 1993.

[23] R. K. Shyamasundar. Analyzing Cryptographic Protocols in a Reactive Framework. In *VMCAI*, pages 46–64, 2002.

[24] M. Tatebayashi, N. Matsuzaki, and D. Neuman. Key distribution protocol for digital mobile communication systems. In *Proc. CRYPTO' 89*, number 90, pages 324–333. Springer Verlag, 1989.